СОДЕРЖАНИЕ

ВВЕДЕНИЕ 8

1. ЛОГИЧЕСКИЕ ЭЛЕМЕНТЫ, ИХ ПОСТРОЕНИЕ НА ТРАНЗИСТОРЕ. СИНТЕЗ ЦИФРОВЫХ СХЕМ 10

1.1 Введение 10

1.2 Построение логических элементов 10

1.3 Синтез цифровых схем по таблице истинности 16

1.3.1 Карты Карно 18

1.4 Заключение 21

1.5 Варианты заданий 21

2. КОМБИНАЦИОННЫЕ СХЕМЫ, ЭЛЕМЕНТЫ ПАМЯТИ, СЧЕТЧИКИ .. 25

2.1 Введение 25

2.2 Комбинационные схемы 25

2.3 Триггеры 28

2.4 Счетчики 35

2.5 Заключение 43

2.6 Варианты заданий 43

3. ОПЕРАЦИОННОЕ УСТРОЙСТВО ЦП 48

3.1 Введение 48

3.2 Регистры и память ОУ 48

3.3 Арифметико-логическое устройство 50

3.4 Запись управляющего слова ОУ 53

3.5 Заключение 57

3.6 Варианты заданий 57

4. УСТРОЙСТВО УПРАВЛЕНИЯ ЦП УЧЕБНОГО ОУ 59

4.1 Введение 59

4.2 Устройство управления ЦП 59

4.3 Циклы выполнения команд 63

4.4 Заключение 68
4.5 Варианты заданий 68
5. ОЗУ. ВИРТУАЛЬНАЯ И ФИЗИЧЕСКАЯ ПАМЯТЬ 75
5.1 Введение 75
5.2 Устройство оперативной памяти и ее основные характеристики ... 76
5.3 Взаимосвязь программной и аппаратной частей работы современной ОЗУ 82
5.4 Заключение 89
5.5 Варианты заданий 89
6. ВНЕШНЯЯ ПАМЯТЬ 93
6.1 Введение 93
6.2 Развитие внешних запоминающих устройств 93
6.3 HDD (Hard Disk Drive) 104
6.4 SSD (Solid State Drive) 114
6.5 Тестирование показателей HDD/SSD 123
6.6 Заключение 126
6.7 Варианты заданий 126
7. ОБЩАЯ ОРГАНИЗАЦИЯ ПРЕРЫВАНИЙ 130
7.1 Введение 130
7.2 Аппаратные прерывания 131
7.3 Исключения 132
7.4 Программные прерывания 133
7.5 Общий механизм обработки прерывания 134
7.6 Схема процесса прерывания 136
7.7 Заключение 139
7.8 Варианты заданий 140
8. АРХИТЕКТУРА RISC-V 145
8.1 Введение 145

8.2 Архитектура RISC 145
8.3 Эмулятор процессора RISC-V 147
8.3.1 Установка эмулятора в ОС Windows 148
8.3.2 Установка эмулятора на дистрибутивах Linux 153
8.4 Создание первой программы на системе RISC-V 154
8.5 Инструменты разработки 156
8.5.1 GNU Make 156
8.5.2 GNU CMake 158
8.5.3 Отладчик gdb 160
8.6 Заключение 167
8.7 Варианты заданий 168
9. ВЗАИМОДЕЙСТВИЕ АССЕМБЛЕРА И ВЫСОКОУРОВНЕВЫХ ЯЗЫКОВ 169
9.1 Введение 169
9.2 Системные службы Linux 169
9.3 Взаимодействие ассемблера и C 179
9.4 Взаимодействие ассемблера и Python 187
9.5 Заключение 188
9.6 Варианты задания 189
10. АССЕМБЛИРОВАНИЕ И ДИЗАССЕМБЛИРОВАНИЕ, ОПТИМИЗАЦИЯ ИСХОДНОГО КОДА УЯЗВИМОСТИ В НИЗКОУРОВНЕВЫХ ПРОГРАММАХ 190
10.1 Введение 190
10.2 Оптимизация кода 191
10.3 Ассемблирование и дизассемблирование 196
10.4 Поиск уязвимостей в исходном коде 205
10.5 Заключение 219
10.6 Варианты заданий 219
11. ОРГАНИЗАЦИЯ ПАРАЛЛЕЛЬНЫЕ ВЫЧИСЛЕНИЯ 221

11.1 Введение ... 221
11.2 История возникновения параллельных вычислений ... 221
11.3 Классификация параллельных вычислительных систем ... 222
11.3.1 Таксономия Флинна ... 223
11.3.2 Уровни параллелизма ... 225
11.3.3 Классификация по организации памяти ... 225
11.4 Системы с общей памятью ... 226
11.4.1 Архитектуры систем с общей памятью ... 226
11.4.2 Технология OpenMP ... 227
11.4.3 Синхронизация и проблемы параллельного программирования ... 228
11.5 Системы с рапределенной памятью ... 229
11.5.1 Архитектуры систем с распределенной памятью ... 229
11.5.2 Технология MPI ... 230
11.5.3 Типы коммуникаций в MPI ... 231
11.5.4 Топологии и производительность ... 232
11.6 Метрики и оценка эффективности параллельных систем ... 232
11.6.1 Ускорение, эффективность, избыточность, стоимость, масштабируемость ... 232
11.6.2 Закон Амдала ... 234
11.6.3 Закон Густафсона ... 234
11.6.4 Закон Сана-Ная ... 235
11.7 Заключение ... 235
11.8 Варианты заданий ... 236
12. ГРАФИЧЕСКИЙ ПРОЦЕССОР ... 267
12.1 Введение ... 267
12.2 Составляющие части современных видеокарт ... 270
12.3 Основные различия в архитектурных подходах у компаний AMD, NVIDIA и Intel ... 287

12.4 Несколько слов о блокчейне и майнинге 313
12.5 Заключение 317
12.6 Варианты заданий 317
ЗАКЛЮЧЕНИЕ 346
СПИСОК ИСТОЧНИКОВ И ЛИТЕРАТУРЫ 347
СВЕДЕНИЯ ОБ АВТОРАХ 349

ВВЕДЕНИЕ

Учебно-методическое пособие предназначено для изучения дисциплины «Архитектура вычислительных машин и систем» и направлено на формирование целостного представления о принципах построения современных вычислительных устройств – от физического уровня до уровня прикладного программного обеспечения. В основе пособия лежит системный подход, отражающий эволюцию вычислительной техники и логическую взаимосвязь её компонентов.

Материал курса последовательно раскрывает переход от фундаментальных основ электроники к более высоким уровням абстракции. Рассматриваются логические элементы и их реализация на транзисторном уровне, что позволяет понять физические принципы работы цифровых устройств. Далее изучаются методы синтеза цифровых схем, а также принципы построения комбинационных схем, элементов памяти и счетчиков. Следующие разделы посвящены архитектуре центрального процессора, включая операционное устройство, арифметико-логическое устройство и устройство управления, а также особенностям выполнения команд. Рассматриваются способы организации оперативной и внешней памяти, графических процессоров. Поднимаются вопросы организации механизмов взаимодействия аппаратного и программного обеспечения, включая систему прерываний, а также современные архитектурные решения, такие как RISC-V. Практическая направленность курса обеспечивается изучением инструментов разработки, основ ассемблерного программирования и взаимодействия низкоуровневых и высокоуровневых языков. Отдельные разделы посвящены вопросам оптимизации программ, анализу уязвимостей и особенностям низкоуровневой разработки. Также описывается организация архитектуры параллельных вычислений и особенности их применения. Общая схема изучаемых тем отражена на рисунке 1.

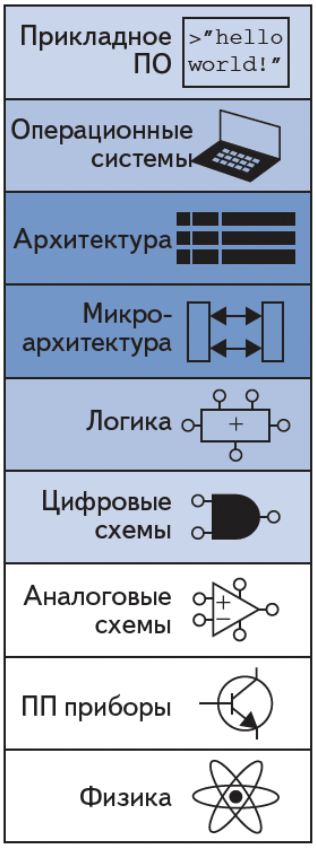


Рисунок 1 – Идеология рассмотрения архитектуры вычислительных машин

Таким образом, пособие охватывает полный путь – от физических основ работы полупроводниковых приборов и схемотехники до архитектуры вычислительных систем, операционных механизмов и прикладного программирования. Такой подход позволяет сформировать комплексное понимание принципов функционирования вычислительных систем и подготовить обучающихся к решению практических задач в области разработки и анализа современных компьютерных систем.

1. ЛОГИЧЕСКИЕ ЭЛЕМЕНТЫ, ИХ ПОСТРОЕНИЕ НА ТРАНЗИСТОРЕ. СИНТЕЗ ЦИФРОВЫХ СХЕМ

1.1. Введение

Вычислительные машины и системы строятся на основе логических элементов, реализованных с использованием электронных компонентов, главным из которых является транзистор. Понимание принципов построения базовых логических элементов и методов синтеза цифровых схем является фундаментом архитектуры современных процессоров, микроконтроллеров и других вычислительных устройств.

В ходе данной практической работы будет изучена структура и работа n- и p-канальных МОП-транзисторов, их использование в составе комплементарных логических элементов (КМОП). Рассматриваются схемы построения основных логических элементов – «НЕ», «И-НЕ», «ИЛИ-НЕ», «И», «ИЛИ», «Исключающее ИЛИ» – и их функциональные возможности. Кроме того, изучаются методы синтеза цифровых схем на основе таблиц истинности, включая построение совершенных нормальных форм (СДНФ и СКНФ), а также минимизированных форм (МДНФ и МКНФ).

Освоение этих навыков позволяет понять, каким образом из простейших элементов строятся сложные логические устройства, и как методы минимизации помогают сокращать количество аппаратных ресурсов, снижая энергопотребление и повышая производительность.

1.2. Построение логических элементов

Основой цифровой электроники являются **транзисторы**, которые можно использовать в качестве управляемых ключей. Схематически транзистор имеет три вывода: **исток**, **сток** и **затвор**. В зависимости от типа канала различают два основных типа МОП-транзисторов: **n-канальные (N-MOS)** и **p-канальные (P-MOS)**, при этом:

- для **n-канального МОП-транзистора** (рис. 1.1а) ключ замыкается (транзистор открывается), если на затвор подать высокий потенциал (логическая «1»). При низком уровне на затворе транзистор разомкнут;
- для **p-канального МОП-транзистора** (рис. 1.1б) ситуация обратная: при низком уровне на затворе транзистор открыт, при высоком – закрыт.

Благодаря противоположным свойствам N-MOS и P-MOS транзисторов их удобно использовать **совместно** в так называемой **КМОП-технологии (CMOS – Complementary MOS)**. Один и тот же входной сигнал одновременно открывает n-канальный транзистор и закрывает p-канальный, или наоборот. Это позволяет строить схемы, которые **не потребляют статический ток** и переключаются только при изменении входного состояния. Такая структура обеспечивает минимальное энергопотребление и высокую плотность интеграции.

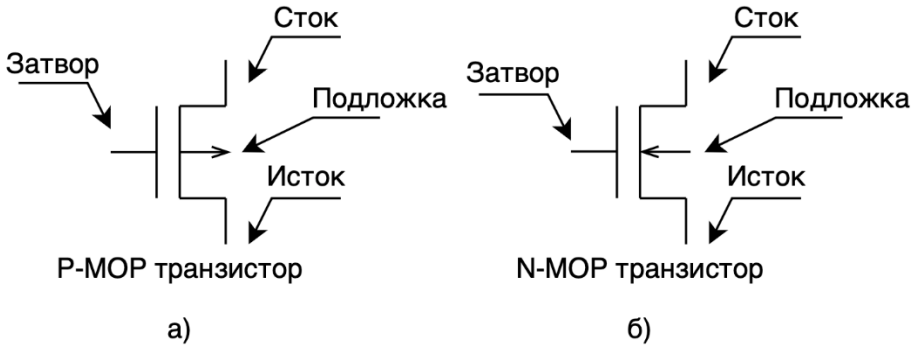


Рисунок 1.1. p-MOS и n-MOS транзисторы

Логический элемент «НЕ» (инвертор). Простейшим логическим элементом является инвертор (рис. 1.2). Он реализуется на одном p- и одном n-канальном транзисторе:

- если на вход подается логическая «1», то n-транзистор открыт, а p-транзистор закрыт, и на выходе формируется «0»;
- если на вход подается «0», то p-транзистор открыт, а n-транзистор закрыт, и на выходе формируется «1».

Инвертор необходим для изменения входного сигнала на противоположный.

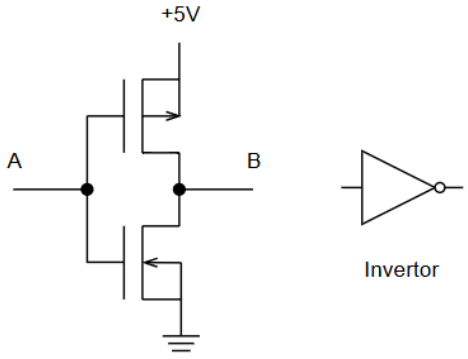


Рисунок 1.2. Схема элемента НЕ

Таблица 1.1 – Таблица истинности элемента «НЕ»

A	B
0	1
1	0

Логический элемент «И-НЕ» (NAND). Элемент «И-НЕ» можно построить на основе КМОП-транзисторов (рис. 1.3).

- в верхнем плече (p-канал) транзисторы соединяются **параллельно**;
- в нижнем плече (n-канал) транзисторы соединяются **последовательно**.

На выходе будет формироваться логическая «1», если хотя бы на одном из входов есть «0». Только при обоих входах равных «1» выход принимает значение «0». Таким образом, схема реализует функцию NAND.

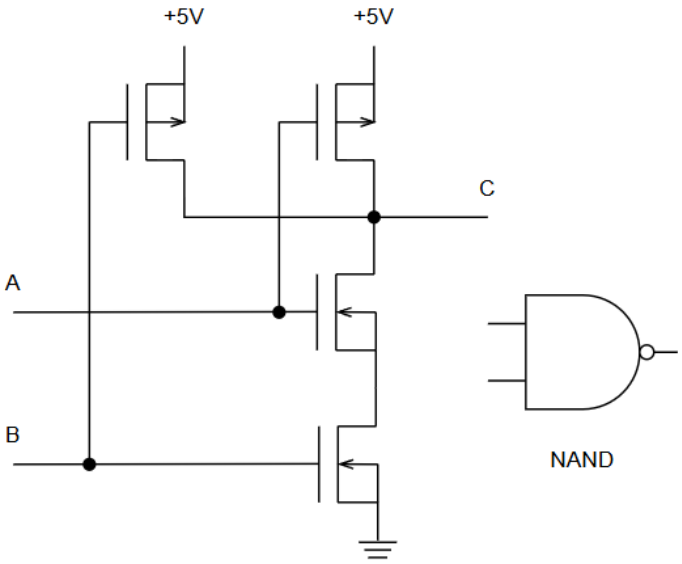


Рисунок 1.3. Схема элемента NAND

Таблица 1.2 – Таблица истинности элемента «И-НЕ»

A	B	C
0	0	1
0	1	1
1	0	1
1	1	0

Логический элемент «ИЛИ-НЕ» (NOR). Для построения элемента «ИЛИ-НЕ» используется обратное соединение транзисторов (рис. 1.4).

- в верхнем плече (p-канал) транзисторы соединяются **последовательно**;
- в нижнем плече (n-канал) транзисторы соединяются **параллельно**.

На выходе формируется логическая «1» только тогда, когда все входы равны «0». При появлении хотя бы одной «1» выход становится «0». Это соответствует функции NOR.

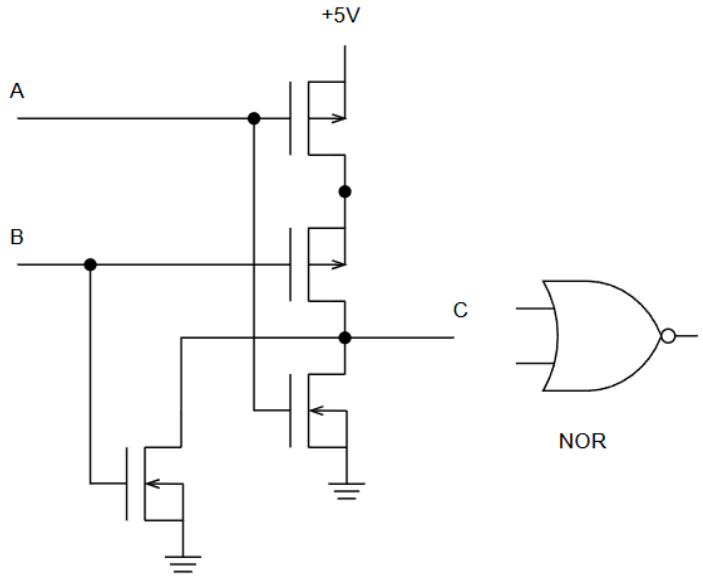


Рисунок 1.4. Схема элемента NOR

Таблица 1.3 – Таблица истинности элемента «ИЛИ-НЕ»

A	B	C
0	0	1
0	1	0
1	0	0
1	1	0

Логический элемент «И» (AND). Так как КМОП-топология естественным образом реализует NAND, то элемент «И» удобно получить на его основе: достаточно **подключить инвертор к выходу NAND** (рис. 1.5). Таким образом, выполняется преобразование «двойное отрицание», и схема реализует функцию логического умножения.

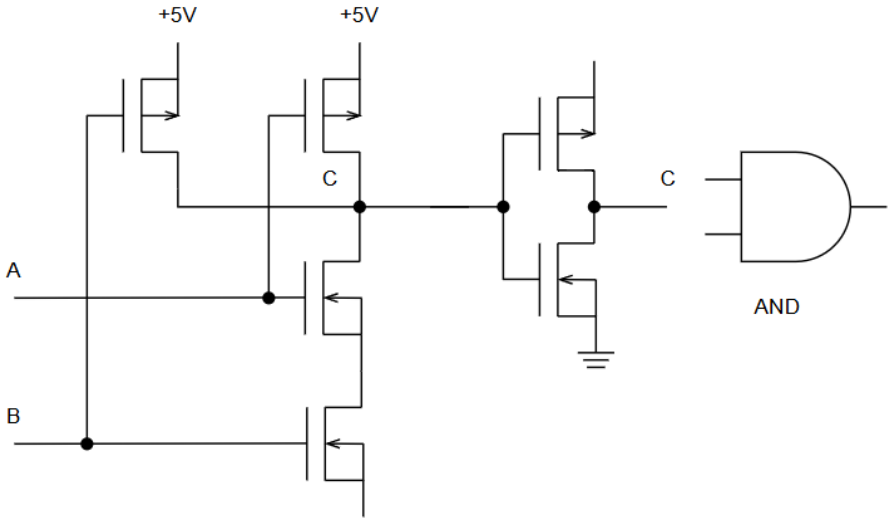


Рисунок 1.5. Схема элемента AND

Таблица 1.4 – Таблица истинности элемента «И»

A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

Логический элемент «ИЛИ» (OR). Аналогично элемент «ИЛИ» можно синтезировать из базовой схемы NOR. Если к выходу NOR добавить инвертор (рис. 1.6), то схема реализует функцию OR. Таким образом, OR строится как инверсия NOR.

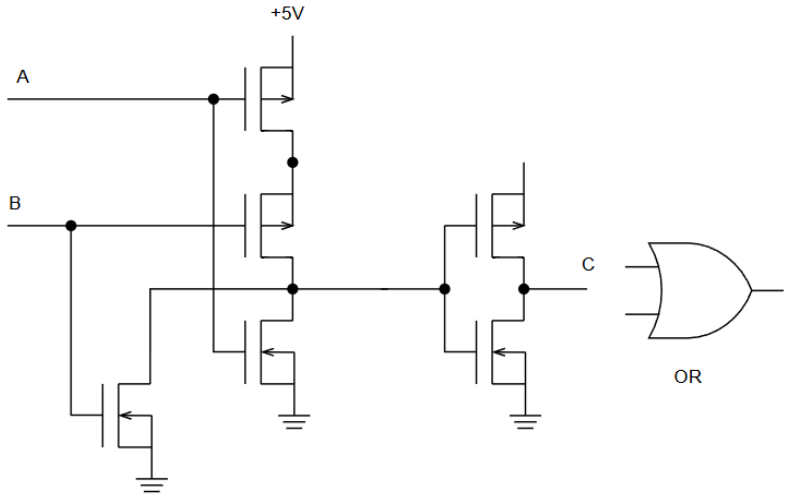


Рисунок 1.6. Схема элемента OR

Таблица 1.5 – Таблица истинности элемента «ИЛИ»

A	B	C
0	0	0
0	1	1
1	0	1
1	1	1

Логический элемент «Исключающее ИЛИ» (XOR). Синтез элемента «XOR» удобен через базисы ИЛИ-НЕ или И-НЕ (рис. 1.7а – базис И-НЕ, б – базис ИЛИ-НЕ). Реализуется через функцию (1.1):

$$F = x_1 \& \overline{x_2} \,||\, \overline{x_1} \& x_2 \tag{1.1}$$

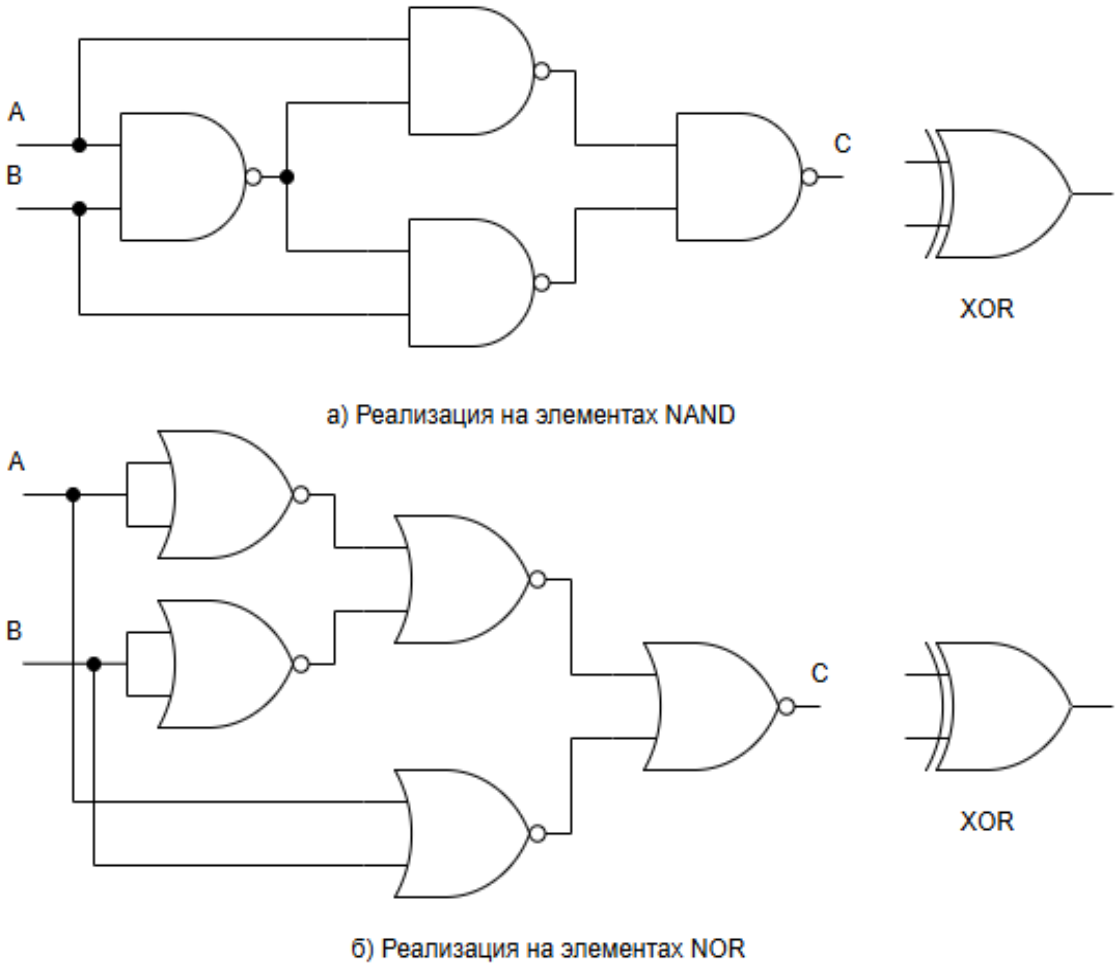


Рисунок 1.7. Схема элемента XOR

Таблица 1.6 – Таблица истинности элемента «Исключающее ИЛИ»

A	B	C
0	0	0
0	1	1
1	0	1
1	1	0

То есть выход равен «1», только если входы различны, в остальных случаях выход равен «0». На практике XOR строится из комбинации транзисторных цепочек, однако в интегральных микросхемах он реализуется как комбинация более простых базовых шлюзов.

1.3. Синтез цифровых схем по таблице истинности

Синтез цифровых схем начинается с построения таблицы истинности, которая задает поведение логической функции. В таблице перечислены все возможные комбинации входных переменных и соответствующие им значения выходной функции. На основе этой информации можно получить аналитическое выражение функции в различных нормальных формах. Наиболее часто используются дизъюнктивная нормальная форма (ДНФ) и конъюнктивная нормальная форма (КНФ). В ДНФ функция представляется как сумма (дизъюнкция) произведений (конъюнкций), а в КНФ – как произведение (конъюнкция) сумм (дизъюнкций).

Для точного описания логики вводятся совершенные формы. Совершенная дизъюнктивная нормальная форма (СДНФ) составляется из конъюнкций, каждая из которых включает все переменные функции в прямом или инверсном виде. Каждая такая конъюнкция соответствует строке таблицы, где функция равна единице. Аналогично, совершенная конъюнктивная нормальная форма (СКНФ) строится по строкам с нулевым значением функции, где каждая дизъюнкция также содержит все переменные. Таким образом, СДНФ и СКНФ обеспечивают однозначное и строгое задание логической функции через таблицу истинности. Рассмотрим пример, пусть логическая функция задана шестнадцатеричным вектором F=ADEF, построим таблицу истинности – табл. 1.7.

Таблица 1.7 – Таблица истинности логической функции F=ADEF

a	b	c	d	F	
0	0	0	0	1	A
0	0	0	1	0	
0	0	1	0	1	
0	0	1	1	0	
0	1	0	0	1	D
0	1	0	1	1	
0	1	1	0	0	
0	1	1	1	1	
1	0	0	0	1	E
1	0	0	1	1	
1	0	1	0	1	
1	0	1	1	0	
1	1	0	0	1	F
1	1	0	1	1	
1	1	1	0	1	
1	1	1	1	1	

Составим СКНФ (1.2) и СДНФ (1.3) для функции из табл. 1.7.

$$F_{\text{СДНФ}} = \bar{a}\&\bar{b}\&\bar{c}\&\bar{d}||\bar{a}\&\bar{b}\&c\&\bar{d}||\bar{a}\&b\&\bar{c}\&\bar{d}||\bar{a}\&b\&\bar{c}\&d||\bar{a}\&\bar{b}\&\bar{c}\&\bar{d}||\bar{a}\&b\&c\&d||$$
$$a\&\bar{b}\&\bar{c}\&\bar{d}||a\&\bar{b}\&\bar{c}\&d||a\&b\&\bar{c}\&\bar{d}||a\&b\&\bar{c}\&d||a\&b\&c\&\bar{d}||a\&b\&c\&d \quad (1.2)$$

$$F_{\text{СКНФ}} = (a||b||c||\bar{d})\&(a||b||c||\bar{d})\&(a||b||\bar{c}||\bar{d})\&(\bar{a}||b||\bar{c}||\bar{d}) \quad (1.3)$$

Однако такие формы, как правило, избыточны и содержат слишком много членов, что делает их непрактичными для реализации в цифровых схемах. Поэтому на практике широко применяется минимизация логических функций. Минимизированные формы называют минимальной ДНФ (МДНФ) и минимальной КНФ (МКНФ). Их цель – сократить число логических операций и тем самым уменьшить количество элементов при построении схемы. Существует несколько основных методов минимизации. Примерами

таких методов являются карты Карно и Квайна-МакКласки. При этом метод Квайна-МакКласки более универсален и подходит для большого числа переменных, хотя и сложнее в реализации. В этом методе все минтермы представляются в двоичном виде, затем группируются по числу единиц, после чего объединяются с устранением различий в одной переменной. Постепенно образуются более общие термы, а лишние исключаются. В итоге получается минимальная форма, которая совпадает с результатом карт Карно, но вычисляется строго по алгоритму и может быть автоматизирована.

1.3.1 Карты Карно

Карты Карно представляют собой графический способ группировки минтермов (в случае ДНФ) или макстермов (в случае КНФ). За счет объединения таких клеток удается упростить выражение, устраняя избыточные переменные. Метод карт Карно хорошо работает для функций с четырьмя или пятью переменными, так как остается наглядным и менее громоздким.

Рассмотрим пример, пусть логическая функция задана шестнадцатеричным вектором F=ADEF (табл. 1.7).

Переменные на карте Карно располагают по осям в порядке Грея, чтобы соседние клетки отличались ровно одной переменной. Для двух переменных карта имеет размер 2×2, для трех – 2×4, для четырех – 4×4. Такой порядок гарантирует корректное «соседство» и создает предпосылку для объединения единиц в прямоугольные группы, устраняющие лишние переменные. По таблице истинности значения функции переносят на карту: в клетки, соответствующие наборам входов с F=1, ставят единицы, в прочие – нули.

Ключевая идея карты — понятие соседства и «обертки» границ. Соседние клетки – те, что отличаются значением лишь одной переменной, причем крайние столбцы и строки являются соседями благодаря циклическому зацеплению карты (геометрически карта устроена как тор). Это означает, что разрешено объединять клетки через левую и правую границы, верхнюю и нижнюю, а также по углам, если это приводит к корректному прямоугольнику степенной площади. Минимизация сводится к покрытию всех единиц прямоугольными группами площадей 1, 2, 4, 8, 16 и т. д. (всегда степени двойки). Чем больше группа, тем больше переменных «поглощается» и исчезает из соответствующего терма. Например, группа из двух соседних единиц устраняет одну переменную, из четырех – две, из восьми – три. Допускается перекрытие групп и повторное покрытие одной и той же

единицы, если это помогает получить меньшее число более крупных групп. Чтобы группировать системно, ищут максимальные прямоугольники единиц, которые невозможно расширить, – это простые импликанты. Алгоритмически удобно «раздувать» каждый потенциальный прямоугольник до предела по всем направлениям, пока он остается полностью единичным. Получившийся набор прямоугольников фиксирует множество всех простых импликант, из которых затем выбирают нужные для минимального покрытия.

Среди простых импликант выделяют существенные: это такие группы, которые покрывают хотя бы одну единичную клетку, не покрытую ни одной другой группой. Существенные импликанты включают в решение обязательно. После их выбора некоторые единицы останутся еще не покрытыми – для них подбирают дополнительные простые импликанты так, чтобы закрыть все оставшиеся единицы минимальным числом групп. Получив набор групп, читают логическое выражение. Каждая группа порождает конъюнкцию из тех переменных, которые в пределах группы не меняют значение. Если переменная в группе всегда равна 1, она входит без инверсии; если всегда равна 0 – входит с отрицанием; если меняется, из терма исключается. Дизъюнкция всех таких конъюнкций дает минимальную ДНФ (МДНФ, 1.4). Двойственный процесс по нулям дает МКНФ (1.5): вместо единиц группируют нули, читают инвариантные переменные как члены дизъюнкций и берут конъюнкцию полученных скобок.

На рис. 1.8 – 1.13 показаны интервалы конъюнктов и дизъюнктов для построения МДНФ и МКНФ описанной в табл. 1.7 функции.

ab\cd	00	01	11	10
00	1	0	0	1
01	1	1	1	0
11	1	1	1	1
10	1	1	0	1

Рисунок 1.8. Область конъюнктов 1

ab\cd	00	01	11	10
00	1	0	0	1
01	1	1	1	0
11	1	1	1	1
10	1	1	0	1

Рисунок 1.9. Область конъюнктов 2

ab\cd	00	01	11	10
00	1	0	0	1
01	1	1	1	0
11	1	1	1	1
10	1	1	0	1

Рисунок 1.10. Область конъюнктов 3

ab\cd	00	01	11	10
00	1	0	0	1
01	1	1	1	0
11	1	1	1	1
10	1	1	0	1

Рисунок 1.11. Область конъюнктов 4

ab\cd	00	01	11	10
00	1	0	0	1
01	1	1	1	0
11	1	1	1	1
10	1	1	0	1

Рисунок 1.12. Область конъюнктов 5

ab\cd	00	01	11	10
00	1	0	0	1
01	1	1	1	0
11	1	1	1	1
10	1	1	0	1

Рисунок 1.13. Область дизъюнктов 1

ab\cd	00	01	11	10
00	1	0	0	1
01	1	1	1	0
11	1	1	1	1
10	1	1	0	1

Рисунок 1.14. Область дизъюнктов 2

ab\cd	00	01	11	10
00	1	0	0	1
01	1	1	1	0
11	1	1	1	1
10	1	1	0	1

Рисунок 1.15. Область дизъюнктов 3

$$F_{\text{МДНФ}} = a\&\bar{c}||b\&d||\bar{b}\&\bar{d}\big||a\&b\big|b\&\bar{c} \quad (1.4)$$

$$F_{\text{МКНФ}} = \left(b||\bar{c}||\bar{d}\right)\&(a||b||\bar{d})\&(a||\bar{b}||\bar{c}||d) \quad (1.5)$$

1.4. Заключение

Выполнение практической работы позволило закрепить знания о принципах работы МОП-транзисторов и их применении для построения логических элементов в КМОП-технологии. Студенты познакомились с реализацией основных логических функций и убедились, что на их основе можно построить любую цифровую схему.

Приобретенные навыки синтеза цифровых функций по таблице истинности и построения СДНФ, СКНФ, а также минимизированных форм (МДНФ, МКНФ) имеют ключевое значение в архитектуре вычислительных машин и систем. Минимизация функций позволяет оптимизировать цифровые устройства, что на практике означает уменьшение числа транзисторов, снижение энергопотребления и повышение надежности вычислительных систем.

Таким образом, выполненная работа формирует базовые компетенции для дальнейшего изучения архитектуры процессоров, организации памяти и построения логических узлов, являющихся основой вычислительных машин и современных информационных технологий.

1.5. Варианты заданий

Для выполнения заданий можно использовать различные специализированные среды, например, Logisim, Quartus, ViIDT (VAD)o.

1. Построить элемент «И-НЕ» на транзисторах. Построить СДНФ и МКНФ функции, заданной шестнадцатеричным вектором F=BABE. Продемонстрировать результат преподавателю.

2. Построить элемент «И-НЕ» на транзисторах. Построить СДНФ и МКНФ функции, заданной шестнадцатеричным вектором F=AAFD. Продемонстрировать результат преподавателю.

3. Построить элемент «ИЛИ-НЕ» на транзисторах. Построить СКНФ и МДНФ функции, заданной шестнадцатеричным вектором F=1F0A. Продемонстрировать результат преподавателю.

4. Построить элемент «ИЛИ» на транзисторах. Построить СДНФ и МКНФ функции, заданной шестнадцатеричным вектором F=3C7E. Продемонстрировать результат преподавателю.

5. Построить элемент «Исключающее ИЛИ» на транзисторах. Построить СКНФ и МДНФ функции, заданной шестнадцатеричным вектором F=00FF. Продемонстрировать результат преподавателю.

6. Построить элемент «И-НЕ» на транзисторах. Построить СДНФ и МКНФ функции, заданной шестнадцатеричным вектором F=1234. Продемонстрировать результат преподавателю.

7. Построить элемент «ИЛИ-НЕ» на транзисторах. Построить СКНФ и МДНФ функции, заданной шестнадцатеричным вектором F=B7A9. Продемонстрировать результат преподавателю.

8. Построить элемент «ИЛИ» на транзисторах. Построить СДНФ и МКНФ функции, заданной шестнадцатеричным вектором F=7E3B. Продемонстрировать результат преподавателю.

9. Построить элемент «Исключающее ИЛИ» на транзисторах. Построить СКНФ и МДНФ функции, заданной шестнадцатеричным вектором F=F0F0. Продемонстрировать результат преподавателю.

10. Построить элемент «И-НЕ» на транзисторах. Построить СДНФ и МКНФ функции, заданной шестнадцатеричным вектором F=9ABC. Продемонстрировать результат преподавателю.

11. Построить элемент «ИЛИ-НЕ» на транзисторах. Построить СКНФ и МДНФ функции, заданной шестнадцатеричным вектором F=0F0F. Продемонстрировать результат преподавателю.

12. Построить элемент «ИЛИ» на транзисторах. Построить СДНФ и МКНФ функции, заданной шестнадцатеричным вектором F=CAFE. Продемонстрировать результат преподавателю.

13. Построить элемент «Исключающее ИЛИ» на транзисторах. Построить СКНФ и МДНФ функции, заданной шестнадцатеричным вектором F=5AA5. Продемонстрировать результат преподавателю.

14. Построить элемент «И-НЕ» на транзисторах. Построить СДНФ и МКНФ функции, заданной шестнадцатеричным вектором F=55AA. Продемонстрировать результат преподавателю.

15. Построить элемент «ИЛИ-НЕ» на транзисторах. Построить СКНФ и МДНФ функции, заданной шестнадцатеричным вектором F=D34E. Продемонстрировать результат преподавателю.

16. Построить элемент «ИЛИ» на транзисторах. Построить СДНФ и МКНФ функции, заданной шестнадцатеричным вектором F=8E1C. Продемонстрировать результат преподавателю.

17. Построить элемент «Исключающее ИЛИ» на транзисторах. Построить СКНФ и МДНФ функции, заданной шестнадцатеричным вектором F=7777. Продемонстрировать результат преподавателю.

18. Построить элемент «И-НЕ» на транзисторах. Построить СДНФ и МКНФ функции, заданной шестнадцатеричным вектором F=2468. Продемонстрировать результат преподавателю.

19. Построить элемент «ИЛИ-НЕ» на транзисторах. Построить СКНФ и МДНФ функции, заданной шестнадцатеричным вектором F=9F00. Продемонстрировать результат преподавателю.

20. Построить элемент «ИЛИ» на транзисторах. Построить СДНФ и МКНФ функции, заданной шестнадцатеричным вектором F=3B2D. Продемонстрировать результат преподавателю.

21. Построить элемент «Исключающее ИЛИ» на транзисторах. Построить СКНФ и МДНФ функции, заданной шестнадцатеричным вектором F=E0E1. Продемонстрировать результат преподавателю.

22. Построить элемент «И-НЕ» на транзисторах. Построить СДНФ и МКНФ функции, заданной шестнадцатеричным вектором F=F1E2. Продемонстрировать результат преподавателю.

23. Построить элемент «ИЛИ-НЕ» на транзисторах. Построить СКНФ и МДНФ функции, заданной шестнадцатеричным вектором F=0A0B. Продемонстрировать результат преподавателю.

24. Построить элемент «ИЛИ» на транзисторах. Построить СДНФ и МКНФ функции, заданной шестнадцатеричным вектором F=4C4C. Продемонстрировать результат преподавателю.

25. Построить элемент «Исключающее ИЛИ» на транзисторах. Построить СКНФ и МДНФ функции, заданной шестнадцатеричным вектором F=AB12. Продемонстрировать результат преподавателю.

26. Построить элемент «И-НЕ» на транзисторах. Построить СДНФ и МКНФ функции, заданной шестнадцатеричным вектором F=C0FF. Продемонстрировать результат преподавателю.

27. Построить элемент «ИЛИ-НЕ» на транзисторах. Построить СКНФ и МДНФ функции, заданной шестнадцатеричным вектором F=1BAD. Продемонстрировать результат преподавателю.

28. Построить элемент «ИЛИ» на транзисторах. Построить СДНФ и МКНФ функции, заданной шестнадцатеричным вектором F=FACE. Продемонстрировать результат преподавателю.

29. Построить элемент «Исключающее ИЛИ» на транзисторах. Построить СКНФ и МДНФ функции, заданной шестнадцатеричным вектором F=00F1. Продемонстрировать результат преподавателю.

30. Построить элемент «И-НЕ» на транзисторах. Построить СДНФ и МКНФ функции, заданной шестнадцатеричным вектором F=3E3E. Продемонстрировать результат преподавателю.

31. Построить элемент «ИЛИ-НЕ» на транзисторах. Построить СКНФ и МДНФ функции, заданной шестнадцатеричным вектором F=B00B. Продемонстрировать результат преподавателю.

2. КОМБИНАЦИОННЫЕ СХЕМЫ, ЭЛЕМЕНТЫ ПАМЯТИ, СЧЕТЧИКИ

2.1. Введение

Основываясь на знаниях, полученных в ходе выполнения предыдущей работы, необходимо рассмотреть и более сложные устройства, которые входят в состав современных вычислительных систем. В данной работе предстоит рассмотреть основные комбинационные схемы, триггеры, регистры и счетчики. Построение этих устройств будет происходить с использованием простейших элементов (И, ИЛИ, НЕ).

2.2. Комбинационные схемы

Дешифратор (DC) – это комбинационная схема, которая имеет n входов и 2^n выходов. Условное графическое обозначение (УГО) дешифратора представлено на рис. 2.1, а его таблица истинности – табл. 2.1.

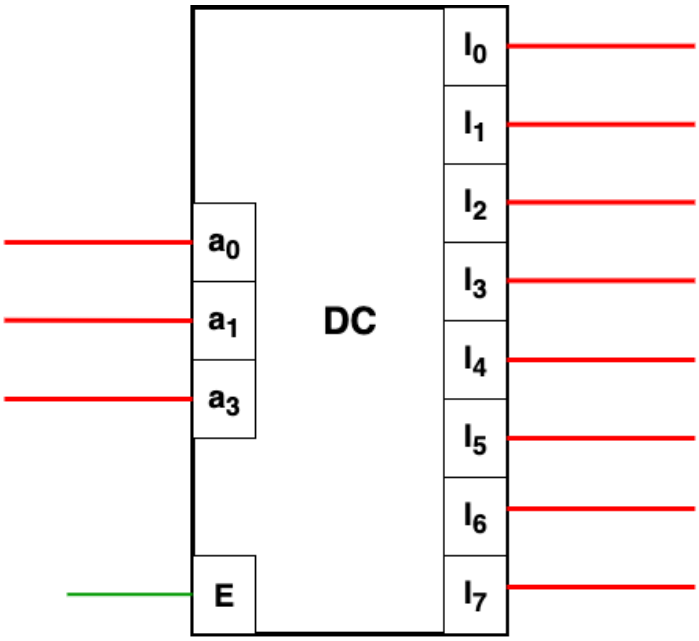


Рисунок 2.1. 3-х разрядный дешифратор

В зависимости от сигнала, поданного на вход дешифратора, формируется выходной сигнал «1» на заданном выходе, все остальные в этот момент времени равны «0». Вход «E» – это разрешающий вход, его задача предотвратить формирование сигнала на выходе, если на нем установлен «0». Наличие разрешающего входа у данного устройства не обязательно, как и у ряда других, которые будут рассмотрены в этой работе.

Таблица 2.1 – Таблица истинности дешифратора 3-8

	Вход			**Выход**							
				7	6	5	4	3	2	1	0

Обратное дешифратору устройство – это шифратор (Coder). Комбинационная схема, которая имеет 2^n входов n выходов. УГО шифратора представлено на рис. 2.2, его таблица истинности – в табл. 2.2.

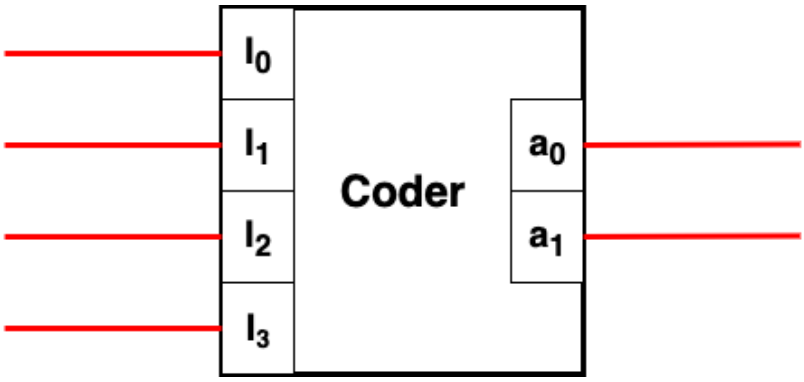


Рисунок 2.2. 2-х разрядный шифратор

Задача шифратора – это преобразование десятичного числа в двоичное представление. Важно сразу отметить, что шифраторы бывают двух типов: приоритетные и неприоритетные. Разница между ними в том, что у первого варианта сигналов «1» на входах может быть несколько, и восприниматься будет лишь тот, который имеет большее значение. Так, например, при подаче сигналов на 1-ю и 3-ю линии первой отработает 3-я, так как она имеет больший приоритет, в результате чего на выходе получается сигнал «11», что соответствует десятичному представлению числа «3». Во втором случае, на входе шифратора может быть только одна единица, а ее позиция на входе будет определять двоичное представление числа, как и в первом случае.

Таблица 2.2 – Таблица истинности шифратора 4-2

Вход				**Выход**	
3	2	1	0		

Следующее устройство – мультиплексор. Мультиплексор – это комбинационная схема, имеющая n адресных входов, 2^n информационных входов и только один выход. В зависимости от комбинации, поданной на адресные входы, на выход отправляется цифровой сигнал только с одного информационного входа. УГО данного устройства изображено на рис. 2.3, его таблица истинности – в табл. 2.3. В различных источниках мультиплексор может именоваться как «MS» или «MX».

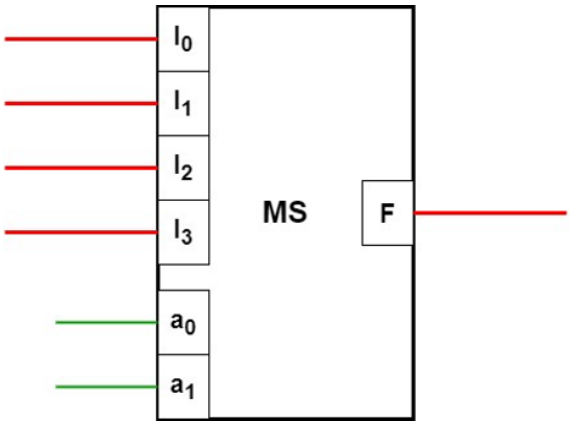


Рисунок 2.3. Мультиплексор 4-1

Таблица 2.3 – Таблица истинности мультиплексора 4-1

Вход						**Выход**
3	2	1	0	1	0	
						0
						1
						2
						3

Существует также и демультиплексор (DMUX) – рис. 2.4. Его принцип работы объединяет работу мультиплексора и дешифратора. Если задача мультиплексора передать сигнал с одной из многих входных информационных

шин на один выход, то задача демультиплексора передать сигнал с единственной входной шины на один из многих выходов, адресуемых адресными входами. На выходе этого устройства получается единица на одном из 2^n выходов, что соответствует работе дешифратора.

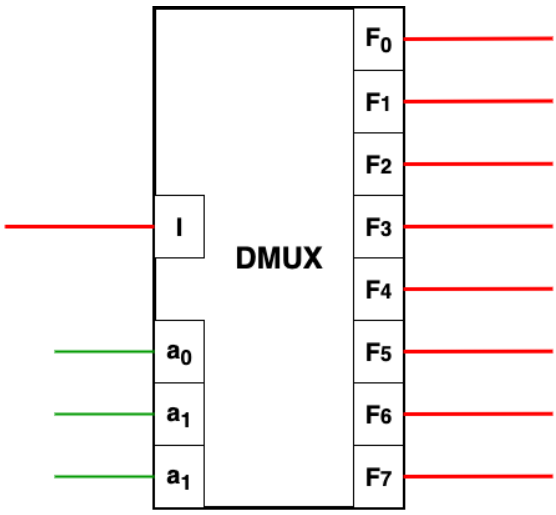


Рисунок 2.4. Демультиплексор 3-8

На основании этого можно составить таблицу истинности описанного устройства – табл. 2.4. Для простоты понимания, установим входной сигнал на информационном входе (I) в «1», если же сигнал на информационном входе будет «0», то и на адресованных выходах также будет «0».

Таблица 2.4 – Таблица истинности демультиплексора 3-8

Вход				**Выход**							
				7	6	5	4	3	2	1	0

2.3. Триггеры

Следующими не менее важными элементами являются триггеры. Триггер – это наименьшая ячейка памяти, которая способна хранить 1 бит информации. Существует множество видов триггеров, многие из них

подразделяются на синхронные и асинхронные версии. Синхронные версии переключаются только по приходу нужного фронта тактового сигнала на синхронном входе (обычно обозначается как «c» или «clk»). Асинхронные, соответственно, не требуют для своей работы этого сигнала, и переключаются по факту изменения входных сигналов.

RS-триггер, англ. Reset-Set trigger (Триггер Сброса/Установки) – самый первый и самый простой вид триггеров, который имеет 2 входных сигнала и 2 выходных сигнала. На рис. 2.5 показано условное графическое изображение, и его внутреннее устройство.

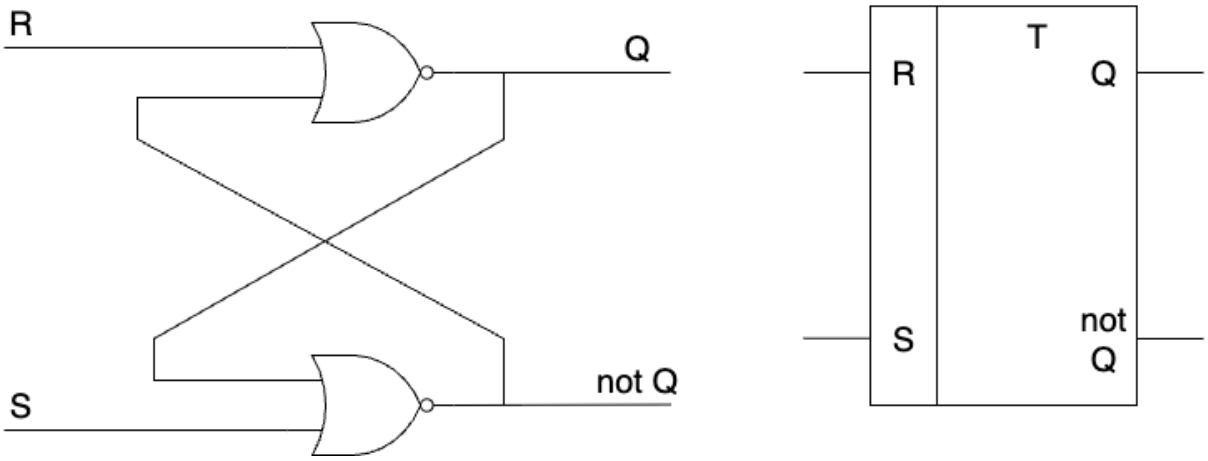


Рисунок 2.5. RS-триггер асинхронный

Для этого триггера существуют 3 разрешенных состояния и 1 запрещенное, в котором получается неопределенное состояние. Отобразим все состояния этого триггера в таблице истинности – табл. 2.5.

Таблица 2.5 – Таблица переходов асинхронного RS-триггера

R	**S**	**Q**	**Q (t+1)**
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	X
1	1	1	X

Для того, чтобы как-то предотвратить появление запрещенных комбинаций, стали добавлять синхросигнал, который позволил предупредить

запрещенные входные сигналы, в случае их формирования. Рассмотрим схему и условное графическое обозначение этого триггера – рис. 2.6, а также его таблицу переходов – табл. 2.6.

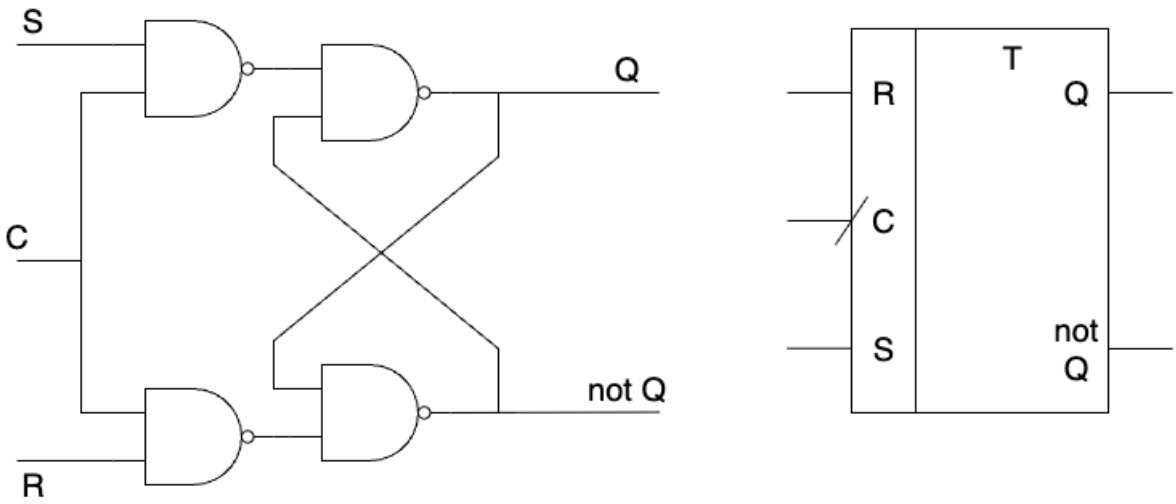


Рисунок 2.6. RS-триггер синхронный

Переключение состояния триггера происходит только, когда приходит требуемый фронт, графически он изображается на самом входе С. Фронтом сигнала (так как ток имеет волновую, прямоугольную структуру в цифровых схемах) называют повышение напряжения с низкого (0) до высокого (1) или в обратную сторону, понижение с высокого (1) до низкого (0). В первом случае при повышении фронт называется передним, во втором случае задним. Если, синхросигнал на УГО устройства имеет символ «/» то переключение происходит на переднем фронте, если же «\» то на заднем.

Таблица 2.6 – Таблица переходов синхронного RS-триггера

C	R	S	Q	Q (t+1)
0	*	*	0	0
0	*	*	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	X
1	1	1	1	X

Существуют также и двухступенчатые версии RS-триггеров, их преимущество заключается в дополнительной защите, за счет добавления этой самой второй ступени, при этом запрещенные комбинации в них все равно присутствуют. Задача двухступенчатого RS-триггера быть основой для более сложных версий, таких как T или JK триггеров, о которых речь пойдет дальше. Изобразим структурную схему и УГО синхронного двухступенчатого RS-триггера – рис. 2.7 – 2.8.

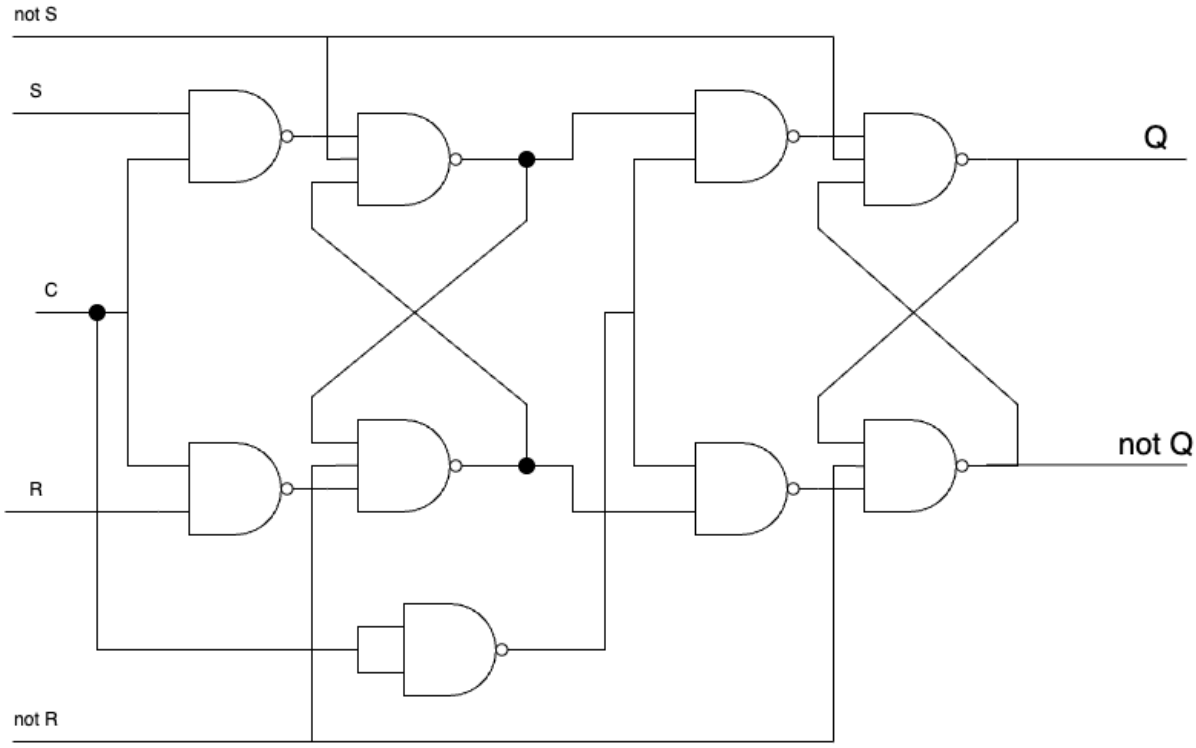


Рисунок 2.7. RS-триггер синхронный двухступенчатый

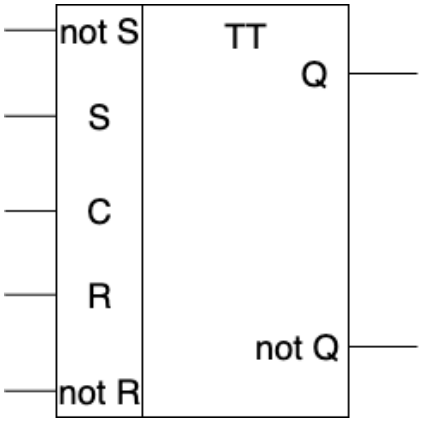


Рисунок 2.8. RS-триггер синхронный двухступенчатый УГО

D-триггер. Delay-trigger (Триггер задержки) – второй и, по совокупности, самый распространенный триггер, который, зачастую, применяется для построения более сложных устройств, таких как, например, счетчики и регистры. Составим его таблицу переходов, структурную схему и условное графическое обозначение – рис. 2.9, табл. 2.7.

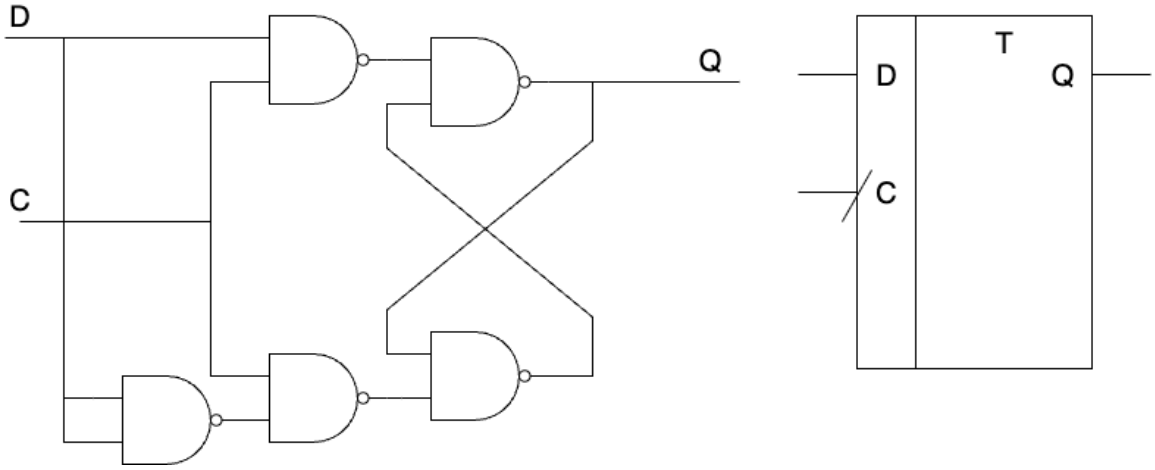


Рисунок 2.9. D-триггер

Данный триггер строится на базе одноступенчатого синхронного RS-триггера, но, в отличие от него не имеет запрещенных комбинаций. На представленной схеме нет инверсного выхода, однако, он может быть добавлен при необходимости (данное пояснение касается и остальных триггеров). Таблица переходов D-триггера выглядит следующим образом.

Таблица 2.7 – Таблица переходов D-триггера

C	D	Q	Q (t+1)
0	*	0	0
0	*	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Более сложной разновидностью триггеров является Т-триггер, его еще называют счетным триггером. В его основе лежит упомянутый ранее двухступенчатый RS-триггер, но с некоторыми доработками в лице отсутствующих R и S входов, а также наличием обратных связей от второй ступени к первой. Изобразим его структурную схему и УГО – рис. 2.10 – 2.11.

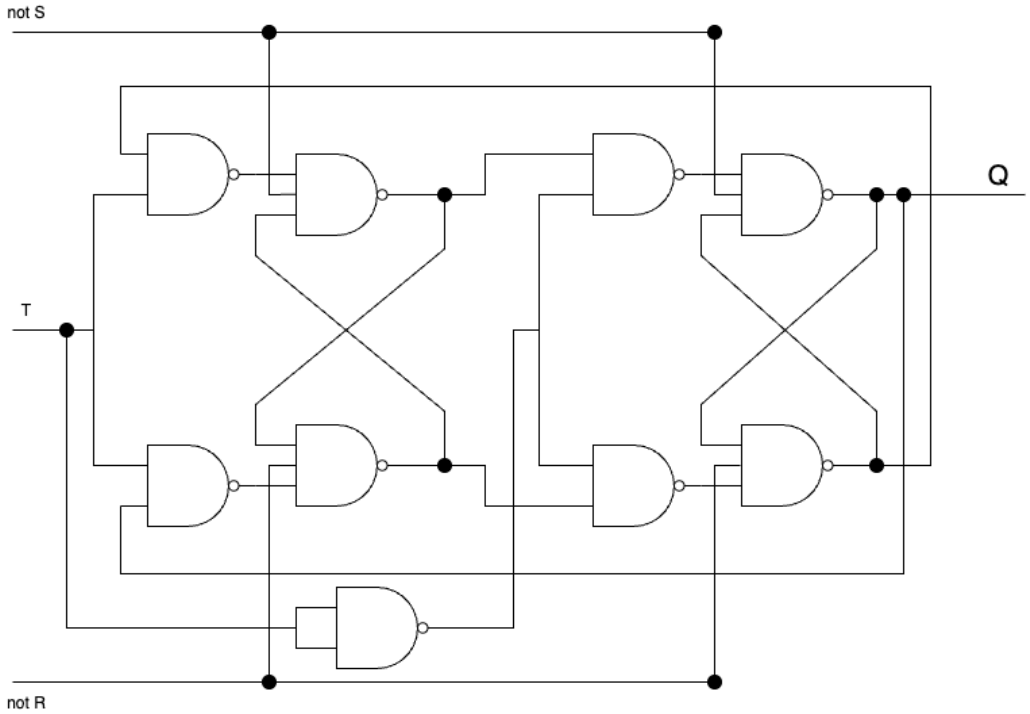


Рисунок 2.10. Т- триггер – асинхронный

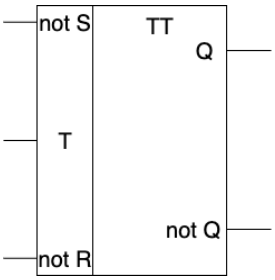


Рисунок 2.11. Т-триггер УГО

Таблица переходов (табл. 2.8) данного триггера не имеет запрещенных комбинаций, а основное предназначение этого триггера – это быть простейшим счетчиком, или быть частью более крупного счетчика.

Таблица 2.8 – Таблица переходов Т-триггера

T	T(t+1)	Q	Q (t+1)
0	0	0	0
0	0	1	1
1	1	0	0
1	1	1	1
0	1	0	0
0	1	1	1
1	0	0	1

1	0	1	0

Некоторым пиком эволюции можно считать универсальный JK-триггер. Свою универсальность он оправдывает тем, что сочетает в себе режимы работы сразу двух триггеров – двухступенчатого RS и T. В обычном режиме JK-триггер работает как RS, а если подать на его входы две 1, что является для второго запрещенной комбинацией, то он переходит в режим работы T триггера. За счет своей универсальности, такие триггеры нередко используются для построения регистров и счетчиков как альтернатива предыдущим двум вариантам. Построение данного автомата можно производить различными способами, но самым простым из них является добавление дополнительных входов к T-триггеру – рис. 2.12 – 2.13.

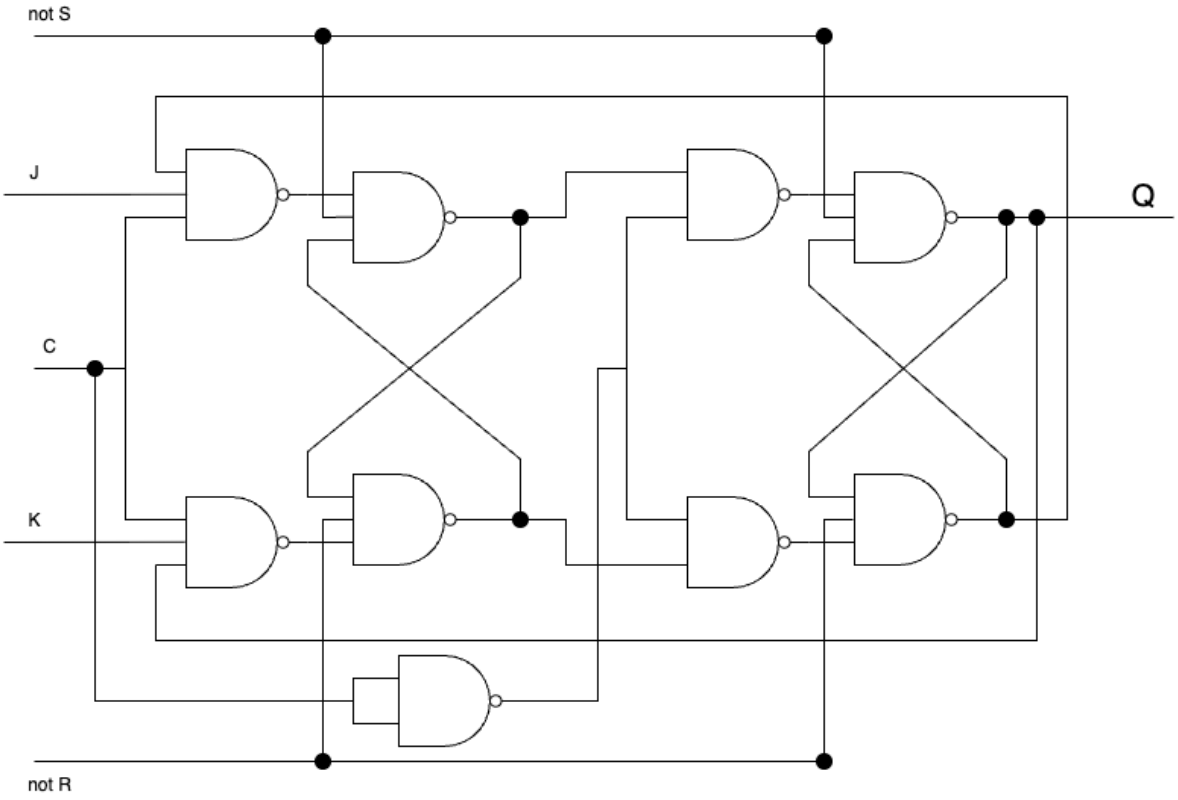


Рисунок 2.12. JK-триггер

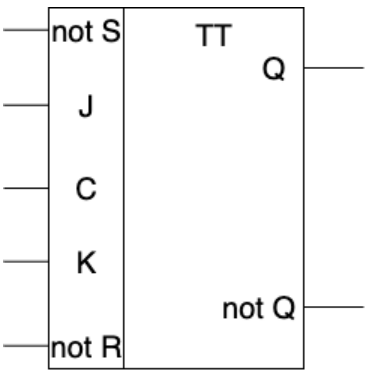


Рисунок 2.13. JK-триггер УГО

2.4. Счетчики

Счетчики – это узлы ЭВМ, предназначенные для подсчета количества входных сигналов, как правило выполняется на наиболее популярных триггерах, таких как D или JK-триггерах в режиме Т-триггера.

Счетчики могут быть: двоичные с модулем счета (M = 2^n или M ≠ 2^n); суммирующие; вычитающие; реверсивные; по организации цепей переноса (с последовательным переносом, со сквозным переносом, с параллельным переносом); кольцевые счетчики. Все счетчики могут быть синхронными или асинхронными (в зависимости от триггеров, на которых они строятся). По типу подсчета счетчики подразделяются на счетчики с естественным и произвольным порядком счета. При естественном порядке счета результат счета изменяется на единицу при поступлении на вход счетчика каждого сигнала (импульса). При произвольном порядке счета, в этом случае счетчики называют также пересчетными устройствами, при поступлении входных сигналов результат счета может изменяться произвольно в соответствии с заданным законом. Предназначение счетчиков в современных ЭВМ – осуществление последовательного выполнения команд программ, подсчета числа циклов выполненных операций, образования адресов при обращении к запоминающим устройствам, в качестве делителей частоты в цифровых электронных часах и частотомерах и др.

Основным параметром счетчиков является модуль счета (коэффициент пересчета, емкость счетчика). Модулем счета называют число состояний, которые он приобретает под действием входных сигналов. Если счетчик начал считать с начального состояния, то через каждые n входных сигналов он снова возвращается в начальное состояние, а на его выходе появляется сигнал переноса.

Рассмотрим построение счетчика произвольного счета с модулем 16 на примере варианта №1 из табл. 2.11. Пусть граф переходов между состояниями задается следующим образом – рис. 2.14.

Для примера построения счетчика возьмем вариант под номером 1. Для того чтобы удобнее было отслеживать переходы счетчика, нам необходимо в изначальный граф (рис. 2.14) вставить значения из варианта, на соответствующие позиции. Изобразим граф переходов с установленными состояниями – рис. 2.15.

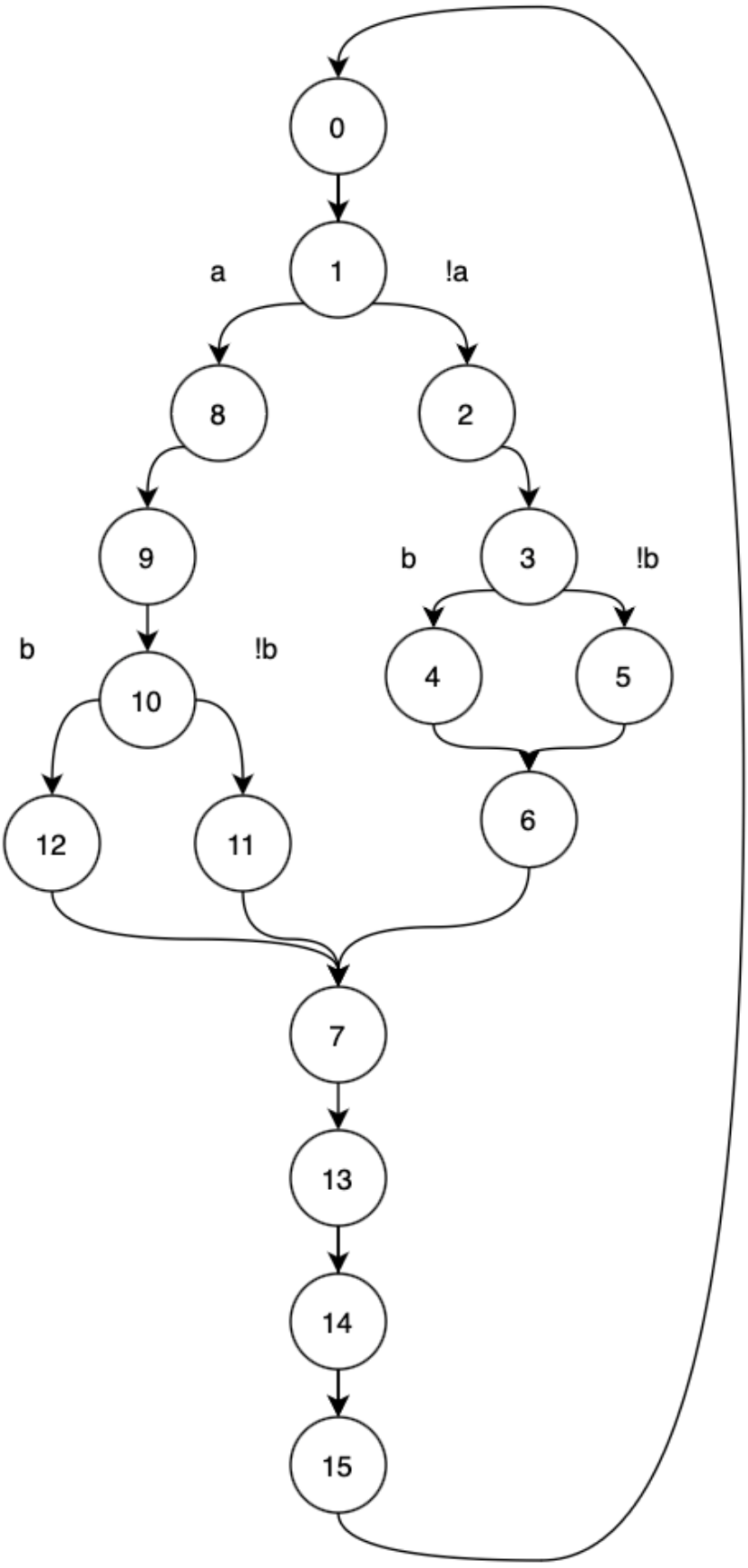


Рисунок 2.14. Граф переходов состояний

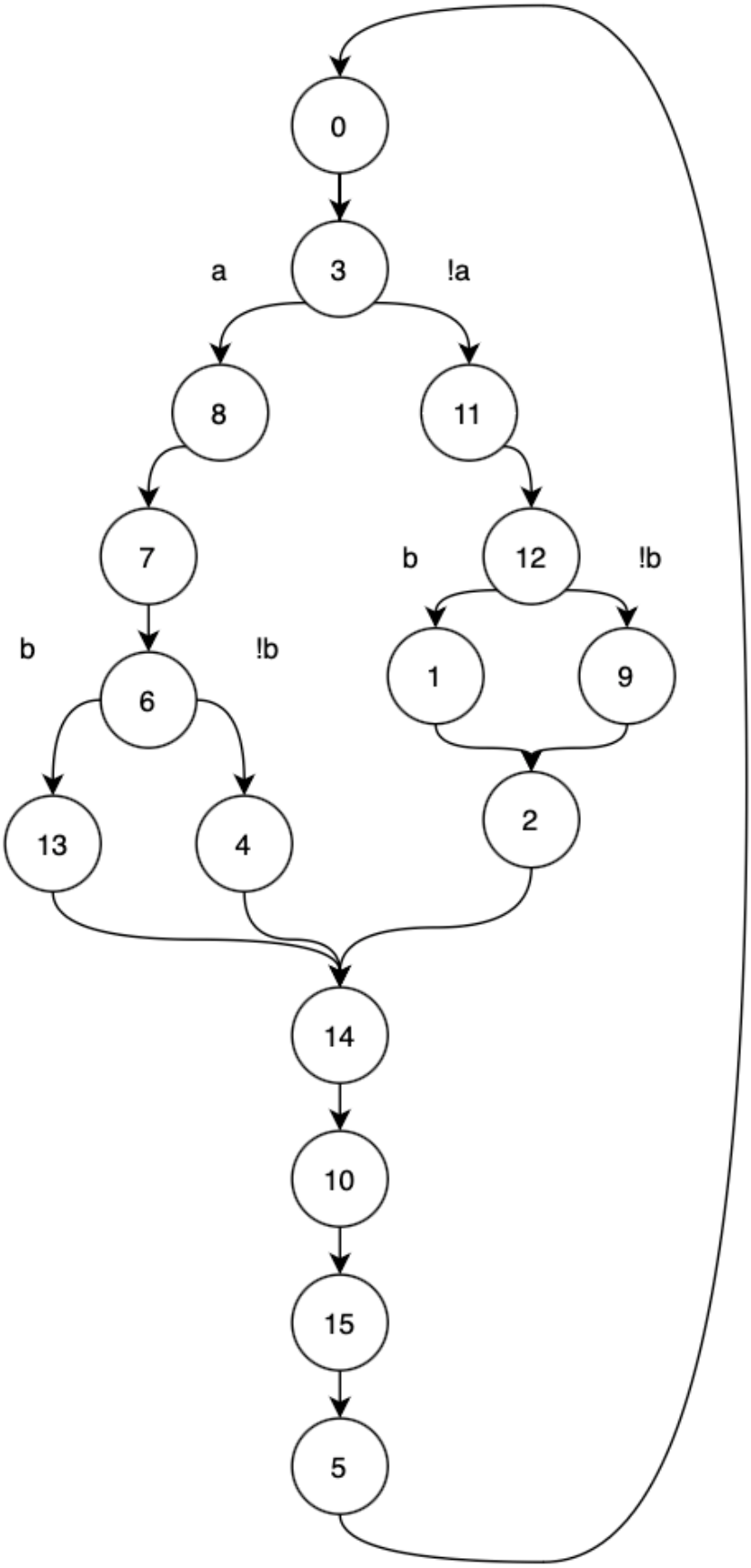


Рисунок 2.15. Граф переходов состояний счетчика для варианта №1

Теперь можно перейти к составлению таблицы переходов для нашего счетчика (табл. 2.9). Для этого составим таблицу истинности на 4 элемента, так как модуль счета у нас 16. В ней нужно отобразить зависимость номера состояния от истинного значения счетчика, например, на 5 такте (номере) счетчика должен принять значение 9, а на 6 такте уже значение 2, и так далее. Несмотря на произвольность счета, начинаться он все равно будет с 0, как и в обычном счетчике, хотя допускается возможность изначальной установки стартового числа.

Таблица 2.9 – Таблица состояний счетчика

№ состояния	**№ состояния из табл.1**	**Двоичный код**

На основании таблицы переходов, а также графа, в котором присутствуют еще и условия, можем составить итоговую таблицу истинности нашего счетчика с учетом всех переходов.

Как уже было сказано начальное значение нашего счетчика равняется 0, следовательно, с него и начнем. Состояния счетчика будем выписывать сразу в двух видах в десятичном и двоичном, чтобы их легче было отслеживать. Согласно графу и таблице переходов, следующее значение будет 3, заносим его в правую часть таблицы, где располагается новое состояние. Для первого варианта число 3 является местом разветвления счетчика, добавляется условие, в нашем случае значение переменной a, в зависимости от которого

счетчик примет либо значение 8, либо значение 11. Добавляем в таблицу оба варианта в столбец старого состояния и переходим к «правой ветке» графа куда движутся наши переходы, согласно изначальному графу, то есть по ветке где значение переменной a = 1. При таких значениях следующее значение графа будет число 11, его и заносим в столбец нового состояния счетчика, а там, где значение a будет равняться 0, установим значение 8. Переходим далее, за числом 11 следует еще одна «развилка» уже на 12 и при условии значения числа b. Повторяем предыдущее действие, занося в таблицу две строки с числом 12 и переходами в 1 и 9, соответственно. Дальнейшее заполнение таблицы подчиняется описанной логике, пока не дойдем до конца графа и не покроем все значения в нем. Итоговая таблица переходов нашего автомата показана в табл. 2.10.

Таблица 2.10 – Таблица истинности счетчика

Старое состояние		**Условие**	**Новое состояние**	
№	Код		№	Код

Имея таблицу переходов, можно переходить к синтезу схемы в Logisim. Для построения счетчика нам потребуется взять четыре D-триггера по одному на каждый разряд. На входном сигнале D-триггера будут приходить новые

состояния, согласно описанной выше таблице истинности. Исходя из таблицы истинности непосредственно D-триггера – переключение в 0 будет происходить при соответствующем сигнале, равно как и переключение в 1, что позволит при приходе нужного фронта включать нужное состояние. Все необходимые триггеры и их подключения изображены на рис. 2.16.

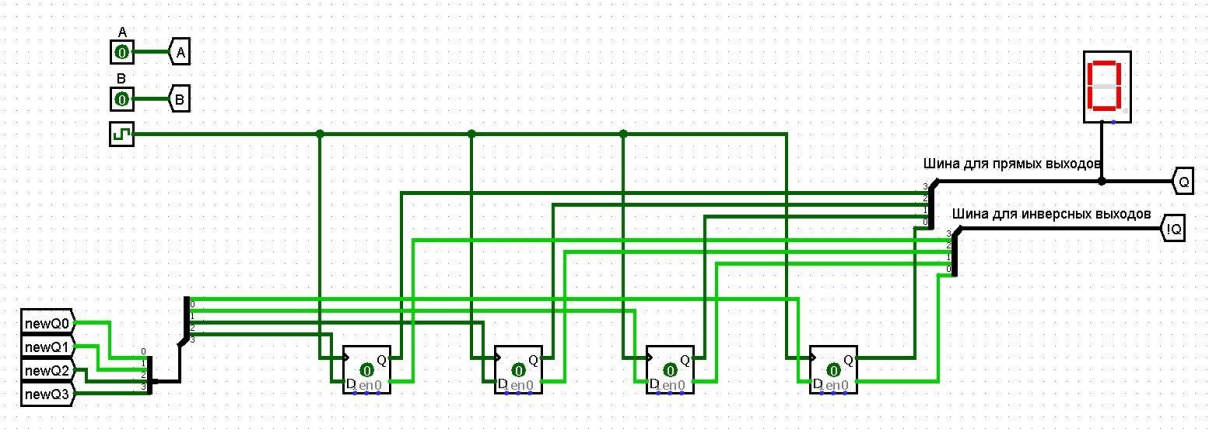


Рисунок 2.16. Подключение триггеров для синтеза счетчика

Для синтеза состояний придется построить все изначальные состояния от 0 до 15 для того, чтобы генерировать новые состояния. Логика построения состояний показана на рис. 2.17.

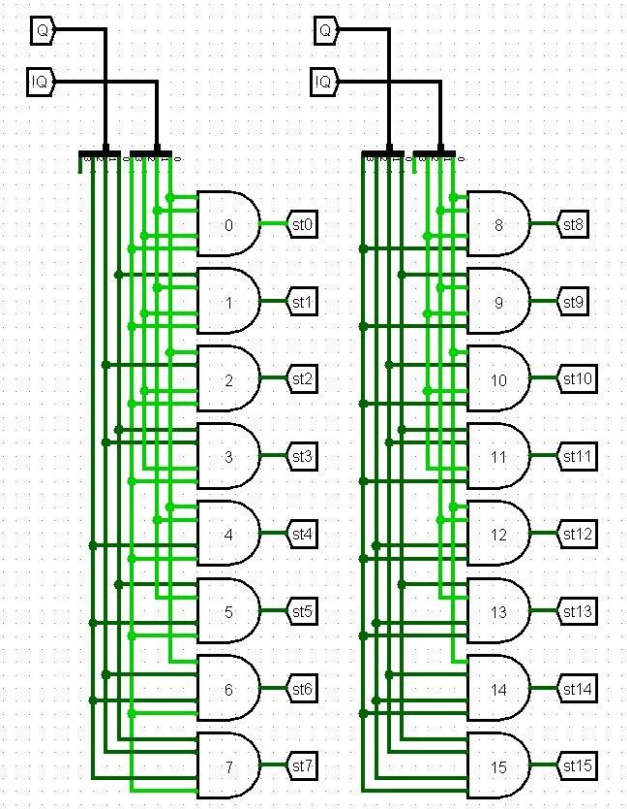


Рисунок 2.17. Состояния от 0 до 15

После чего необходимо учесть «развилки», где есть зависимость от переменных a или b. Для этого предусмотрена простая схема с элементом «И» (рис. 2.18), который будет объединять сигнал от переменной и от состояния, в котором происходит разветвление.

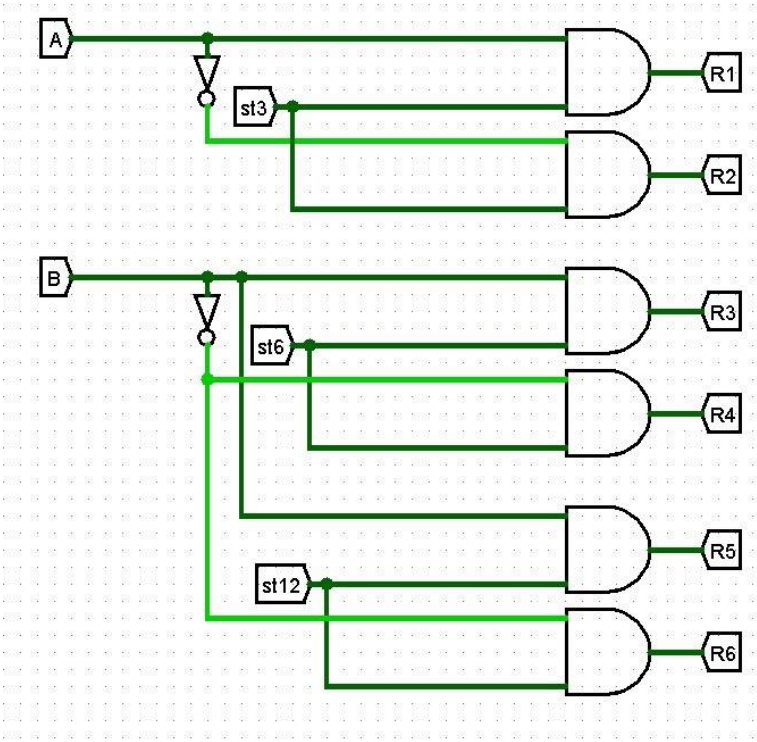


Рисунок 2.18. Схемы коррекции состояния a и b

После заданных приготовлений можно перейти к вычислению нового состояния для каждого бита, для этого будем использовать элементы ИЛИ, которые соединим с основными состояниями. В местах развилок на вход элемента ИЛИ будет подаваться результат логического «И» из рисунка выше (R1-R6). Все последовательности объединений новых битов состояния показаны на рис. 2.19 – 2.22.

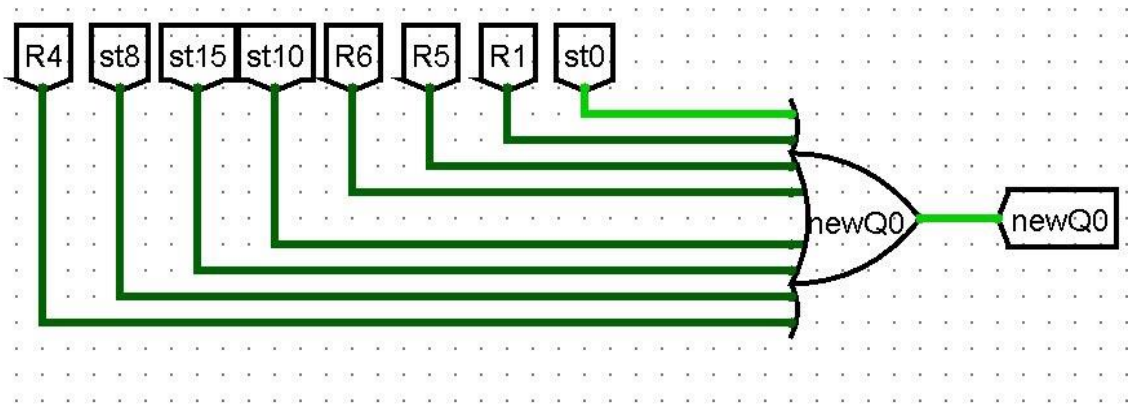


Рисунок 2.19. Схема подключения нового состояния бита 1

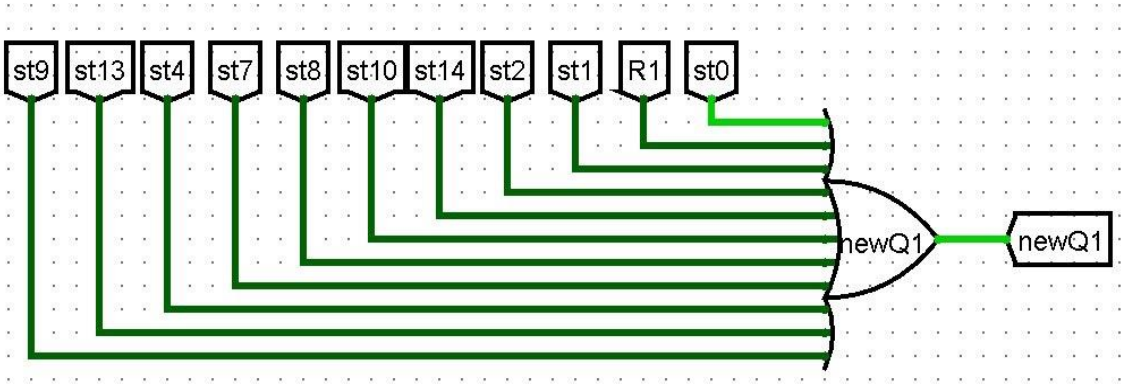


Рисунок 2.20. Схема подключения нового состояния бита 2

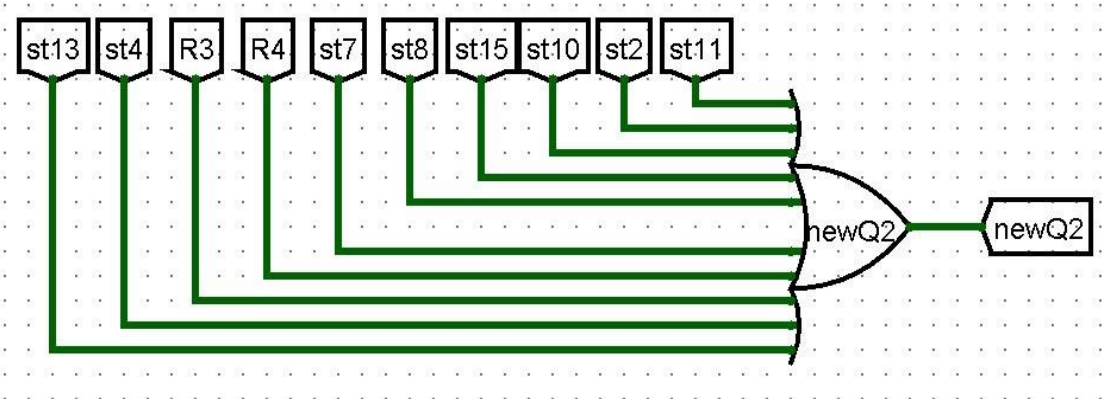


Рисунок 2.21. Схема подключения нового состояния бита 3

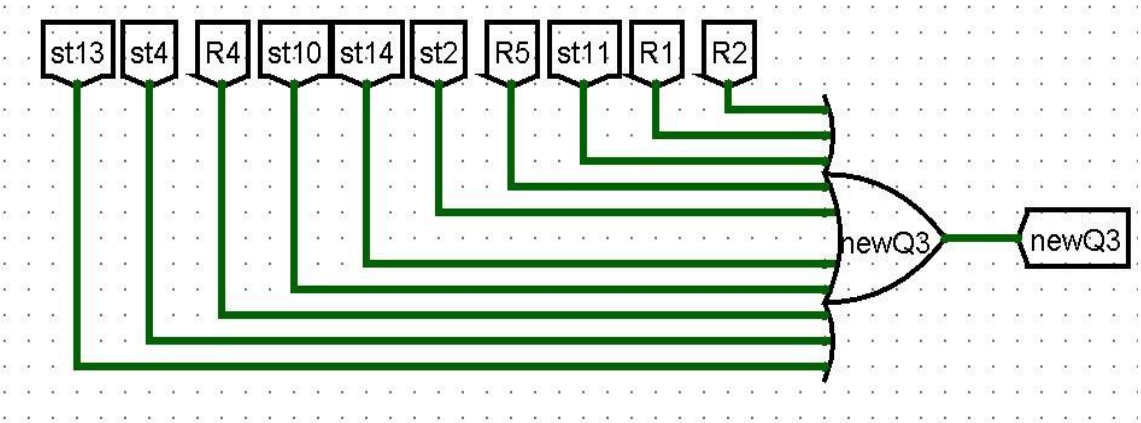


Рисунок 2.22. Схема подключения нового состояния бита 4

Результат построения счетчика будет изображен на рис. 2.23. Для упрощения подключений и общего вида схемы большая часть проводки была убрана в инструмент «тоннель» и помечена соответствующей меткой. Из дополнительных элементов у нас выступает шестнадцатеричный индикатор для отображения текущего состояния счетчика.

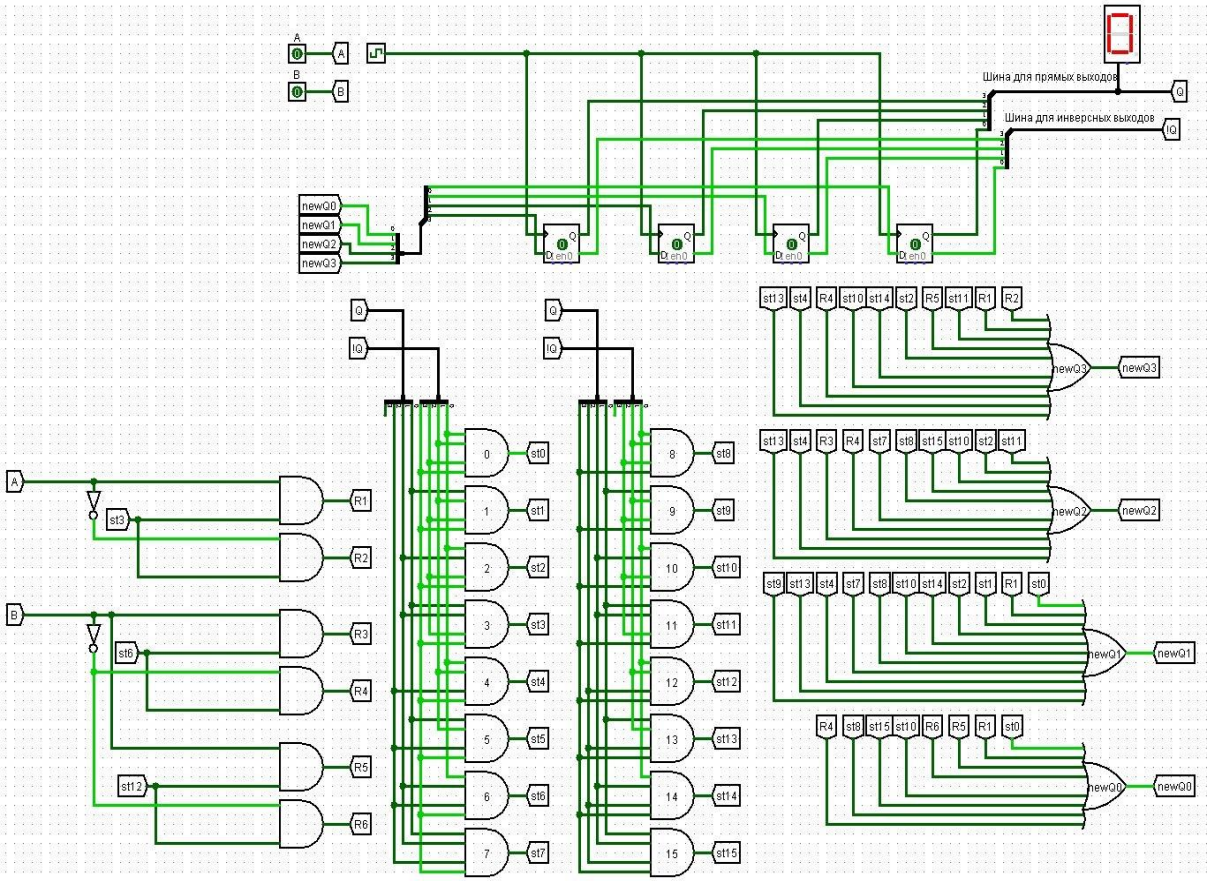


Рисунок 2.23. Счетчик по модулю 16 с шагом 1 и произвольным порядком счета

2.5. Заключение

В процессе выполнения данной работы студенты научились синтезировать более сложные и комплексные схемы устройств и автоматов, используя логические элементы, таблицы истинности и булеву алгебру. Изучение процесса построения заданных устройств позволит лучше понять логику работы каждого из них, а также позволит использовать изученные и построенные элементы для построения еще более сложных комплексных компонентов компьютера.

2.6. Варианты заданий

Для выполнения заданий можно использовать различные специализированные среды, например, Logisim, Quartus, Vivado. Заданием к практической работе является: построить в среде LogiSim комбинационную схему, триггер, счетчик.

Таблица 2.11 – Варианты для построения счетчиков

№ вар	Состояния счетчика															
	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
1	0	3	11	12	1	9	2	14	8	7	6	4	13	10	15	5
2	5	0	3	11	12	1	9	2	14	8	7	6	4	13	10	15
3	15	5	0	3	11	12	1	9	2	14	8	7	6	4	13	10
4	10	15	5	0	3	11	12	1	9	2	14	8	7	6	4	13
5	13	10	15	5	0	3	11	12	1	9	2	14	8	7	6	4
6	4	13	10	15	5	0	3	11	12	1	9	2	14	8	7	6
7	6	4	13	10	15	5	0	3	11	12	1	9	2	14	8	7
8	7	6	4	13	10	15	5	0	3	11	12	1	9	2	14	8
9	8	7	6	4	13	10	15	5	0	3	11	12	1	9	2	14
10	14	8	7	6	4	13	10	15	5	0	3	11	12	1	9	2
11	2	14	8	7	6	4	13	10	15	5	0	3	11	12	1	9
12	9	2	14	8	7	6	4	13	10	15	5	0	3	11	12	1
13	1	9	2	14	8	7	6	4	13	10	15	5	0	3	11	12
14	12	1	9	2	14	8	7	6	4	13	10	15	5	0	3	11
15	11	12	1	9	2	14	8	7	6	4	13	10	15	5	0	3
16	3	11	12	1	9	2	14	8	7	6	4	13	10	15	5	0
17	0	5	15	10	13	4	6	7	8	14	2	9	1	12	11	3
18	3	0	5	15	10	13	4	6	7	8	14	2	9	1	12	11
19	11	3	0	5	15	10	13	4	6	7	8	14	2	9	1	12
20	12	11	3	0	5	15	10	13	4	6	7	8	14	2	9	1
21	1	12	11	3	0	5	15	10	13	4	6	7	8	14	2	9
22	9	1	12	11	3	0	5	15	10	13	4	6	7	8	14	2
23	2	9	1	12	11	3	0	5	15	10	13	4	6	7	8	14
24	14	2	9	1	12	11	3	0	5	15	10	13	4	6	7	8
25	8	14	2	9	1	12	11	3	0	5	15	10	13	4	6	7
26	7	8	14	2	9	1	12	11	3	0	5	15	10	13	4	6
27	6	7	8	14	2	9	1	12	11	3	0	5	15	10	13	4
28	4	6	7	8	14	2	9	1	12	11	3	0	5	15	10	13
29	13	4	6	7	8	14	2	9	1	12	11	3	0	5	15	10
30	10	13	4	6	7	8	14	2	9	1	12	11	3	0	5	15

Студентам предоставляется готовый счетчик, построенный по варианту 1, задача состоит в том, чтобы правильно составить таблицы переходов и контакты newQ, под каждый бит. Шаблон графа состояний счетчика показан на рис. 2.14. После выполнения работы продемонстрировать результат преподавателю. Варианты:

1. Разобрать дешифратор с 3 входными сигналами и D-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 2.

2. Разобрать дешифратор с 2 входными сигналами и D-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 3.

3. Разобрать дешифратор с 4 входными сигналами и T-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 4.

4. Разобрать шифратор с 2 входными сигналами и JK-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 5.

5. Разобрать шифратор с 4 входными сигналами и D-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 6.

6. Разобрать шифратор с 4 входными сигналами и T-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 7.

7. Разобрать мультиплексор с 2 адресными входными сигналами и JK-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 8.

8. Разобрать мультиплексор с 3 адресными входными сигналами и D-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 9.

9. Разобрать мультиплексор с 4 адресными входными сигналами и T-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 10.

10. Разобрать демультиплексор с 2 адресными входными сигналами и JK-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 11.

11. Разобрать демультиплексор с 3 адресными входными сигналами и D-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 12.

12. Разобрать демультиплексор с 4 адресными входными сигналами и T-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 13.

13. Разобрать дешифратор с 2 входными сигналами и D-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 14.

14. Разобрать шифратор с 2 входными сигналами и T-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 15.

15. Разобрать мультиплексор с 2 адресными входными сигналами и JK-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 16.

16. Разобрать демультиплексор с 2 адресными входными сигналами и D-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 17.

17. Разобрать дешифратор с 3 входными сигналами и T-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 18.

18. Разобрать шифратор с 4 входными сигналами и JK-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 19.

19. Разобрать мультиплексор с 3 адресными входными сигналами и D-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 20.

20. Разобрать демультиплексор с 3 адресными входными сигналами и T-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 21.

21. Разобрать дешифратор с 4 входными сигналами и JK-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 22.

22. Разобрать шифратор с 8 входными сигналами и D-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 23.

23. Разобрать мультиплексор с 4 адресными входными сигналами и T-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 24.

24. Разобрать демультиплексор с 4 адресными входными сигналами и JK-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 25.

25. Разобрать дешифратор с 2 входными сигналами и T-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 26.

26. Разобрать шифратор с 2 входными сигналами и JK-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 27.

27. Разобрать мультиплексор с 2 адресными входными сигналами и D-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 28.

28. Разобрать демультиплексор с 2 адресными входными сигналами и T-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 29.

29. Разобрать дешифратор с 3 входными сигналами и JK-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 30.

30. Разобрать шифратор с 4 входными сигналами и D-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 2.

31. Разобрать мультиплексор с 3 адресными входными сигналами и Т-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 3.

32. Разобрать демультиплексор с 3 адресными входными сигналами и JK-триггер, составить таблицу истинности. Реализовать счетчик согласно варианту 4.

3. ОПЕРАЦИОННОЕ УСТРОЙСТВО ЦП

3.1. Введение

Центральный процессор (ЦП) – главный элемент вычислительной машины, который реализует выполнение команд, хранящихся в памяти. Его структура обычно делится на устройство управления (УУ) и операционное устройство (ОУ). УУ отвечает за формирование управляющих сигналов, организацию последовательности выполнения команд и микропрограмм. ОУ выполняет преобразование данных, арифметические и логические операции, пересылку и хранение промежуточных результатов. В данной практической работе рассмотрим операционное устройство.

3.2. Регистры и память ОУ

Регистр – это минимальная ячейка памяти процессора, построенная на триггерах. В простейших схемах используют RS- и D-триггеры, которые позволяют фиксировать одно двоичное значение. В современных процессорах применяются синхронные D-триггеры, работающие по тактовому сигналу, что обеспечивает синхронность всех операций. Современные ЦП содержат десятки или даже сотни регистров: регистры общего назначения (GPR), регистры адресов, регистры состояния, сегментные регистры, векторные и SIMD-регистры. Далее рассмотрим основные регистры и элементы памяти ЦП.

PC (Program Counter) – счетчик команд. Хранит адрес следующей инструкции, которая должна быть выполнена. После выборки команды из памяти его значение автоматически увеличивается, а при переходах или вызовах подпрограмм – изменяется на новое значение. В современных процессорах PC может быть расширен за счет буферов предвыборки и предсказания переходов.

IR (Instruction Register) – регистр команды. Содержит код текущей выполняемой команды. Этот код поступает на дешифратор, который определяет, какие управляющие сигналы нужно сгенерировать. В современных архитектурах IR часто дополняется декодером, а также поддержкой микропрограммного управления.

AC (Accumulator) – аккумулятор. Один из старейших регистров, предназначенный для хранения промежуточных результатов вычислений. В простейших машинах все операции выполнялись через аккумулятор. В

современных ЦП вместо одного АС используется целый набор регистров общего назначения (GPR).

MAR (Memory Address Register) – регистр адреса памяти. Хранит адрес ячейки памяти, к которой нужно обратиться. Адрес формируется УУ и подается на шину адреса. MAR обеспечивает связь между процессором и оперативной памятью (RAM).

MDR (Memory Data Register) – регистр данных памяти. Используется для временного хранения данных, загружаемых из памяти или записываемых в нее. Таким образом, MAR определяет, куда, а MDR – что именно будет передано.

InpR и OutR – регистры ввода и вывода. Служат для обмена информацией с внешними устройствами. InpR хранит поступающие извне данные, а OutR – данные, подготовленные для передачи наружу. В современных архитектурах регистры ввода-вывода заменяются контроллерами и буферами, которые управляются через специальные инструкции.

ОЗУ (RAM, память) – рабочая память процессора, может быть реализована также на регистрах. В учебных моделях память рассматривается как часть ЦП, но в реальных системах оперативная память вынесена во внешние микросхемы и связана с процессором через шину или контроллер памяти. Современные процессоры содержат многоуровневую систему кэш-памяти (L_1, L_2, L_3), которая выполняет роль «ускоренного ОЗУ». Общая схема ОЗУ показана на рис. 3.1, в следующих практических работах тема устройства ОЗУ будет рассмотрена подробнее. Для нашего учебного ОУ ОЗУ будет хранить информацию о командах, а также операндах, которые АЛУ будет использовать в процессе своей работы.

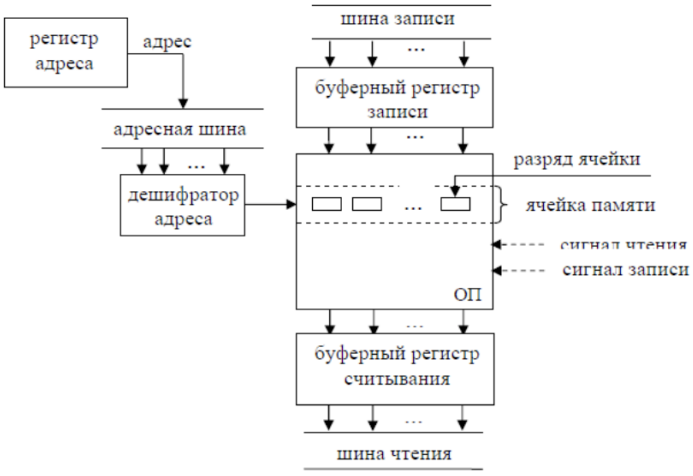


Рисунок 3.1. Упрощенная схема ОЗУ

3.3. Арифметико-логическое устройство

Арифметико-логическое устройство (АЛУ, от англ. Arithmetic Logic Unit) – это центральный компонент процессора, выполняющий арифметические и логические операции над данными. Его работа определяется комбинацией управляющих сигналов, которые поступают от устройства управления. На входы АЛУ подаются два операнда, чаще всего загруженные из регистров, например, аккумулятора и регистра данных, а результат вычислений возвращается обратно в регистры или передается на шину данных. Помимо формирования результата, АЛУ обновляет специальные признаки состояния в регистре флагов, которые отражают свойства полученного значения и необходимы для последующего выполнения программы.

Работа АЛУ обычно складывается из нескольких этапов. Сначала устройство принимает два входных числа и управляющий код операции, который определяет вид вычисления. Далее выполняется сама операция: это может быть арифметическое действие, такое как сложение, вычитание, инкремент или декремент, или логическая операция – «И», «ИЛИ», «НЕ», «исключающее ИЛИ». В ряде случаев выполняются операции сдвига и вращения битов, которые позволяют эффективно работать с бинарными данными. После завершения обработки результат помещается в регистр результата или аккумулятор, а в регистр состояния записываются флаги, которые помогают процессору анализировать последствия вычислений.

Дополнительно существует система флагов АЛУ. Наиболее часто используются следующие признаки: флаг Zero (Z) показывает, равен ли результат нулю, что важно при условных переходах; флаг Sign (S) отражает знак результата и помогает различать положительные и отрицательные значения; флаг Carry (C)фиксирует наличие переноса при сложении или заема при вычитании, что особенно важно при работе с длинными числами; флаг Overflow (V) указывает на переполнение, то есть ситуацию, когда результат не помещается в заданное количество разрядов и искажается его знак. В ряде архитектур также применяются дополнительные признаки, такие как Parity (P), показывающий четность результата, или Auxiliary Carry (AC), который используется в двоично-десятичной арифметике. Таким образом, сами данные при вычислении не изменяются, но система флагов позволяет процессору принимать решения в условных инструкциях и корректно обрабатывать нестандартные ситуации.

Если рассматривать АЛУ обобщенно, то его входами служат операнды, поступающие из регистров или по системной шине, а также код управляющего сигнала, определяющий операцию. Внутри устройства располагаются такие функциональные блоки, как сумматор для выполнения арифметики, логический блок для работы с булевыми операциями, схема сдвигов, а также мультиплексоры, выбирающие результат. На выходе формируется вычисленное значение, а вместе с ним обновляются флаги состояния, которые связывают работу АЛУ с общей логикой процессора. Общая упрощенная АЛУ показана на рис. 3.2 – 3.3.

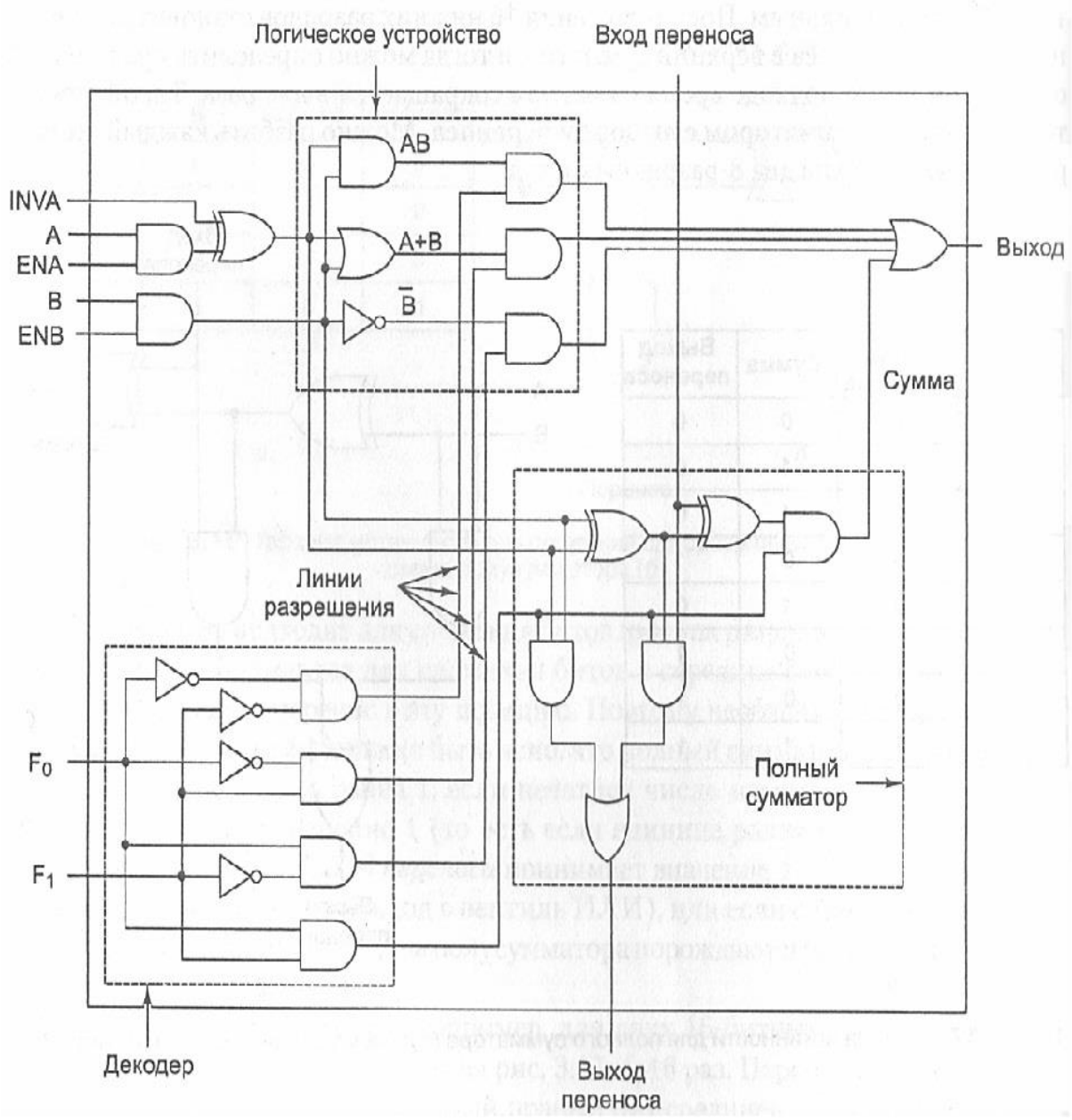


Рисунок 3.2. Упрощенная схема АЛУ

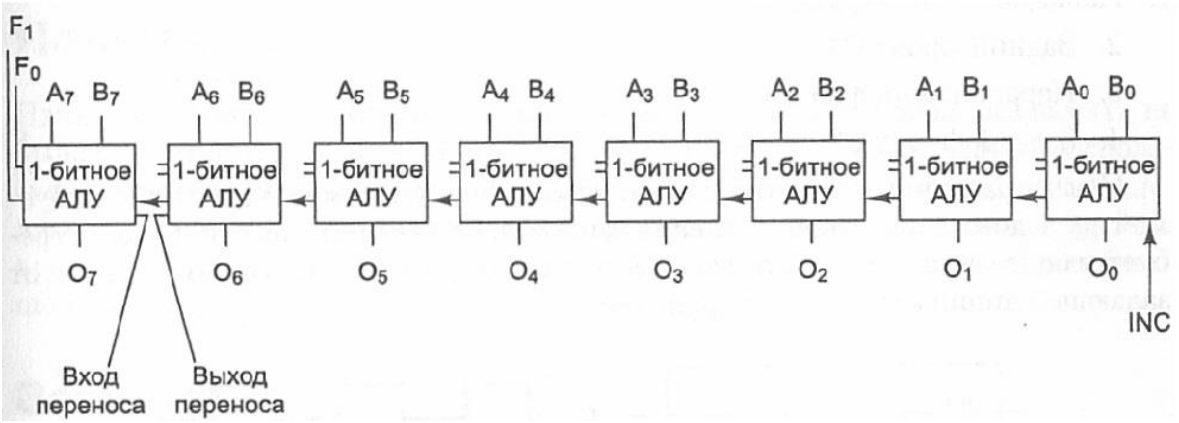


Рисунок 3.3. Упрощенная схема 8-разрядного АЛУ

Учебные модели АЛУ обычно представляются в виде комбинационных схем, построенных из одноразрядных элементов. Каждый такой элемент выполняет операцию над отдельным битом, а их объединение образует n-разрядное устройство. Для сложения используется схема из полусумматоров и полных сумматоров, где переносы передаются от младших разрядов к старшим. Логическая часть реализуется простыми логическими элементами «И», «ИЛИ», «НЕ» и «XOR». Выбор необходимой функции обеспечивают мультиплексоры, которые переключают линии данных в зависимости от управляющих сигналов. Такая учебная конструкция проста и наглядна, она демонстрирует, как из элементарных компонентов строится сложная схема, и позволяет изучить принципы работы вычислительных устройств.

Современные АЛУ намного сложнее и отличаются от учебных моделей не только количеством поддерживаемых функций, но и организацией работы. В современных процессорах они являются частью исполнительного конвейера и могут включать несколько параллельных блоков. Помимо работы с целыми числами, часто используется отдельный модуль для операций с плавающей точкой – FPU (Floating Point Unit). Аппаратная реализация сдвигов и вращений значительно ускоряет операции, связанные с обработкой двоичных данных и криптографией. Широкое применение нашли технологии SIMD (Single Instruction Multiple Data), позволяющие выполнять одну инструкцию сразу над несколькими числами, что делает АЛУ особенно эффективным в задачах мультимедиа, машинного обучения и обработки сигналов.

Кроме того, современные процессоры нередко содержат несколько АЛУ, которые могут работать параллельно, что позволяет выполнять несколько инструкций за один такт и реализует параллелизм на уровне команд. В архитектуре ARM, например, поддерживаются условные инструкции, которые исполняются только при определенных значениях

флагов, что позволяет сократить количество условных переходов и повысить производительность. Таким образом, современные АЛУ превратились из простых сумматоров и логических схем в многофункциональные исполнительные модули, обеспечивающие высокую производительность и гибкость вычислений.

Таким образом, арифметико-логическое устройство – это не просто компонент для выполнения простых операций, а сложный и многогранный блок, эволюционировавший от простейших комбинационных схем до мощных параллельных систем. Оно стало основой вычислительных процессов в современных ЭВМ и играет ключевую роль в обеспечении высокой производительности процессоров.

3.4. Запись управляющего слова ОУ

Перейдем к описанию учебного ОУ. Учебное операционное устройство (ОУ) является моделью центрального процессора, предназначенной для демонстрации работы регистров, АЛУ и управления потоками данных через мультиплексоры и дешифраторы. Управление устройствами ОУ осуществляется с помощью **управляющего слова**, которое содержит 18 разрядов и задает действия процессора над данными в регистрах.

В состав учебного ОУ входят следующие регистры:

- регистр PC (Program Counter) – счетчик команд. Хранит адрес текущей команды в памяти;
- регистр AC (Accumulator) – аккумулятор. Хранит промежуточные результаты вычислений. В простейших машинах все операции выполнялись через AC; в современных ЦП вместо одного AC используется набор регистров общего назначения (GPR);
- регистр IR (Instruction Register) – регистр команд. Хранит закодированную инструкцию для выполнения;
- регистр MAR (Memory Address Register) – регистр адреса памяти. Хранит адрес ячейки ОЗУ, с которой нужно работать, обеспечивая связь процессора с памятью;
- регистр MDR (Memory Data Register) – регистр данных памяти. Временно хранит данные, считываемые из памяти или записываемые в нее.
- регистр InpR (Input Register) – регистр ввода. Хранит данные, поступающие из внешних устройств (например, клавиатуры);
- регистр OutR (Output Register) – регистр вывода. Хранит данные, передаваемые на внешние устройства (например, на экран).

Дополнительно добавлены регистры флагов для операций сравнения:

- флаг Z Флаг нуля, 1 если результат = 0;
- флаг S Флаг знака, 1 если результат < 0;
- флаг C Флаг переноса/заема;
- флаг V Флаг переполнения.

Кроме того, используется ОЗУ (RAM), которая в учебной модели может быть реализована на регистрах процессора, а в реальных системах – как внешняя память, связанная с процессором через шину и контроллер. Современные ЦП имеют многоуровневую систему кэш-памяти (L_1, L_2, L_3) для ускоренного доступа.

Для управления потоками данных используются:

- дешифратор выбора регистров. Разрешает запись в один из внутренних регистров ОУ на основе двоичного адреса. Например, если нужно записать результат в MAR, на адресный вход подается 011;
- мультиплексоры A и B. Позволяют выбрать, какие регистры подать на входы АЛУ как операнды A и B. Левый мультиплексор подает операнд A, правый – операнд B. Для формирования управляющего слова каждому мультиплексору назначается поле MS(A) или MS(B) (по 3 бита), задающее выбор регистра;
- мультиплексор MS(M). Управляет, какое значение будет записано в MDR – из АЛУ или из памяти.

На табл. 3.1 показаны микрооперации учебного ОУ. На рис. 3.4 приведена общая схема ОУ. Обычно подпись у входов АЛУ следующая:

- вход F Основной выбор функции АЛУ: обычно 0 = арифметика, 1 = логика/сдвиги. Иногда используется как дополнительный бит расширения для выбора новых групп операций;
- вход M Бит выбора источника данных или режима (например, 0 = обычные операции с регистрами A/B, 1 = работа с регистрами памяти или сдвиги);
- вход S2, S1, S0 Код операции внутри выбранного режима (арифметика или логика);
- вход C0 Бит «перенос/заем», который может участвовать в арифметических операциях, например, для сложения с переносом A+B+C0 или вычитания с заемом.

В нашем учебном АЛУ при F = 0 – арифметика и стандартные логические операции, при F = 1 – расширенные логические/сдвиговые операции.

Таблица 3.1 – микрооперации учебного АЛУ

						Микрооперация	**Наименование**
							Передача A
							Инкремент A
							Сумма
							Сумма с переносом
						A−B−1	Разность с заемом
						A−B	Разность
						A−1	Декремент A
							Передача A (альтерн.)
						∨ B	Логическое ИЛИ
						⊕ B	Исключающее ИЛИ
						∧ B	Логическое И
							Инверсия A
							Сдвиг A влево
							Сдвиг A вправо
							Циклический сдвиг влево
							Циклический сдвиг вправо
						B−A	Разность B−A
						B−A−1	Разность B−A с заемом
							Умножение на 2
							Деление на 2
							Логическое НЕ-И
							Логическое НЕ-ИЛИ
							Исключающее ИЛИ с инверсией
							Циклический сдвиг влево на 2
							Циклический сдвиг вправо на 2

Для удобства использования управляющее слово ЦП запишем в виде таблицы (табл. 3.2), которую также необходимо использовать при выполнении практической работы. Рассмотрим запись управляющего слова для команды MAR ← IR + AC. Сигналы для ALU определяются по табл. 3.1. В качестве адресов для MS A и MS B нужно подать значение, соответствующее коду информационных входов каждого из операнда (для IR – 101, AC – 110). Сигнал wr во всех примерах не используется, можно оставить x. Сигнал MS M нужен только при записи в MDR (при необходимости на адрес подать 0). В табл. 3.3 показана запись машинного слова ОУ для рассмотренной операции.

Таблица 3.2 – Машинное слово ОУ

Таблица 3.3 – Машинное слово ОУ(заполненное)

MS A			MS B			ALU						DC				MSM	WR
a2	a1	a0	a2	a1	a0	F	M	S2	S1	S0	C	E	x2	x1	x0	a0	wr
1	0	1	1	1	0	0	0	0	0	1	0	1	1	0	0	x	x

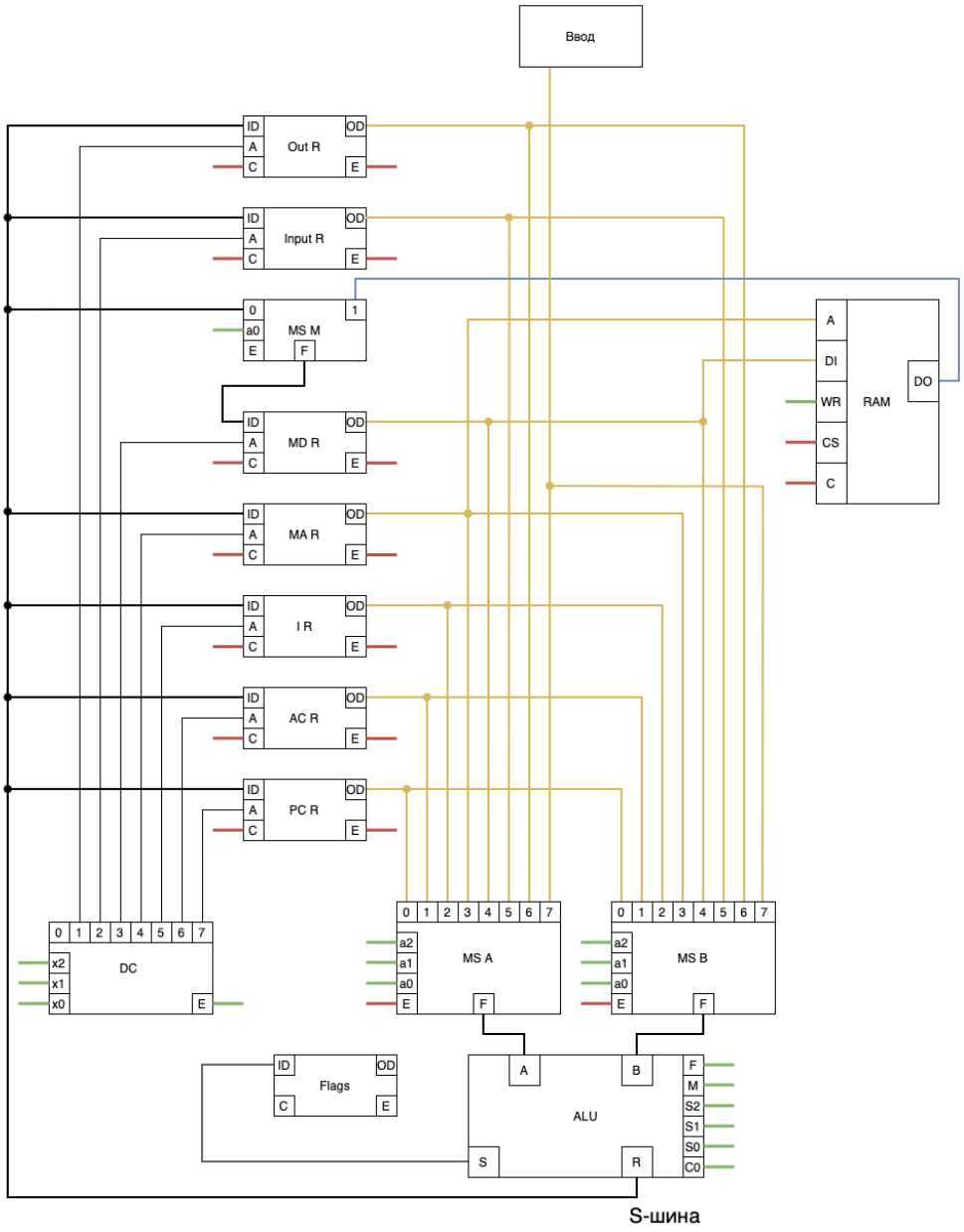


Рисунок 3.4. Схема учебного ОУ

3.5. Заключение

В ходе выполнения практической работы были изучены принципы работы операционного устройства центрального процессора, его составных элементов – регистров, АЛУ и системы флагов, а также механизм управления потоками данных с помощью мультиплексоров и дешифраторов. Были рассмотрены ключевые регистры (PC, IR, AC, MAR, MDR, InpR, OutR) и их роль в организации вычислительного процесса, а также функции микроопераций АЛУ, обеспечивающих выполнение арифметических и логических операций. Особое внимание уделено структуре управляющего слова ОУ, которое позволяет задавать операции над регистрами и контролировать запись в память и обновление флагов состояния. Полученные знания демонстрируют, как из элементарных компонентов формируется сложная вычислительная система и обеспечивается взаимодействие между регистрами, АЛУ и памятью, что является основой работы современных процессоров.

3.6. Варианты заданий

Варианты для выполнения практической работы:

1. S ← INPR ∧ IR;
2. AC ← AC / 2;
3. IR ← MAR ∨ OUTR;
4. OUTR ← AC − MDR – 1;
5. MDR ← PC + INPR;
6. S ← INPR + 1;
7. OUTR ← AC NAND MAR;
8. MAR ← OUTR − AC – 1;
9. AC ← IR ⊕ MDR;
10. INPR ← IR + MAR;
11. AC ← AC * 2;
12. IR ← MAR + OUTR + 1;
13. PC ← MAR ∧ PC;
14. MDR ← IR ⊕ AC;
15. S ← AC >>> 1;
16. OUTR ← INPR;
17. MAR ← INPR – AC;
18. IR ← MDR ∨ IR;
19. AC ← IR − PC – 1;

20. PC ← INPR ⊕ OUTR;
21. INPR ← MAR + OUTR;
22. MDR ← OUTR + IR + 1;
23. OUTR ← AC NOR OUTR;
24. S ← MDR ∧ IR;
25. AC ← AC <<< 1;
26. PC ← MAR ⊕ PC;
27. IR ← OUTR + PC;
28. S ← AC * 2;
29. OUTR ← AC * 2;
30. S ← INPR <<< 1;
31. MDR ← AC XNOR MDR;
32. OUTR ← AC − MDR.

4. УСТРОЙСТВО УПРАВЛЕНИЯ ЦП УЧЕБНОГО ОУ

4.1. Введение

В предыдущей работе при знакомстве с ОУ было рассмотрено выполнение некоторой абстрактной команды. Теперь рассмотрим более сложный, но, в то же время, более реальный способ работы схемы под управлением управляющего устройства (УУ), которое отвечает за формирование управляющих сигналов, организацию последовательности выполнения команд и микропрограмм.

4.2. Устройство управления ЦП

Рассмотрим детальнее работу учебной вычислительной машины (ВМ) на основе управляющего устройства (УУ).

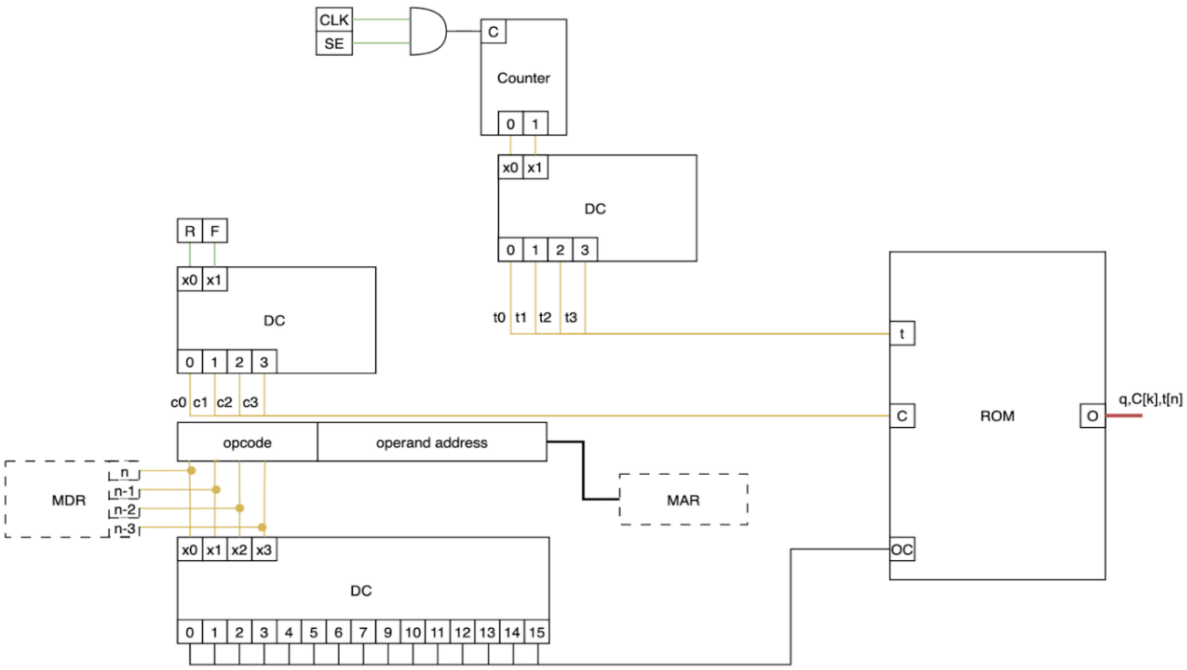


Рисунок 4.1. Схема учебного УУ

Схема УУ показана на Рисунке 4.1. Для формирования сигналов, управляющих учебным ОУ, используется устройство управления. В основе УУ лежит постоянное запоминающее устройство (ПЗУ), в котором хранятся управляющие слова. Сигналы, формируемые на его выходах, задают работу регистров, мультиплексоров, АЛУ и памяти. Генератор ВМ вырабатывает синхросигнал. Дополнительно используется сигнал SE (Start/Enable) – сигнал разрешения работы. Если SE = 0, то сигналы генератора блокируются, и ВМ находится в состоянии останова. Если SE = 1, то сигналы поступают на вход

счетчика тактов, соединенного с дешифратором. В учебной ВМ возможно до четырех тактов выполнения микрокоманд. Дешифратор команд работает по старшим разрядам регистра IR (Instruction Register), которые содержат код операции. Так как код имеет 4 разряда, всего поддерживается 16 различных команд.

В табл. 4.1 показан список доступных команд учебной ВМ. AD – адрес операнда, хранящийся в регистре адреса памяти (MAR). Для команд 12–15 второй операнд не требуется, поэтому в поле адреса записывается FFF (все единицы).

Таблица 4.1 – Команды ВМ

№	Обозн.	Команда	Код операции	Адрес	Расшифровка команды
q0					Загружает содержимое ячейки памяти по адресу AD в аккумулятор: **AC ← M[AD]**
q1					Сохраняет содержимое аккумулятора в ячейку памяти с адресом AD: **M[AD] ← AC**
q2					Складывает содержимое аккумулятора с операндом из памяти по адресу AD и результат записывает в AC: **AC ← AC + M[AD]**
q3					Вычитает из аккумулятора содержимое ячейки памяти по адресу AD: **AC ← AC – M[AD]**
q4					Побитовое логическое И между аккумулятором и содержимым памяти по адресу AD: **AC ← AC ∧ M[AD]**
q5					Побитовое логическое ИЛИ аккумулятора и содержимого памяти: **AC ← AC ∨ M[AD]**
q6					Побитовое исключающее ИЛИ аккумулятора и содержимого памяти: **AC ← AC ⊕ M[AD]**
q7					Побитовое отрицание содержимого памяти по адресу AD с записью в AC (обратный код): **AC ← ¬M[AD]**
q8					Формирование дополнительного кода (отрицание числа): **AC ← ¬M[AD] + 1**

Продолжение табл. 4.1

q					Увеличивает содержимое ячейки памяти по адресу AD на единицу: **AC ← M[AD] + 1**
q					Уменьшает содержимое ячейки памяти по адресу AD на единицу: **AC ← M[AD] – 1**
q11					Безусловный переход: в PC загружается адрес AD, и выполнение продолжается с этой ячейки: **PC ← AD**
q12					Обнуление аккумулятора: **AC ← 0**
q13					Увеличение аккумулятора на единицу: **AC ← AC + 1**
q14					Ввод данных: содержимое регистра ввода INPR копируется в аккумулятор: **AC ← INPR**
q15					Вывод данных: содержимое аккумулятора передается в регистр вывода OUTR: **OUTR ← AC**

Далее рассмотрим принцип формирования сигналов циклов и работу процесса выборки/выполнения инструкции. В УУ состояния циклов формируются комбинацией двух триггеров (битов состояния), обозначенных как F и R. Их выходы подаются на дешифратор тактов 2-4 (cycle-decoder), который в каждом такте дает одну «значащую единицу» (one-hot) – сигнал соответствующего цикла. Поскольку на дешифратор можно подать только 2 входных сигнала, то это дает 4 возможные комбинации – система поддерживает четыре логических цикла работы:

- C0 – выборка команды (fetch);
- C1 – дешифрация команды (decode);
- C2 – выполнение команды (execute);
- C3 – обслуживание прерывания / служебный цикл (interrupt / service).

Типичное соответствие состояний триггеров и циклов можно представить в виде таблицы (пример кодирования) – табл. 4.2.

Таблица 4.2 – Циклы УУ

F	R	Активный цикл
0	0	C0 (выборка)
0	1	C1 (дешифрация)
1	0	C2 (выполнение)
1	1	C3 (прерывание)

Генератор ВМ формирует последовательность синхроимпульсов («тактов»), которые по каналу управления либо доходят до счетчика тактов, либо блокируются. Для управления этим потоком используется сигнал разрешения SE и так называемая «схема совпадения». Поведение следующее:

- сигнал SE = 0 – схема совпадения блокирует проход импульсов от генератора, счетчик тактов не получает тактовых входов и УУ находится в состоянии останова; микрокоманды не выполняются;
- сигнал SE = 1 – схема совпадения пропускает импульсы генератора на вход двухразрядного счетчика, и УУ последовательно переключает тактовые стадии в соответствии с кодом триггеров F, R и дешифратором циклов.

Переходы из одного цикла в другой управляются логикой микрокоманд (полями «FR» в микрокоде или отдельными управляющими сигналами). Каждая микрокоманда на своем такте может задавать значения, которые будут записаны в триггеры F и R (то есть задает следующий активный цикл). Таким образом, УУ реализует конечный автомат: в зависимости от выполняемой микрокоманды и текущего кода операции выбирается, оставаться ли в цикле выборки, переходить на дешифрацию, перейти к выполнению или к обслуживанию прерываний.

Полезно понимать, что набор из четырех циклов – это общая архитектурная возможность. На практике, в простых учебных ВМ многие инструкции выполняются при минимальном наборе циклов: чаще всего используются только C0 (выборка) и C2 (выполнение). Цикл C1 (дешифрация) нужен, если необходимо дополнительное время для разборки формата инструкции или если поддерживаются разные методы адресации (например, косвенная, индексная и т.п.). Цикл C3 резервируется для обработки прерываний и служебных мероприятий. В учебной ВМ, где используется прямая адресация (адрес операнда уже находится в поле команды IR), цикл

дешифрации часто пропускают – после загрузки IR управление сразу переходит к циклу выполнения (C2).

4.3. Циклы выполнения команд

Рассмотрим подробнее циклы выполнения команд. Начнем с цикла выборки – обязательной первой стадии любой инструкции. Для выполнения выборки последовательно выполняются следующие микрооперации (по тактам):

- C0 t0: MAR ← PC – Счетчик команд (PC) передает адрес в регистр адреса памяти (MAR). На аппаратном уровне мультиплексор адреса и дешифратор шины адреса подключаются к выходам PC;
- C0 t1: MDR ← M[MAR] (инициализация чтения) – по адресу MAR выполняется обращение в ОЗУ; на шине данных приходит слово из памяти и загружается в MDR. Этот такт включает сигнал чтения памяти (memory read) и разрешение записи в MDR;
- C0 t2: IR ← MDR – содержимое MDR (машинная инструкция: код операции + поле адреса) копируется в регистр команд IR. Одновременно могут быть обновлены внутренние шины, и старшие биты IR становятся доступны дешифратору команды;
- C0 t3: PC ← PC + 1; установка F/R для перехода – выполняется инкремент PC (подготовка адреса следующей инструкции). На этом такте микрокоманда также задает значения триггеров F и R (через поле FR): либо оставаться в цикле выборки, либо перейти к следующему циклу (обычно – к выполнению). В учебных микропрограммах в конце цикла выборки часто устанавливают комбинацию триггеров, которая переводит УУ в состояние C2 (выполнение), если это требуется.

Если используется прямая адресация, то после C0t2 поле кода операции в IR уже известно и дополнительной дешифрации может не потребоваться – поэтому переход сразу в C2 допустим и экономичен.

Перейдем к циклу дешифрации (C1). Цикл C1 выполняет разбор формата инструкции и формирование дополнительной информации для исполнения: выбор способа адресации, вычисление косвенных адресов, подготовка индексных смещений, вычисление адреса микропрограммы и т. п. В C1 обычно выполняются микрооперации вида: извлечение полей из IR, выбор таблиц в ПЗУ микрокода, установка управляющих битов для

последующих тактов. В учебной ВМ, если набор адресации прост (прямая), этот цикл может быть пустым и пропускается.

Цикл выполнения (C2) рассмотрим на примере команды сложения ADD. Для конкретной инструкции набор микроопераций в C2 определяется самой командой. Рассмотрим пошагово ADD (код 0010, семантика AC ← AC + M[AD]):

- q2 C2 t0: MAR ← IR[AD] – из регистра IR берутся младшие биты (поле адреса AD) и передаются в MAR – подготовка к обращению к операнду;
- q2 C2 t1: MDR ← M[MAR] – чтение содержимого памяти по MAR; результат записывается в MDR;
- q2 C2 t2: AC ← AC + MDR (ALU: A + B) на этом такте АЛУ выполняет операцию сложения: в качестве операндов подаются AC (A) и MDR (B). Результат записывается обратно в AC; одновременно обновляются флаги (Z, S, C, V) в зависимости от результата;
- q2 C2 t3: NOP; и F, R ← (возврат в выборку) – в большинстве реализаций t3 не выполняет полезной работы (NOP) и служит для синхронизации; микрокоманда на этом такте обычно устанавливает триггеры F и R в состояние, соответствующее следующему циклу – чаще всего 00 (C0), то есть переход обратно к выборке следующей команды.

Таким образом, последовательность тактов C0…C2 обеспечивает: выборку инструкции, чтение операнда и выполнение арифметики с корректным обновлением состояния процессора.

Цикл завершения/обслуживания (C3). Цикл C3 предназначен для обработки прерываний, сервисных операций или особых служебных сценариев (например, ввод/вывод, сохранение контекста). Его содержимое зависит от архитектуры: в C3 могут выполняться действия по сохранению PC, переключению режимов, вызову обработчика прерываний и т. п. В учебной ВМ C3, как правило, реализуется опционально и активируется только при наличии запроса прерывания.

Рассмотрим работу учебной ВМ на примере выполнения команд ADD и INCA. Цикл выборки команды (одинаков для всех рассматриваемых команд) из памяти ВМ представлен в табл. 4.3.

Цикл дешифрации (также общем) представлен в табл. 4.4.

Далее идет цикл выполнения команды. Для выполнения команды ADD (0010, сложение содержимого аккумулятора с содержимым памяти по адресу операнда) запись цикла показана в табл. 4.5, для однотактовой команды INCA – табл. 4.6.

Таблица 4.3 – Цикл выборки команд

Такт	**Микрокоманда**	**Описание**
t0	MAR ← PC	Счетчик команд (PC) передает адрес следующей команды в регистр адреса памяти (MAR).
t1	MDR ← M[MAR]	Чтение команды из памяти в регистр данных (MDR).
t2	IR ← MDR	Передача команды в регистр команд (IR) для выполнения.
t3	PC ← PC + 1, F,R ← 0,1	Инкремент счетчика команд, подготовка к следующей выборке; триггер F устанавливается для перехода к циклу выполнения.

Таблица 4.4 – Цикл дешифрации команды

Такт	**Микрокоманда**	**Описание**
C1t0	DECODER ← IR[OP]	Дешифрация кода операции из старших разрядов IR
C1t1	ADDRESS MODE ← IR[AD]	Определение адреса операнда и типа адресации
C1t2	F,R←1,0	Подготовка к циклу выполнения C2
C1t3	NOP	Такт ожидания (иногда используется для синхронизации)

Таблица 4.5 – Запись цикла выполнения команды ADD

К,Ц,Т	**Микрокоманда**	**Описание**
q2C2t0	MAR ← IR[AD]	Перенос адреса операнда из поля IR в регистр адреса памяти.
q2C2t1	MDR ← M[MAR]	Чтение второго операнда из памяти в регистр MDR.
q2C2t2	AC ← AC + MDR	Сложение содержимого аккумулятора и MDR, запись результата в AC.
q2C2t3	F,R ← 1,1	Переход к циклу завершения C3

Таблица 4.6 – Запись цикла выполнения команды INCA

К,Ц,Т	Микрокоманда	Описание
q13C2t0	AC ← AC + 1	Инкремент аккумулятора
q13C2t1	NOP	Такт ожидания (иногда используется для синхронизации)
q13C2t2	NOP	Такт ожидания (иногда используется для синхронизации)
q13C2t3	F,R ← 0,0	Переход к циклу завершения C3

Цикл C3 выполняет следующие функции:

- фиксация результата операции в целевых регистрах или памяти;
- обновление флагов состояния (FR);
- подготовка ВМ к переходу к следующей команде.

Для большинства команд учебной ВМ этот цикл интегрирован в последний такт выполнения (q2C2t3 для ADD, q13C2t3 для INCA), поэтому отдельного процесса не требуется.

Запишем управляющее слово (табл. 4.7) для команды ADD при условии:

- адрес команды: 5A8;
- адрес операнда: F12;
- значение операнда в памяти: 731F;
- значение в аккумуляторе: 8721.

Таблица 4.7 – Управляющее слово для всех четырех циклов (ADD)

К,Ц,Т	Микрокоманда	MS(A)	MS(B)	АЛУ	DC	MS(M)	WR	FR
q2C0t0	MAR ← PC	000	xxx	000000	1100	x	0	00
q2C0t1	MDR ← M[MAR]	xxx	xxx	xxxxxx	1011	1	0	00
q2C0t2	IR ← MDR	100	xxx	000000	1101	x	0	00
q2C0t3	PC ← PC+1, F,R←0,1	000	xxx	000001	1111	x	0	01
q2C1t0	DECODER ← IR[OP]	xxx	xxx	xxxxxx	0xxx	x	0	01
q2C1t1	ADDRESS MODE ← IR[AD]	xxx	xxx	xxxxxx	0xxx	x	0	01
q2C1t2	F,R←1,0	xxx	xxx	xxxxxx	0xxx	x	0	01
q2C1t3	NOP	xxx	xxx	xxxxxx	0xxx	x	0	10

Продолжение табл. 4.7

q2C2t0	MAR ← IR[AD]	010	xxx	000000	1100	x	0	10
q2C2t1	MDR ← M[MAR]	xxx	xxx	xxxxxx	1101	1	0	10
q2C2t2	AC ← AC + MDR	001	100	000010	1110	x	0	10
q2C2t3	F,R ← 1,1	xxx	xxx	xxxxxx	0xxx	x	0	11
q2C3t0	WR ←1, AC → MEM	xxx	xxx	xxxxxx	0xxx	x	1	11
q2C3t1	UPDATE FLAGS	xxx	xxx	xxxxxx	0xxx	x	0	11
q2C3t2	NOP	xxx	xxx	xxxxxx	0xxx	x	0	11
q2C3t3	F, R←0,0	xxx	xxx	xxxxxx	0xxx	x	0	00

В табл. 4.8 показано содержимое регистров на каждом такте при выполнении команды ADD. Обратим внимание, что в MDR хранится значение «код операции и адрес операнда». Код операции сложения – «2», поэтому в MDR записано значение 2F12.

Таблица 4.8 – Состояние регистров по всем циклам (пример для ADD)

К,Ц,Т	**Микрокоманда**	**PC**	**MAR**	**IR**	**MDR**	**AC**	**INPR**	**OUTR**
Исходное состояние		5A8	—	—	—	8721	—	—
q2C0t0	MAR ← PC	5A8	5A8	—	—	8721	—	—
q2C0t1	MDR ← M[MAR]	5A8	5A8	—	2F12	8721	—	—
q2C0t2	IR ← MDR	5A8	5A8	F12	2F12	8721	—	—
q2C0t3	PC ← PC+1, F←1	5A9	5A8	F12	2F12	8721	—	—
q2C1t0	DECODER ← IR[OP]	5A9	5A8	F12	2F12	8721	—	—
q2C1t1	ADDRESS MODE ← IR[AD]	5A9	5A8	F12	2F12	8721	—	—
q2C1t2	F←2	5A9	5A8	F12	2F12	8721	—	—
q2C1t3	NOP	5A9	5A8	F12	2F12	8721	—	—
q2C2t0	MAR ← IR[AD]	5A9	F12	F12	2F12	8721	—	—
q2C2t1	MDR ← M[MAR]	5A9	F12	F12	731F	8721	—	—
q2C2t2	AC ← AC + MDR	5A9	F12	F12	731F	FA40	—	—
q2C2t3	F ← 3	5A9	F12	F12	731F	FA40	—	—

Продолжение табл. 4.8

q2C3t0	WR ← AC → MEM	5A9	F12	F12	731F	FA40	—	—
q2C3t1	UPDATE FLAGS	5A9	F12	F12	731F	FA40	—	—
q2C3t2	NOP	5A9	F12	F12	731F	FA40	—	—
q2C3t3	F←0	5A9	F12	F12	731F	FA40	—	—

4.4. Заключение

В ходе работы было изучено устройство управления учебного операционного устройства. Рассмотрены основные поля управляющего слова и их назначение. Выделены четыре цикла выполнения команд: выборки, дешифрации, исполнения и завершения. На примерах команд учебной ВМ показано формирование микроопераций и управляющих слов. Составлена таблица команд виртуальной машины, отражающая структуру учебного процессора. Практическая работа позволила закрепить понимание принципов работы микропрограммного управления и роли управляющего устройства в организации вычислительного процесса.

4.5. Варианты заданий

Заполнить шаблоны табл. 4.9 – «Управляющее слово», и табл. 4.10 – «Состояние регистров» выполнения команды, заданной персональном вариантом.

Таблица 4.9 – Управляющее слово

К,Ц,Т	**Микрокоманда**	**MS(A)**	**MS(B)**	**АЛУ**	**DC**	**MS(M)**	**WR**	**FR**
C0t0								
C0t1								
C0t2								
C0t3								
C2t0								
C2t1								
C2t2								
C2t3								

Таблица 4.10 – Состояние регистров

К,Ц,Т	**Микрокоманда**	**PC**	**MAR**	**IR**	**MDR**	**AC**	**INPR**	**OUTR**
Исходное состояние								
C0t0								
C0t1								
C0t2								
C0t3								
C2t0								
C2t1								
C2t2								
C2t3								

Варианты:

1. Записать микрооперации цикла выборки и выполнения команды LDA. По адресу 101 записана команда LDA с адресом 2A3. По этому адресу команды записан операнд 1F2C. В регистре-аккумуляторе – операнд 3B7D. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

2. Записать микрооперации цикла выборки и выполнения команды STA. По адресу 210 записана команда STA с адресом 4D6. По этому адресу команды записан операнд 9A1F. В регистре-аккумуляторе – операнд C34B. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

3. Записать микрооперации цикла выборки и выполнения команды ADD. По адресу 305 записана команда ADD с адресом 5B7. По этому адресу команды записан операнд 7C2A. В регистре-аккумуляторе – операнд 2F3E. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

4. Записать микрооперации цикла выборки и выполнения команды SUB. По адресу 412 записана команда SUB с адресом 38F. По этому адресу команды записан операнд 8E4D. В регистре-аккумуляторе – операнд FF12.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

5. Записать микрооперации цикла выборки и выполнения команды AND. По адресу 523 записана команда AND с адресом 6C9. По этому адресу команды записан операнд AB7F. В регистре-аккумуляторе – операнд 704B. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

6. Записать микрооперации цикла выборки и выполнения команды OR. По адресу 634 записана команда OR с адресом 2E1. По этому адресу команды записан операнд C13A. В регистре-аккумуляторе – операнд 1F7C. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

7. Записать микрооперации цикла выборки и выполнения команды XOR. По адресу 745 записана команда XOR с адресом 3D4. По этому адресу команды записан операнд BA21. В регистре-аккумуляторе – операнд CD56. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

8. Записать микрооперации цикла выборки и выполнения команды COM. По адресу 856 записана команда COM с адресом 507. По этому адресу команды записан операнд 52E8. В регистре-аккумуляторе – операнд 2A6C. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

9. Записать микрооперации цикла выборки и выполнения команды JMP. По адресу 967 записана команда JMP с адресом 89A. По этому адресу команды записан операнд E1F2. В регистре-аккумуляторе – операнд B2E7. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

10. Записать микрооперации цикла выборки и выполнения команды DEC. По адресу A78 записана команда DEC с адресом 36F. По этому адресу команды записан операнд 6B4C. В регистре-аккумуляторе – операнд 56F1. Определить информацию, которая будет записана в регистрах PC, MAR, MDR,

IP, AC после выполнения этой команды. Показать процесс выполнения команды.

11. Записать микрооперации цикла выборки и выполнения команды INC. По адресу B89 записана команда INC с адресом D2B. По этому адресу команды записан операнд 5217. В регистре-аккумуляторе – операнд A7D4. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

12. Записать микрооперации цикла выборки и выполнения команды NEG. По адресу C9A записана команда NEG с адресом 406. По этому адресу команды записан операнд AD31. В регистре-аккумуляторе – операнд 2B0D. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

13. Записать микрооперации цикла выборки и выполнения команды CLEA. По адресу DAB записана команда CLEA с адресом FFF. По этому адресу команды записан операнд FFFE. В регистре-аккумуляторе – операнд F657. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

14. Записать микрооперации цикла выборки и выполнения команды INCA. По адресу EBC записана команда INCA с адресом FFF. По этому адресу команды записан операнд FFFD. В регистре-аккумуляторе – операнд F23F. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

15. Записать микрооперации цикла выборки и выполнения команды INP. По адресу FCD записана команда INP с адресом FFF. По этому адресу команды записан операнд FFFF. В регистре-аккумуляторе – операнд 5A4C. Регистры INPR=AB, OUTR=DE. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC, INPR, OUTR после выполнения этой команды. Показать процесс выполнения команды.

16. Записать микрооперации цикла выборки и выполнения команды OUT. По адресу 1DE записана команда OUT с адресом FFF. По этому адресу команды записан операнд FFFF. В регистре-аккумуляторе – операнд AB13. Регистры INPR=BC, OUTR=AE. Определить информацию, которая будет

записана в регистрах PC, MAR, MDR, IP, AC, INPR, OUTR после выполнения этой команды. Показать процесс выполнения команды.

17. Записать микрооперации цикла выборки и выполнения команды LDA. По адресу 2EF записана команда LDA с адресом 504. По этому адресу команды записан операнд C34B. В регистре-аккумуляторе – операнд 5542. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

18. Записать микрооперации цикла выборки и выполнения команды STA. По адресу 3F0 записана команда STA с адресом 15F. По этому адресу команды записан операнд 1F16. В регистре-аккумуляторе – операнд CE14. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

19. Записать микрооперации цикла выборки и выполнения команды AND. По адресу 4F1 записана команда AND с адресом 14C. По этому адресу команды записан операнд 477B. В регистре-аккумуляторе – операнд 2270. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

20. Записать микрооперации цикла выборки и выполнения команды SUB. По адресу 5F2 записана команда SUB с адресом 235. По этому адресу команды записан операнд 2AB6. В регистре-аккумуляторе – операнд FF46. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

21. Записать микрооперации цикла выборки и выполнения команды AND. По адресу 6F3 записана команда AND с адресом 67C. По этому адресу команды записан операнд 377E. В регистре-аккумуляторе – операнд 557B. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

22. Записать микрооперации цикла выборки и выполнения команды OR. По адресу 7F4 записана команда OR с адресом 2CD. По этому адресу команды записан операнд CCFE. В регистре-аккумуляторе – операнд FAC2. Определить информацию, которая будет записана в регистрах PC, MAR, MDR,

IP, AC после выполнения этой команды. Показать процесс выполнения команды.

23. Записать микрооперации цикла выборки и выполнения команды XOR. По адресу 8F5 записана команда XOR с адресом 50D. По этому адресу команды записан операнд AB46. В регистре-аккумуляторе – операнд FF35. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

24. Записать микрооперации цикла выборки и выполнения команды COM. По адресу 9F6 записана команда COM с адресом 60A. По этому адресу команды записан операнд 54FE. В регистре-аккумуляторе – операнд 3CD0. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

25. Записать микрооперации цикла выборки и выполнения команды JMP. По адресу AF7 записана команда JMP с адресом 99C. По этому адресу команды записан операнд CF13. В регистре-аккумуляторе – операнд AB56. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

26. Записать микрооперации цикла выборки и выполнения команды DEC. По адресу BF8 записана команда DEC с адресом 458. По этому адресу команды записан операнд 7B12. В регистре-аккумуляторе – операнд 5440. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

27. Записать микрооперации цикла выборки и выполнения команды INC. По адресу CF9 записана команда INC с адресом A13. По этому адресу команды записан операнд B457. В регистре-аккумуляторе – операнд A114. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

28. Записать микрооперации цикла выборки и выполнения команды NEG. По адресу DF0 записана команда NEG с адресом 60B. По этому адресу команды записан операнд CB65. В регистре-аккумуляторе – операнд CCAA. Определить информацию, которая будет записана в регистрах PC, MAR, MDR,

IP, AC после выполнения этой команды. Показать процесс выполнения команды.

29. Записать микрооперации цикла выборки и выполнения команды CLEA. По адресу EF1 записана команда CLEA с адресом FFF. По этому адресу команды записан операнд FFFF. В регистре-аккумуляторе – операнд 458D. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

30. Записать микрооперации цикла выборки и выполнения команды INCA. По адресу FF2 записана команда INCA с адресом FFF. По этому адресу команды записан операнд FFFF. В регистре-аккумуляторе – операнд AB20. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды. Показать процесс выполнения команды.

31. Записать микрооперации цикла выборки и выполнения команды INP. По адресу 10F3 записана команда INP с адресом FFF. По этому адресу команды записан операнд FFFF. В регистре-аккумуляторе – операнд 1730. Регистры INPR=FE, OUTR=F6. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC, INPR, OUTR после выполнения этой команды. Показать процесс выполнения команды.

32. Записать микрооперации цикла выборки и выполнения команды OUT. По адресу 11F4 записана команда OUT с адресом FFF. По этому адресу команды записан операнд FFFF. В регистре-аккумуляторе – операнд CD1F. Регистры INPR=D1, OUTR=7B. Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC, INPR, OUTR после выполнения этой команды. Показать процесс выполнения команды.

5. ОЗУ. ВИРТУАЛЬНАЯ И ФИЗИЧЕСКАЯ ПАМЯТЬ

5.1. Введение

Рассмотрев устройство учебного вычислительного устройства, можно переходить к рассмотрению более сложных видов запоминающих элементов. В современном компьютере существует несколько основных уровней памяти, каждая из них имеет свои особенности, характеристики и задачи, для которых она используется. Все виды памяти в компьютере можно изобразить в виде пирамиды, на вершине которой располагается самая быстрая, малая по объему и самая дорогая память, а в основании, медленная, большая по объему и сравнительно дешевая память (рис. 5.1).



Рисунок 5.1. Иерархия памяти

Следующим, по иерархии памяти будет Оперативное Запоминающее Устройство (ОЗУ). Главная задача ОЗУ быть мостом между внешней памятью и КЭШ памятью процессора. ОЗУ компьютера – это энергозависимая память, то есть память, способная хранить информацию только пока есть напряжение, при выключении компьютера эта память полностью освобождается до следующего включения, в результате чего для долговременного хранения эта память не подходит. В данной практической работе студентам предоставляется возможность изучить физическое устройство ОЗУ, его основные характеристики, а также взаимосвязь программной и аппаратной составляющих в функционировании этого вида памяти.

5.2. Устройство оперативной памяти и ее основные характеристики

Рассмотрение строения оперативной памяти ПК начинается с рассмотрения наименьшего элемента – одной ячейки. Для построения одной ячейки памяти потребуется два компонента: МОП-транзистор и конденсатор. МОП-транзистор, в данном случае будет выполнять роль ключа и позволит току попадать на конденсатор, тем самым заряжая его. Конденсатор, в свою очередь, сохранит этот заряд (логическую 1), на некоторое непродолжительное время (ввиду его физических особенностей), в результате чего, для увеличения времени хранения, а также при работе с памятью в целом, заряд потребуется регулярно обновлять.

Для операций «Записи» или «Чтения» в одну ячейку ОЗУ, требуется два сигнала «Word-line» и «Bit-line» (Рисунок НИЖЕ). Первая линия является управляющей, через нее подается «адрес» ячейки. Вторая линия, используется для непосредственной записи информации, в ячейку (зарядка конденсатора). Данное правило актуально для единственной ячейки, в глобальной схеме организации памяти функционал у этих линий несколько отличается. Схема подключения для одной ячейки изображена на рис. 5.2.

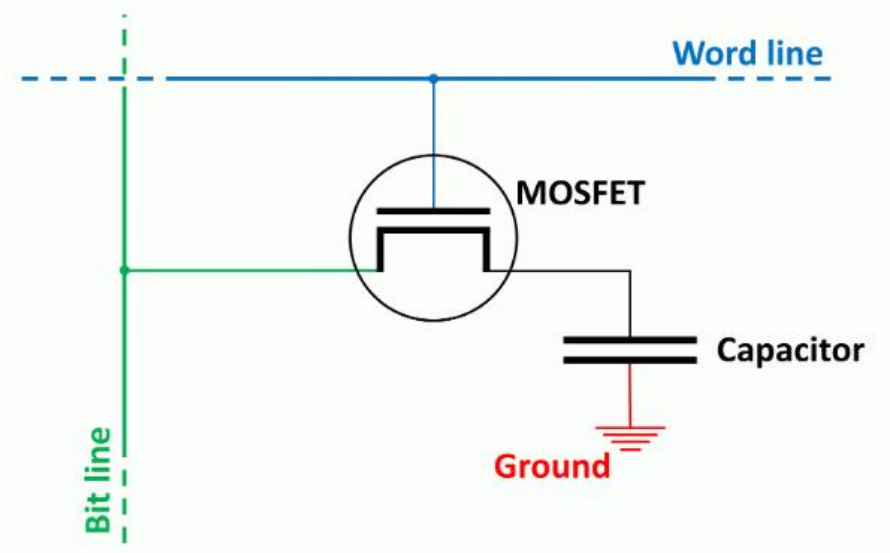


Рисунок 5.2. Схема подключения одной ячейки памяти

Для организации хранения нескольких бит, данное подключение дополняется огромным множеством таких же подключений, для того чтобы поместить большее количество информации. При этом, учитывая, что все ячейки зависят от управляющей «Word-line», то ячейки объединяются в группы по n элементов, образуя страницы. Рис. 5.3 показывает схему

организации двух страниц в ОЗУ. Обе страницы, в данном примере, имеют два бита информации.

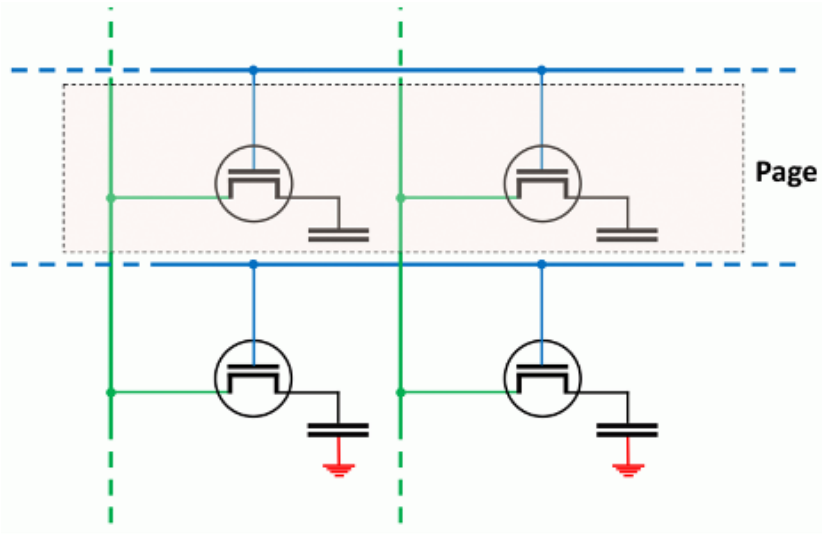


Рисунок 5.3. Схема подключения двух ячеек памяти

Как и было сказано в итоговой организации памяти принцип немного меняется, у «Word-line» задача выбрать строку в памяти (целиком всю страницу), у «Bit-line» – задача выбрать столбец в памяти. На полученном пересечении формируется записываемый/читаемый бит. Чем больше бит в одной странице, тем больше конденсаторов потребуется для их хранения, и в современных системах стандартный размер страницы 4К Байт. Совокупность множества страниц, адресуемых строками и столбцами, образует один банк памяти, рис. 5.4.

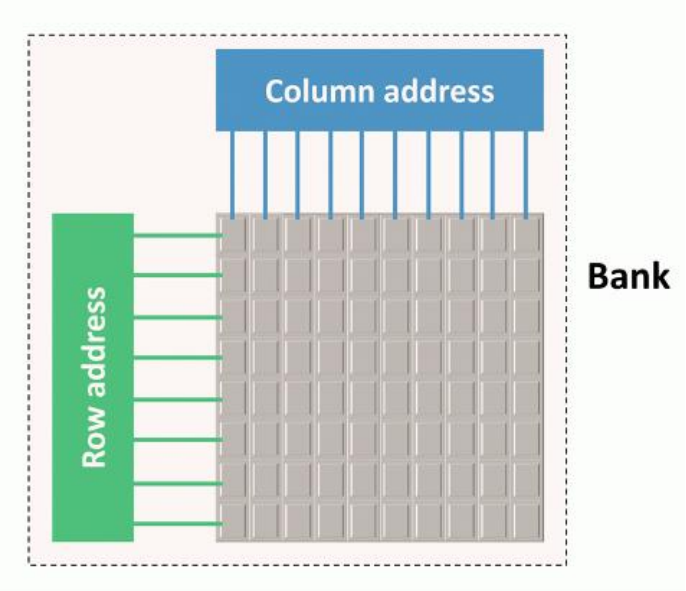


Рисунок 5.4. Банк памяти

Банки памяти – это по-сути матрицы, где на пересечении строк и столбцов которых хранятся биты, соответствующие хранимой информации. По своему построению и структуре, банки памяти могут быть прямоугольными или квадратными, каждый производитель решает сам, какие банки он будет использовать, и узнать наверняка какие – не представляется возможным. Высшей точкой в составлении архитектуры планки памяти ОЗУ является – модуль (чип) памяти. Чип памяти – это совокупность банков памяти, соединенных тремя основными шинами, с помощью которых осуществляется взаимодействие с ними (рис. 5.5). Командная шина (Command bus) указывает на операцию с данными, шина адреса (Address bus) выбирает нужную строку или столбец, шина данных (Data bus) передает значения в процессор или, наоборот, из него после обработки.

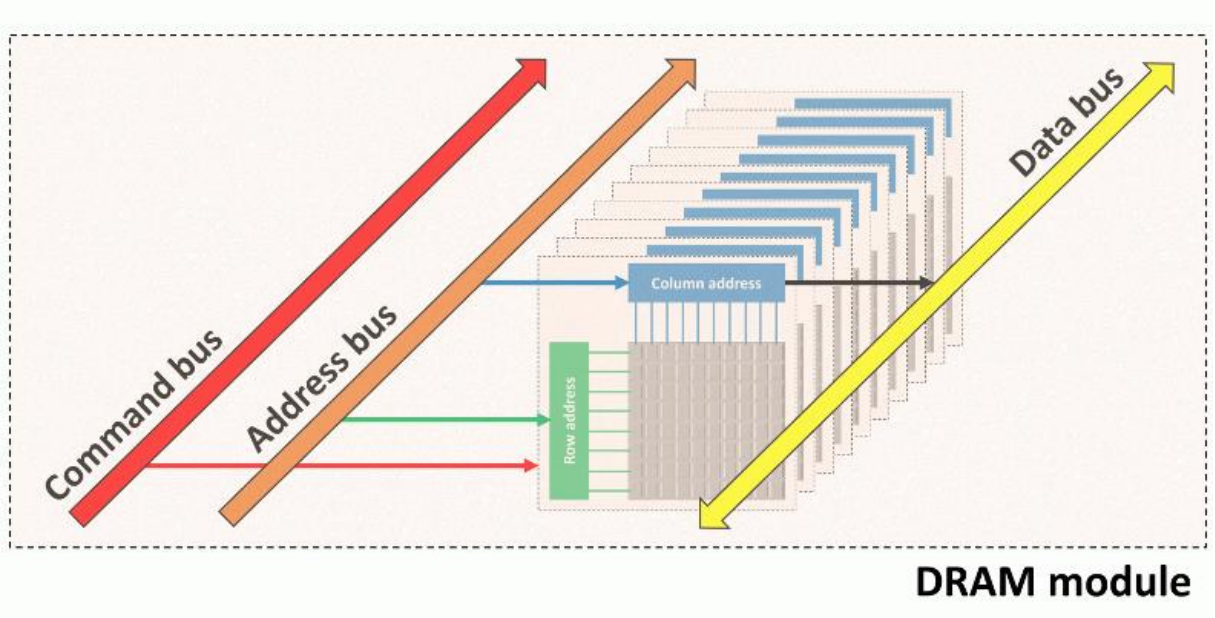


Рисунок 5.5. Схема DRAM модуля

Принципы работы модулей памяти менялись по мере развития технологий. Первым стандартом стала SDR-SDRAM (1993 г.), Single Data Rate Synchronous Dynamic RAM – Синхронная Динамическая память с произвольным доступом с одинарной передачей данных. Она работала на тактовых частотах 66 – 133 МГц и выполняла одну операцию за такт, передавая 8 бит данных. В 1998 г. Samsung предложила новый подход – DDR SDRAM, где передача данных осуществляется дважды за такт, по фронтам (на повышении и спаде сигнала), данный принцип показан на рис. 5.6. Такой способ позволил существенно ускорить обработку данных, и даже при столь

низких частотах (66 – 133 МГц). Данный метод лег в основу всей ОЗУ и используется по сей день с некоторыми изменениями.

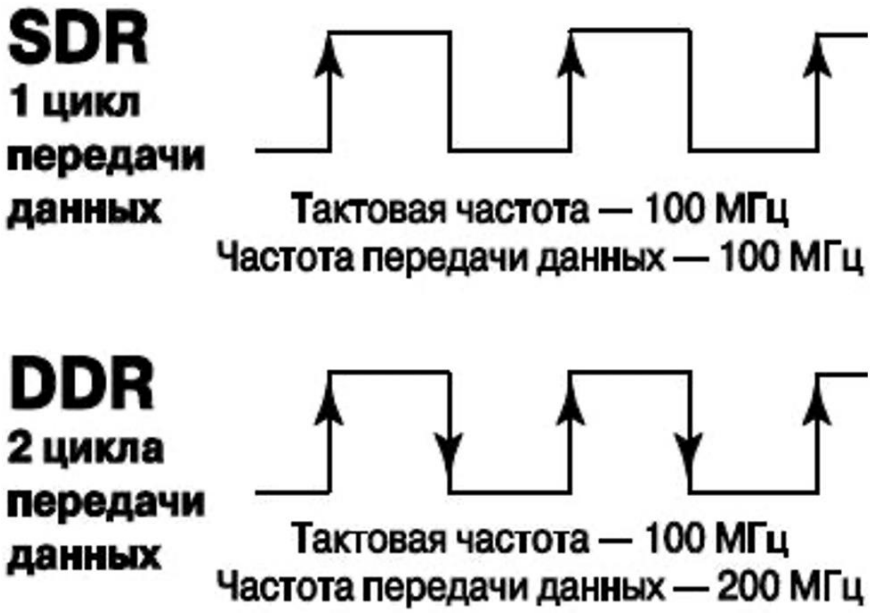


Рисунок 5.6. Принцип передачи данных DDR SDRAM

В спецификациях к ОЗУ, стандартно указывают «эффективную» частоту передачи данных (например, 3200 МГц для стандарта DDR4), которая вдвое выше реальной частоты. При просмотре частот в ОС Windows, например, тоже отображается «эффективная» частота, реальную же частоту можно найти путем деления на 2 «эффективной» частоты, либо через выполнение команды, или же с помощью специализированных программ, таких как AIDA 64 или ее аналоги.

С 2000 года DDR-память прошла пять поколений – от DDR1 (DDR SDRAM или просто DDR) до DDR5. Каждое новое поколение повышало реальную частоту, увеличивало объем чипов памяти и снижало рабочее напряжение. Рабочее напряжение в свою очередь зависит от способа и количества организации банков памяти. Чем больше в них банков памяти, и чем больше страницы в них, тем большее напряжение требуется подать, чтобы эту страницу прочитать. Базовые характеристики ОЗУ фиксирует стандарт JEDEC (Joint Electron Devices Engineering Council – Сообщество инженеров электронных устройств), а для разгона применяются XMP-профили, позволяющие поднять характеристики выше заявленных в стандарте без значительной потери ресурса. Эти параметры хранятся непосредственно на планке памяти и могут регулироваться вручную через BIOS/UEFI. Сводная таблица характеристик ОЗУ за все поколения – табл. 5.1.

Таблица 5.1 – Характеристики DDR памяти по поколениям

Характеристика	**DDR1**	**DDR2**	**DDR3**	**DDR4**	**DDR5**
Год выпуска	2000	2003	2007	2014	2020
Рабочее напряжение	2.5–2.6 В	1.8 В	1.5–1.65 В	1.2 В	1.1 В
Частота (МГц)	200–400	400–1066	800–2133	1600–3200	3200–6400+
Пропускная способность (ГБ/с)	1.6–3.2	3.2–8.5	6.4–17	12.8–25.6	25.6–51.2+
Количество контактов	184	240	240	288	288
Объем одного модуля (макс.)	1 ГБ	4 ГБ	16 ГБ	64 ГБ	128 ГБ
Поддержка ECC	Нет	Опционально	Опционально	Опционально	Да

Важная характеристика производительности любой ОЗУ – тайминги (задержки при обращении к данным). Возникают они из-за физических основ построения ОЗУ, описанных выше. Для того чтобы что-то прочитать/записать информацию, нужно подать напряжение на всю страницу (а не на отдельные биты), и зарядить/разрядить конденсаторы, после чего восстановить заряды, на тех разрядах, которые не были задействованы, чтобы не потерять имеющуюся информацию. Перечислим наиболее важные виды из них:

1. tRCD (Time Row to Column Delay) – задержка при обращении к строке;
2. tCL (CAS Latency – Time Column Latency) – задержка при доступе к столбцу и непосредственно операции чтения/записи;
3. tRAS (Row Active Time) – минимальное время работы со страницей;
4. tRP (Row Precharge) – время восстановления страницы (зарядов на конденсаторах).

Принцип обращения к ОЗУ, а также в какой момент времени задействуется тот или иной тайминг, показано на следующей схеме (рис. 5.7).

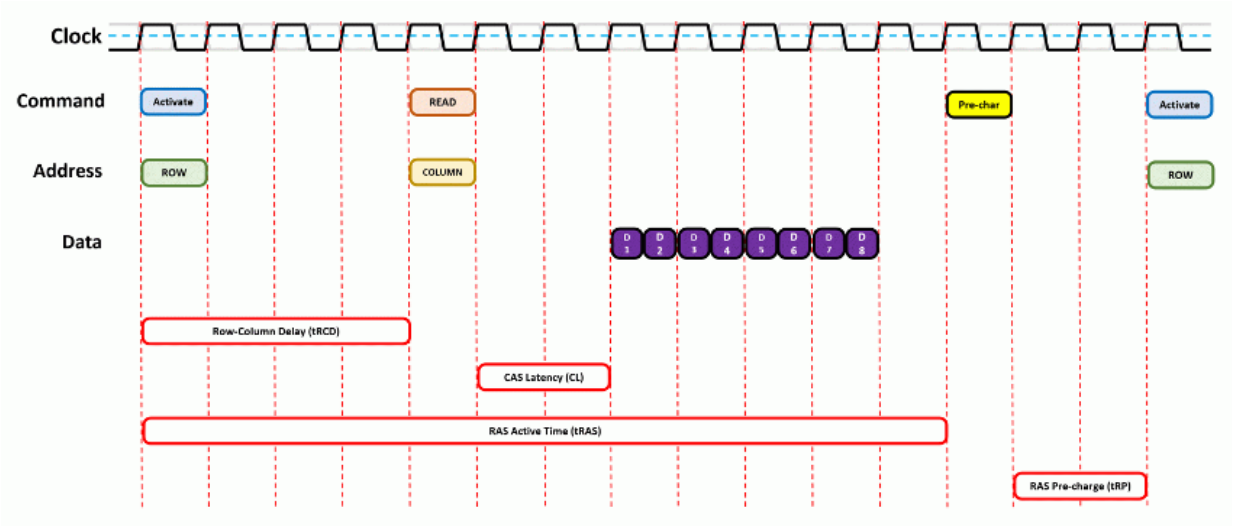


Рисунок 5.7. Временная диаграмма обращения к ОЗУ

Благодаря развитию технологий, частота все время увеличивается, однако, вопреки логичному предположению, что тайминги будут уменьшаться, они увеличиваются в числовом выражении. Но из-за уменьшения длительности такта обращения, новая память, все равно работает быстрее (пример: DDR5 при таймингах 40-40-40-77 будет эффективнее по пропускной способности поколения DDR4 с 16-18-18-36), но задержка у DDR5 будет выше чем у DDR4.

Еще важная характеристика, влияющая на производительность – это конфигурация каналов на материнской плате (точнее физическое подключение планок ОЗУ к ней). Процессор связан с ОЗУ двумя физическими каналами (A и B). При установке одной планки ОЗУ работает только один канал, при двух – активируется двухканальный режим, что удваивает пропускную способность и еще больше ускоряет обмен данными процессора с ОЗУ. В случае, когда в наличии только одна планка памяти, процессор будет взаимодействовать только с ней, и данные, которые будут в ней храниться, будут запрашиваться поочередно. Однако, если вместо одной планки памяти, например, на 8 ГБ, установить две планки по 4 ГБ, то процессор сможет распределить данные уже по двум планкам памяти, и тем самым брать данные с двух каналов одновременно, что вдвое увеличит пропускную способность, и как следствие общее быстродействие в пользовательских приложениях.

Кроме того, модули различаются по рангам – наборам чипов памяти. Контроллер работает с одним рангом одновременно, но может переключаться между ними. Чаще всего, в бытовых устройствах, встречаются одноранговые и двухранговые планки (рис. 5.8). Двухранговые планки удобны при ограниченном числе слотов или ограниченности физических размеров (SO-

DIMM), или, например, для обеспечения большего объема памяти. Для серверов доступны многоранговые модули (например, 4-х) с поддержкой ECC (коррекция ошибок) и буферизацией, так как там в приоритете сохранность данных.

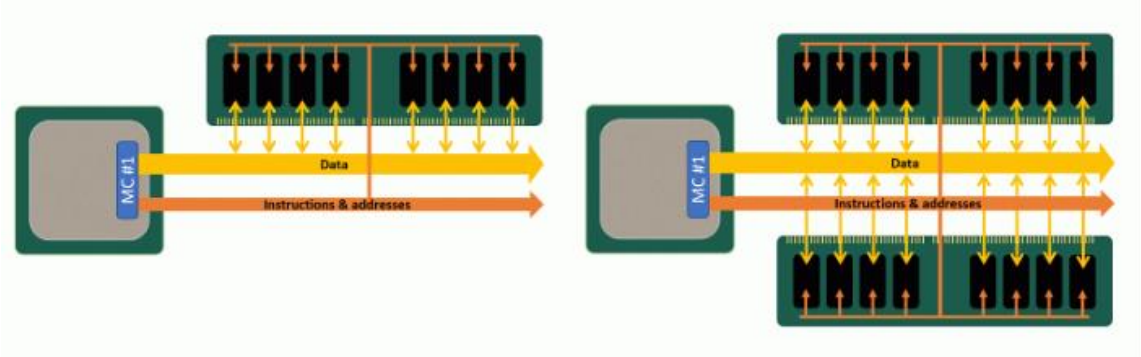


Рисунок 5.8. Планки памяти

Последней важной особенностью является форм-фактор ОЗУ. Существует два основных вида: DIMM (стандартный модуль для ПК) и SO-DIMM (компактный для ноутбуков). Современные модули DIMM поддерживают до 128 ГБ на одну планку, SO-DIMM – до 64 ГБ, такие ограничения актуальны из-за физических размеров самих планок.

5.3. Взаимосвязь программной и аппаратной частей работы современной ОЗУ

Современные программы требуют огромные объемы данных для своего корректного исполнения, и, зачастую, эти объемы превышают доступные объемы оперативной памяти компьютера. Немаловажной особенностью является и то количество процессов (например, ОС), которые выполняются в компьютере помимо нужной пользователю программы, а ведь самих прикладных программ может быть несколько. Из-за всего вышеописанного возникают проблемы с размещением всех требуемых данных в ограниченной физической памяти таким образом, чтобы это не сильно влияло на общую производительность, и, при этом, адресное пространство должно быть настроено таким образом, чтобы одни процессы не «залезали» области других процессов. Понятно, что изменение одной или нескольких «чужих» страниц в ОЗУ может привести как к потере важных пользовательских данных, так и к, возможному, краху операционной системы. Для грамотного решения озвученных проблем был придуман механизм разделения на виртуальное и физическое адресные пространства. Разберем его чуть более подробно.

Физическое адресное пространство (PA) – это понятный и разобранный ранее механизм ОЗУ. Есть некоторое количество страниц, хранимых в банках памяти и готовых к передаче в КЭШ для скорого исполнения на процессоре.

Виртуальное пространство (VA) – это некоторая, абстрактная изолированная среда для каждой отдельной программы. Для нее (программы) – это единая неделимая область, некоторое подобие собственной ОЗУ, в которой есть своя правильная адресация и только ей разрешено работать и изменять данные. Первое, что дает этот механизм – это доступ к существенно большему объему памяти, чем есть в физической ОЗУ в ПК. Второе – позволяет не загружать всю программу целиком в ОЗУ, а загружать информацию отдельными частями, по мере необходимости. Третье, что напрямую вытекает из второго – это факт того, что одновременно может выполняться существенно больше программ, благодаря такой оптимизации хранения. Четвертое, возможно, главное из всех – это отсутствие необходимости соблюдать последовательность адресации, для сохранения целостности программ. Каждая программа записывает свои страницы по физическому адресу в случайном порядке в первый освободившийся «слот» и таким же образом выгружает их, не мешая остальным программам.

Совокупность описанных особенностей механизма виртуализации памяти, в первую очередь, позволяет упростить работу программистов, так как им не требуется работать с реальными физическими адресами (которые скорее всего будут отличаться от устройства к устройству) на накопителях и ОЗУ, и позволяет сосредоточиться на грамотном разграничении адресного пространства разрабатываемой программы и ее данных, а все остальные задачи по их загрузке ОЗУ для корректной работы на себя возьмет ОС и совокупность различных аппаратных механизмов.

Однако, такой механизм предполагает ряд проблем, которые связаны с распределением этих адресов, а главное с переводов из PA в VA и обратно. Здесь стоит упомянуть, что программа со своей «виртуальной ОЗУ» никак не взаимодействует с физическим адресом и аппаратной стороной ПК, более того, она даже не «знает» о его существовании, а все задачи по распределению данных находятся на ОС и устройстве MMU (Memory Management Unit) – блоке управления памятью. MMU – это специальное аппаратно-программное решение, часть современных процессоров, которая позволяет преобразовывать виртуальные адреса запросов на чтение/запись памяти, выдаваемых программами, в физические для сохранения в постоянной памяти. Данный механизм открыл новые возможности в управлении памятью.

Современные операционные системы используют различные техники, позволяющие избежать многократной загрузки общего кода в ОЗУ, который может применяться разными программами, например, стандартной библиотеки языка «С». Для этого код единожды помещается в определенную область физической памяти, а затем MMU настраивается так, чтобы каждая программа «видела» его в своем виртуальном адресном пространстве. Еще один прием, применяемый современными ОС (разные ОС делают это по-разному), заключается в том, чтобы выгружать часть физической памяти процесса на внешний диск в так называемый «файл подкачки» (или его аналог) и освобождать ресурсы ОЗУ для других задач. При этом MMU перенастраивается так, чтобы при обращении процесса к выгруженным данным, ОС получала сигнал подгружать их обратно с диска. Это особенно удобно, так как у подавляющего большинства программ есть код, необходимый только на этапе запуска – в большинстве случаев его можно безопасно выгрузить во внешнюю память и больше не держать в оперативной памяти, пока программа не будет перезапущена.

Способов организации подкачки данных программ в физическое пространство ОЗУ несколько, и за время своего существования каждый из них показал как свои недостатки, так и свои преимущества в тех или иных случаях. Всего существует 3 типа организации связи между физическим и виртуальным пространствами: страничный, сегментный и сегментно-страничный, разберем суть их организации.

Страничная организация памяти. Первым, и по совокупности, наиболее распространенным является страничный метод организации (рис. 5.9).

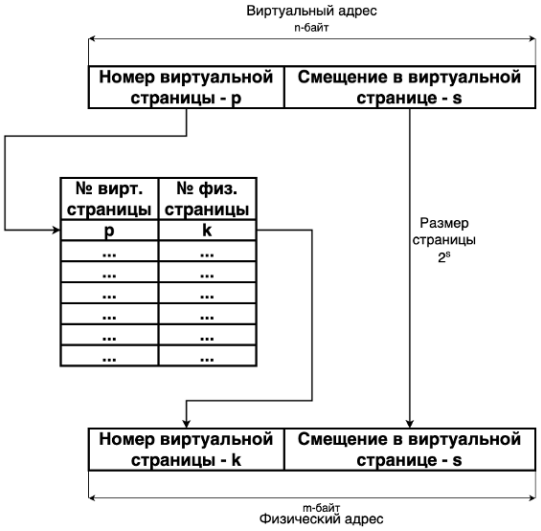


Рисунок 5.9. Организация сегментной памяти

Всякий раз, когда процессору потребуется получить доступ к виртуальной странице процесса некоторой программы, он должен транслировать виртуальный адрес в соответствующий физический адрес, для корректного обращения. Здесь, следует упомянуть, что сам по себе виртуальный адрес и его получение – это комплексный процесс, если отобразить его в виде некоторой схемы, то получится следующее (рис. 5.10), или следующее (рис. 5.11).

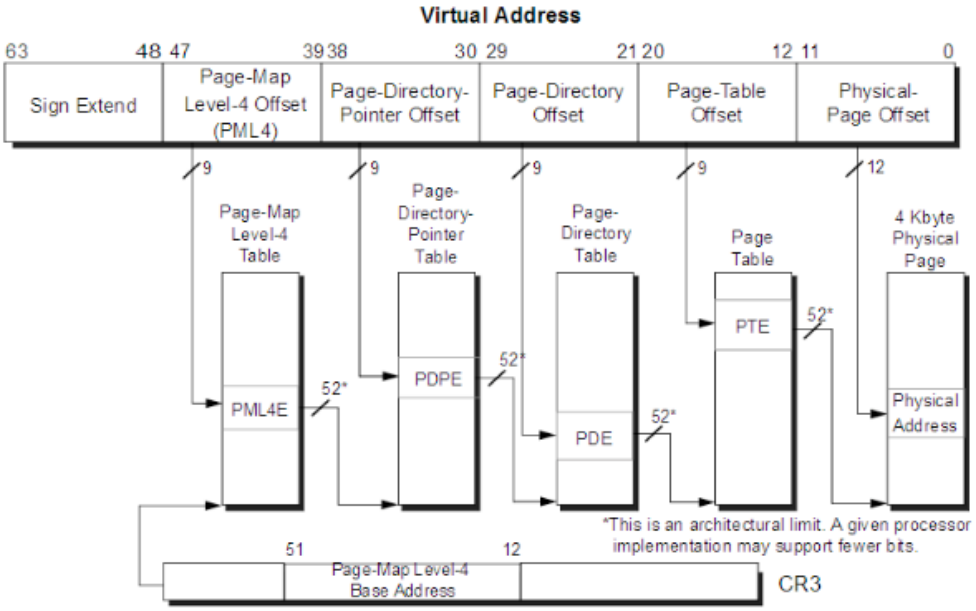


Рисунок 5.10. Организация виртуальной памяти

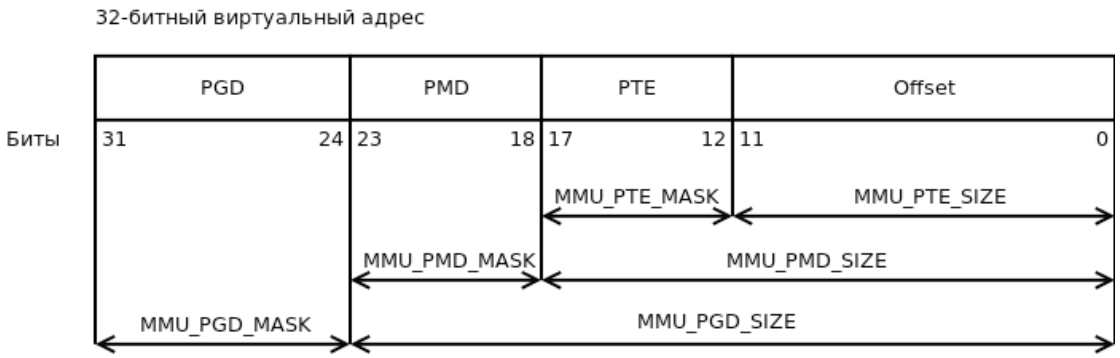


Рисунок 5.11. 32-битный виртуальный адрес

На данном рисунке (верхний) изображена сама структура многоуровневой страничной организации памяти (в данном случае 4), на примере архитектуры AMD64, ниже, для сравнения, приведена 3-х уровневая структура для ОС Embox (на базе Linux). На вершину такого Отличия могут быть в том, как та или иная ОС управляет таблицами страниц: Linux использует свои структуры (PGD, PMD, PTE и т.д.), а Windows – свои (Page

Directory, Page Table и пр.). Но на уровне процессора этот механизм одинаков и процесс трансляции (page walk) адреса неизменен. PGD (Page Global Directory) – таблица самого высокого уровня, каждая запись в ней – ссылка на Page Middle Directory (PMD), записи которой, в свою очередь, ссылаются на таблицу Page Table Entry (PTE). Записи в PTE ссылаются на реальные физические адреса, а также хранят флаги состояния страницы. Общий посыл построения многоуровневой страничной организации памяти – един, но сами записи, и хранимые там биты отличаются в зависимости от конкретной архитектуры.

Сам адрес занимает как правило 48 бит (может быть и меньше или больше), из которых 4 уровня по 9 бит, составляющих непосредственно адрес и 12 бит, которые отведены на смещение внутри страницы. Вместо того чтобы каждый раз, при обращении к данным, проходить полностью трансляцию нужной виртуальной страницы по заданному пути, и была создана ассоциативная страничная таблица Translation Lookaside Buffer (TLB), куда заносятся готовые для использования адреса. Стоит также отметить, что для некоторых архитектур характерна запись и некоторых промежуточных данных процесса трансляции. TLB в рамках процессора – это отдельный элемент, реализованный как ассоциативная память. С точки зрения построения такая память может быть разбита на 2 буфера: один более быстрый с наиболее востребованными страницами, другой – с менее востребованными страницами и, следовательно, более медленный. В данном случае все зависит от конкретной архитектуры и производителя. В случае отсутствия в таблице информации для трансляции виртуальной страницы в физический фрейм (TLB-промах, или же «Page fault»), процессору потребуется обратиться непосредственно к процессу программы и получить физический адрес страницы от него, путем его трансляции через схему выше, после чего добавить в TLB полученную информацию. Сей процесс может быть реализован как аппаратно, так и программно – через обработчик исключения. Сами страницы как правило имеют фиксированный размер, например, 4096 Байт для 64-битных систем архитектуры x86 (может быть и больше, так как есть поддержка «сверхбольших» страниц, вплоть до 1 ГБ), что позволяет эффективно работать в обоих адресных пространствах. Для правильной загрузки/выгрузки размеры соблюдаются как в физическом, так и в виртуальном адресных пространствах.

Среди преимуществ такого подхода к организации памяти, в первую очередь можно выделить его быстродействие, так как после инициализации

TLB, достаточно просто оперировать ассоциативными записями для своевременной подгрузки требуемой информации. Вторым, не менее весомым, является малая, если и вовсе отсутствующая фрагментация, ввиду равного размера страниц. Однако, даже при многоуровневой организации, есть недостаток в лице недостатка прав доступа к той или иной информации, что накладывает ряд ограничений.

Сегментная организация памяти. Следующий вариант организации связи VA и PA – сегментный (рис. 5.12).

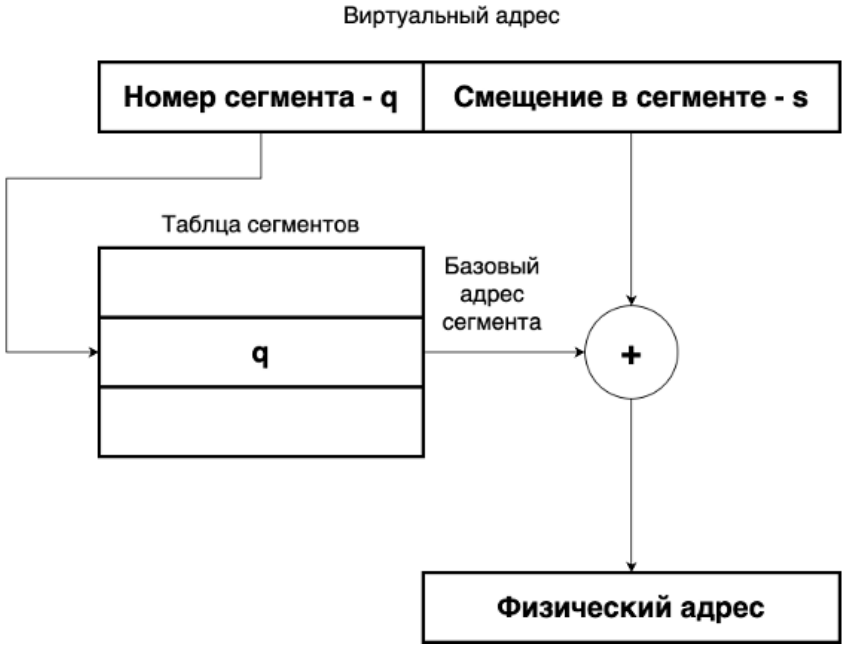


Рисунок 5.12. Схема сегментарной памяти

При таком распределении появляется возможность добавить права доступа и логически поделить всю информацию на блоки (код, данные, инструкции и тп.). Как и в предыдущем случае MMU создает таблицу, которая содержит уже не страницы, а сегменты, заданного размера. Отличие заключается в том, что сегменты (в отличие от страниц) имеют различный размер и находятся в виртуальной памяти в случайном порядке. Поэтому при вычислении физического адреса необходимо производить вынужденные арифметические вычисления, учитывая базовый адрес сегмента в ОЗУ и смещение внутри сегмента. Эти вычисления, естественно, требуют больше времени по сравнению с ассоциативной таблицей адресов при страничном распределении, которое осуществляется посредством конкатенации номера страницы и смещения внутри страницы, что замедляет общую работу ПК. Однако самым крупным недостатком является агрессивная фрагментация, в

виду размерности сегментов, а это в свою очередь приводит к появлению «дыр» в ОЗУ, которые, зачастую, нечем закрыть, так как подходящего по размеру сегмента может не быть. На практике данный подход в классической реализации не применяется, так как есть более продвинутый вариант, сочетающий в себе как преимущества текущего и предыдущего подходов.

Странично-сегментная организация памяти. Пиком эволюции связи виртуального и физического пространств, можно по праву считать именно данный вариант (рис. 5.13).

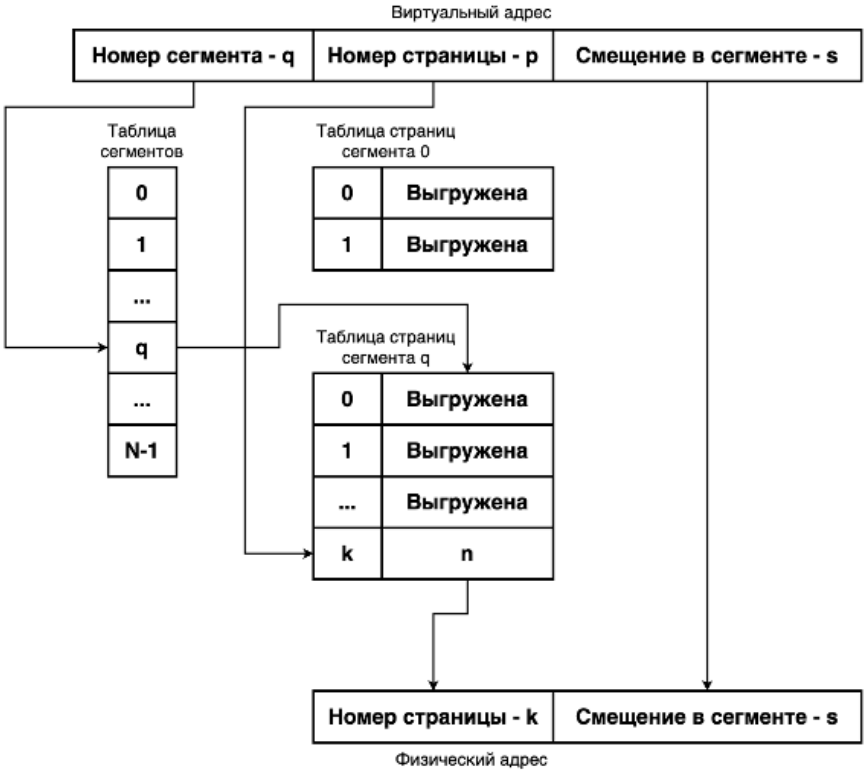


Рисунок 5.13. Странично-сегментная организация памяти

По общей идеологии, программы в виртуальном пространстве разбиваются на сегменты, а сегменты, в свою очередь, разделяются на страницы фиксированного размера. Для применения данного метода, как видно из схемы, потребуется в 2 раза больше ресурсов, так как хранить нужно и адрес сегмента, и адрес страницы (а при всем при этом, их еще нужно изначально вычислить), что уже говорит о том, что в плане производительности, данный метод куда медленнее двух предыдущих. Преимущества в плане разграничения прав доступа и быстрого общения с физическими адресами благодаря страницам не смогли перевесить быстродействия страничного распределения, в результате чего данный метод также не применяется в современных реалиях.

5.4. Заключение

В результате выполнения данной работы студенты познакомились со строением оперативного запоминающего устройства (ОЗУ), изучили основные его характеристики и их влияние на производительность современных персональных компьютеров. Была подробно рассмотрена взаимосвязь аппаратной и программной стороны работы оперативной памяти, рассмотрены основные виды организации этого взаимодействия, приведены соответствующие схемы.

5.5. Варианты заданий

В рассмотренной работе, для записи информации на ОЗУ, требуется использовать таблицу TLB, которая содержит записи о виртуальной и физической странице. Согласно страничной организации памяти в TLB хранится номер страницы и смещение внутри страницы. В полученном варианте рассматривается ситуация записи конкретной виртуальной страницы на некоторый физический фрейм, даны соответствующие номера VPN – Virtual Page Number и PFN – Physical Frame Number, дан размер страницы и виртуальный адрес, требуется на основании полученных данных вычислить физический адрес. Также требуется указать возможна ли такая трансляция или же в результате вычисления адреса произойдет «Page Fault» и будет вызвано прерывание.

Рассмотрим пример: VPN = 3 → PFN = 11, размер страницы 4 КБ, виртуальный адрес 0x5142. Сначала определим, что у нас есть и что требуется найти. В задаче даны:

- VPN (Virtual Page Number) = 3;
- PFN (Physical Frame Number) = 11;
- размер страницы = 4 КБ (4096 байт);
- виртуальный адрес = 0x5142.

Нужно вычислить физический адрес, соответствующий данному виртуальному, используя таблицу TLB. Также нужно проверить, возможно ли такое преобразование, или произойдёт «Page Fault».

Разделение виртуального адреса на компоненты. Виртуальный адрес в страничной памяти состоит из двух частей:

1. Номер страницы (VPN) – старшие биты, указывающие на страницу.
2. Смещение внутри страницы (Offset) – младшие биты, указывающие конкретное место в странице.

Так как размер страницы 4 КБ, а 4 КБ = 4096 = 2^{12}, для смещения потребуется 12 бит. То есть младшие 12 бит виртуального адреса – это смещение, а остальные старшие биты – номер виртуальной страницы.

Выделение смещения из виртуального адреса. Вычисляем смещение внутри страницы:

- Виртуальный адрес = 0x5142 (в шестнадцатеричной системе).
- Преобразуем в двоичную или просто разделим на страницу. Размер страницы = 0x1000 (4096). Смещение = 0x5142 / 0x1000 = 0x5142 - 0x5000 = 0x142. Таким образом, Offset = 0x142.

Определение VPN из виртуального адреса. Теперь определим номер страницы виртуального адреса: VPN = виртуальный адрес // размер страницы = 0x5142 // 0x1000 = 0x5 = 5.

В нашей TLB записан только VPN = 3 → PFN = 11. Сравниваем VPN виртуального адреса (5) с VPN из TLB (3). Перейдем к проверке наличия страницы в TLB. В таблице TLB хранится информация только для VPN = 3. Наш виртуальный адрес принадлежит VPN = 5, а записи с VPN = 5 в TLB нет. Следовательно, обращение к этому адресу приведёт к Page Fault, и процессору нужно вызвать обработчик исключения для трансляции страницы в память.

Формула вычисления физического адреса (если бы страница была в TLB). Если бы VPN совпадал с записью в TLB, физический адрес вычислялся бы по формуле (5.1).

$$Physical\ Address = PFN \times Page\ Size + Offset Physical\ Address = PFN \times Page\ Size + Offset. \quad (5.1)$$

Например, если бы мы использовали VPN = 3 → PFN = 11 и Offset = 0x142 – формула (5.2).

$$Physical\ Address = 11 \times 0x1000 + 0x142 = 0xB000 + 0x142 = 0xB142. \quad (5.2)$$

Подытожим: виртуальный адрес 0x5142 принадлежит VPN = 5. В TLB есть запись только для VPN = 3. Следовательно, трансляция невозможна без обращения к странице в основной памяти, произойдёт Page Fault. Если бы страница была в TLB, физический адрес вычислялся бы по указанной формуле.

Варианты:

1. VPN = 5 → PFN = 12, размер страницы 4 КБ, виртуальный адрес 0x5143.
2. VPN = 2 → PFN = 8, размер страницы 1 КБ, виртуальный адрес 0x234.
3. VPN = 9 → PFN = 4, размер страницы 2 КБ, виртуальный адрес 0x2501.
4. VPN = 3 → PFN = 15, размер страницы 512 Б, виртуальный адрес 0x6B2.
5. VPN = 12 → PFN = 20, размер страницы 4 КБ, виртуальный адрес 0xC145.
6. VPN = 7 → PFN = 2, размер страницы 2 КБ, виртуальный адрес 0x1D20.
7. VPN = 1 → PFN = 9, размер страницы 1 КБ, виртуальный адрес 0x3F5.
8. VPN = 4 → PFN = 11, размер страницы 8 КБ, виртуальный адрес 0x8421.
9. VPN = 10 → PFN = 5, размер страницы 4 КБ, виртуальный адрес 0xA0F3.
10. VPN = 6 → PFN = 13, размер страницы 512 Б, виртуальный адрес 0x6C9.
11. VPN = 14 → PFN = 30, размер страницы 2 КБ, виртуальный адрес 0xE356.
12. VPN = 0 → PFN = 7, размер страницы 1 КБ, виртуальный адрес 0x0C2.
13. VPN = 8 → PFN = 21, размер страницы 4 КБ, виртуальный адрес 0x8234.
14. VPN = 11 → PFN = 25, размер страницы 8 КБ, виртуальный адрес 0xB345.
15. VPN = 2 → PFN = 6, размер страницы 512 Б, виртуальный адрес 0x2F1.
16. VPN = 13 → PFN = 3, размер страницы 2 КБ, виртуальный адрес 0xD678.
17. VPN = 5 → PFN = 18, размер страницы 1 КБ, виртуальный адрес 0x534.
18. VPN = 9 → PFN = 27, размер страницы 4 КБ, виртуальный адрес 0x9345.

19. VPN = 7 → PFN = 2, размер страницы 2 КБ, виртуальный адрес 0x7B24.
20. VPN = 3 → PFN = 14, размер страницы 512 Б, виртуальный адрес 0x3E8.
21. VPN = 15 → PFN = 5, размер страницы 4 КБ, виртуальный адрес 0xF876.
22. VPN = 1 → PFN = 9, размер страницы 8 КБ, виртуальный адрес 0x1456.
23. VPN = 4 → PFN = 10, размер страницы 2 КБ, виртуальный адрес 0x4C12.
24. VPN = 6 → PFN = 19, размер страницы 1 КБ, виртуальный адрес 0x672.
25. VPN = 12 → PFN = 31, размер страницы 512 Б, виртуальный адрес 0xCFA.
26. VPN = 10 → PFN = 22, размер страницы 4 КБ, виртуальный адрес 0xA2C1.
27. VPN = 2 → PFN = 15, размер страницы 2 КБ, виртуальный адрес 0x2580.
28. VPN = 8 → PFN = 5, размер страницы 8 КБ, виртуальный адрес 0x8F23.
29. VPN = 11 → PFN = 17, размер страницы 1 КБ, виртуальный адрес 0xB45.
30. VPN = 9 → PFN = 28, размер страницы 4 КБ, виртуальный адрес 0x9E31.
31. VPN = 0 → PFN = 6, размер страницы 512 Б, виртуальный адрес 0x03C.
32. VPN = 14 → PFN = 20, размер страницы 2 КБ, виртуальный адрес 0xE204.

6. ВНЕШНЯЯ ПАМЯТЬ

6.1. Введение

После рассмотрения основных компонентов, непосредственно влияющих на производительность персонального компьютера, и плавно двигаясь по иерархии памяти в сторону увеличения ее объема, необходимо рассмотреть следующую ступень – внешнюю память. В данной работе студентам предстоит познакомиться с устройством и функционированием основных внешних запоминающих устройств, которые применялись или применяются в современных компьютерах. В данной работе будет представлен разбор основных характеристик внешних запоминающих устройств, а также технологии S.M.A.R.T.

6.2. Развитие внешних запоминающих устройств

Внешние запоминающие устройства прошли долгий путь эволюции, меняясь, ускоряясь, увеличивая свой объем, надежность и ресурс. Например, еще 20 лет назад никто и не думал об облачных хранилищах твердотельных накопителях со скоростью чтения в 7000 МБ/с, а сейчас это обыденность, которой пользуются все большее число владельцев ПК. Коротко рассмотрим эволюцию накопителей их характеристики, преимущества и недостатки.

Перфокарты. Первыми носителями информации, удобными для хранения и использования стали перфокарты. Перфокарта – это старинный способ хранения и кодирования информации, уходящий корнями в 1800-е года, когда Жозеф Мари Жаккар применил сменные перфокарты в своем ткацком станке (рис. 6.1). Задача таких перфокарт заключалась в сохранении узора, а также для его усложнения, так как некоторые узоры были крайне трудны при ручном вышивании.



Рисунок 6.1. Сменные перфокарты

Следующим этапом развития и применения перфокарт стала аналитическая машина Чарльза Бэббиджа (рис. 6.2). Основными частями Аналитической машины являлись: «склад» – устройство для хранения чисел, то есть память в современной терминологии; «мельница» – устройства для выполнения арифметических действий (Арифметическое устройство); устройство, управляющее операциями машины; устройства ввода и вывода. В этой архитектуре можно увидеть прообраз современного компьютера с памятью, процессором (мельница + устройство управления) и устройствами ввода вывода. «Шину обмена» данными между АЛУ и памятью представлял собой набор зубчатых реек. Объем оперативной памяти должен был составлять тысячу чисел, около 50 десятичных знаков. Для числа из 50 десятичных разрядов со знаком необходимо 168 бит, то есть объём ОЗУ был чуть больше 20 килобайт. На этой машине предполагались все основные математические операции, такие как сложение, вычитание, умножение и деление, особенности реализации которых рассматривать не стоит. Перфокарты использовались в этой машине для кодирования операции и для осуществления переноса информации между «мельницей» и «складом».

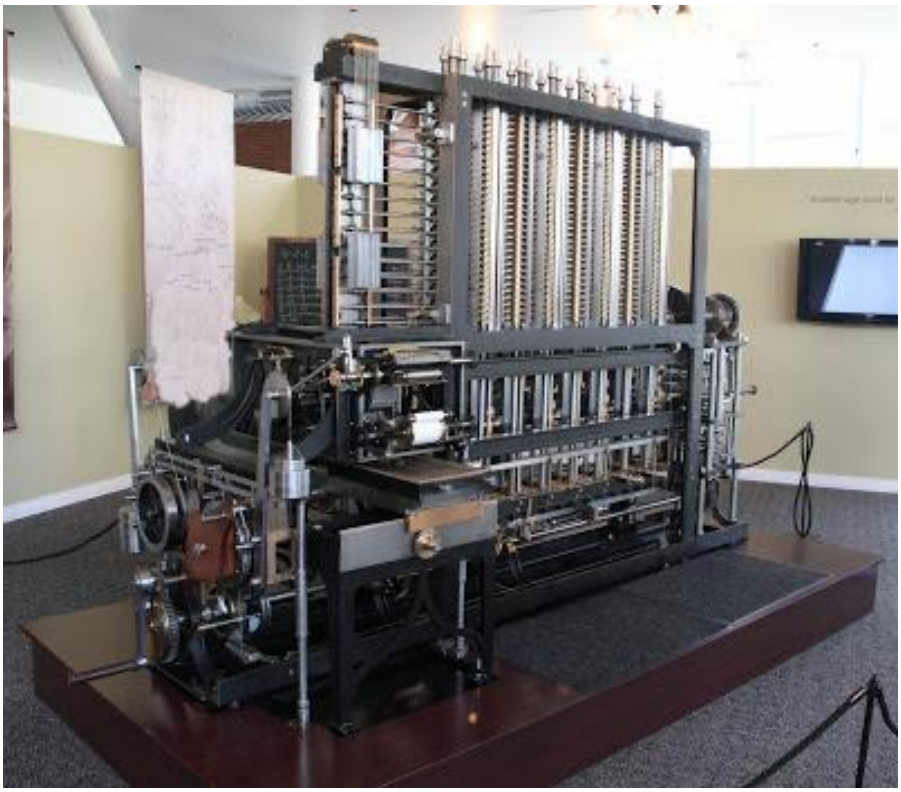


Рисунок 6.2. Машина Чарльза Бебиджа

Значительный вклад во внедрение перфокарт в компьютеры, на тот момент уже электронные внесла компания IBM, которая разработала для них

собственный стандарт IBM-80. Физически такой носитель представлял собой картонную карточку заданного размера с набором прорезей, который использовался для кодирования данных в двоичном виде (рис. 6.3). Однако уже тогда разметка была сложной, так как на карточке, помимо самой информации (числа или чисел), содержалось большое количество служебной информации. Классическая перфокарта могла хранить не более 80 байт данных, поэтому достаточно быстро пришла потребность в переходе на более емкие накопители.

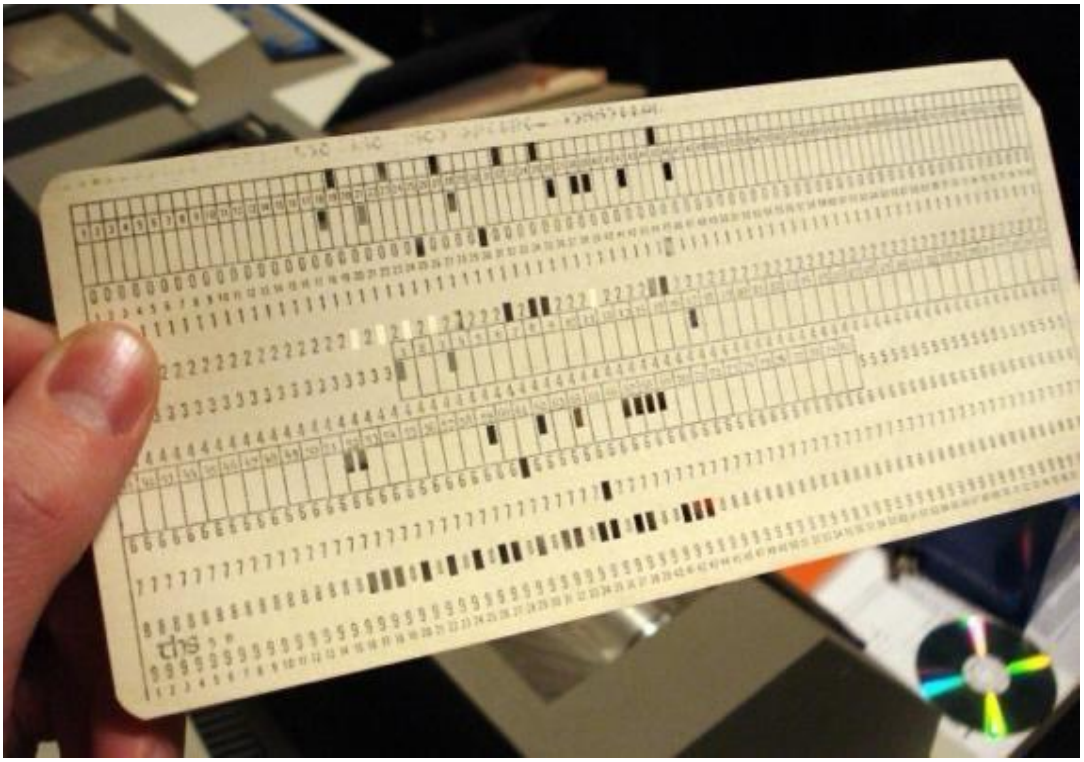


Рисунок 6.3. Накопитель стандарта IBM-80

Магнитные ленты. Более емким накопителем, а по совместительству и следующим этапом развития внешних накопителей стали магнитные ленты – рис. 6.4 (НМЛ) или стримеры. Первые упоминания НМЛ имеются в 1928 году. Изобретены они были в Германии, использовали бумажную основу и тонкий слой оксида железа, примерно в это же время была показана и машина для записи информации на магнитную ленту. Однако использовать их стали не сразу, так как возможности перфокарт полностью покрывали все необходимости в хранении информации, массовое внедрение произошло только в 1951 году, с изобретением ЭВМ UNIVAC-1. На магнитной ленте, длиной в полкилометра (450м) и весом около 11 кг, можно было сохранить порядка 1 МБ данных, что равнялось 10 тысячам перфокарт.



Рисунок 6.4. Магнитные ленты

Принцип работы НМЛ довольно прост, и, по сути, заложил основу для большинства накопителей в будущем. Для записи магнитная лента проходит через записывающую головку, которая создает магнитное поле. Под воздействием этого поля участки магнитного слоя на ленте намагничиваются в определенном направлении. Это направление – север-юг или юг-север – соответствует двоичным данным, то есть нулям и единицам. Таким образом, информация кодируется в виде намагниченных областей, разделенных на дорожки. Количество дорожек и ширина ленты зависят от того, какая информация должна быть занесена на ленту, и какой объем должен быть.

Например, в звукозаписи как правило использовали ленты шириной 3,81 и 6,25 мм и толщиной 9, 12, 18, 27,37 и 55 мкм (кассетные и катушечные бытовые магнитофоны, студийные магнитофоны). Видеозапись осуществлялась на ленты шириной 50,8 и 25,4 мм и толщиной 37 мкм (студийные видеомагнитофоны), 6,25 и 12,7 мм при толщине 37 мкм (бытовые видеомагнитофоны). В запоминающих устройствах применяли магнитные ленты шириной 12,7 мм и толщиной 37 мкм.

Также для записи, контроллер, должен получить от процессора полный блок записываемой информации, вся необходимая служебная информация добавляется также контроллером по мере необходимости. Поскольку запись блока остановлена быть не может, вся информация от процессора должна быть получена до начала выполнения операции. Чтение информации, происходит по схожему принципу, только идет считывание уже имеющихся полей на ленте, преобразование в двоичные данные и в сигналы. Перезапись,

аналогична команде записи, но с рядом ограничений и особенностей. Во-первых, НМЛ не контролирует и не гарантирует, что перезаписываемый блок имеет ту же длину, что и ранее записанный, за это отвечают программы и программист, что требует внимательности и аккуратности со стороны второго. Во-вторых, накопитель может, но не обязан, скорректировать в сторону уменьшения времена старта и стопа, чтобы не испортить последующий блок. Самый пик своего применения магнитные ленты нашли в 1960–1990 годы. Использовались они в кассетах для хранения музыки (компакт-кассеты) и кино (VHS). Несмотря на свою привлекательность (простота, долговечность и дешевизна хранения информации), магнитные ленты были не сильно устойчивы к внешним воздействиям, что осложняло их хранение. И по сей день накопители на магнитных лентах все еще применяются в компьютерной технике, в основном для хранения архивных данных. Причиной тому является гораздо больший объем информации, который они могут хранить, по сравнению с магнитными дисками или твердотельными накопителями. В 2020 году компаниям Fujifilm и IBM, смогли создать магнитную ленту емкостью 580 ТБ, при поддержке шифрования и сжатия информации.

Дискета. Следующий шаг в развитии внешних запоминающих устройств – это дискеты (Floppy disc), появившиеся в 1971 году (рис. 6.5). Дискета стала первым устройством для хранения данных, доступным для широкой аудитории. До изобретения и массового распространения оптических дисков, флоппи был стандартным устройством для хранения и распространения и данных, и ПО на уже персональных компьютерах.



Рисунок 6.5. Дискеты

Дискета – накопитель на гибком магнитном диске или Floppy. Дискета состоит из круглой полимерной подложки, покрытой с обеих сторон магнитным окислом и помещенной в пластиковый каркас, на внутренней поверхности этого каркаса имеется слой мягкого фетро–подобного (на основе войлока) материала, помогающего в защите дискеты. Он служит как для смягчения ударов, так и для улавливания пыли. Дискеты выпускались в основном двух размеров: диаметром 5.25 дюйма (около 13 см) и 3.5 дюйма (около 9 см). На ранних этапах еще встречались 8-дюймовые дискеты (около 20 см). В каркасе имеется три отверстия, каждое из которых имеет свое собственное назначение:

- Первое отверстие (в центре дискеты) предназначено для захвата диска приводом дисковода. Через это отверстие механизм дисковода захватывает гибкий диск, чтобы привести его во вращение.
- Второе отверстие представляет собой продолговатую прорезь, закрытую металлической шторкой, через которую осуществляется доступ к дискете головок чтения/записи. Через эту прорезь происходит процесс чтения и записи информации.
- Третье отверстие – квадратная прорезь в углу корпуса дискеты – служит в качестве признака защиты от записи. Практически все магнитные носители используют те или иные признаки защиты от записи.

При установке в накопитель дискета автоматически в нем фиксируется, после чего механизм накопителя раскручивается до частоты вращения 360 об/мин, при всем при этом в накопителе вращается сама дискета, а не диск внутри каркаса, магнитные головки также остаются неподвижными, стоит также отметить, что дискета вращается только при обращении к ней.

Принцип хранения информации на дискете, а также ее записи и чтения, стала неким стандартом для целого ряда последующих накопителей, принцип работы которых разберем на примере именно дискеты. Основная идея похожа на магнитные ленты, за исключением того, что в угоду компактности и удобства, форма непосредственно носителя стала дискообразной. Поверхность диска, как и поверхность ленты, покрывается окисью металла, что позволяет создавать на ней заряды. После чего все плоскость делится на дорожки, которые в свою очередь делятся на сектора (рис. 6.6).

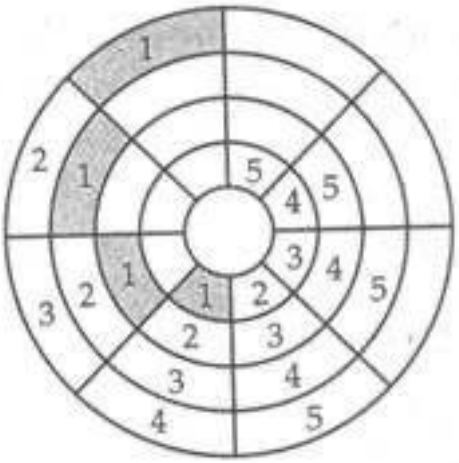


Рисунок 6.6. Дорожки дискет

С физической точки зрения обе поверхности всех магнитных дисков содержат дорожки. В дискетах, например, число дорожек равно 80, а число дорожек того же жесткого диска достигает нескольких тысяч. Дорожки, как и головки, идентифицируются собственным номером (внешняя дорожка и верхняя головка, в случае с жестким диском, имеет нулевой номер). Количество дорожек на диске определяется поверхностной плотностью записи. Для работы с дискетой достаточно иметь только одну магнитную головку, так как магнитный диск в ней только один. Хотя каждой рабочей поверхности соответствует своя головка, по которым нумеруются поверхности. Физически максимально допустимое число головок за всю историю производства магнитных накопителей было равно 11, но в современных накопителях более 6 головок не используется.

Дорожки, в свою очередь, разбиваются на сектора, являющие минимальными физическими элементами хранения и адресации данных. Чаще всего, сектора на каждой дорожке имеют фиксированный угловой размер, благодаря чему на всех дорожках располагается одинаковое количество секторов. Для стандартной дискеты 3,5”, каждая дорожка содержит 18 секторов. Жесткий диск, напротив, имеет обычно от 17 до 63 секторов. Реально же на дорожке современного накопителя содержится около 100 секторов, а максимальное их количество равно 256. Размер сектора определен в 512 байт, а их нумерация на дорожках начинается с единицы, а не с нуля, как это обычно бывает. Объем дискет как правило составлял 720 КБ, 1440 КБ, 2880 КБ.

В каждом секторе имеются не только данные, но и некоторая служебная информация. В начале каждого сектора записывается его заголовок (prefix), по

которому определяется начало и номер сектора, а в конце – заключение (suffix), в котором находится контрольная сумма (checksum, CRC), необходимая для проверки целостности данных. Заголовок сектора включает в себя идентификатор (ID) сектора, первую CRC (контрольная сумма) и интервал включения записи. Идентификатор содержит информацию о номере цилиндра, головки и сектора. Далее следует интервал включения записи, после которого следует 512 байт данных. За данными располагается вторая CRC и интервал между записями (секторами), необходимый для того, чтобы застраховать следующий сектор от записи на предыдущий. Это может произойти из-за неравномерной скорости вращения диска. Завершает сектор предындексный интервал, который имеет размер от 693 байт, служит для компенсации неравномерности скорости вращения диска. Таким образом, размер сектора увеличивается до 571 байта, из которых 512 байт составляют данные.

Вся эта информация записывается на заводе при низкоуровневом форматировании, используя специальные программные средства или команды DOS. Кроме промежутков между секторами существуют еще и промежутки между самими дорожками. Префиксы, суффиксы и промежутки как раз и составляют то пространство диска, которое теряется при форматировании.

Оптические диски (CD/DVD/Blu-Ray). Следующий этап развития методов хранения информации – оптические диски (рис. 6.7).



Рисунок 6.7. Оптические диски

По своему принципу действия это продолжатели магнитных дисков, только в более тонком и удобном формате (пластиковая основа с зеркальным напылением, рис. 6.8), и в основе схожий с магнитными дисками принцип работы, только уже основанный на оптических технологиях. Хотя следует

отметить, что изначально компакт-диски не применялись для хранения информации, файлов и ПО, а являлись неким логическим продолжением виниловых пластинок.

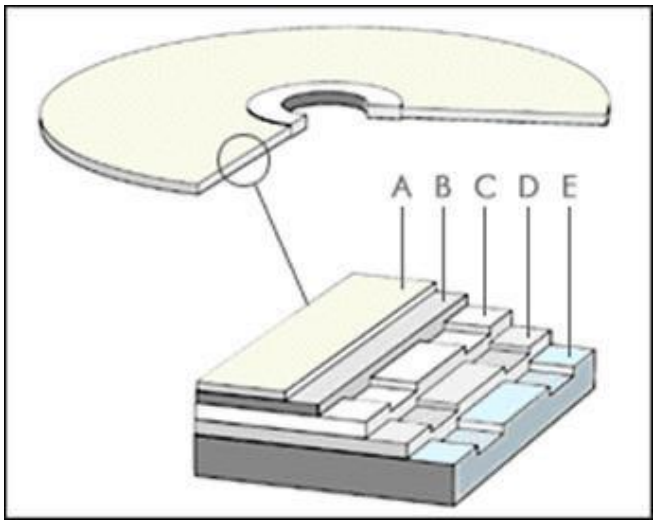


Рисунок 6.8. Дорожки оптических дисков

Основа большинства CD – поликарбонат (на рисунке отмечено как E), на которую наносятся все остальные слои, он же придает необходимую прочность и форму всей конструкции. Следующим наносится активный слой (D) (dye), который позволяет осуществлять запись на диск и отвечает за его надежность, и качество считывания информации в дальнейшем (в простейших CD-ROM дисках данный слой отсутствует, так как запись нужной информации происходит непосредственно на предприятии-изготовителе). После того, как на поликарбонатовую заготовку был нанесен dye, диск покрывается специальным слоем светоотражающего материала (C). Для CD-ROM применяется алюминий, в CD-R дисках же применяется чистое серебро, позволяющее добиться 65–80%-го коэффициента отражения. Завершающим этапом изготовления диска является нанесение защитного слоя (B), на который в дальнейшем возможно нанесение изображений, как правило отражающих содержимое диска (A). Наиболее распространенным и простым в изготовлении защитным слоем является специальный лак. Лакировка диска не дает большой гарантии сохранности данных, однако защищает от легких механических и химических воздействий.

Виды оптических дисков и их предназначение:

- CD-Audio – первый вид, задача которого хранить аудиозаписи в цифровом формате.

- CD-ROM – первый компакт-диск задачей которого стало хранение любых файлов в цифровом виде. ROM – Read Only Memory – только для чтения, такой диск использовался для распространения различного ПО.
- CD-R – Recordable – компакт-диск с возможностью одноразовой записи на него информации.
- CD-RW – ReWriteable – компакт-диск с возможностью записи и перезаписи информации на нем. Бесконечно перезаписывать диск не получится, так как поддерживается около 1000 циклов перезаписи.

Возможностей по записи и перезаписи информации удалось достичь путем добавления промежуточного слоя между отражающим слоем и пластиковой основой, который может изменять свои свойства под действием лазерного излучения определенной длины волны, а на поликарбонате печатаются пустые дорожки. Существуют и другие виды компакт-дисков для специальных задач, например: CD-Extra, CD-ROM XA, CD-G, CD-Text и др., однако сильно массового применения они не нашли.

Следующей ступенью эволюции уже компакт-дисков стали DVD и, впоследствии Blu-Ray (BD). По своей сути это такие же компакт диски, но с существенно большей емкостью, до 17 ГБ в случае с DVD и до 100 ГБ, в случае с Blu-Ray диском, у CD максимальная емкость достигала лишь 900 МБ. Достичь подобного стало возможно благодаря возросшей плотности записи и появления слоев, до двух на одной стороне в случае с DVD и до 4 слоев в случае с BD. Еще одним аспектом, который стал лучше – скорость считывания и записи, за счет применения лучшей оптической системы. Также возросла устойчивость к механическим повреждениям.

DVD, равно как и BD диски имеют различные виды и подвиды, такие как DVD-R, DVD-RW, DVD-ROM, HD-DVD, BD-R, BD-RE и другие. И в первом, и во втором случаях, такие носители преимущественно использовались и используются для хранения видеозаписей и кино, видеоигры. На основании этого однозначной расшифровки аббревиатуры DVD до сих пор нет. В некоторых источниках указывается Digital Video Disk – цифровой видео диск, в других Digital Versatile Disk – цифровой многофункциональный диск.

Принцип функционирования всех вышеописанных дисков неизменен до сих пор. Как уже было сказано принцип сильно схож с принципом работы магнитных накопителей, диск, установленный в дисковод, раскручивается и на него подается лазерный луч, определенный длины (Рисунок 6.9). Луч движется по концентрическим дорожкам, которые идут по спирали, от центра

к краю, в которых имеются микроскопические углубления, называемые «питы» или ямы, которые отвечают за логические «0» и «1» (Рисунок 6.10) для того, чтобы контроллер (декодер) через фотодатчик мог отличить эти «питы», делались физические прорези в поликарбонате, им соответствующие. В случае с CD размеры 1 минимального углубления составляют 330 нм в ширину и 680 нм в длину, при этом расстояние между дорожками ~930 нм. Достижение больших объемов, стало возможно путем уменьшения длины луча, увеличением плотности нанесения «питов», их непосредственного размера, и добавления слоев. В случае с перезаписываемыми дисками, как уже было сказано, добавленный слой позволял менять записанную информацию и записывать новую, однако это повысило необходимость в защите диска от внешнего воздействия, особенно от прямого света, так как лучи могут испортить записанную информацию на носителе.

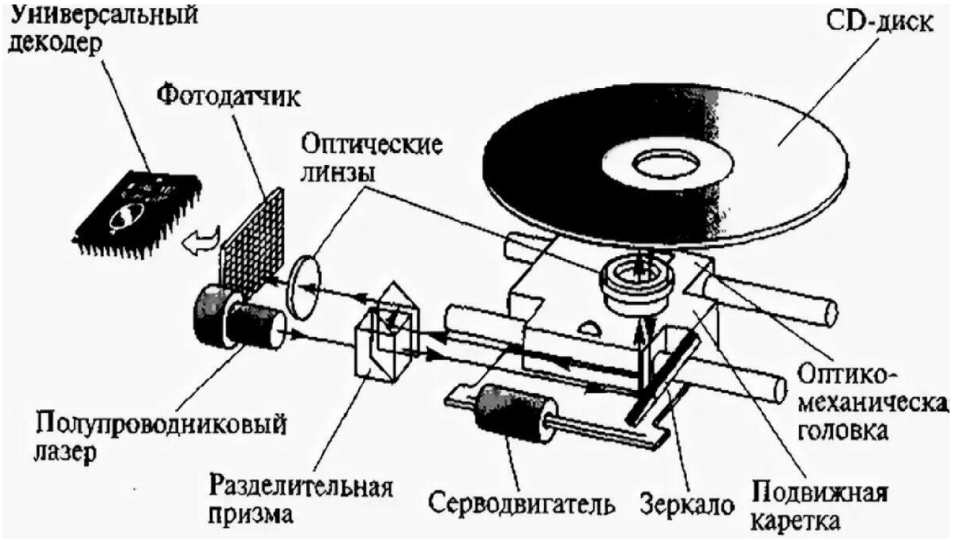


Рисунок 6.9. Конструкция для считывания информации с оптического диска

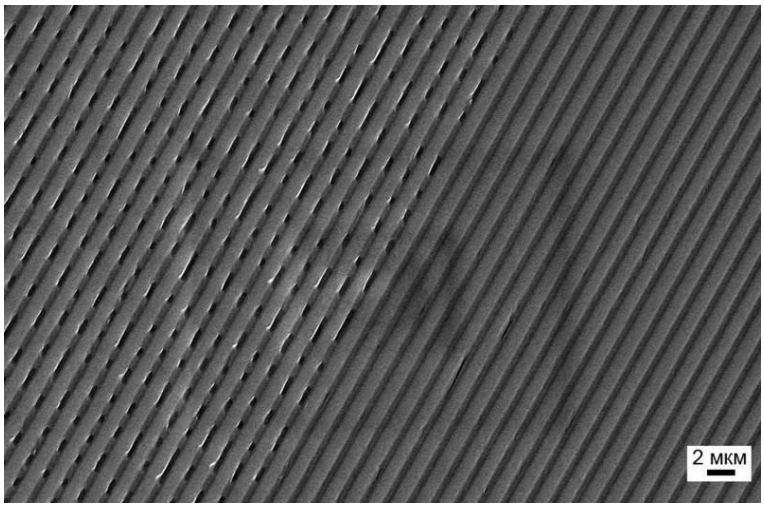


Рисунок 6.10. Дорожки на оптическом диске

6.3. HDD (Hard Disk Drive)

Продолжателями магнитных дисков, а по совокупности пиком их развития на данный момент стали накопители на жестких магнитных дисках (Hard Disc Drive – HDD). Во-первых, вся механическая часть для работы с непосредственно диском (контроллер, привод, сам диск, головка чтения/записи), переехала в единый неделимый корпус (рис. 6.11), называемым гермоблоком, что позволило еще больше сэкономить на физическом пространстве. Несмотря на свое название, корпус не герметичен на 100%, назван он так в силу того, что сами диски изолированы от внешнего вредоносного воздействия пользователя, хотя в самых дорогих и профессиональных версиях HDD, действительно может быть организован вакуум, для продления срока службы и уменьшения силы трения при работе.



Рисунок 6.11. Пример HDD накопителя

Во-вторых, вместо гибкого полимерного диска, установлен твердый диск из металлических сплавов или стекла, также покрытый ферромагнитным слоем, для сохранения информации. В-третьих, благодаря отказу от пластикового ненадежного корпуса, и переходу к металлическому, устройство имеет существенно большую устойчивость к механическим повреждениям. В-четвертых, такой корпус, позволил устанавливать сразу несколько магнитных дисков и несколько головок чтения/записи одновременно на один шпиндель, что существенно увеличило объем хранимой информации. Сектора,

находящиеся друг над другом в пакете дисков, на которые одновременно может быть спозиционирован блок головок, называющийся цилиндром. В связи с тем, что накопитель имеет несколько дисков, расположенных друг под другом, то их разбиения на сектора идентичны.

Первые жесткие диски появились лишь немного позже, чем НМЛ, но были также неудобны в применении и велики в размерах (первый жесткий диск Ramac I весил около тонны, при возможности хранения лишь 3.5 МБ). К классическому своему виду жесткие диски пришли только в 1983 году, когда появился стандарт 3.5”, его емкость тогда составляла 10 МБ. Чуть позже, в 1991 году появляется новый формат 2,5”, который в дальнейшим будет применяться в портативных устройствах, например, в ноутбуках. Примечательно, что размер непосредственно магнитного диска остался прежним, дальнейшее развитие пошло по экспоненте, к 1995 году объем HDD составлял уже 2 ГБ, к 1997 году целых 10 ГБ, а в 2000 году появились первые HDD на 40 – 80 ГБ. В настоящее время емкость одного жесткого диска может составлять 36 ТБ, в нем установлен цилиндр на 10 дисков по 3ТБ каждый.

Принцип работы был разобран ранее на примере флоппи, стоит лишь отметить ряд основных отличий и улучшений. Первое – это увеличение размера одного сектора, если у флоппи и ранних HDD это было 512 Байт, то у современных накопителей это 4 КБ (4096 Байт). Вторым важным изменением стало усложнение контроллера, так как работать теперь приходится с несколькими дисками в одном корпусе, а не с одним. Далее увеличились скорости вращения, если для флоппи это было 360 об/мин, то сейчас это значение доходит до 7200 об/мин. Увеличение скорости вращения, и применение более быстрых интерфейсов позволило увеличить скорости чтения и записи. Также изменились методы записи информации. Например, на ранних этапах это была продольная магнитная запись (LMR – Longitudinal Magnetic Recording), где магнитная головка намагничивает домены (ячейки способные хранить заряд) продольно оси вращения диска. Далее появилась перпендикулярная магнитная запись (CMR – Conventional Magnetic Recording), где домены располагались перпендикулярно оси вращения диска, что позволило существенно увеличить плотность записи. Следующим вариантом магнитной записи стал черепичный метод записи (SMR – Shingled Magnetic Recording), в данном случае поверхность записи больше, чем поверхность считывания, а дорожки, частично перекрывают друг друга, подобно черепичной кровле.

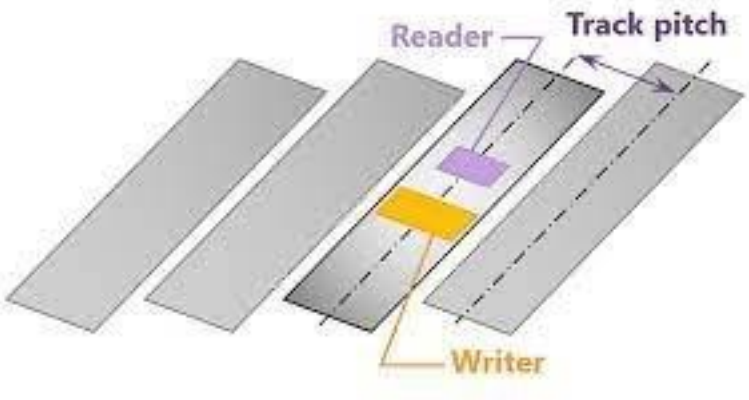


Рисунок 6.12. Чтение и запись на HDD

Среди перспективных технологий, существует тепловая магнитная запись (HAMR – Heat-Assisted Magnetic Recording), ее принцип заключается в точечном нагреве областей диска, проход магнитной головки намагничивает эти точечные нагретые области что при охлаждении позволяет закрепить намагниченность в определенном значении, в 2023 году первые такие образцы были представлены на коммерческом рынке компанией Seagate, емкость таких накопителей составила около 30 ТБ. Далее была разработана микроволновая магнитная запись (MAMR – Microwave-Assisted Magnetic Recording) – технология крайне сложная и основана на изменении магнитного момента в момент записи, что позволяет уменьшить зону намагничивания без опасения возникновения суперпарамагнитного эффекта (когда намагниченным становится целиком весь домен, а не отдельная его часть). Последним на данный момент существующим методом записи информации, который также находится на стадии прототипирования – это BPM (Bit-Patterned Media) – это метод записи, который использует в качестве головки массив магнитных ячеек, каждый из которых соответствует 1 биту информации. По предварительным предположениям плотность записи должна составить около 2.5–3 Терабит на 1 квадратный дюйм.

Магнитные диски – это долговечные и надежные накопители, способные проработать несколько лет, при этом стоимость хранения 1 ГБ информации, по сравнению, например, со следующим видом накопителей гораздо меньше. Также немаловажно, чтобы магнитные диски были постоянно в работе, так как, при постоянной работе они обновляют свой магнитный слой, что позволяет им дольше работать и надежнее хранить данные, при хранении на полке срок жизни жесткого диска может сильно сократиться.

Переходя к долголетию хранения данных на магнитных дисках, то важной частью современных накопителей (не только магнитных) является технология S.M.A.R.T., которая отслеживает жизненные показатели накопителя и позволяет вовремя предупредить его возможную внезапную кончину. Расшифровывается эта аббревиатура как Self-Monitoring, Analysis and Reporting Technology – Технология самоконтроля, анализа и отслеживания параметров, ее первые наработки были представлены еще 1995 году. От момента производства накопитель имеет о себе некоторую информацию, которая хранится в его служебной зоне, доступ к которой возможен только с помощью специализированного ПО поставляемого разработчиком конкретного жесткого диска. Параметры в этой зоне с течением времени меняются, отражая текущее состояние жесткого диска, проходя путь от наилучших к наихудшим. В S.M.A.R.T. предусмотрены индикаторы и параметры (атрибуты), для начала определимся с индикаторами:

- ID – идентификатор, или номер атрибута в системе;
- Описание – словесное описание атрибута, за что он отвечает в устройстве;
- Value – текущее значение атрибута;
- Worst – худший параметр за время эксплуатации диска;
- Threshold – пороговое значение атрибута, при достижении которой накопитель будет сигнализировать о скорой кончине;
- RAW – значение атрибута, как правило в 16-ной системе, не несущее явной информации для пользователя;
- Статус – словесная оценка состояния атрибута.

Атрибуты – непосредственно характеристики жесткого диска, влияющие на его работу и долговечность.

- Raw Read Error Rate – количество ошибок диска при чтении, возникших из-за аппаратных неполадок, меньше значение параметра – лучше состояние диска;
- Spin-Up Time – время, необходимое для раскрутки шпинделя, чем меньше, тем лучше;
- Start/Stop Count – количество полных остановок и раскруток шпинделя;
- Reallocated Sectors count – количество переназначенных секторов, отвечает за износ магнитного слоя диска, чем больше этот параметр, тем хуже;
- Seek Error Rate – количество ошибок позиционирования головок, чем выше значение, тем хуже;

- Power-On Time Count – общее время наработки накопителя.

Естественно, это не все параметры, а только основные из них, современные накопители хранят о себе гораздо больше информации, однако основные атрибуты остаются неизменными, так как они имеют больше всего влияния на диск. Получить доступ к ним и посмотреть состояние диска, можно с помощью специальных приложений, следящих за «здоровьем» диска, одной из таких программ является CrystalDiskInfo (рис. 6.13).

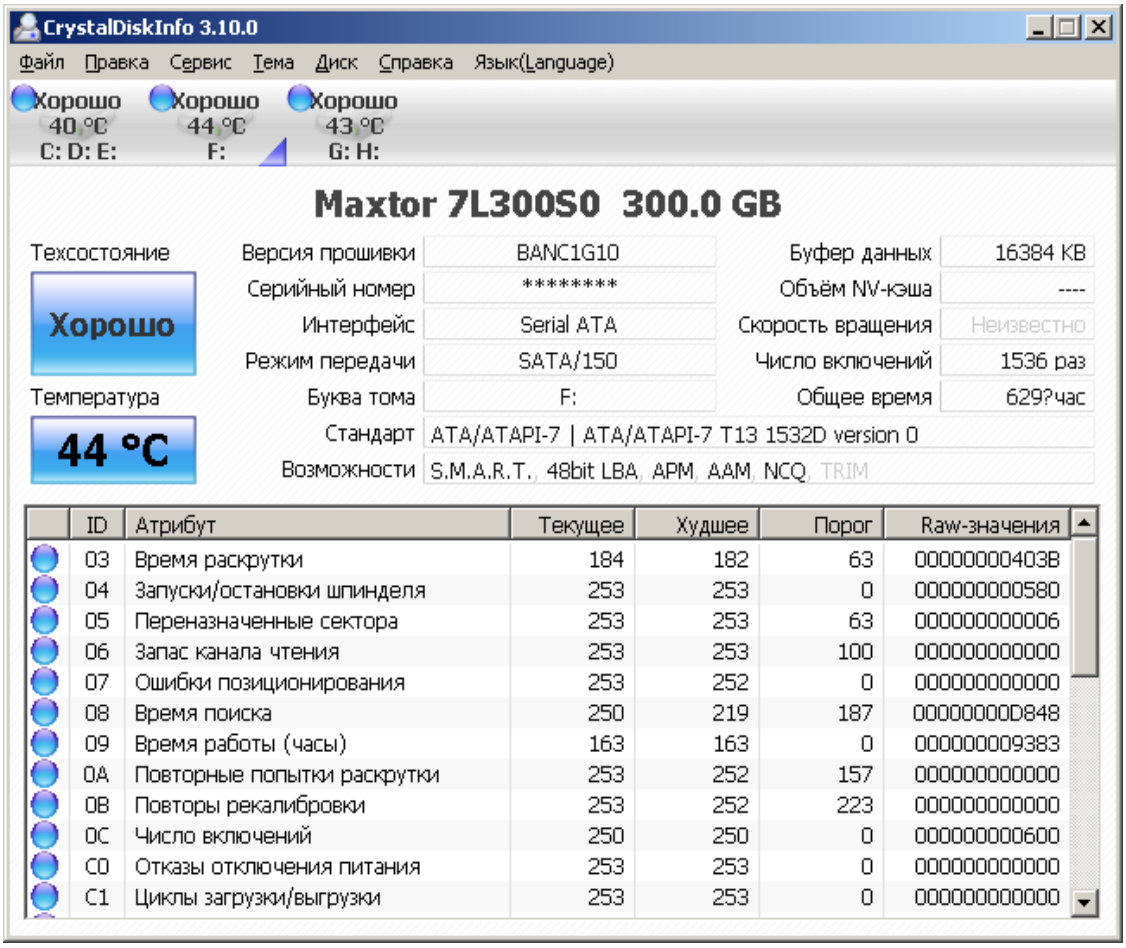


Рисунок 6.13. Интерфейс CrystalDiskInfo

RAID-массивы. Еще одна новая функция, недоступная ранее ни лентам, ни флоппи, ни, тем более, оптическим накопителем – это объединение нескольких жестких дисков (HDD) в одно единое хранилище, называемое RAID-массивом. Коротко разберемся в его назначении, видах и устройстве. RAID-массив расшифровывается как: Redundant Array of Independent Disks – избыточный массив независимых дисков – то есть объединение двух и более накопителей в единый логический элемент, с целью повышения его надежности. Область применения RAID-массивов – это крупные базы и хранилища данных, где сохранность информации гораздо важнее скорости

доступа к ней. Степень защищенности данных зависит от способа организации самого массива. Всего существуют 7 видов RAID-массивов (0-6 уроовни, и комбинированне 10, 50, 60), среди которых широко используются около 2-х.

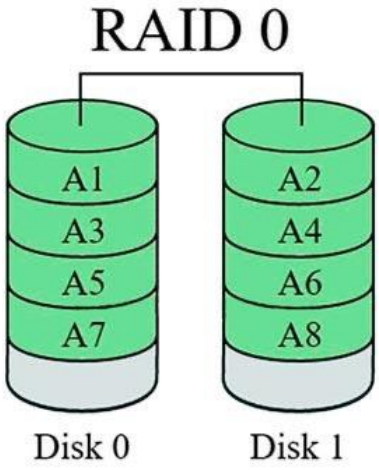


Рисунок 6.14. Структура RAID-0

RAID-0 – Массив без отказоустойчивости (рис. 6.14) – первый и самый простой способ организации. Данный вид массива предполагает простое объединение двух и более дисков в один. Такая организация еще называется «чередованием», так как при записи информации она разбивается на части (блоки) фиксированного размера и записывается поочередно на все собранные в массив диски. На практике такой вид организации массива используется очень редко. Среди достоинств можно выделить среднее быстродействие и легкость реализации. Из недостатков – невозможность реализации отказоустойчивости, следовательно, повышенный риск потери всей информации.

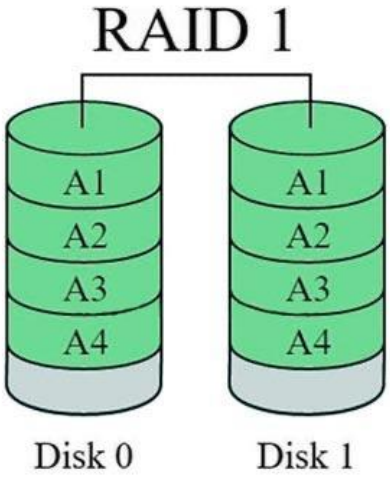


Рисунок 6.15. Структура RAID-1

RAID-1 – Массив с зеркалированием (рис. 6.15) – эта организация создана объединением двух и более дисков в первую очередь для обеспечения отказоустойчивости путем дублирования информации одновременно на два диска. Главным достоинством является гарантия сохранения данных в результате отзеркалирования на второй диск, а также незначительное увеличение быстродействия (за счет параллельности). При этом часть диска отдается под служебную информацию, что снижает доступный объем дисковой памяти. Еще одним важным моментом, является то, что для системы видимым остается только один диск, второй (зеркало) остается доступным только контроллеру. При условии смерти одного из дисков, требуется оперативная его замена и восстановление данных.

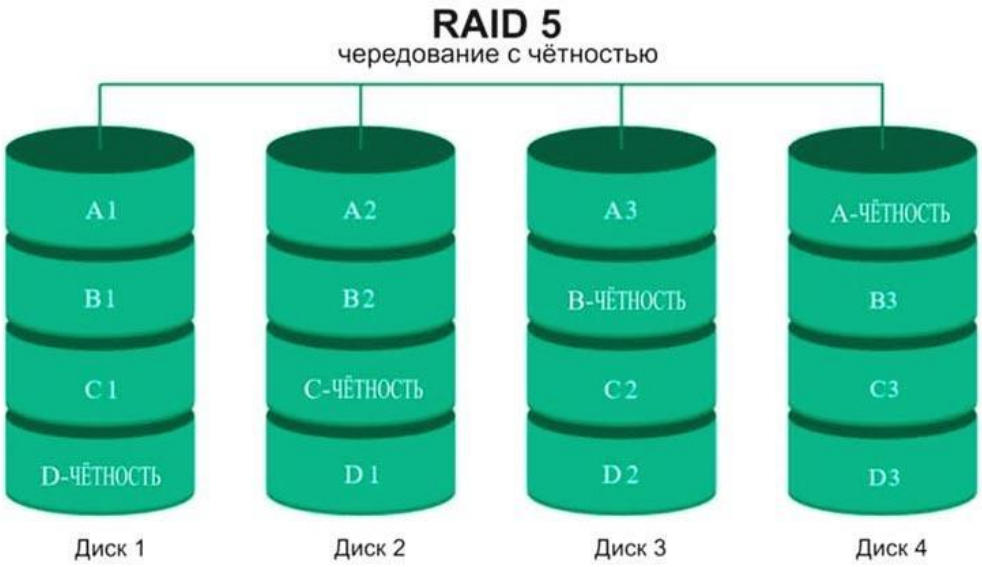


Рисунок 6.16. Структура RAID-5

RAID-5 – Отказоустойчивый массив с распределенной четностью (рис. 6.16). Здесь стоит упомянуть, что между RAID-1 и RAID-5 существуют еще RAID-2/3/4, которые на практике особо не применяются. В RAID-2 используются коды коррекции ошибок Хемминга, минимум дисков для организации такого массива – 7, причем большая часть из них будет отдана под коды Хемминга, так как их хранение занимает больше памяти чем хранение собственно информации. Преимуществом является возможность быстрого обнаружения и коррекции ошибок, недостатком – его неэкономичность, особенно для небольших хранилищ данных, в результате чего его вытеснили более старшие уровни массивов. RAID-3/4 – RAID-2, только без кодов Хемминга, за место них используются контрольные суммы,

а сам файл дробится на мелкие части, которые записываются в сами диски, один из дисков выделяется под хранение этих самых контрольных сумм, для такой организации необходимый минимум – 3 диска. Данные варианты также не используются, так как существует RAID-5, который и вытеснил предыдущие варианты. RAID-5 по совместительству еще и самый распространенный вариант организации, он объединяет преимущества своих предшественников, и делает их лучше и эффективнее. Файл дробится на блоки равного размера и высчитываются контрольные суммы, которые следят за корректностью хранимой информации, только хранятся контрольные суммы не на одном носителе, а равномерно распределяются по всем дискам, каждый из которых хранит свою часть. Еще одним преимуществом этой реализации является возможность создания ее сугубо на аппаратной стороне, равно как и программной. Существенным недостатком такой реализации является возможность внезапного ухода из жизни одного из накопителей, в этом случае нагрузка на остальные накопители существенно возрастает, что, в свою очередь, подвергает опасности остальные накопители и увеличивает вероятность выхода из строя еще одного или нескольких накопителей. Для предупреждения таких ситуаций используются технологии S.M.A.R.T. и использовании Hot Spare. Hot Spare – технология горячего резервирования, когда при смерти одного из дисков автоматически подхватывает резервный диск, и начинает процесс восстановления.

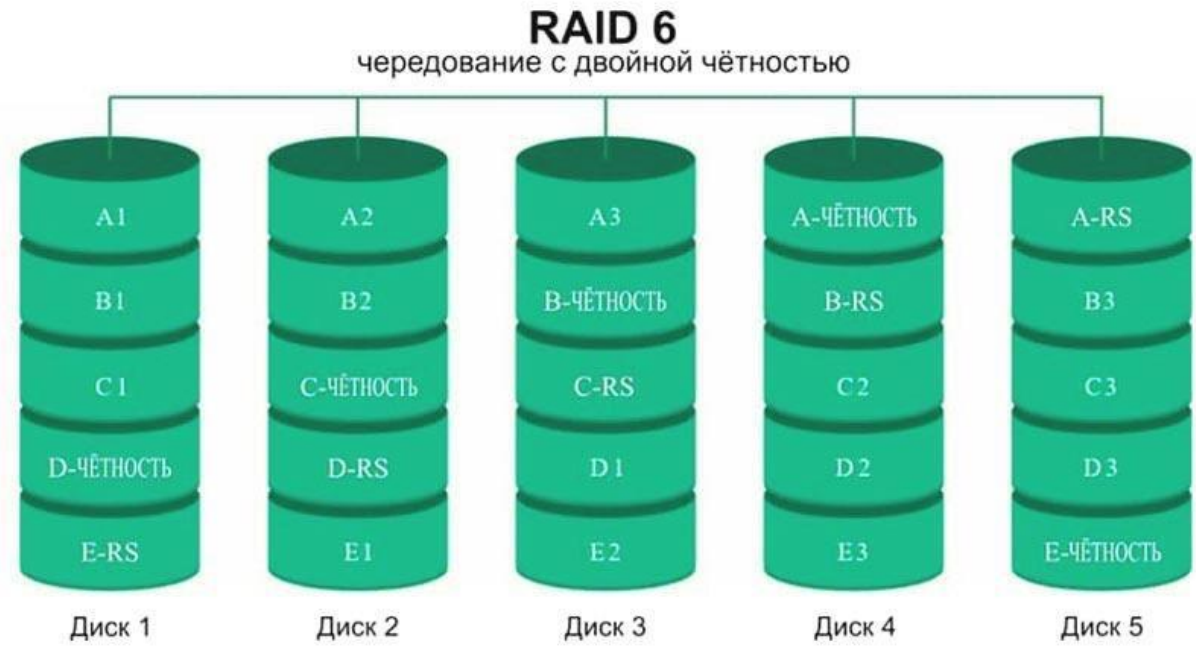


Рисунок 6.17. Структура RAID-6

RAID-6 – отказоустойчивый массив с двойной распределенной четностью (рис. 6.17). Улучшенная версия RAID-5, где контрольные суммы дублируются, что еще больше повышает надежность и немного уменьшает общий объем, при этом общее количество дисков также немного увеличивается.

Для дальнейших улучшений в надежности и долговечности данных стали применять комбинированные уровни RAID, которые объединяют преимущества друг друга.

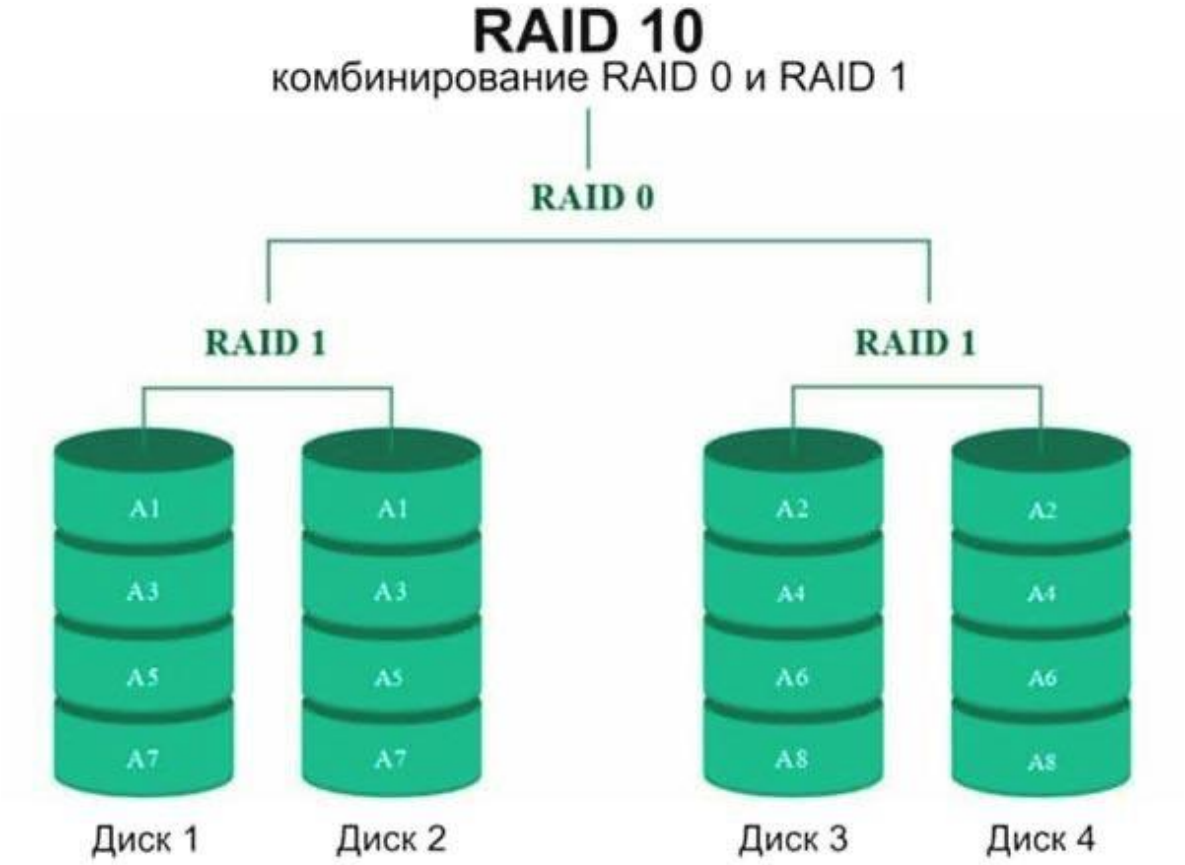


Рисунок 6.18. Структура RAID-10

RAID-10 – вариант, при котором в массив типа RAID-0 объединяются не на просто диски, а массивы RAID-1 (рис. 6.18). Такой способ организации хоть и надежен, но крайне дорог в реализации, да и с экономностью и объемами имеются проблемы, так как половина дисков просто дублирует существующие данные.

RAID-50 (RAID-60) – похожий по смыслу на предыдущий, только с RAID-5(6) внутри, в зависимости от количества накопителей и сложности

системы, позволяет пережить потерю до 4-х дисков, без ущерба хранимой информации.

Однако для вышеописанных комбинированных вариантов необходим качественный и сложный контроллер, который будет работать с массивами. Физически внешний RAID массив выглядит примерно следующим образом – рис. 6.19.



Рисунок 6.19. Дисковый шкаф для RAID

Контроллеры для RAID-массивов также бывают разными, в зависимости от задач и их сложности. Они могут быть интегрированы в материнскую плату, могут быть программными, могут быть аппаратными и быть выполнены в виде платы расширения или вовсе отдельного устройства. Подключение дисков, как правило осуществляется через стандартный SATA интерфейс, подключение к материнской плате через PCIe. У контроллеров разных фирм, видов, форм-факторов могут отличаться количество поддерживаемых дисков, наличие отсутствие ПЗУ на флэш-памяти, размер (наличие) кэш-памяти, пропускная способность интерфейсов.

6.4. SSD (Solid State Drive)

Пиком эволюции внешних запоминающих устройств на данный момент можно считать твердотельные накопители. Устройства полностью отличающиеся по принципу работы от своих предшественников, имеющие лучшие характеристики по скоростям чтения и записи, но недолговечные и дорогие в производстве из-за своего устройства и законов физики. Разнообразные форм-факторы этой памяти позволили создавать как компактные переносные хранилища для практически любой информации, известные всему миру как флэшки и SD-карты, так и конвенциональные устройства формата 2.5”, а позже и устройства еще более компактные под новый стандарт M2. Углубимся в их устройство, принципы работы, виды, а также выделим основные преимущества и недостатки такого вида памяти.

Краткая историческая справка: первым в мире твердотельным накопителем стало творение компании Dataram под названием Bulk Core, представленное ими еще в 1976 году. Как и все в то время, это было нечто огромное и очень тяжелое, устройство на стальном шасси, которое содержало 8 планок обыкновенной энергозависимой RAM-памяти, каждая из которых содержала 256 КБ данных, что было весьма внушительно. Устройство нашло свое применение в наукоемких отраслях. Всего через 6 лет появился первый потребительский твердотельный накопитель от компании Axlon, который был уже существенно меньше в габаритах и имел на своем борту 320 КБ памяти. Так как память эта была энергозависимой, то в комплекте предусматривался блок батарей, на случай отключения основного питания, которого хватало на 3 часа непрерывной работы. Ненадежность вышеперечисленных устройств, а также энергозависимость ограничивала их применение, так, как например жесткие диски, гибкие диски, магнитные ленты и в конце концов перфокарты не требовали энергии для хранения информации. Решением стала flash-память, первый прототип которой был представлен в 1984 году, японским инженером-физиком Фудзио Матсуоко, который представил прототип энергонезависимой флэш-памяти. Его флэш-память была основана на транзисторах с дополнительным плавающим затвором, расположенным между p-слоем и управляющим затвором, что позволило создавать действительно миниатюрные чипы. В 1988 году на основе этой технологии было развернуто производство чипов компании Intel. Чуть позже, 1989 году была представлена первая NAND-память. Вместе с этим компания Digipro представила первый твердотельный накопитель Flashdisk, это был 1995 год (рис. 6.20).

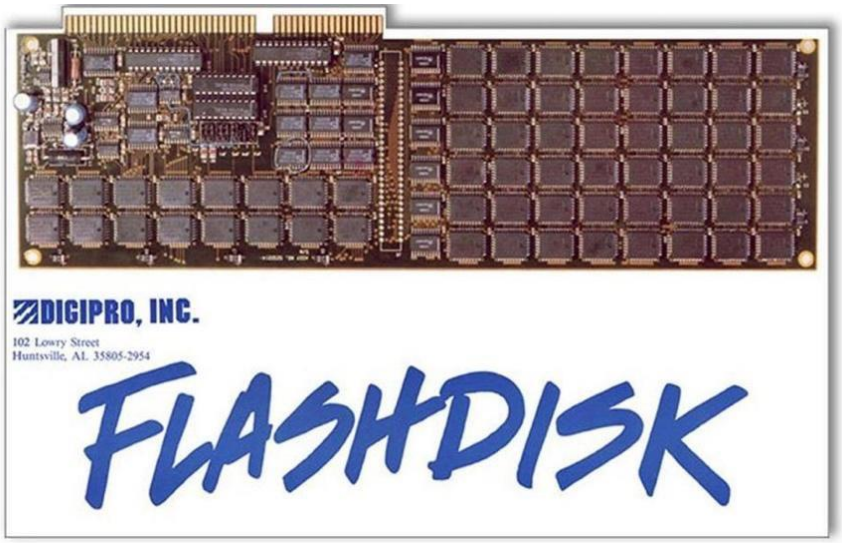


Рисунок 6.20. NAND-память

Он имел привычный на то время формат 3.5”, нес 16 МБ данных. Создание подобных чипов было крайне дорогим удовольствием, поэтому массовое распространение такая память получила только к 2000 году, когда стало понятно, что затраты на подобные устройства могут окупаться. В этом же году компания IBM представила Disk-On-Chip, первую, в современном понимании, флэшку. На тот момент это устройство хранило 8 МБ, что равнялось 8 дискетам. Первым коммерческим твердотельным накопителем стал Transcend IDE Flash Module (рис. 6.21), имевший интерфейс Parallel ATA (PATA) и 512 МБ на борту.

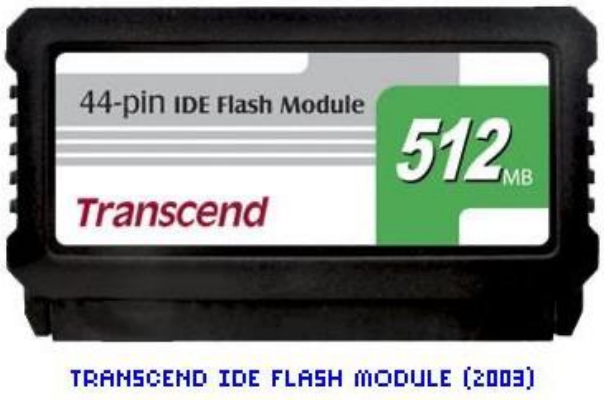


Рисунок 6.21. Первый коммерческий твердотельный накопитель

Первый серийный SATA накопитель появился спустя год, Adtron Corporation выпустила накопитель Flash SSD формата 2.5”, ставшего впоследствии стандартом, стоимостью около 11000 $ и объемом 32 ГБ

(розничная цена была около 700 $). В следующие несколько лет технология просто совершенствовалась, удешевляясь, ускоряясь, становясь надежней. За время своего существования SSD накопители приобрели несколько различных форм-факторов, которые показаны на рис. 6.22.

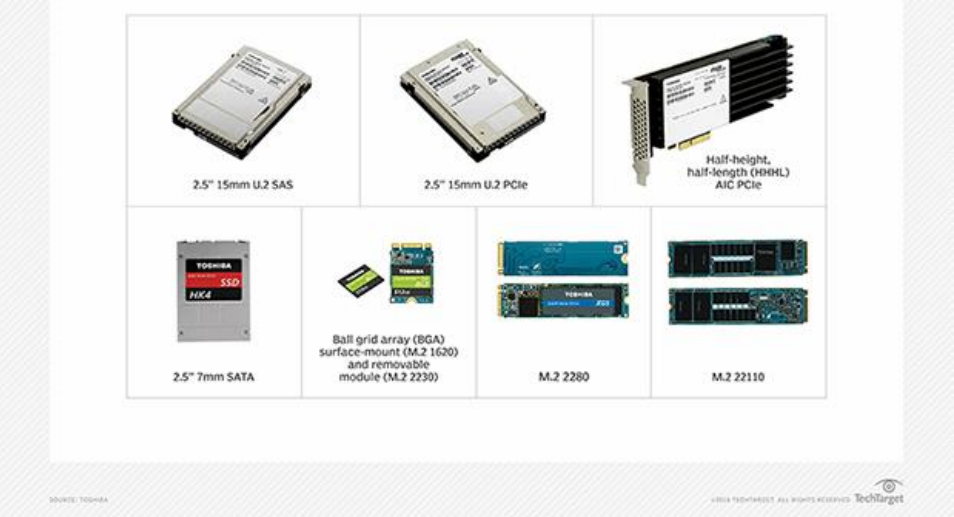


Рисунок 6.22. Форм-факторы SSD

Относительно новым этапом в их развитии стал 2012 год, когда был представлен накопитель с интерфейсом, использующим шину PCIe вместо классического SATA. А технология получила название NVMe (Non-Volatile Memory Express, или быстрая энергонезависимая память). К 2015 году компания Samsung представила первый накопитель потребительского сегмента под новую технологию, и формат M2. Для своей работы накопители этого формата используют шину PCIe x4, которая существенно быстрее шины SATA. Важное замечание, так как накопитель использует общую шину PCIe, а в материнской плате, как известно она одна и к ней может подключаться несколько устройств, главным из которых является видеокарта, а при наличии производительной видеокарты и большого количества M2 накопителей, при сильной нагрузке могут снижаться их характеристики работы, и не доходить до заявленных производителем цифр.

Основа твердотельных накопителей – хорошо знакомые нам транзисторы, но с дополнительным плавающим затвором (рис. 6.23), которые соединяются в матрицы, хранящие слова данных по адресам. Основными подходами являются хорошо знакомые NOR и NAND, схемы которых представлены на рис. 6.23.

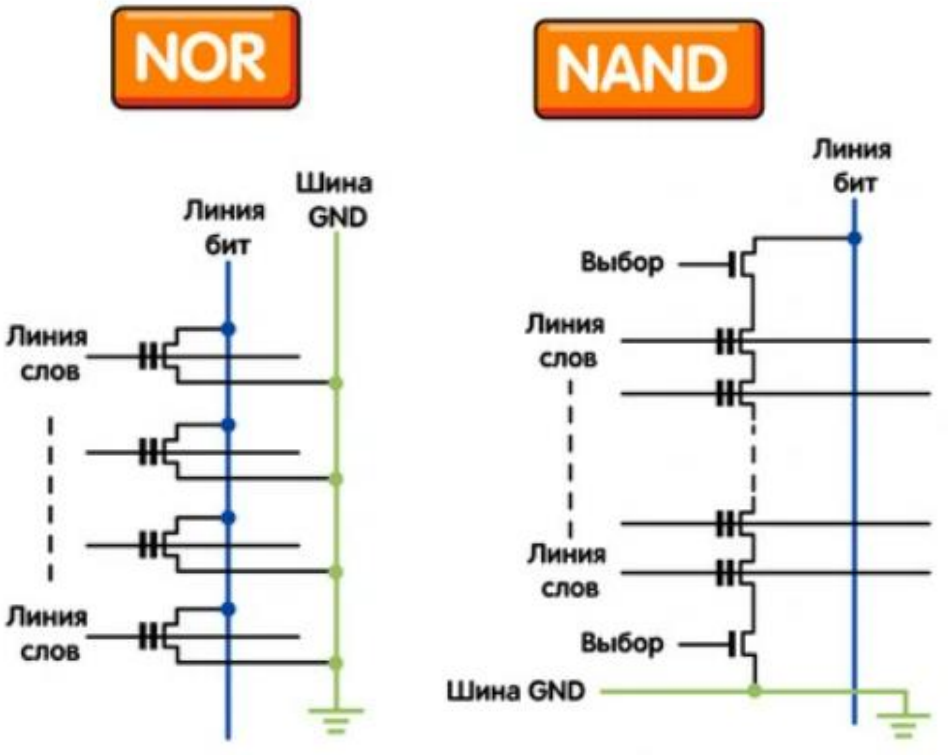


Рисунок 6.23. Транзисторы с плавающим затвором

Построение матрицы по схеме NOR удобно тем, что можно просто получить доступ к любой конкретной ячейке и записать информацию именно в нее. Схема NAND – это несколько одиночных ячеек памяти соединенных последовательно, для записи состояние «нуля» в одну из них, надо, чтобы все другие были открыты и пропускали ток. Поэтому стирание информации в микросхемы NAND-памяти производится поблочно, а для того, чтобы записать новые данные, обновляют информацию сразу для множества ячеек (осуществляют запись «постранично»). Но зато такая схемотехника позволила значительно упростить топологию и сократить размеры ячеек памяти на кристалле, поэтому в современной микроэлектронике именно NAND-память является основной.

Сам по себе транзистор – это обычный n-транзистор, о котором было многократно сказано ранее, но к которому добавили еще один затвор «плавающий» (рис. 6.24). Затвор и полупроводник, представляющий собой канал транзистора между стоком и истоком, разделяет тонкий слой диэлектрика. Электроны при воздействии положительного напряжения к затвору, смогут не только направиться по своему обычному пути внутри полупроводника, но и «перескочить» с помощью инжекции или туннелирования через слой диэлектрика в плавающий затвор. Сделать это могут не все электроны, а только их часть – те, которые получили наибольшую

энергию. При этом они не пробивают слой диэлектрика в физическом смысле, а в соответствии со своими свойствами «перепрыгивают». Вернуться обратно «перепрыгнувшие» электроны не могут, так как у них для этого не хватает энергии. Храниться там они могут достаточно долго, что в реальных условиях эксплуатации равно нескольким годам.

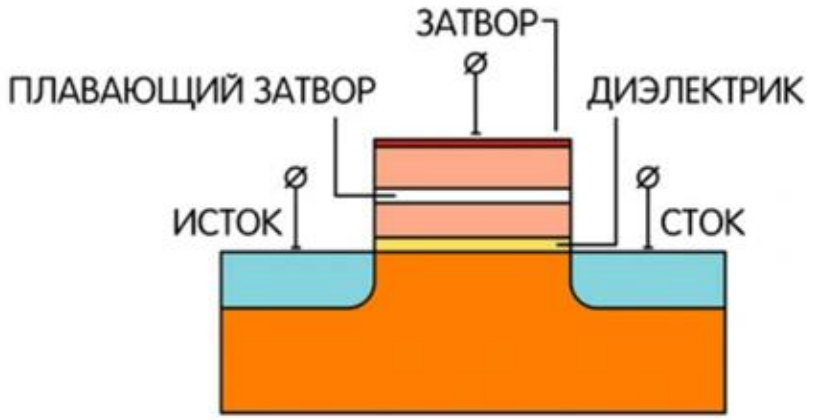


Рисунок 6.24. Схема транзистора с плавающим затвором

Видов организации такой памяти – много, и она с каждым вариантом становилась сложнее, но при всем при этом дешевле в реализации. Рассмотрим основные технологии организации хранения памяти и выделим основные их особенности (рис. 6.25).

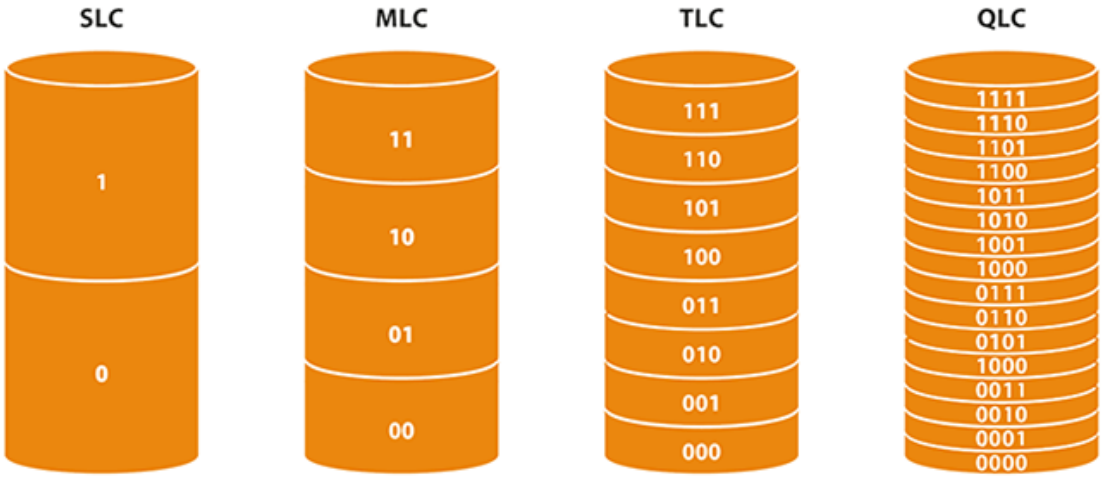


Рисунок 6.25. Типы хранения памяти в SSD

Первым типом памяти, который был представлен, стал SLC -Single Level Cell – одноуровневая ячейка памяти. Такая организация памяти позволяет в одной ячейке хранить только один бит информации. В результате чего

обеспечивается самая быстрая запись и чтение информации среди конкурентов. Этот же тип памяти наиболее долговечный и поддерживает до 100000 циклов перезаписи, однако емкость такой памяти не очень высокая, что накладывает ряд ограничений. В потребительском сегменте эта память не используется, в первую очередь из-за чрезмерной стоимости, поэтому сфера ее применения сугубо специализированная: военная либо научная. Следующий вид организации памяти – MLC – Multi-Level Cell – многоуровневая ячейка памяти, в этот раз инженерам удалось уместить на один транзистор 2 бита памяти. Такая организация стала дешевле в производстве (несмотря на усложнение конструкции), позволила хранить больший общий объем памяти, но в угоду скорости доступа к информации, которая стала ниже, и долговечности, около 10000 циклов перезаписи. Следующая ступень – TLC – Triple Level Cell – трехуровневая ячейка памяти. В данный момент самая распространенная технология изготовления твердотельных накопителей, сочетающая оптимальную скорость работы, долговечность и стоимость. Хотя по сравнению с предыдущим вариантом характеристики стали еще хуже. Количество циклов перезаписи упало до 3000–5000, хотя теоретически в зависимости от условий эксплуатации может быть и больше, что актуально и для всех предыдущих. Последним на данный момент является технология QLC (Quad Level Cell) – четырехуровневая ячейка памяти. Самый дешевый (по стоимости производства и для потребителя) вид памяти, с худшими, по сравнению с предшественниками характеристиками, и самой малой продолжительностью жизни (1000 циклов перезаписи). Эта технология претендует на замену обычных жестких дисков, где не требуется активное взаимодействие с накопителем, а необходимо хранение данных.

Следует отметить, что при переходе от одноуровневой организации к многоуровневым, сам транзистор не меняется, а вот его восприимчивость меняется, что приводит к квадратичному росту ячеек памяти. Если говорить с точки зрения физических процессов, то эффект записи нескольких бит в одну ячейку достигается путем сдвига плавающего затвора в определенный момент времени. При этом необходимо не только смотреть на факт проводимости транзистора, а на характеристику проводимости, что и позволяет более точно и контролируемо заряжать плавающий затвор чтобы получить больше информации при записи в одну ячейку.

Еще одной, важной особенностью твердотельных накопителей является их многослойность. Различают три основных, на данный момент, вида построения банков памяти: 2D NAND, 3D NAND и 3D XPOINT (рис. 6.26).

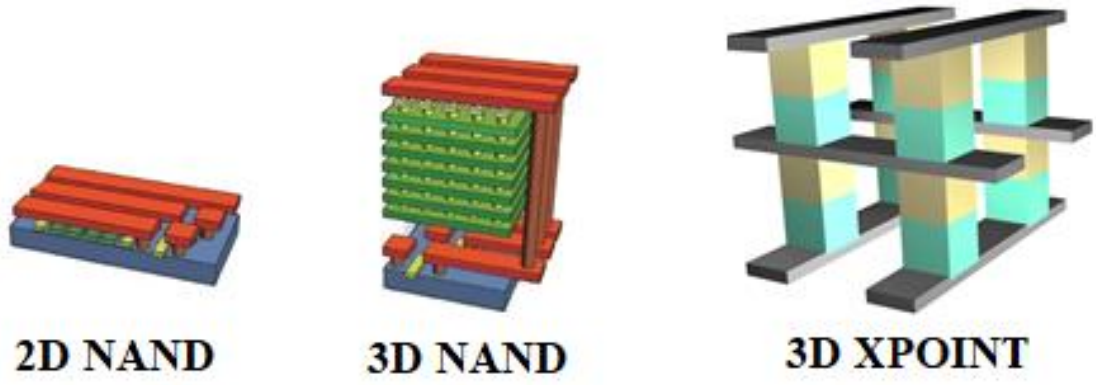


Рисунок 6.26. Построение банков памяти

В ситуации с 2D NAND памятью все просто, транзисторы располагаются горизонтально на поверхности кристалла, рядом друг с другом. Такое расположение имеет ряд ограничений, первый из которых объем, второй, который вытекает из первого – размер, дальнейшее уменьшение транзисторов (меньше 15 нм), не представляется возможным, по причине физических ограничений и явлений, следовательно, для повышения емкости планарной памяти необходимо увеличивать физический размер чипа. Смотря на это, инженеры пришли к идее создания 3D NAND (V-NAND, в зависимости от источников) памяти, где ячейки памяти получилось расположить в вертикальной плоскости, таким образом увеличить максимальный общий объем, и сильно сэкономить на размерах самих чипов. Вертикальная компоновка содержит: проводящие и изолирующие слои, которые напыляются на кристалл слоями. Далее формируются каналы, на стенки которых наносятся структуры изоляторов и плавающих затворов – в результате мы получаем миллионные столбцы кольцеобразных полевых транзисторов. Поддерживается установка до 48 подобных слоев. Однако, с течением времени компанией Western Digital было предложено новое решение, позволяющее увеличить количество слоев в 1.4 раза, до 64, а позже и до 96. Таким решением стало BICS 3D NAND (Bit Cost Scalable 3D NAND). Основным отличием стало применение технологии CTF – Charge Trap Flash, где вместо классических транзисторов, использовалась их измененная версия, где записываемая информация хранится не на затворе, а в специальной изолированной области (рис. 6.27).

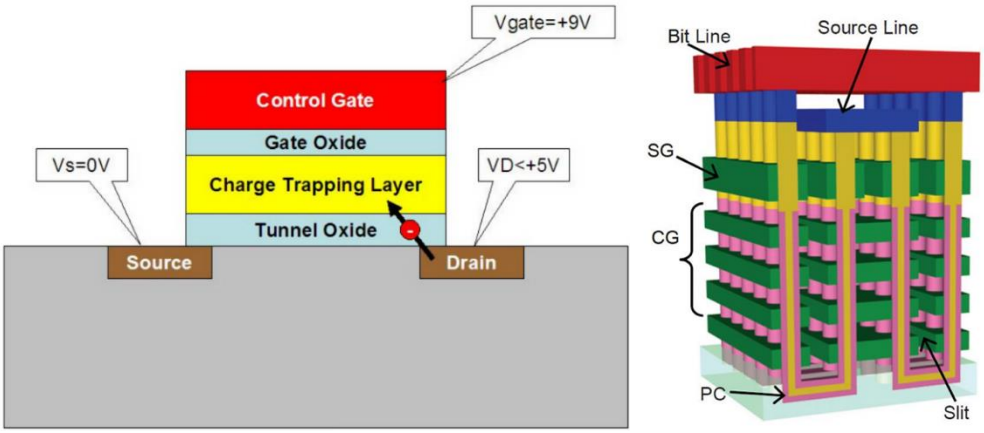


Рисунок 6.27. Схема транзистора с выделенной областью для хранения информации

Что еще входит в состав современного твердотельного накопителя? Помимо ячеек памяти со своей организации для грамотной работы в любом накопителе присутствует контроллер, DRAM буфер, интерфейс ввода/вывода и чип с прошивкой для контроллера. Последнее нередко является причиной гибели твердотельных накопителей, в ситуации, когда он сгорает, контроллер не имеет возможности общаться ни с ячейками памяти, ни с центральным процессором. Интерфейсов ввода вывода существует много, однако наиболее распространенными являются SATA или PCIe на 4 линии, второй является стандартном для M2 накопителей, первый же хоть и используется, но в основном для подключения жестких дисков. Задача DRAM-буфера сопоставима с функциями кэш памяти, в него помещаются обрабатываемые файлы перед записью на диск, а также наиболее часто используемые файлы, для сокращения времени доступа к ним. Теоретически наличие буфера не обязательно, и в ранних версиях их и не было, а в случае с накопителями на SLC памяти он и не нужен вовсе, однако в более дорогих моделях он нередко встречается. DRAM буфер, как ясно из названия – это память, на основе ОЗУ, которая была рассмотрена ранее, иногда такой временный кэш может строится на SLC памяти. Контроллер – не менее сложное устройство, содержащее собственную оперативную память, процессор (на RISC архитектуре), блок коррекции ошибок (ECC), блок S.M.A.R.T. (в твердотельных накопителях такая технология тоже применима), интерфейсы для взаимодействия с NAND памятью и с CPU, блок шифрования и др.

Подводя некоторый итог – твердотельные накопители – это компактные устройства, с возможностью очень быстрой работы с данными, они не сильно пригодны для задач долговременного хранения информации, на эту роль

больше подходят жесткие диски и магнитные ленты, в виду более надежного способа хранения. Исходя из этого скорой повсеместной замены жестких дисков на твердотельные накопители ждать не стоит, так как долговременная сохранность данных гораздо важнее скорости их получения. Перспективы развития твердотельных накопителей тоже достаточно туманные, как минимум перспективная технология пятиуровневой ячейки памяти, но это еще сильнее уменьшит срок жизни накопителя и еще сильнее его замедлит. А дальнейшее уменьшение, как и в случае с процессорами выходит на другой физический уровень.

При рассмотрении внешних накопителей хотелось бы сказать несколько слов о гибридных жестких дисках (SSHD – Solid State Hard Drive, рис. 6.28) и, причинах по которым они не стали популярными. В 2007 году компания Seagate, представила широкой публике первый гибридный накопитель данных – Momentus PSD.



Рисунок 6.28. Гибридные накопители

Идея состояла в том, чтобы взять жесткие диски с их недостатками в виде медленной скорости чтения и записи, и твердотельные накопители с их недолговечной и малой по объему памятью и соединить в одно единое устройство, лишив такой накопитель практически всех недостатков. При этом получившееся творение стоило достаточно дорого, а функционировало не сильно лучше конкурентов. Рассмотрим по подробнее его устройство.

По своему внешнему виду, SSHD не отличался от обычного HDD, формата 3.5”. Внутри же, помимо цилиндра дисков располагалась плата с флэш памятью, все это объединял контроллер, который должен был работать на два фронта (и с флэш памятью и с магнитными дисками). Идея такого накопителя заключалась в том, чтобы записывать часто используемые файлы и служебную информацию на SSD, а редко используемые файлы и основной массив данных на жесткий диск. Однако, реализация подкачала, пользователю не было предоставлено возможности контроля над этим процессом. Существовало только два предзаготовленных варианта: либо ОС (Host-hinted) решала куда и какие файлы поместить, либо сам диск брал на себя эту задачу

(self-optimized). При этом скорости чтения/записи, конечно, выросли относительно обычного жесткого диска, но не были на столько же быстрые как у твердотельных накопителей, а прирост скорости с учетом стоимости самого накопителя была неоправданна, и система, содержащая по отдельности HDD и SSD, были гораздо более выгодной как в плане быстродействия, так и в плане стоимости. Из-за совокупности описанных особенностей массового распространения такие накопители не получили. А скорый выход накопителей формата M2 и их стремительное развитие и вовсе заставило большинство отказаться от этой технологии, хотя на данный момент они производятся, и применяются, хотя ограничено.

6.5. Тестирование показателей HDD/SSD

Помимо отслеживания жизненных показателей внешних накопителей, можно отследить еще и параметры производительности, которые напрямую укажут на эффективность работы устройства, установленного в ПК. Называется данный метод – FIO-тест (Flexible Input/Output). Следует сразу отметить, что при неграмотном использовании этой команды можно **потерять данные**, хранящиеся на устройстве, особенно, если тестирование проводится на домашнем ПК, на конкретном функционирующем диске. На основании этого не рекомендуется применять эту команду в домашних условиях, при отсутствии полной уверенности в корректности и безопасности ее выполнения. Если же проводить тест при определенных обстоятельствах, то проблем возникать не должно, а в результате его прохождения можно увидеть текущую производительность накопителя.

Приведем пример безопасного FIO-теста, который может быть применен для оценки количества IOPS (Input-Output Per Second) – количества операций ввода/вывода в секунду, а также задержки (Latency). Безопасные тесты проводятся с некоторым фиктивным файлом, который занимает некоторый объем (в нашем случае 1 ГБ). В общем случае при установленной и доступной команде в ОС, ее запуск выглядит как название команды (fio), и перечисление набора параметров, которые задают тонкости проведения теста. Пусть команда будет выглядеть следующим образом (Листинг 6.1):

Листинг № 6.1

```
    fio --name=randread --filename=testfile --rw=randread --bs=4k
--size=1G --ioengine=libaio --iodepth=32 --direct=1 --runtime=30s
--time_based --group_reporting
```

Разберем эту команду на составляющие части и определим какие параметры за что отвечают – табл. 6.1.

Таблица 6.1 – Параметры команды FIO

Параметр	Расшифровка	Что делает
`fio`	Flexible I/O Tester	Запускает утилиту FIO
`--name=randread`	Имя теста (job name)	Задает имя для проводимого теста, которое будет отображаться в отчете.
`--filename=testfile`	Файл, с которым проводится тест	FIO создаёт (или использует уже созданный) файл testfile в текущей директории. Все операции чтения/записи происходят только с ним.
`--rw=randread`	Тип операции (read/write)	Определяет режим работы: randread – случайное чтение (Random Read). То есть утилита читает данные из случайных мест файла, имитируя работу реальных программ.
`--bs=4k`	Размер блока (block size)	Указывает размер каждого блока чтения: 4 килобайта. Это стандартный размер блока для тестов IOPS (имитация доступа к мелким файлам, как в реальных приложениях).
`--size=1G`	Размер тестового файла	Задание размера используемого файла 1 Гигабайт в нашем случае
`--ioengine=libaio`	Механизм ввода-вывода	Определяет, как FIO обращается к диску. libaio – это Linux Asynchronous I/O (асинхронный ввод-вывод), позволяет запускать несколько операций одновременно.
`--iodepth=32`	Глубина очереди запросов	Указывает, сколько операций ввода-вывода выполняется одновременно. Чем выше значение – тем сильнее нагружается диск (и можно оценить производительность в многопоточном режиме).

Продолжение табл. 6.1

`--direct=1`	Обход системного кэша	Устанавливает вид работы с устройством, в нашем случае, напрямую с устройством, минуя кэш операционной системы. Это нужно, чтобы измерить **реальную** скорость диска, а не скорость оперативной памяти.
`--runtime=30s`	Время выполнения теста	Задание длительности теста, по истечении времени он автоматически завершится.
`--time_based`	Ориентация по времени, а не по размеру	Обычно FIO работает, пока не пройдёт весь файл --size. С этим параметром – наоборот: работает ровно указанное время (--runtime), даже если файл не прочитан до конца.
`--group_reporting`	Сводный отчёт	Если выполняется несколько потоков или задач, результаты будут объединены и показаны одной суммарной строкой.

После прохождения теста, на экран (или в файл), будет выведена некоторая информация о его результатах. Выглядеть она может следующим образом (Листинг 6.2).

Листинг №6.2

```
read: IOPS=85000, BW=332MiB/s (348MB/s)(10.0GiB/30s)
   lat (usec): min=60, max=900, avg=145.2, stdev=20.7
```

В данном тесте проводился тест на количество операций ввода/вывода и задержку, соответственно показатели получились следующие: IOPS – 85000, то есть за 1 секунду диск производит такое количество операций, для блоков по 4 КБ (стандартная страница, как в ОЗУ). BW – это показатель средней пропускной способности, то есть скорости чтения. Нижняя строка отвечает за показатель задержки между запросом на чтение/запись и непосредственно ответом – в нашем случае это 145 мс, что является крайне хорошим показателем для современных дисков.

Если говорить о некоторых средних значениях для современных накопителей, то: для HDD – характерными являются значения около 100-200

IOPS, для SSD, подключаемые через SATA, около 80000-100000, для NVME SSD – около 700000 и более.

Приведенная команда – это лишь один из тестов, которые можно провести, существуют и другие параметры, которые можно указать в команде для получения новых показателей для оценки тех или иных показателей диска в соответствующих условиях.

6.6. Заключение

В ходе рассмотрения данной работы студенты познакомились с устройством внешних запоминающих устройств, историей их развития, и основными видами, и их характеристиками, которые влияют на производительность ПК, и удобство пользователя. Были рассмотрены основные инструменты, которые позволяют отслеживать основные характеристики HDD/SSD, на примере S.M.A.R.T., а также основной показательный тест FIO, с помощью которого можно оценить текущие характеристики накопителя и его общую производительность.

6.7. Варианты заданий

Для оценки некоторых характеристик современных внешних накопителей студентам предоставляется возможность проведения безопасного FIO теста – варианты в табл. 6.2. Задача сводится к тому, чтобы выполнить тестирование, и на основании полученных результатов сделать выводы о производительности диска. Расписать полученные результаты.

Таблица 6.2 – Варианты заданий

№	Название	Команда
1	Последовательное чтение	fio.exe --name=seq_read --filename=testfile --rw=read --bs=1M --size=4G --ioengine=windowsaio --direct=1 --runtime=30s --time_based --group_reporting
2	Последовательная запись	fio.exe --name=seq_write --filename=testfile --rw=write --bs=1M --size=4G --ioengine=windowsaio --direct=1 --runtime=30s --time_based --group_reporting
3	Случайное чтение 4К	fio.exe --name=rand_read_4k --filename=testfile --rw=randread --bs=4k --size=1G --ioengine=windowsaio --iodepth=32 --direct=1 --runtime=30s --time_based --group_reporting

Продолжение табл. 6.2

4	Случайная запись 4К	fio.exe --name=rand_write_4k --filename=testfile --rw=randwrite --bs=4k --size=1G --ioengine=windowsaio --iodepth=32 --direct=1 --runtime=30s --time_based --group_reporting
5	Смешанная случайная нагрузка 70/30	fio.exe --name=randrw_70_30 --filename=testfile --rw=randrw --rwmixread=70 --bs=4k --size=2G --ioengine=windowsaio --iodepth=64 --numjobs=4 --direct=1 --runtime=40s --time_based --group_reporting
6	Низкая глубина очереди	fio.exe --name=single_io --filename=testfile --rw=randread --bs=4k --size=1G --ioengine=windowsaio --iodepth=1 --direct=1 --runtime=20s --time_based --group_reporting
7	Высокая параллельность	fio.exe --name=single_io --filename=testfile --rw=randread --bs=4k --size=1G --ioengine=windowsaio --iodepth=1 --direct=1 --runtime=20s --time_based --group_reporting
8	Чтение большого блока (128К)	fio.exe --name=seq_128k --filename=testfile --rw=read --bs=128k --size=4G --ioengine=windowsaio --direct=1 --runtime=30s --time_based --group_reporting
9	Короткий быстрый тест	fio.exe --name=quick_test --filename=testfile --rw=randread --bs=4k --size=512M --ioengine=windowsaio --iodepth=16 --direct=1 --runtime=10s --time_based --group_reporting
10	Тест только чтения некоторого существующего файла	fio.exe --name=readonly_test --filename=C:\fio_tests\large_existing_file.iso --rw=read --bs=1M --ioengine=windowsaio --direct=1 --readonly --runtime=20s --time_based --group_reporting
11	Последовательная запись с fsync	fio.exe --name=seq_write_sync --filename=testfile --rw=write --bs=64k --size=2G --ioengine=windowsaio --fsync=1 --direct=0 --runtime=30s --time_based --group_reporting
12	Замер IOPS со стандартными блоками	fio.exe --name=iops_write_test --filename=testfile --rw=randwrite --bs=4k --size=1G --ioengine=windowsaio --iodepth=64 --numjobs=2 --direct=1 --runtime=30s --time_based --group_reporting
13	Замер работы кэша системы	fio.exe --name=cache_test --filename=testfile --rw=randread --bs=4k --size=1G --ioengine=windowsaio --iodepth=32 --direct=0 --runtime=30s --time_based --group_reporting
14	Чтение маленьких блоков	fio.exe --name=512b_rand --filename=testfile --rw=randread --bs=512 --size=1G --ioengine=windowsaio --iodepth=64 --direct=1 --runtime=30s --time_based --group_reporting
15	Последовательная запись больших блоков	fio.exe --name=seq_write_large --filename=testfile --rw=write --bs=128k --size=2G --ioengine=windowsaio --direct=1 --runtime=30s --time_based --group_reporting

Продолжение табл. 6.2

16	Чтение/запись блоков 16К	fio.exe --name=randrw_16k --filename=testfile --rw=randrw --rwmixread=50 --bs=16k --size=2G --ioengine=windowsaio --iodepth=16 --direct=1 --runtime=30s --time_based --group_reporting
17	Многопоточность, последовательное чтение	fio.exe --name=multi_seq --filename=testfile --rw=read --bs=1M --size=4G --ioengine=windowsaio --numjobs=4 --direct=1 --runtime=30s --time_based --group_reporting
18	Длительный тест	fio.exe --name=soak_test --filename=testfile --rw=randrw --rwmixread=50 --bs=4k --size=4G --ioengine=windowsaio --iodepth=64 --numjobs=4 --direct=1 --runtime=600s --time_based --group_reporting
19	Работа с мелкими файлами	fio.exe --name=smallfile_test --filename=testfile --rw=randrw --rwmixread=50 --bs=4k --size=500M --ioengine=windowsaio --iodepth=32 --numjobs=2 --direct=1 --runtime=30s --time_based --group_reporting
20	Тест нагрузки с iodepth = 1	fio.exe --name=low_depth --filename=testfile --rw=randread --bs=4k --size=500M --ioengine=windowsaio --iodepth=1 --direct=1 --runtime=20s --time_based --group_reporting
21	Смешанная нагрузка 1К	fio.exe --name=mixed_1k --filename=testfile --rw=randrw --rwmixread=60 --bs=1k --size=1G --ioengine=windowsaio --iodepth=64 --numjobs=4 --direct=1 --runtime=30s --time_based --group_reporting
22	Проверка целостности с verify	fio.exe --name=write_verify --filename=testfile --rw=write --bs=128k --size=1G --ioengine=windowsaio --direct=1 --verify=crc32 --runtime=30s --time_based --group_reporting
23	Сохранение результата в файл	fio.exe --name=save_output --filename=testfile --rw=randread --bs=4k --size=1G --ioengine=windowsaio --iodepth=32 --direct=1 --runtime=30s --time_based --group_reporting --output=results.txt
24	Работа с малым файлом при длительности теста больше размера файла	fio.exe --name=time_vs_size --filename=testfile --rw=randread --bs=4k --size=200M --ioengine=windowsaio --iodepth=32 --direct=1 --runtime=45s --time_based --group_reporting
25	Сравнение чтения блоков 4К и 64К	fio.exe --name=compare_4k --filename=testfile --rw=randread --bs=4k --size=1G --ioengine=windowsaio --iodepth=32 --direct=1 --runtime=30s --time_based --group_reporting fio.exe --name=compare_64k --filename=testfile --rw=randread --bs=64k --size=1G --ioengine=windowsaio --iodepth=32 --direct=1 --runtime=30s --time_based --group_reporting

Продолжение табл. 6.2

26	Последовательные тесты	fio.exe --name=jobA --filename=testfile --rw=read --bs=1M --size=2G --ioengine=windowsaio --direct=1 --runtime=20s --time_based --group_reporting --stonewall fio.exe --name=jobB --filename=testfile --rw=write --bs=1M --size=2G --ioengine=windowsaio --direct=1 --runtime=20s --time_based --group_reporting
27	Сравнение глубин очередей 1,8,32	fio.exe --name=depth1 --filename=testfile --rw=randread --bs=4k --size=1G --ioengine=windowsaio --iodepth=1 --direct=1 --runtime=20s --time_based --group_reporting fio.exe --name=depth8 --filename=testfile --rw=randread --bs=4k --size=1G --ioengine=windowsaio --iodepth=8 --direct=1 --runtime=20s --time_based --group_reporting fio.exe --name=depth32 --filename=testfile --rw=randread --bs=4k --size=1G --ioengine=windowsaio --iodepth=32 --direct=1 --runtime=20s --time_based --group_reporting
28	Симуляция базы данных	fio.exe --name=db_sim --filename=testfile --rw=randwrite --bs=8k --size=1G --ioengine=sync --fsync=1 --direct=0 --iodepth=1 --runtime=30s --time_based --group_reporting
29	NVME – стресс тест	fio.exe --name=nvme_stress --filename=testfile --rw=randrw --rwmixread=50 --bs=4k --size=2G --ioengine=windowsaio --iodepth=256 --numjobs=8 --direct=1 --runtime=30s --time_based --group_reporting
30	Энергоэффективная низкая нагрузка	fio.exe --name=low_load --filename=testfile --rw=randread --bs=4k --size=500M --ioengine=windowsaio --iodepth=1 --runtime=20s --time_based --group_reporting
31	Стандартный тест со случайным доступом	fio.exe --name=win_rand4k --filename=testfile --rw=randrw --rwmixread=50 --bs=4k --size=1G --ioengine=windowsaio --iodepth=32 --direct=1 --runtime=30s --time_based --group_reporting
32	Последовательная запись	fio.exe --name=win_seq_write --filename=testfile --rw=write --bs=1M --size=2G --ioengine=windowsaio --direct=1 --runtime=30s --time_based --group_reporting

7. ОБЩАЯ ОРГАНИЗАЦИЯ ПРЕРЫВАНИЙ

7.1. Введение

Прерывания – важный механизм, применяемый в компьютерах для обработки более приоритетных и важных задач (действий), которые требуют скорейшего выполнения. Также этот механизм применяется для обработки ошибок, возникающих при работе процессора. В общем смысле задача данного механизма, насильно прервать выполняемую (в текущий момент) программу на процессоре, чтобы выполнить некоторые действия, которые являются более приоритетными (с точки зрения пользователя), после чего вернуться к своим изначальным задачам. По своей природе прерывания делятся на аппаратные, программные и исключения (табл. 7.1).

Таблица 7.1 – Виды прерываний

Тип прерывания	**Источник прерывания**	**Примеры**	**Краткое описание**
Аппаратные	Физические устройства: ввода/вывода, внутренние таймеры, контроллеры, диски и др.	Клавиатура, сетевая карта, таймер системных тиков, плоттер, жесткий диск	Сигнал поступает по физической линии от некоторого устройства
Программные	Программы и ОС	int 0x80 в Linux, syscall в x86_64	Помещаются программистом в текст программы сознательно с целью выполнения системных вызовов
Исключения	Процессор	Деление на ноль, «page fault», недопустимая инструкция	Возникают при ошибках выполнения инструкций

Вне зависимости от природы прерывания, его возникновение и дальнейшая обработка происходит при помощи контроллера прерываний, который находится в ЦП. Его задача определить прерывание, сигнализировать о нем процессору, чтобы тот подгрузил программу-обработчик и обработал возникшее прерывание. Общий алгоритм обработки любых прерываний таков: возникновение прерывания, реакция контроллера и отправка сигнала процессору, сохранение состояния (контекста) процессора, подгрузка из памяти программы-обработчика, выполнение программы, возврат к выполнению исходной программы со следующего адреса. Главной и важной особенностью данного механизма является гарантированное и обязательное сохранение всех данных, которые были в процессоре до возникновения прерывания для его последующей корректной работы. Разберем механизм прерывания подробнее.

7.2. Аппаратные прерывания

Первый и наиболее распространенный вид прерываний – это аппаратные прерывания. Данные прерывания возникают во внешних, периферийных, устройствах, таких как клавиатура, принтер, мышь, жесткие диски и прочие. Самым ярким примером внешнего аппаратного прерывания, будет работа пользователя с клавиатурой. Разберем соответствующий пример: на процессоре выполняется некоторая прикладная программа, предположим, что она перестала отвечать (повисла и не реагирует на пользовательские действия). Пользователь нажимает сочетание клавиш «Win+Shift+Tab», что позволяет прервать выполнение этой программы, подгрузить новую программу – диспетчер задач, через который пользователь принудительно завершает зависшую программу. Факт нажатия/отпуска пользователем соответствующих клавиш на клавиатуре является аппаратным внешним прерыванием, которое подгружает действие (программу обработчик) с ним связанное. Реакция ОС на нажатие этих клавиш (последовательность их кодов) вызывает диспетчер задач, данный процесс уже к прерываниям не относится.

Другим примером внешнего аппаратного прерывания будет работа внешних дисков, например, оповещение процессора о завершении операции чтения/записи информации в память. Аналогично можно сказать про другие периферийные устройства.

Характерно для подобных прерываний то, что большая часть из них – асинхронна и независима, относительно текущей работы процессора. Важен также и тот факт, что реакция на действия пользователя должна происходить

в «реальном времени», то есть время от отправки сигнала о прерывании до непосредственно завершения его обработки не должно превышать некоторой предельной величины. Значение времени обработки того или иного сигнала зависит от типа прерывания и устройства, которое его выдало. Основные примеры внешних аппаратных прерываний сведем в табл. 7.2.

Таблица 7.2 – Аппаратные внешние прерывания

№	Устройство	Событие	Линия / Тип
	Системный таймер	Истек интервал времени	
	Клавиатура	Нажатие/отпуск клавиши	
	Мышь	Движение/щелчок	IRQ12 или MSI
	Диск	Завершена операция ввода/вывода	
	Сетевая карта	Поступил или отправлен пакет	

7.3. Исключения

Несмотря на нестандартное название, под исключениями понимаются внутренние аппаратные прерывания, которые вызываются непосредственно самим ЦП, при нехватке данных, нестандартных результатах или командах. Ярким примером таких прерываний может служить изученный ранее «page fault», который разбирался в теме про ОЗУ. При отсутствии необходимой записи в TLB (или ее некорректности) вызывается прерывание, которое подгружает программу-обработчик для вычисления потерянного адреса и записи его в TLB. В результате успешного вычисления адреса, процессор, естественно, вернется к выполнению прикладной программы, однако данная ошибка нередко приводит к более серьезным проблемам. Другими примерами могут служить деление на ноль (указанное в таблице ранее), переполнение, или попытка выполнение неподдерживаемой инструкции. Все подобные прерывания делятся по уровням «опасности»:

- Faults – ошибки, как правило не критические, ведущие к повторному исполнению инструкции;
- Traps – программные прерывания, о которых будет сказано позже, после которых исключение просто игнорируется и выполняется со следующей инструкции;
- Aborts – серьезные ошибки, после которых выполнение программы – невозможно.

Совокупность исключении и других аппаратных прерываний зачастую может вызвать общеизвестный «Синий экран смерти» – что само по себе не является исключением, но является реакцией ОС на их совокупность и невозможность обработки. Ниже, в табл. 7.3 приведены наиболее распространенные исключения.

Таблица 7.3 – Распространенные исключения

№	Код	Название	Причина	Тип	Действие ОС
			Деление на ноль		Ошибка арифметики
			Недопустимая страница памяти		Подкачка или ошибка
			Нарушение защиты памяти		Завершение процесса
			Недопустимая инструкция		Ошибка выполнения
			Исключение во время исключения		Остановка системы
			Попытка использовать FPU без инициализации		Включение
			Инструкция INT 3 (точка останова)		Переход в отладчик
			Переполнение при арифметике		Сигнал отладчику

7.4. Программные прерывания

По своей сути это не совсем прерывания, в прямом понимании этого слова, а программные ловушки. Источником данного типа прерываний является сама исполняемая на ЦП программа, а прерывания в ней известны заранее и закладываются в исполняемый код программистом на этапе проектирования приложения. Программные прерывания – это механизм, который позволяет контролируемо и безопасно вызывать исключения, и, в случае необходимости их игнорировать, чтобы не прерывать функционирование программы. На текущий момент основной задачей программных прерываний, является выполнение системных вызовов. Иными словами – это способ обращения к ядру ОС, для выполнения какой-либо

привилегированной операции – например, чтения или записи файла, выделения памяти, создания процесса или вывода данных на экран. К системным вызовам можно отнести работу с файловой системой, вводом/выводом информации, синхронизацию, работу с внутренними процессами (их создание, исполнение или удаление) и др. Важно отметить, что несмотря на видимую связь программных и аппаратных прерываний общим для них является только механизм обработки, а коды и данные – различаются. Примеры программных прерываний показаны в табл. 7.4.

Таблица 7.4 – Примеры программных прерываний

Инструкция	**Архитектура**	**Назначение**	**Применение**
SYSENTER / SYSEXIT	x86 (Intel)	Быстрый системный вызов	Windows NT/XP/7, Linux (32-бит)
SYSCALL / SYSRET	x86-64 (AMD и Intel)	Быстрый системный вызов в 64-битных ОС	Linux, Windows 10/11 x64
SVC #imm (Supervisor Call)	ARM / ARM64	Аналог INT – вызов ядра	Android, iOS, Embedded-системы
HVC #imm (Hypervisor Call)	ARM	Вызов гипервизора (виртуализация)	Виртуальные машины ARM
ECALL	RISC-V	Вызов системной функции из пользовательского режима	Linux для RISC-V, микроконтроллеры
EBREAK	RISC-V	Отладочная ловушка (аналог INT 3)	Отладка и трассировка
VMCALL / VMMCALL	x86 с виртуализацией	Вызов гипервизора	VMware, Hyper-V, KVM

7.5. Общий механизм обработки прерывания

Учитывая полученные знания по прерываниям, следует предметно рассмотреть их механизм обработки. Проще всего это сделать на примере аппаратного вида, как наиболее распространенного и частого. В начале

последовательность обработки была приведена, но кратко, расширим и дополним ее до более правильного вида:

- возникновение прерывания (нажатие пользователем клавиши), отправка устройством сигнала;
- приход сигнала на контроллер прерываний (PIC или APIC);
- формирование адреса программы обработчика в таблице векторов прерываний (IDT);
- формирование разрешения на прерывание, уведомление процессора;
- завершение выполнения текущей инструкции в ЦП;
- сохранение контекста;
- выполнение процессором программы обработчика по адресу из полученного вектора;
- реакция ОС на прерывание;
- восстановление контекста и продолжение выполнения программы.

Разберем данный процесс поподробнее. Первым этапом является возникновение прерывания, пользователь нажал на клавишу для ввода некоторого значения. В устройстве генерируется сигнал прерывания, который по специальной линии IRQ0 (Interrupt Request №), отправляется на контроллер прерываний PIC (Programmable Interrupt Controller) или, что более актуально для современных процессоров, в APIC (Advanced PIC). PIC – это устаревший контроллер, который поддерживал до 8 линий прерываний (IRQ0 – IRQ7), что достаточно быстро стало недостаточным. Одним из решений стало каскадное объединение двух таких контроллеров, что расширило количество линий до 15, однако и это количество достаточно быстро исчерпалось. В PIC были реализованы все необходимые функции, в числе которых было и маскирование прерывания (разграничение приоритетов), благодаря которому одни прерывания могли прерывать другие, если у них был больший приоритет.

В современных процессорах применяется APIC – эволюционное развитие обычного PIC. Первым отличием является то, что он разделен на две логические структуры, Local APIC и I/O APIC. Local APIC находится непосредственно в процессоре в каждом ядре и отвечает за обработку пришедшего прерывания на конкретном ядре, при учете доступности ядра и приоритета прерывания. I/O APIC располагается на чипсете материнской платы и отвечает за получение прерываний с периферийных устройств и перенаправление их в соответствующие свободные локальные контроллеры для последующей обработки. Вторым отличием является существенно

больший хранимый объем адресов (255). За это отвечает специальная таблица IDT (Interruption Descriptor Table) – таблица векторов прерываний. Она хранит вектора прерываний (номера/адреса) и имеет логическое разделение на исключения (0-31 адреса) и на аппаратные прерывания или системные вызовы (32-255 адреса). Для вторых актуально динамическое переназначение, в зависимости от приоритета и ОС. Некоторые устройства, например, видеокарта или диск, не имеет физических линий IRQ, а использует механизм отправки сообщений, напрямую через шину PICe или SATA. Современный механизм, который называется MSI/ MSI-X (Message Signaled Interrupts) – прерывания, сигнализирующие сообщениями. Такой механизм быстрее, не требует отдельной физической линии, а инициализация прерывания происходит путем записи необходимого значения на соответствующий регистр в ЦП (IDTR – IDT Register – который будет хранить данные из таблицы прерываний).

Описанные контроллеры, формируют требуемый адрес и сигнализируют процессору о наличии прерывания. Они же решают можно ли его выполнить сейчас, на каком ядре это удобнее всего сделать. Если прерывание все-таки разрешено, то процессор завершает текущую инструкцию, и сохраняет свое состояние (состояние регистров, аккумулятора, программного счетчика, флагов и пр.). Как правило сохранение происходит в стековой памяти. Из области памяти подгружается программа-обработчик и процессор начинает ее выполнение. В процессе выполнения (в нашем случае нажатия на клавишу) происходит реакция ОС и соответствующей программы в ней (получение кода нажатой клавиши), что приводит к появлению символа на экране, ну или прерыванию выполнения кода (в случае с комбинацией клавиш «Ctrl + C»).

После завершения программы обработчика, процессор восстанавливает свое состояние, которое было ДО прерывания и продолжает работу со следующей инструкции.

Механизм обработки программных прерываний и исключений полностью совпадает с описанным процессом для аппаратных прерываний, за исключением непосредственно источников и кодов этих прерываний.

7.6. Схема процесса прерывания

Рассмотрим пример вычислительного процесса при обработке внешних аппаратных прерываний. В качестве внешних устройств будут выступать рядовые периферийные устройства: жесткий диск, принтер, клавиатура и

мышь. В конечном итоге, для рассмотрения процесса обработки прерывания названия внешних устройств не важно, так как они все-равно будут иметь свою линию IRQ, которая и будет играть главенствующую роль.

Программы обработки прерываний (ПОП) находятся по адресам от 0x0065 (101) до 0x01F4 (500), то есть в служебной, верхней части ОЗУ. Для программы обработки диска выделим адреса 0x0065–0x00C8 (линия IRQ0), для программ обработки принтера 0x00C9–0x012C (линия IRQ1), для программы обработки мыши 0x012D–0x0191 (линия IRQ2) и для клавиатуры 0x0192–0x01F4 (линия IRQ3). Как можно заметить для удобства под каждое периферийное устройство выделено по 100 ячеек памяти программы обработчика прерывания (ПОП). Исполняемый код основной программы будет располагаться в области памяти 0x1000–0x8000, то есть в первой половине ОЗУ. Так как при прерывании будет необходимо запоминать адрес команды и состояния регистров, добавим стековую память. Для корректного отображения процесса обработки прерывания составим схему (рис. 7.1).

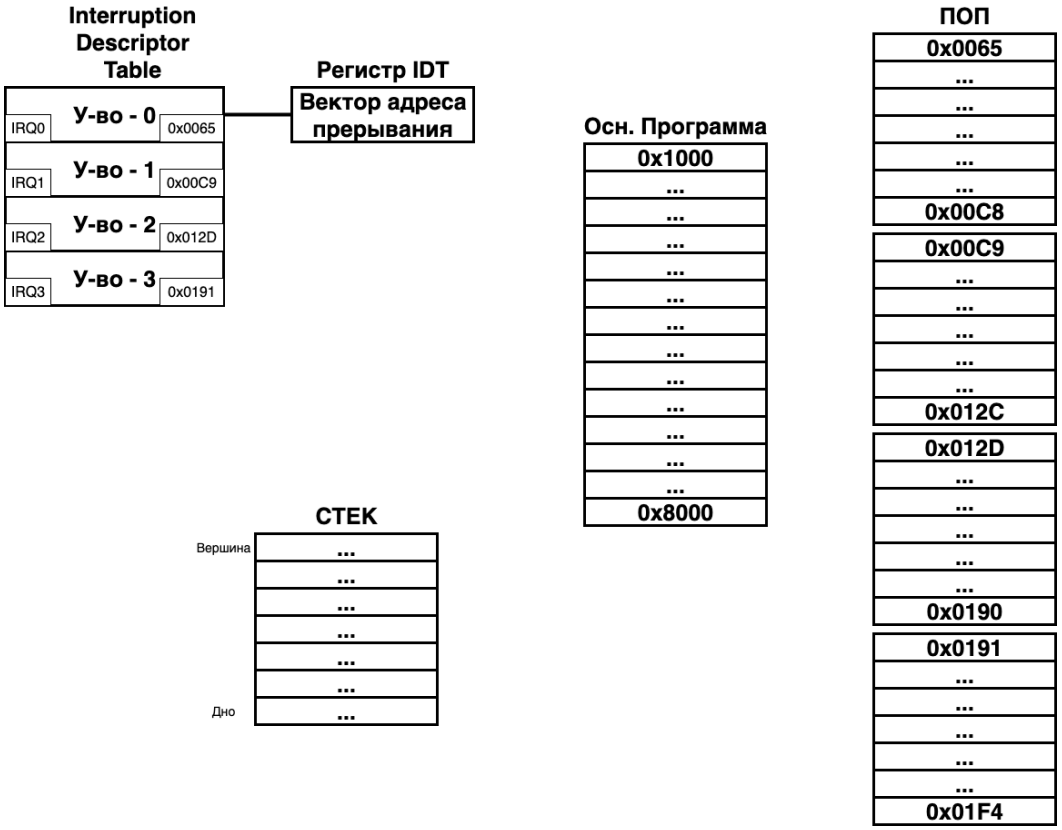


Рисунок 7.1. Шаблон схемы для последовательности обработки прерывания

Рассмотрим конкретный пример: пусть при выполнении некоторой команды по адресу 0x1329, основной программы, пришло прерывание от клавиатуры. Затем, при выполнении команды по адресу 0x01BA ПОП

клавиатуры, пришло прерывание от принтера. Изобразить последовательность обработки прерывания, используя заготовленную ранее схему (рис. 7.1).

Проще всего данный процесс будет изобразить последовательностью стрелок. Известно, что первое прерывание было получено от устройства «Клавиатура», при выполнении команды 0x1329. Основная программа располагается в диапазоне 0x1000–0x8000. Значит нулевой этап – это последовательное выполнение программы, до адреса 0x1329. При поступлении сигнала о прерывании необходимо сделать следующие действия: закончить выполнение текущей команды, вычислить адрес следующей команды, и сохранить контекст (состояние регистров процессора). В нашем случае адрес следующей команды – 0x132A. Всю вышеописанную информацию складываем в стэк. Важно отметить, в данной схеме не учитываются состояния регистров, как до появления прерывания, так и после, в контексте понимания последовательности обработки прерывания, их значения не важны, поэтому в стеке будем отражать только адреса команд. Далее, необходимо узнать, по какому адресу находится программа-обработчик прерывания для устройства «Клавиатура», для этого необходимо обратиться к таблице векторов прерываний IDT (VAD) (в регистр IDTR), на котором к этому моменту уже сформировался необходимый адрес программы обработчика, и считать его, в нашем случае, адрес будет только один, потому что только одно устройство запросило прерывание. Для клавиатуры это адрес 0x0191.

В процессор загружается команда, лежащая по адресу 0x0191 – первая инструкция программы обработчика. По условию задачи, через некоторое время приходит еще одно прерывание, на этот раз от устройства «Принтер». Предположим, что это устройство вызвало прерывание ранее, а сейчас хочет сообщить, о том, что задача выполнена. Принтер в условии задачи имеет приоритет 1, а клавиатура приоритет 3, что означает, что прерывание от принтера должно быть обработано раньше. Повторяем процесс сохранения адреса. Завершаем выполнение команды 0x01BA, заносим в стек адрес 0x01BB. В таблице векторов IDT (VAD) (на регистре IDTR) сформировался адрес 0x00C9, который соответствует началу программы обработчика принтера. Команда загружается на процессор, начинается обработка прерывания.

По условию задачи, никакие устройства больше прерываний не запрашивали, следовательно, полностью выполняем ПОП принтера, доходя до адреса 0x012C. Так как прерывание обработано, необходимо восстановить

состояние процессора, которое было до его появления. Значит обращаемся в стек, восстанавливаем состояния регистров, и берем верхний адрес, которым оказывается адрес 0x01BB, переходим по нему, и выполняем до конца ПОП клавиатуры. Когда выполнение ПОП клавиатуры дойдет до адреса 0x01F4, повторим чтение адреса из стека (и восстановление регистров), и это позволит вернуться к выполнению основной программы, начиная с адреса 0x132A. Ход процесса обработки прерывания от двух устройств с разными приоритетами продемонстрирован на рис. 7.2.

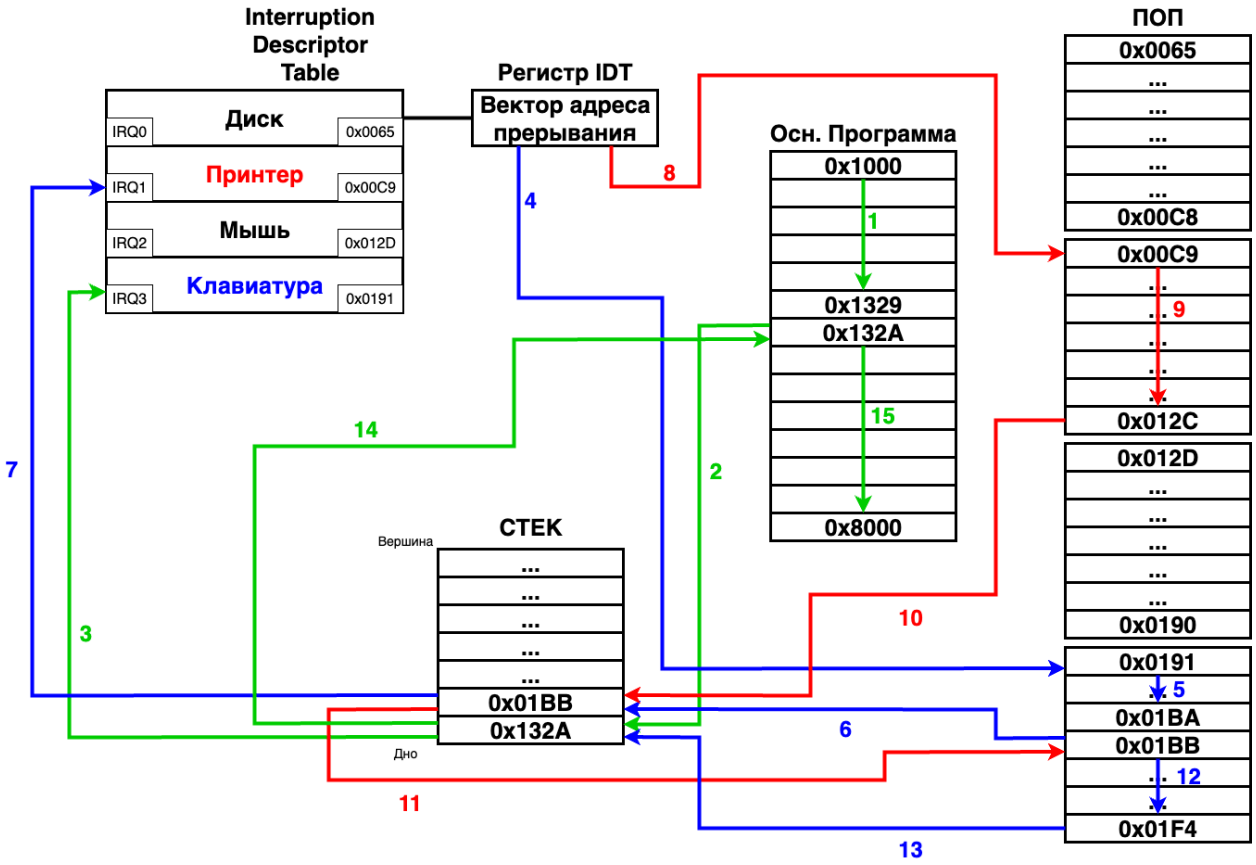


Рисунок 7.2. Последовательность обработки прерывания для задачи из примера

Естественно, данный процесс актуален и для программных прерываний и исключений, за той лишь разницей, что инициатором прерывания выступают другие источники.

7.7. Заключение

В процессе рассмотрения данной работы, студенты познакомились с прерываниями, их видами, и процессом их обработки. Данный механизм лежит в основе взаимодействия пользователя с персональным компьютером. Благодаря этому механизму можно отлаживать работу прикладных программ,

вводить и выводить данные в реальном времени и обрабатывать исключительные ситуации оперативно и безопасно. В современных процессорах, учитывая частоты и многозадачность прерывания не так критически влияют на работу ПК, и незаметны.

7.8. Варианты заданий

Используя шаблон схемы для последовательности обработки прерываний (рис. 7.2), необходимо изобразить процесс обработки прерываний согласно полученному варианту.

1. Периферийные устройства: SSD; Мышь; Клавиатура; Принтер.

При выполнении команды 0x217A прикладной программы пришло прерывание от Принтера. Далее, при выполнении команды 0x01A8 ПОП Принтера пришло прерывание от Клавиатуры. Изобразить на рисунке ход выполнения вычислительного процесса.

2. Периферийные устройства: HDD; Тачпад; Геймпад; Сканер.

При выполнении команды 0x6A41 прикладной программы пришло прерывание от Сканера. Далее, при выполнении команды 0x01D3 ПОП Сканера пришло прерывание от Геймпада. Изобразить на рисунке ход выполнения вычислительного процесса.

3. Периферийные устройства: Таймер; Мышь; Клавиатура; Плоттер.

При выполнении команды 0x4C12 прикладной программы пришло прерывание от Плоттера. Далее, при выполнении команды 0x01C2 ПОП Плоттера пришло прерывание от Клавиатуры. Изобразить на рисунке ход выполнения вычислительного процесса.

4. Периферийные устройства: SSD; Тачпад; Геймпад; Принтер.

При выполнении команды 0x3880 прикладной программы пришло прерывание от Геймпада. Далее, при выполнении команды 0x014D ПОП Геймпада пришло прерывание от SSD. Изобразить на рисунке ход выполнения вычислительного процесса.

5. Периферийные устройства: HDD; Мышь; Клавиатура; Сканер.

При выполнении команды 0x5073 прикладной программы пришло прерывание от Клавиатуры. Далее, при выполнении команды 0x013D ПОП Клавиатуры пришло прерывание от HDD. Изобразить на рисунке ход выполнения вычислительного процесса.

6. Периферийные устройства: Таймер; Тачпад; Геймпад; Принтер.

При выполнении команды 0x7B41 прикладной программы пришло прерывание от Принтера. Далее, при выполнении команды 0x01C5 ПОП

Принтера пришло прерывание от Таймера. Изобразить на рисунке ход выполнения вычислительного процесса.

7. Периферийные устройства: SSD; Мышь; Клавиатура; Плоттер.

При выполнении команды 0x68A9 прикладной программы пришло прерывание от Плоттера. Далее, при выполнении команды 0x01E7 ПОП Плоттера пришло прерывание от Клавиатуры. Изобразить на рисунке ход выполнения вычислительного процесса.

8. Периферийные устройства: HDD; Тачпад; Геймпад; Принтер.

При выполнении команды 0x2F10 прикладной программы пришло прерывание от Принтера. Далее, при выполнении команды 0x019A ПОП Принтера пришло прерывание от Геймпада. Изобразить на рисунке ход выполнения вычислительного процесса.

9. Периферийные устройства: Таймер; Мышь; Клавиатура; Сканер.

При выполнении команды 0x5C24 прикладной программы пришло прерывание от Мыши. Далее, при выполнении команды 0x0104 ПОП Мыши пришло прерывание от Таймера. Изобразить на рисунке ход выполнения вычислительного процесса.

10. Периферийные устройства: SSD; Тачпад; Клавиатура; Принтер.

При выполнении команды 0x6431 прикладной программы пришло прерывание от Тачпада. Далее, при выполнении команды 0x00CB ПОП Тачпада пришло прерывание от SSD. Изобразить на рисунке ход выполнения вычислительного процесса.

11. Периферийные устройства: HDD; Мышь; Геймпад; Плоттер.

При выполнении команды 0x4109 прикладной программы пришло прерывание от Мыши. Далее, при выполнении команды 0x00F4 ПОП Мыши пришло прерывание от HDD. Изобразить на рисунке ход выполнения вычислительного процесса.

12. Периферийные устройства: Таймер; Тачпад; Клавиатура; Сканер.

При выполнении команды 0x4E21 прикладной программы пришло прерывание от Сканера. Далее, при выполнении команды 0x01CB ПОП Сканера пришло прерывание от Таймера. Изобразить на рисунке ход выполнения вычислительного процесса.

13. Периферийные устройства: SSD; Мышь; Геймпад; Принтер.

При выполнении команды 0x73A3 прикладной программы пришло прерывание от Геймпада. Далее, при выполнении команды 0x0181 ПОП Геймпада пришло прерывание от SSD. Изобразить на рисунке ход выполнения вычислительного процесса.

14. Периферийные устройства: HDD; Тачпад; Клавиатура; Плоттер.

При выполнении команды 0x3144 прикладной программы пришло прерывание от Клавиатуры. Далее, при выполнении команды 0x0137 ПОП Клавиатуры пришло прерывание от HDD. Изобразить на рисунке ход выполнения вычислительного процесса.

15. Периферийные устройства: Таймер; Мышь; Геймпад; Сканер.

При выполнении команды 0x580F прикладной программы пришло прерывание от Сканера. Далее, при выполнении команды 0x01F0 ПОП Сканера пришло прерывание от Таймера. Изобразить на рисунке ход выполнения вычислительного процесса.

16. Периферийные устройства: SSD; Тачпад; Клавиатура; Принтер.

При выполнении команды 0x7C92 прикладной программы пришло прерывание от Принтера. Далее, при выполнении команды 0x01BC ПОП Принтера пришло прерывание от Клавиатуры. Изобразить на рисунке ход выполнения вычислительного процесса.

17. Периферийные устройства: HDD; Мышь; Геймпад; Сканер.

При выполнении команды 0x2571 прикладной программы пришло прерывание от Геймпада. Далее, при выполнении команды 0x0143 ПОП Геймпада пришло прерывание от HDD. Изобразить на рисунке ход выполнения вычислительного процесса.

18. Периферийные устройства: Таймер; Тачпад; Клавиатура; Принтер.

При выполнении команды 0x3A80 прикладной программы пришло прерывание от Тачпада. Далее, при выполнении команды 0x010E ПОП Тачпада пришло прерывание от Таймера. Изобразить на рисунке ход выполнения вычислительного процесса.

19. Периферийные устройства: SSD; Мышь; Геймпад; Плоттер.

При выполнении команды 0x67F3 прикладной программы пришло прерывание от Плоттера. Далее, при выполнении команды 0x01EA ПОП Плоттера пришло прерывание от Геймпада. Изобразить на рисунке ход выполнения вычислительного процесса.

20. Периферийные устройства: HDD; Тачпад; Клавиатура; Сканер.

При выполнении команды 0x6A01 прикладной программы пришло прерывание от Сканера. Далее, при выполнении команды 0x01E9 ПОП Сканера пришло прерывание от Клавиатуры. Изобразить на рисунке ход выполнения вычислительного процесса.

21. Периферийные устройства: Таймер; Мышь; Геймпад; Принтер.

При выполнении команды 0x3F74 прикладной программы пришло прерывание от Принтера. Далее, при выполнении команды 0x01B2 ПОП Принтера пришло прерывание от Таймера. Изобразить на рисунке ход выполнения вычислительного процесса.

22. Периферийные устройства: SSD; Тачпад; Клавиатура; Плоттер.

При выполнении команды 0x7E55 прикладной программы пришло прерывание от Клавиатуры. Далее, при выполнении команды 0x0171 ПОП Клавиатуры пришло прерывание от SSD. Изобразить на рисунке ход выполнения вычислительного процесса.

23. Периферийные устройства: HDD; Мышь; Геймпад; Сканер.

При выполнении команды 0x5F23 прикладной программы пришло прерывание от Мыши. Далее, при выполнении команды 0x00FA ПОП Мыши пришло прерывание от HDD. Изобразить на рисунке ход выполнения вычислительного процесса.

24. Периферийные устройства: Таймер; Тачпад; Клавиатура; Принтер.

При выполнении команды 0x4B71 прикладной программы пришло прерывание от Клавиатуры. Далее, при выполнении команды 0x014D ПОП Клавиатуры пришло прерывание от Таймера. Изобразить на рисунке ход выполнения вычислительного процесса.

25. Периферийные устройства: SSD; Мышь; Геймпад; Сканер.

При выполнении команды 0x2D09 прикладной программы пришло прерывание от Сканера. Далее, при выполнении команды 0x01CC ПОП Сканера пришло прерывание от SSD. Изобразить на рисунке ход выполнения вычислительного процесса.

26. Периферийные устройства: HDD; Тачпад; Клавиатура; Плоттер.

При выполнении команды 0x5B80 прикладной программы пришло прерывание от Плоттера. Далее, при выполнении команды 0x01B8 ПОП Плоттера пришло прерывание от Клавиатуры. Изобразить на рисунке ход выполнения вычислительного процесса.

27. Периферийные устройства: Таймер; Мышь; Геймпад; Принтер.

При выполнении команды 0x6D70 прикладной программы пришло прерывание от Геймпада. Далее, при выполнении команды 0x0163 ПОП Геймпада пришло прерывание от Таймера. Изобразить на рисунке ход выполнения вычислительного процесса.

28. Периферийные устройства: SSD; Тачпад; Клавиатура; Сканер.

При выполнении команды 0x3C93 прикладной программы пришло прерывание от Сканера. Далее, при выполнении команды 0x01BC ПОП

Сканера пришло прерывание от SSD. Изобразить на рисунке ход выполнения вычислительного процесса.

29. Периферийные устройства: HDD; Мышь; Геймпад; Принтер.

При выполнении команды 0x4D02 прикладной программы пришло прерывание от Принтера. Далее, при выполнении команды 0x01C4 ПОП Принтера пришло прерывание от Геймпада. Изобразить на рисунке ход выполнения вычислительного процесса.

30. Периферийные устройства: Таймер; Тачпад; Клавиатура; Плоттер.

При выполнении команды 0x7A64 прикладной программы пришло прерывание от Тачпада. Далее, при выполнении команды 0x00DE ПОП Тачпада пришло прерывание от Таймера. Изобразить на рисунке ход выполнения вычислительного процесса.

31. Периферийные устройства: SSD; Мышь; Геймпад; Принтер.

При выполнении команды 0x6F80 прикладной программы пришло прерывание от Принтера. Далее, при выполнении команды 0x01AF ПОП Принтера пришло прерывание от Геймпада. Изобразить на рисунке ход выполнения вычислительного процесса.

32. Периферийные устройства: HDD; Тачпад; Клавиатура; Сканер.

При выполнении команды 0x5209 прикладной программы пришло прерывание от Тачпада. Далее, при выполнении команды 0x0122 ПОП Тачпада пришло прерывание от HDD. Изобразить на рисунке ход выполнения вычислительного процесса.

8. АРХИТЕКТУРА RISC-V

8.1. Введение

Почти все современные устройства содержат процессоры, которые выполняют программы и обрабатывают данные, при этом современные компьютеры используют дополнительные процессоры для распределения нагрузки, а модули памяти тоже имеют встроенные процессоры для операций чтения-записи. Основой любого процессора является определенная архитектура команд, такие как CISC, RISC, ARM и их производные, каждая из которых имеет свои преимущества и ограничения, включая необходимость лицензий для ARM и Intel. В отличие от них, RISC-V, разработанная в 2010 году в Университете Беркли, распространяется с открытыми стандартами, подходит для микроконтроллеров и суперкомпьютеров, имеет активное сообщество и обширную литературу, что делает её удобной для изучения программирования на ассемблере. Цель лабораторной работы – изучение принципов архитектуры RISC-V и создание простой программы на ассемблере с использованием инструментов разработки.

8.2. Архитектура RISC

Архитектура RISC V поддерживает 32-, 64-, 128-битные процессоры. В большинстве устройств RISC V процессор выступает в роли системы на кристалле, что позволяет реализовать значительную часть вычислительной системы на одном чипе. Зачастую такой процессор обладает несколькими ядрами, что позволяет обрабатывать инструкции для нескольких программ параллельно. Дополнительно могут включаться следующие модули: сопроцессоры для вычислений с плавающей точкой, графический процессор, модули для реализации протоколов коммуникации.

Набор инструкций в RISC V является модульным. В каждом процессоре используется один или несколько модулей, содержащих базовый набор инструкций. При необходимости добавляются модули, которые содержат дополнительные наборы команд. Наборы команд модулей представлены в табл. 8.1 и 8.2.

Таблица 8.1 – Базовые наборы инструкций

Название	**Описание**
RV32I	Набор 32-битных целочисленных инструкций
RV32E	Набор 32-битных целочисленных инструкций (встроенный)
RV64I	Набор 64-битных целочисленных инструкций

Таблица 8.2 – Расширенные наборы инструкций

Название	**Описание**
M	Расширение для операций целочисленного умножения и деления
A	Расширение для атомарных инструкций
F	Расширение для вычислений с плавающей точкой одинарной точности
D	Расширение для вычислений с плавающей точкой двойной точности
Zicsr	Инструкции для работы с контрольными и статусными регистрами
Zifencei	Инструкции синхронизации потоков команд и данных
G	Инструкции расширения IMAFD Zicsr Zifencei
C	Расширение для инструкций с сокращенными именами

Базовый набор инструкций RISC-V обеспечивает выполнение ключевых операций машинного кода – загрузку и сохранение данных, арифметические и логические операции, управление переходами и вызовами процедур. Расширенные наборы добавляют специализированные функции, например, работу с числами с плавающей точкой, оптимизацию многопоточности и повышение эффективности компилятора. Модульная структура архитектуры позволяет включать только необходимые расширения, что экономит ресурсы и облегчает разработку как встраиваемых микроконтроллеров (RV32E), так и серверных процессоров с полным набором расширений G. Основной принцип RISC заключается в простоте инструкций, их выполнении за один такт и фиксированной длине команд, что упрощает декодирование, повышает предсказуемость исполнения и перекладывает сложность на компилятор. Типы инструкций кратко показаны в табл. 8.3.

RISC-V использует строгую организацию форматов команд, которые унифицируют кодирование и минимизируют аппаратные затраты. Преимущества подхода включают простое проектирование аппаратуры, высокую предсказуемость исполнения, масштабируемость и эффективное использование конвейера, а недостатки – увеличение числа элементарных инструкций для сложных операций, рост размера кода, повышенные требования к памяти и необходимость развитого компилятора.

Таблица 8.3 – Типы инструкций RISC-V

Тип инструкции	Назначение	Пример использования
R-тип	Арифметические и логические операции с регистрами	add x1, x2, x3
I-тип	Команды с непосредственными значениями и операции загрузки	addi x1, x2, 10; lw x1, 0(x2)
S-тип	Сохранение данных в память	sw x1, 0(x2)
U-тип	Работа с константами большой разрядности	lui x1, 0x1000
J-тип	Безусловные переходы	jal x1, offset
B-тип	Условные переходы	beq x1, x2, offset

8.3. Эмулятор процессора RISC-V

Для работы с системами на RISC V существуют следующие популярные решения:

1. Компьютер RISC-V на базе Linux, такой как Starfive Visionfive. Процессор на плате Visionfive 2 поддерживает набор расширений RISC-V под название RV64GC.
2. Эмулятор RISC-V, работающий на компьютере под управлением Windows или Linux на базе Intel/AMD.
3. Микроконтроллер на базе RISC-V: таких микроконтроллеров существует много, например, Espressi ESP32-C3 DevKit. В таком случае все необходим компьютер для компиляции программы и загрузки ее на микроконтроллер.

Существует множество других возможностей, которые тоже работают, но для их использования необходима адаптация инструкции для поддерживаемых систем.

Рассмотрим установку эмулятора на ОС Windows и Linux. Для эмулирования работы системы будет использоваться гипервизор QEMU и linux система на базе RISC V.

8.3.1 Установка эмулятора в ОС Windows

Вначале необходимо установить эмулятор QEMU в операционной системе Windows через установочный файл программы с официального сайта проекта. После загрузки установщика его нужно запустить и подтвердить установку в окне контроля учетных записей Windows. Программа установится в каталог “C:\Program Files\Qemu”, если оставить путь по умолчанию (рис. 8.1 – 8.2).

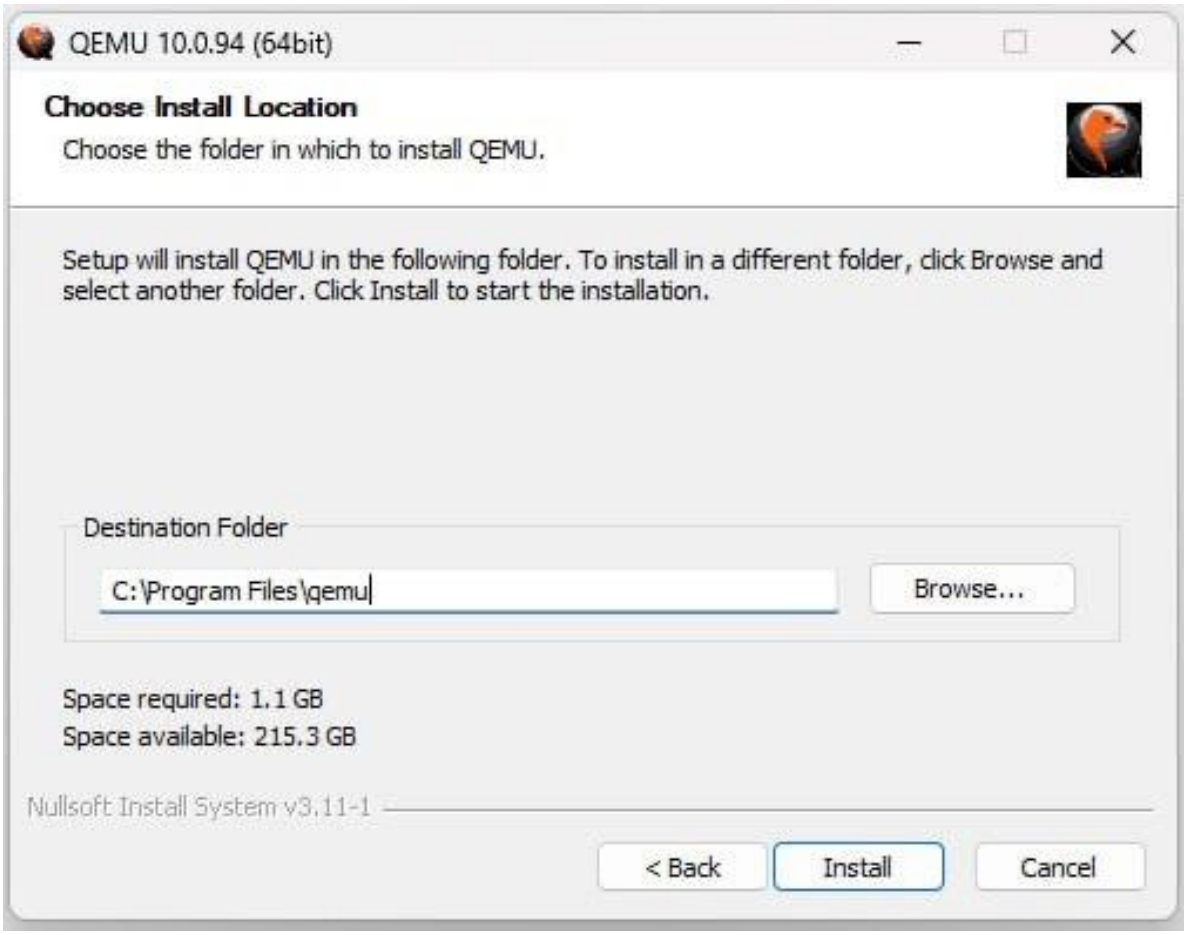


Рисунок 8.1. Выбор директории для установки

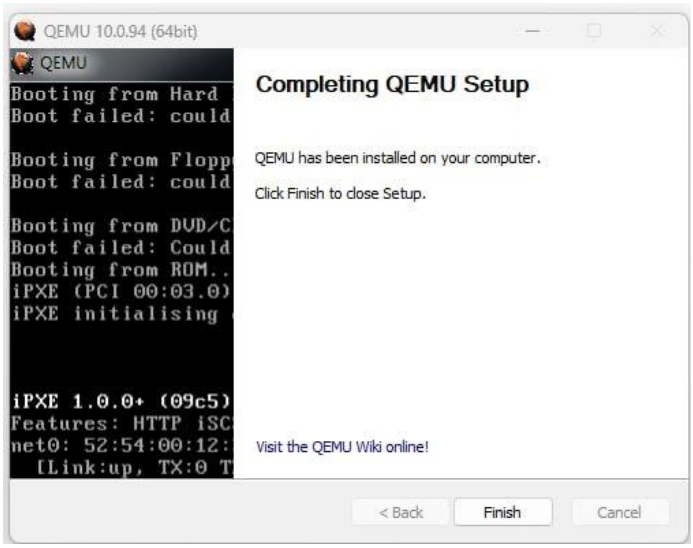


Рисунок 8.2. Завершение установки гипервизора

После установки требуется добавить каталог с исполняемыми файлами QEMU в системный путь. Для этого в свойствах системы открывается раздел «Переменные среды», где в переменной “PATH” указывается путь к папке установки QEMU. Это позволит вызывать команды эмулятора из любой директории командной строки – рис. 8.3.

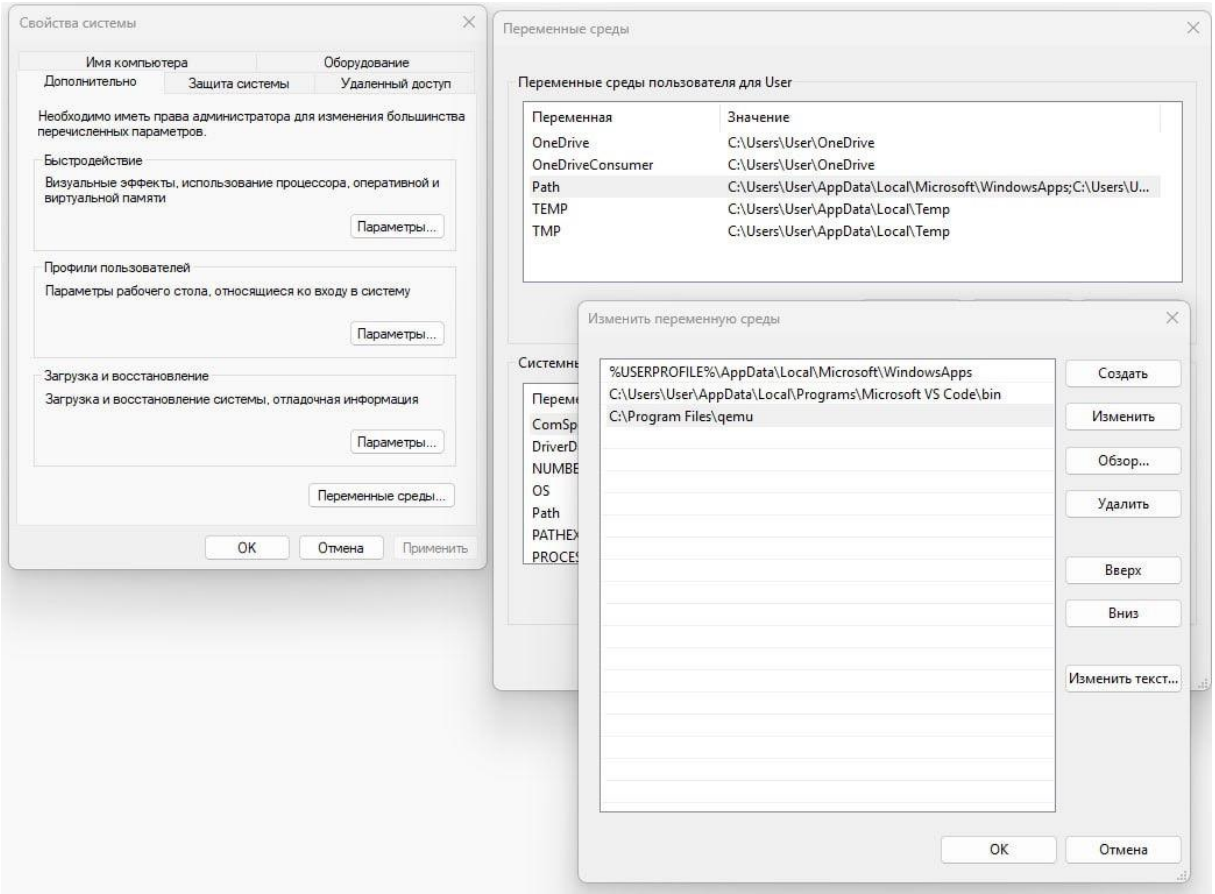


Рисунок 8.3. Установка переменной среды

Далее загружается готовый образ Ubuntu Server для архитектуры RISC-V. Образы можно найти на официальном сайте Ubuntu в разделе загрузок. Файл распространяется в формате xz, поэтому для его распаковки потребуется программа 7-Zip или другой архиватор, поддерживающий данный формат. После извлечения архива получится файл дискового образа с расширением .img – рис. 8.4 – 8.5.

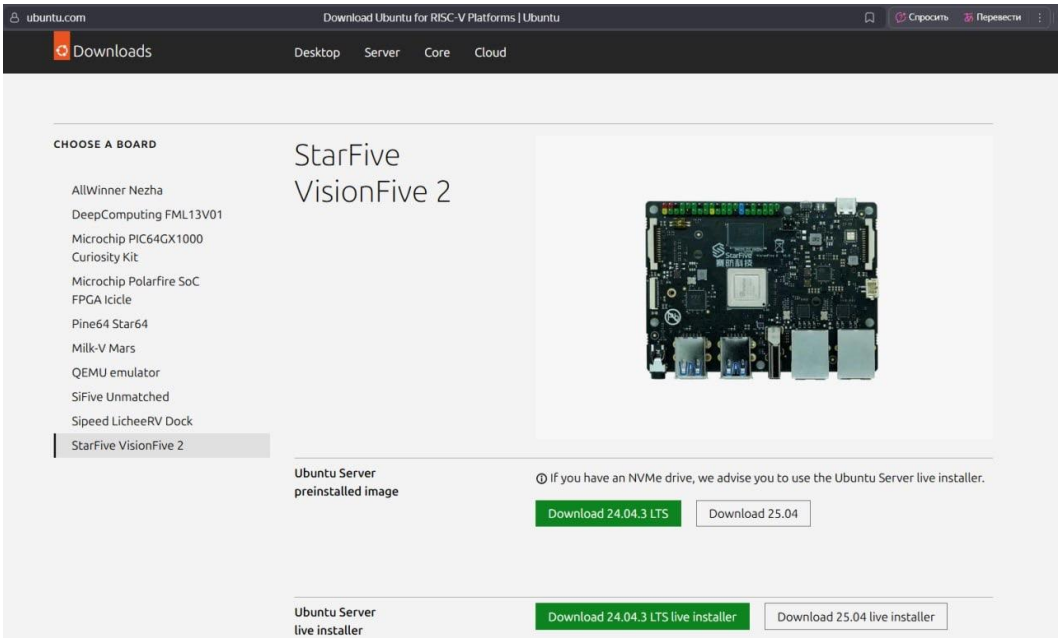


Рисунок 8.4. Образ на официальном сайте Ubuntu

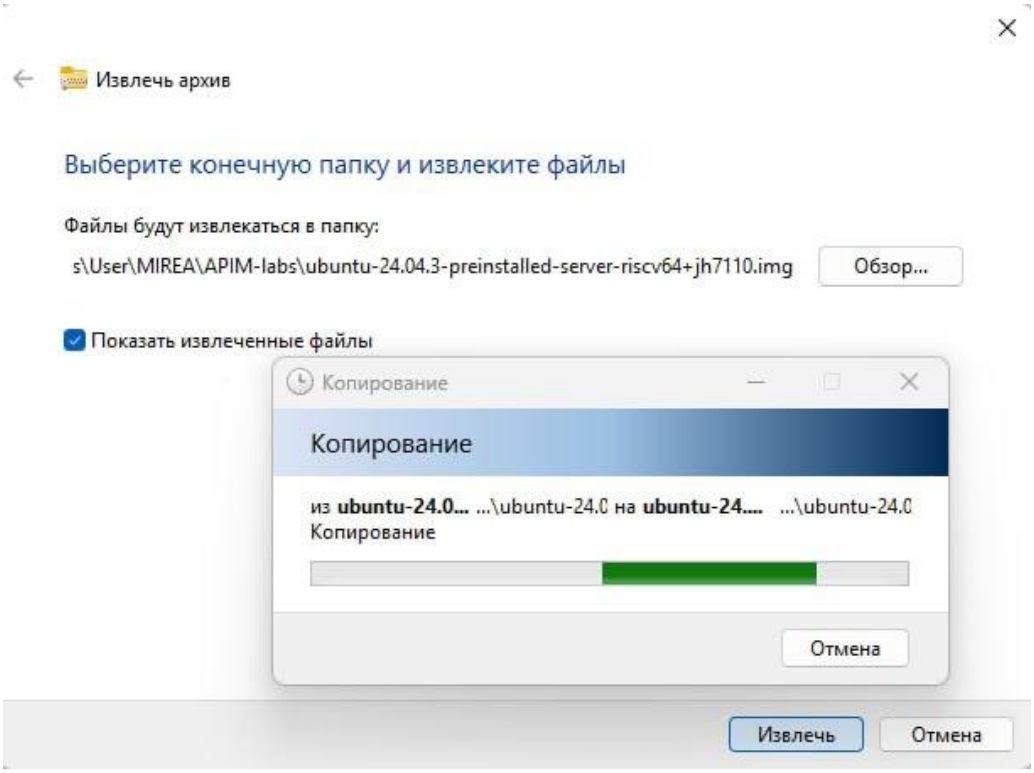


Рисунок 8.5. Распаковка образа

Для удобства дальнейшей работы этот образ рекомендуется переименовать, например, в ubuntu-24.04.3.img. Такое имя проще использовать в командах запуска. После этого к образу можно добавить дополнительное пространство, чтобы внутри системы хватало места для установки программ – рис. 8.6. В результате размер виртуального диска увеличится на 10 гигабайт.

```
PS C:\Users\User\MIREA\APIM-labs> qemu-img.exe resize .\ubuntu-24.04.3.img +10G
WARNING: Image format was not specified for '.\ubuntu-24.04.3.img' and probing guessed raw.
         Automatically detecting the format is dangerous for raw images, write operations on block 0 will be restricted.
         Specify the 'raw' format explicitly to remove the restrictions.
Image resized.
PS C:\Users\User\MIREA\APIM-labs>
```

Рисунок 8.6. Расширения размера диска образа ОС

Чтобы запустить систему, потребуется два вспомогательных файла – fw_jump.elf и uboot.elf. Они выполняют роль BIOS и загрузчика для эмулируемой платы и обеспечивают корректный старт операционной системы. Файлы нужно сохранить в ту же директорию, где расположен образ. На рис. 8.7 показаны необходимые файлы для запуска системы.

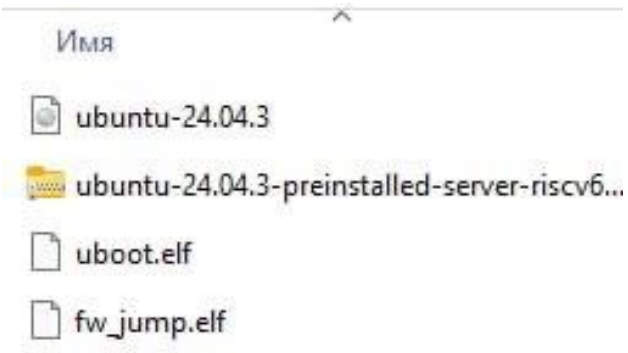


Рисунок 8.7. Исходная директория проекта

Запуск осуществляется при помощи команды qemu-system-riscv64 с набором параметров, определяющих виртуальную машину. Указываются модель процессора rv64, объём памяти в 2 ГБ, подключение дискового образа, виртуальный сетевой адаптер с пробросом порта для SSH-доступа, а также файлы fw_jump.elf и uboot.elf для инициализации системы. В командной строке Windows это выглядит так как показано на рис. 8.8. Команда для запуска: “qemu-system-riscv64 -machine virt -cpu rv64 -m 2G -device virtio-blk-device,drive=hd -drive file=ubuntu-24.04.3.img,if=none,id=hd -device virtio-net-device,netdev=net -netdev user,id=net,hostfwd=tcp::2222-:22 -bios fw_jump.elf -kernel uboot.elf -append "root=LABEL=rootfs console=ttyS0" -nographic”.

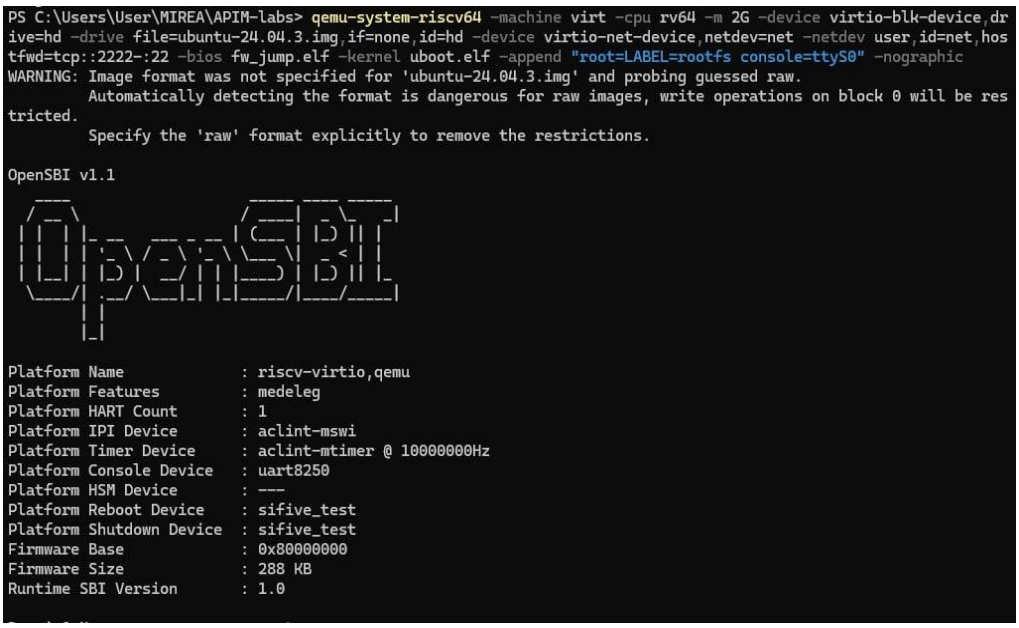


Рисунок 8.8. Запуск эмулятора системы RISC V

После выполнения этой команды эмулятор запустит виртуальную машину. Однако в командной строке Windows могут возникнуть проблемы с отображением символов и работой клавиш. Поэтому для входа в систему рекомендуется использовать SSH-подключение. Для этого в отдельной консоли выполняется команда – рис. 8.9. Команда: “ssh -p 2222 ubuntu@localhost”, при первом входе система может запросить смену пароля, пароль по умолчанию ubuntu. После этого появится приглашение на ввод пароля, и можно будет работать с системой Ubuntu, запущенной под управлением QEMU.

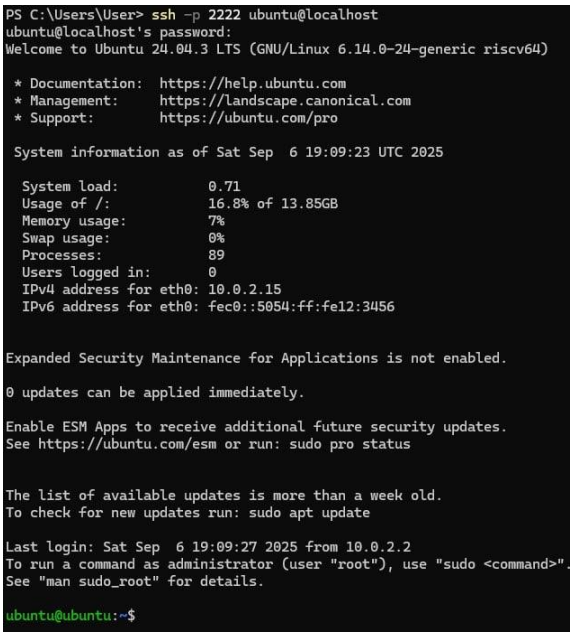


Рисунок 8.9. Подключение к эмулируемой системе RISC V

8.3.2 Установка эмулятора на дистрибутивах Linux

Первым шагом необходимо установить сам эмулятор и необходимые загрузочные компоненты. В дистрибутивах семейства Ubuntu это выполняется при помощи пакетного менеджера apt. В терминале нужно выполнить команды для установки необходимых пакетов – Листинг 8.1.

Листинг 8.1 – Установка необходимых пакетов

```
sudo apt install qemu-system-riscv64
sudo apt install u-boot-qemu opensbi
```

Эти пакеты обеспечат наличие эмулятора QEMU, загрузчика U-Boot и OpenSBI, которые необходимы для корректной инициализации виртуальной системы.

Далее необходимо загрузить предустановленный образ Ubuntu Server для архитектуры RISC-V. На официальном сайте Ubuntu в разделе загрузок можно найти актуальные версии таких образов. После загрузки пользователь получает файл с расширением .img.xz, который упакован в формате xz. Для распаковки образа используется утилита unxz (Листинг 8.2). В результате команда извлекает содержимое архива и создаст файл ubunturv.img, который будет использоваться в качестве виртуального жесткого диска.

После распаковки рекомендуется увеличить размер виртуального диска (Листинг 8.2), чтобы в дальнейшем можно было устанавливать дополнительные программы внутри системы, команда аналогична для запуска в Windows.

Листинг 8.2 – Распоковка кода и увеличение диска

```
unxz <имя_скачанного_файла.img.xz> ubunturv.img
sudo qemu-img resize ubunturv.img +10G
```

Теперь можно переходить к запуску виртуальной машины. Для этого в QEMU указывается модель процессора, объём оперативной памяти, виртуальные устройства, путь к образу, а также файлы OpenSBI и U-Boot, которые играют роль прошивки и загрузчика. В Linux они устанавливаются автоматически вместе с пакетами и располагаются в системных каталогах. Команда для запуска показана в Листинге 8.3.

Листинг 8.3 – Команда для запуска системы в дистрибутивах Linux

```
qemu-system-riscv64 \
-machine virt \
-cpu rv64 \
-m 2G \
-device virtio-blk-device,drive=hd \
-drive file=ubunturv.img,if=none,id=hd \
-device virtio-net-device,netdev=net \
-netdev user,id=net,hostfwd=tcp::2222-:22 \
-bios /usr/lib/riscv64-linux-gnu/opensbi/generic/fw_jump.elf \
-kernel /usr/lib/u-boot/qemu-riscv64_smode/uboot.elf \
-object rng-random,filename=/dev/urandom,id=rng \
-device virtio-rng-device,rng=rng \
-append "root=LABEL=rootfs console=ttyS0" \
-nographic
```

После ввода этой команды QEMU запустит виртуальную систему Ubuntu. В консольном режиме могут быть ограничения по работе с клавишами, поэтому для удобного входа в систему можно использовать SSH. В результате откроется сессия с полноценной поддержкой клавиш управления, и можно будет продолжить работу с виртуальной машиной.

8.4. Создание первой программы на системе RISC-V

Рассмотрим простой пример первой программы на языке ассемблера RISC-V, которая выводит на экран строку «Hello RISC-V World!». В Листинге 8.4 представлена код программа hello_world.S, выполняемой в среде Linux. Программа демонстрирует базовый принцип взаимодействия с системными вызовами операционной системы: используется вызов write для вывода текста и вызов exit для завершения работы. Важно отметить, что после знака # могут располагаться комментарии, поясняющие действия инструкций.

Для того чтобы программа могла быть собрана и запущена, требуется использовать компоновочный файл Листинг 8.5. В этом файле содержатся команды ассемблирования и компоновки: сначала исходный текст на ассемблере преобразуется в объектный файл hello_world.o, а затем с помощью компоновщика формируется исполняемый файл hello_world. Предварительно необходимо установить дополнительные пакеты командой “sudo apt install binutils -y”

После сохранения компоновочного файла необходимо выдать ему права на выполнение с помощью команды “chmod +x build.sh”. Таким образом,

компоновочный файл становится самостоятельным скриптом, который можно запускать напрямую.

Листинг 8.4 – Код программы hello_world на ассемблере

```
#
# Программа на ассемблере RISC-V для вывода строки
# «Hello RISC-V World!» в поток стандартного вывода
#
# a0-a2 - параметры для системных вызовов Linux
# a7 - номер системного вызова Linux
#
.global _start  # Указываем точку входа программы для компоновщика
# Настраиваем параметры для вывода строки
# и вызываем системный вызов Linux.
_start: addi    a0, x0, 1  # a0 = 1  - стандартный вывод (stdout)
     la   a1, helloworld  # a1 = адрес строки “Hello RISC-V World!”
     addi  a2, x0, 20      # a2 = длина строки (20 символов)
     addi  a7, x0, 64      # a7 = 64 - системный вызов write
     ecall                 # выполняем системный вызов (печать строки)
# Настраиваем параметры для завершения программы
# и вызываем системный вызов Linux.
     Addi  a0, x0, 0  # a0 = 0 - код возврата (успешное завершение)
     addi  a7, x0, 93 # a7 = 93 - системный вызов exit
     ecall            # выполняем системный вызов (выход из программы)
.data
helloworld:     .ascii “Hello RISC-V World!\n”  # строка для вывода
```

Листинг 8.5 – Файл сборки проекта

```
#!/bin/bash
as -o HelloWorld.o HelloWorld.S
ld -o HelloWorld HelloWorld.o
```

Сборка программы осуществляется выполнением файла скрипта /build.sh. В результате будет создан исполняемый файл с именем hello_world, который можно запустить отдельной командой. На Рисунке показан результат выполнения программы: строка «Hello RISC-V World!» корректно выводится на стандартное устройство вывода.

На рис. 8.10 показаны этапы создания файлов программы и сборки, их запуск, демонстрация результата.

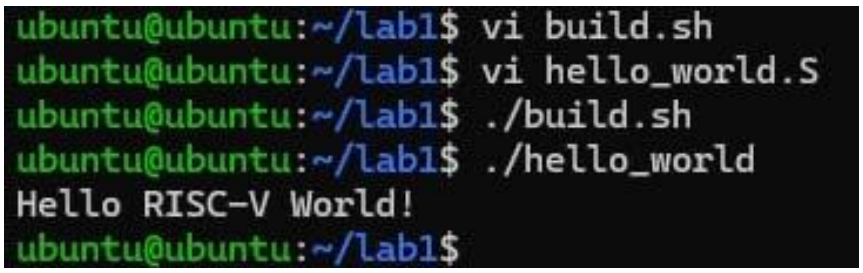


Рисунок 8.10. Сборка и выполнение программы hello_world

Таким образом, данный пример (Листинги 8.4-8.5, а также рис. 8.10) демонстрирует минимальный цикл программирования для архитектуры RISC-V: написание программы, её трансляция и компоновка, а затем выполнение с выводом результата. Этот шаг является основой для последующих работ, где будут изучаться более сложные инструкции ассемблера и взаимодействие с операционной системой Linux.

8.5. Инструменты разработки

8.5.1 GNU Make

В предыдущем разделе для компоновки программы на ассемблере использовался простой скрипт оболочки, который поочередно вызывал ассемблер и компоновщик. Такой способ подходит для минимальных примеров, но при росте числа исходных файлов становится неудобным. В Linux для автоматизации процесса сборки широко применяется утилита GNU Make, которая устанавливается по умолчанию во многих дистрибутивах, для самостоятельной установки можно использовать команду “sudo apt install make -y”.

Основная задача GNU Make заключается в том, чтобы описать правила преобразования одних файлов в другие и зависимости между ними. Каждый проект содержит целевые объекты (например, исполняемую программу), а также список файлов, от которых они зависят (исходный код, объектные файлы). При запуске команды make программа автоматически проверяет даты модификации файлов и определяет, какие части проекта требуют пересборки. Это избавляет программиста от необходимости вручную указывать все команды для каждой перекомпоновки.

Рассмотрим пример программы hello_world, представленной ранее (Листинг 8.4). В Листинге 8.6 показан простой файл makefile, который описывает правила преобразования исходного файла в объектный, а затем объектного файла – в исполняемый.

Листинг 8.6 – Содержимое makefile

```
hello_world: hello_world.o
     ld -o hello_world hello_worLd.o
hello_world.o: hello_world.s
     as -o hello_world.o hello_world.s
```

Подстановочный знак %.S обозначает любой исходный файл с расширением .S, символ $< указывает на этот исходный файл, а $@ заменяется на имя выходного файла. Благодаря таким обозначениям можно избежать дублирования команд и описывать универсальные правила компоновки.

Важно, чтобы строки с командами начинались с символа табуляции, а не с пробелов. Если исполняемый файл уже существует и свежее, чем объектный и исходный файлы, то команда make ничего не выполнит. Для принудительной пересборки используется флаг -B, для передачи makefile'а с именем отличным от "makefile" используется флаг -f.



Рисунок 8.11. Использование make

Чтобы избежать дублирования команд, можно использовать правила компоновки. В Листинге 8.7 приведен вариант makefile, где определено общее правило преобразования файлов .S в .o. Теперь при сборке make автоматически подставляет нужные параметры для любого исходного файла на ассемблере. Результат показан на рис. 8.12.

Листинг 8.7 – Содержимое makefile

```
%.o : %.S
     as $< -o $@
hello_world: hello_world.o
     ld -o hello_world hello_world.o
```

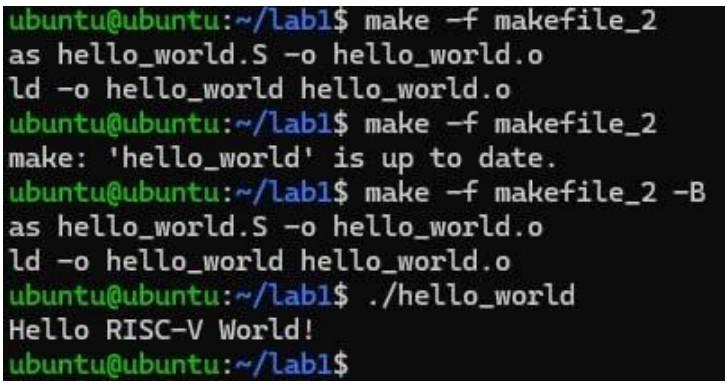


Рисунок 8.12. Использование make с заданным набором правил

В более сложных проектах применяются переменные. В Листинг 8.8 показано, как вынести список объектных файлов в отдельную переменную OBJS. Это упрощает поддержку проекта: при добавлении новых исходников их достаточно указать только в одной строке – рис. 8.13.

Листинг 8.8 – Содержимое makefile с добавлением переменной

```
OBJS = hello_world.o
%.o : %.S
      as $< -o $@
hello_world: $(OBJS)
      ld -o hello_world $(OBJS)
```

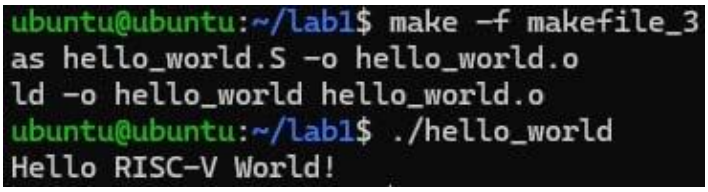


Рисунок 8.13. Использование make с переменной имени файла

Таким образом, использование GNU Make позволяет организовать централизованное управление процессом сборки программ. Этот инструмент автоматизирует рутинные операции, уменьшает вероятность ошибок и облегчает сопровождение кода. В последующих лабораторных работах файлы make будут усложняться: появятся новые параметры компиляции, работа с несколькими исходными файлами и оптимизационные флаги.

8.5.2 GNU CMake

CMake – это кроссплатформенный инструмент автоматизации сборки, который не зависит от конкретного компилятора. Его основная задача заключается в том, чтобы скрыть от разработчика все технические сложности,

связанные с настройкой и использованием различных цепочек инструментов на Linux, Windows или macOS. Для сборки достаточно написать один файл CMakeLists.txt, и на его основе CMake создаст корректный Makefile, который затем выполнит GNU Make. Таким образом, программисту не нужно вручную описывать правила компиляции для каждого файла, как это делается в Make. Рассмотрим на примере ранее написанной программы hello world. CMakeLists.txt в корне проекта показан в Листинге 8.9.

Первая строка указывает минимальную версию CMake, совместимую с проектом. Это важно, так как SDK проверяет версии инструментов, чтобы исключить ошибки сборки. Вторая строка подключает специальный файл project.cmake, который содержит правила сборки и настройки для конкретного процессора, третья – задает имя проекта, в нашем случае – hello_world.

Листинг 8.9 – Содержимое CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(hello_world ASM)
# Добавляем исполняемый файл из исходника на ассемблере
add_executable(hello_world hello_world.S)
# Указать линкеру не использовать стандартные стартовые файлы
target_link_options (hello_world PRIVATE -nostartfiles)
```

В подпапке проекта находится ещё один файл CMakeLists.txt, в котором перечисляются исходные файлы, которые должны быть собраны – Листинг 8.10.

Листинг 8.10 – Содержимое CMakeLists.txt

```
idf_component_register(SRCS "hello_world_main.S"
 INCLUDE_DIRS ".")
```

Здесь с помощью команды idf_component_register указываются исходники проекта. В нашем случае это файл на языке ассемблера hello_world_main.S. Если добавить файлы на C или C++, то CMake автоматически подберет нужные правила компиляции по расширению файлов. Это делает систему гибкой и масштабируемой: для добавления нового исходника достаточно вписать его название в список, а всё остальное выполнит CMake.

На рис. 8.14 – 8.15 показано содержимое директории проекта под сборку с использованием CMake и сама сборка с последующим запуском программы.

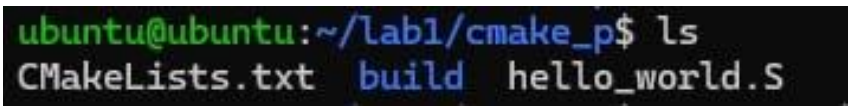


Рисунок 8.14. Директория проекта с исходными файлами

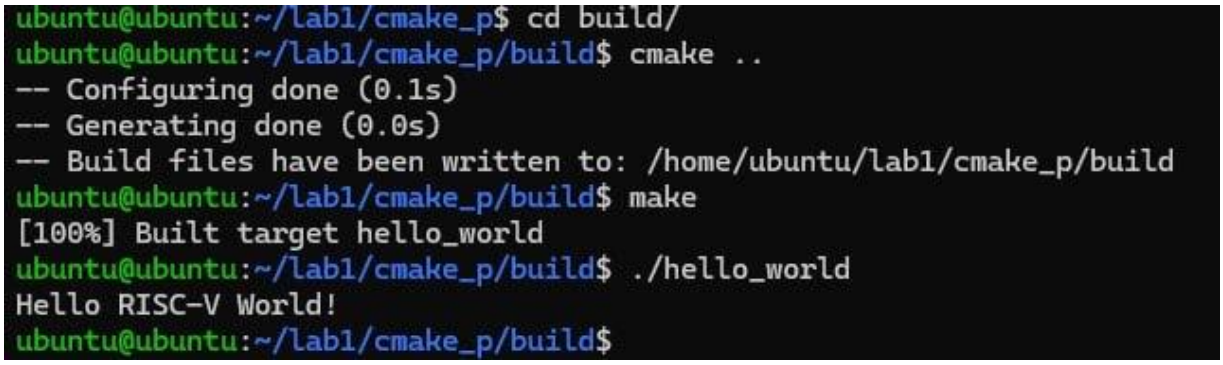


Рисунок 8.15. Сборка и запуск проекта

Таким образом, отличие CMake от Make в том, что разработчик описывает только что должно быть собрано, а не как именно. Внутренние правила компиляции и линковки подбираются автоматически. Например, добавив строку с файлом test.c, можно собрать проект из ассемблерных и C-файлов без ручного написания команд as или gcc.

8.5.3 Отладчик gdb

Большинство языков высокого уровня содержат встроенные средства разработки, которые позволяют легко выводить строковые литералы и числа на консоль, в окно или веб-страницу. Разработчики при этом часто полагаются на стандартные библиотеки языка и редко используют отладчик напрямую. Однако при программировании на ассемблере ситуация иная: здесь важно уметь работать с отладчиком, поскольку он помогает не только находить ошибки, но и глубже понимать работу программы на уровне регистров и инструкций.

Для архитектуры RISC-V важную роль играет знание регистров общего назначения. В ассемблере они имеют имена от x0 до x31, а в отладчике gdb (установить можно командой “sudo apt install gdb”) каждому из них сопоставлено альтернативное имя ABI, которое отражает их назначение в вызовах функций. Например, регистр x10 обозначается как a0 и используется для передачи первого аргумента функции или возврата значения. В табл. 8.4 представлены основные регистры RISC-V и их альтернативные имена.

Таблица 8.4 – Таблица основных регистров

Регистр	**Имя ABI**	**Назначение**
x0	zero	Нулевой регистр
x1	ra	Адрес возврата
x2	sp	Указатель стека
x3	gp	Глобальный указатель
x4	tp	Указатель потока (thread)
x5	t0	Временный регистр 0
x6	t1	Временный регистр 1
x7	t2	Временный регистр 2
x8	s0/fp	Сохраняемый регистр 0 / указатель фрейма
x9	s1	Сохраняемый регистр 1
x10	a0	0-й аргумент функции / 0-е возвращаемое значение
x11	a1	1-й аргумент функции / 1-е возвращаемое значение
x12	a2	2-й аргумент функции
x13	a3	3-й аргумент функции
x14	a4	4-й аргумент функции
x15	a5	5-й аргумент функции
x16	a6	6-й аргумент функции
x17	a7	7-й аргумент функции
x18–x27	s2–s11	Сохраняемые регистры
x28	t3	Временный регистр 3
x29	t4	Временный регистр 4
x30	t5	Временный регистр 5
x31	t6	Временный регистр 6

Для примера рассмотрим простую программу (Листинг 8.11), которую можно использовать для практики работы с отладчиком gdb. Она выполняет базовые арифметические операции и завершает работу через системный вызов exit.

Листинг 8.11 – Пример программы для работы с отладчиком

```
# Простая программа для отладки в gdb
.global _start             # Указываем точку входа
_start:
     li    x5, 0x12345     # загрузить x5 значением 0x12345
     addi  x5, x5, 0x678   # прибавить к x5 значение 0x678
     li    x6, 10          # загрузить x6 = 10
     add   x7, x5, x6      # x7 = x5 + x6
     sub   x8, x7, x6      # x8 = x7 - x6
# Завершение программы через системный вызов
     li    a0, 0           # код возврата
     li    a7, 93          # номер системного вызова exit
     ecall                 # вызвать ядро Linux
```

Создадим также makefile для сборки в debug и обычных режимах – Листинг 8.12. В gdb есть следующие полезные команды:

- layout asm – показать ассемблерный код программы;
- break _start – установить точку останова в начале;
- run – запустить выполнение;
- stepi – выполнить одну инструкцию;
- info registers – вывести содержимое регистров;
- x/10x $sp – просмотреть 10 инструкций из памяти по стеку.

Также для получения справки можно вызвать команду “gdb --help”. Результат выполнения команд на основе Листингов 8.10 – 8.11 показан на рис. 8.16 – 8.19.

Листинг 8.12 – Содержимое makefile с режимом отладки

```
OBJS = simple.o
ifdef DEBUG
DEBUGFLAGS = -g
else
DEBUGFLAGS =
endif
%.o: %.S
     as $(DEBUGFLAGS) $< -o $@
simple: $(OBJS)
     ld -o $@ $(OBJS)
clean:
     rm -f $(OBJS) simple
```

```
ubuntu@ubuntu:~/lab1$ make -f makefile_d -B
as  simple.S -o simple.o
ld -o simple simple.o
ubuntu@ubuntu:~/lab1$ make -B -f makefile_d DEBUG=1
as -g simple.S -o simple.o
ld -o simple simple.o
ubuntu@ubuntu:~/lab1$ make -f makefile_d -B
as  simple.S -o simple.o
ld -o simple simple.o
ubuntu@ubuntu:~/lab1$ gdb ./simple
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "riscv64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./simple...
(No debugging symbols found in ./simple)
(gdb) exit
ubuntu@ubuntu:~/lab1$ make -B -f makefile_d DEBUG=1
as -g simple.S -o simple.o
ld -o simple simple.o
ubuntu@ubuntu:~/lab1$ gdb ./simple
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "riscv64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./simple...
(gdb)
```

Рисунок 8.16. Сборка в debug и обычном режиме с запуском gdb

```
   0x100b0 <_start>        lui     t0,0x12
B+ 0x100b4 <_start+4>      addiw   t0,t0,837
  >0x100b8 <_start+8>      addi    t0,t0,1656
   0x100bc <_start+12>     li      t1,10
   0x100c0 <_start+16>     add     t2,t0,t1
   0x100c4 <_start+20>     sub     s0,t2,t1
   0x100c8 <_start+24>     li      a0,0
   0x100cc <_start+28>     li      a7,93
   0x100d0 <_start+32>     ecall
   0x100d4                 .insn   2, 0x3641
   0x100d6                 .insn   2, 0x
   0x100d8                 .insn   2, 0x7200
   0x100da                 .insn   2, 0x7369
   0x100dc                 bgeu    zero,a6,0x100e8
   0x100e0                 .insn   2, 0x002c
   0x100e2                 .insn   2, 0x
   0x100e4                 .insn   2, 0x7205
native process 1258 (asm) In: _start                                L??   PC: 0x100b8
Breakpoint 1 at 0x100b4
Note: breakpoint 1 also set at pc 0x100b4.
Breakpoint 2 at 0x100b4
(gdb) run
Starting program: /home/ubuntu/lab1/simple

Breakpoint 1, 0x0000000000100b4 in _start ()
(gdb) stepi
0x0000000000100b8 in _start ()
(gdb)
```

Рисунок 8.17. Вывод команд layout asm & run

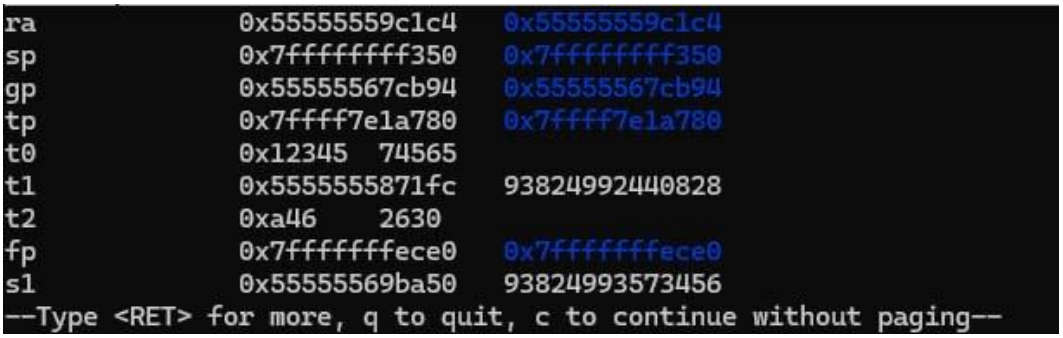


Рисунок 8.18. Вывод состояния регистров

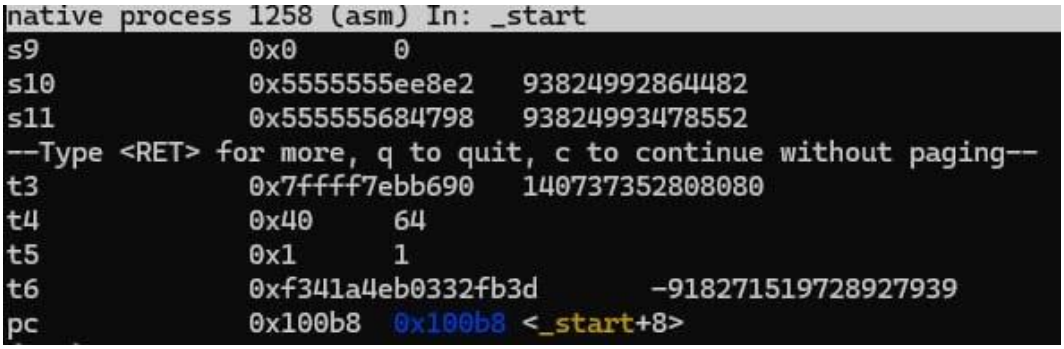


Рисунок 8.19. Вывод состояния регистров продолжение



Рисунок 8.20. Просмотр инструкций из памяти стека

Рассмотрим подробнее возможности отладчика gdb (рис. 8.21 – 8.24) на примере программы из Листинга 8.10. В табл. 8.5 и 8.6 отражены основные команды управления и команды анализа состояния. Также рассмотрим некоторые условные обозначения.

Форматы вывода памяти:

- t – двоичный;
- x – шестнадцатеричный;
- d – десятичный;
- i – инструкция;
- s – строка.

Размеры единиц:

- b – байт;
- h – полуслово (16 бит);
- w – слово (32 бита);
- g – двойное слово (64 бита).

Таблица 8.5 – Основные команды управления

Команда (краткая форма)	Назначение
break <line> или b	Установить точку останова
run или r	Запустить выполнение программы
step или s	Пошаговое выполнение (по одной инструкции)
continue или c	Продолжить выполнение до следующей точки останова
quit или q	Выйти из gdb
Ctrl+C	Прервать выполнение программы

Таблица 8.6 – Команды анализа состояния

Команда (краткая форма)	Назначение
info registers или i r	Просмотреть содержимое регистров
info breakpoints или i b	Просмотреть список точек останова
delete <n>	Удалить точку останова по номеру
disassemble <label>	Дизассемблировать код от метки
x/Nfu <addr>	Просмотреть память (N – количество, f – формат, u – размер единицы)
info registers или i r	Просмотреть содержимое регистров

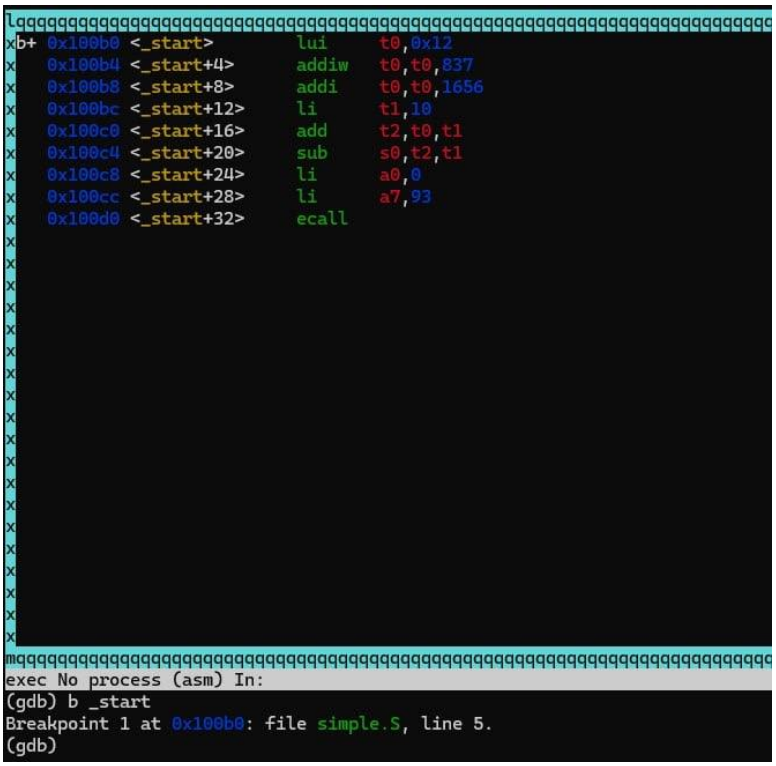


Рисунок 8.21. Установка точки начала программы на метку _start

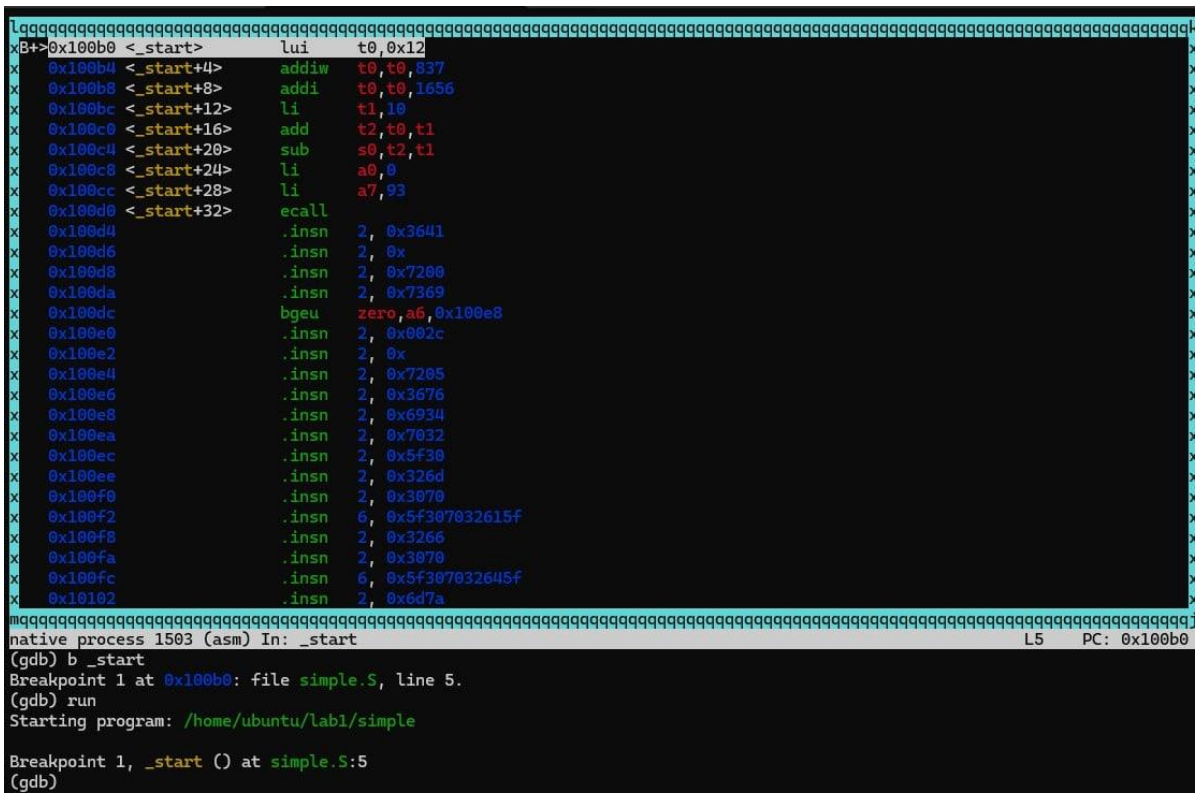


Рисунок 8.22. Запуск выполнения программы

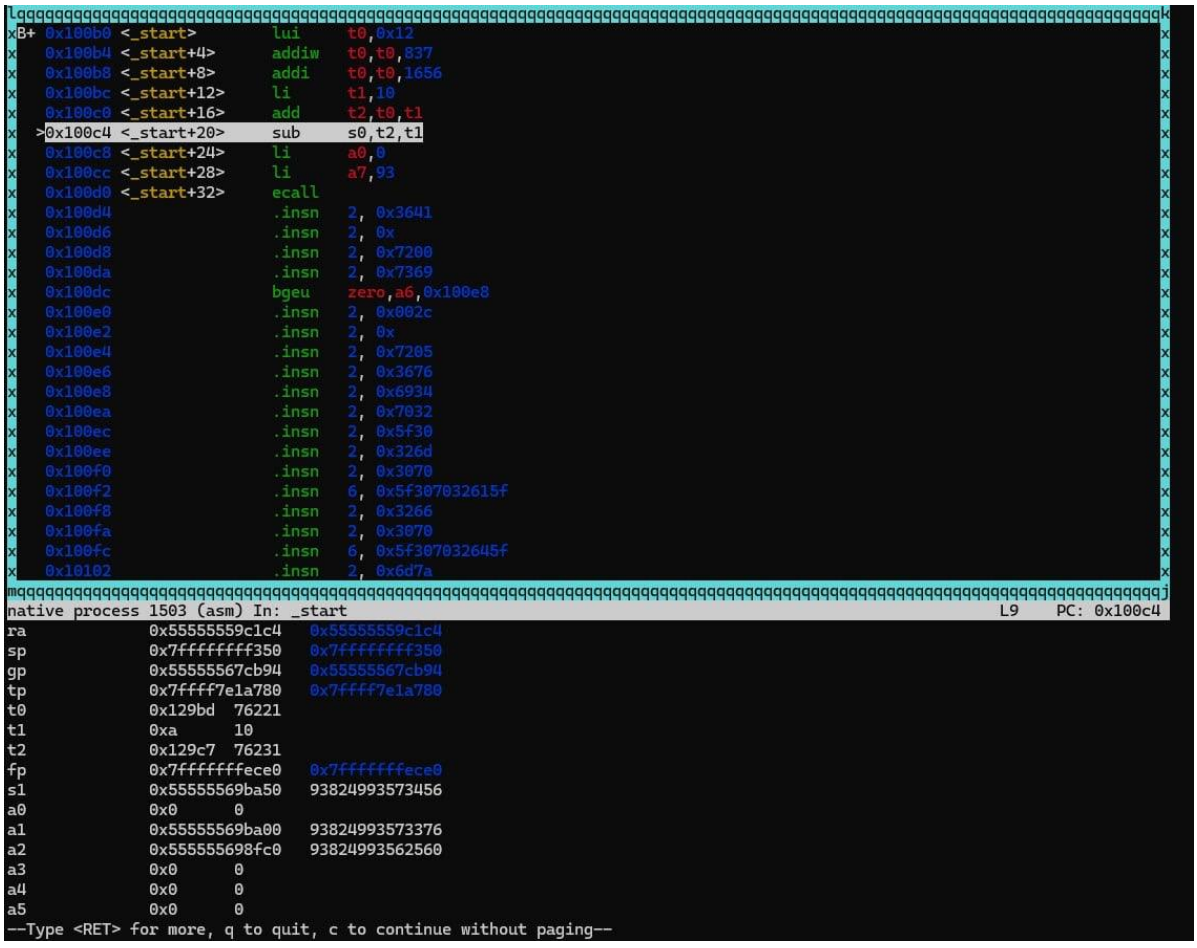


Рисунок 8.23. Выполнение нескольких шагов и вывод содержимого регистров

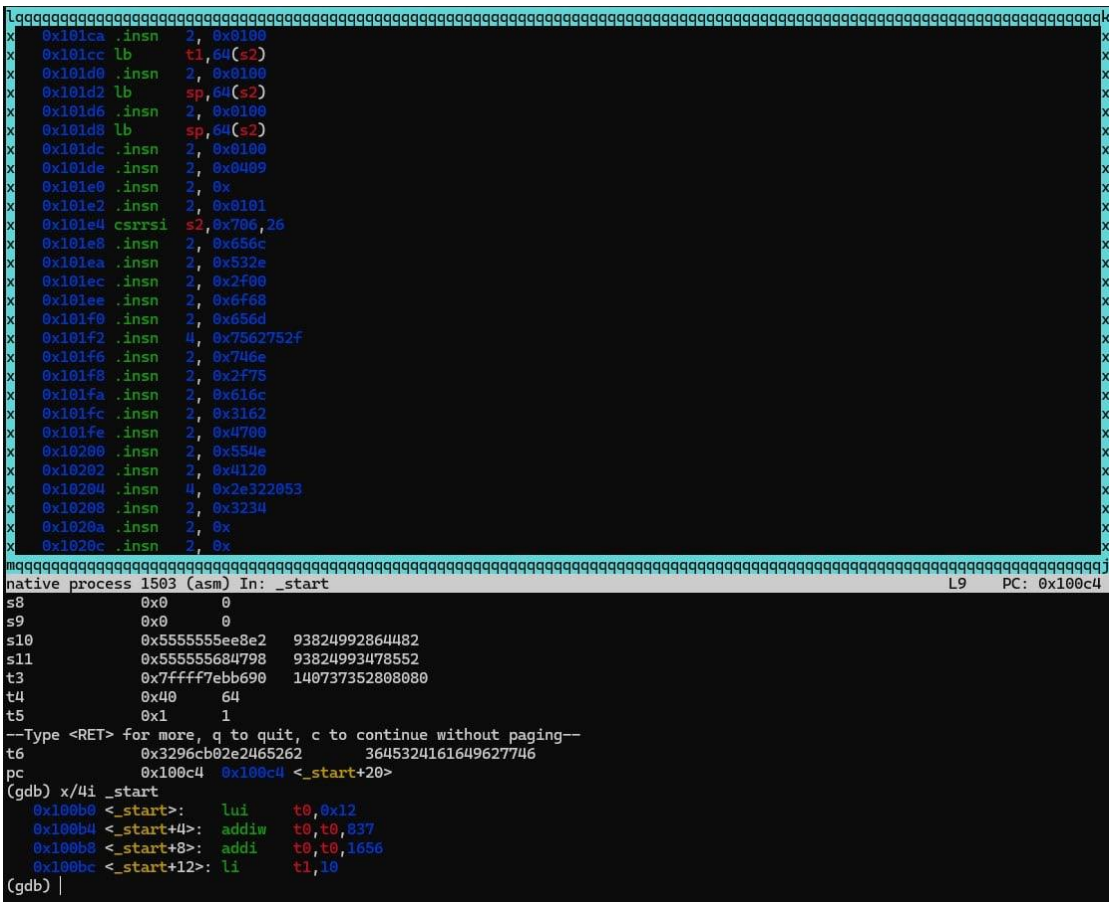


Рисунок 8.24. Просмотр 4 записей в памяти

Таким образом, GNU Debugger позволяет детально отслеживать выполнение программы на ассемблере. С его помощью можно пошагово выполнять инструкции, контролировать состояние регистров, исследовать память и управлять точками останова. Это делает gdb незаменимым инструментом при изучении архитектуры процессора и отладке низкоуровневого кода.

8.6. Заключение

Выполнение практической работы позволило закрепить знания об архитектуре набора команд RISC-V, её модульной структуре и принципах организации базовых и расширенных наборов инструкций. Студент познакомился с процессом настройки среды разработки на основе эмулятора QEMU, освоил создание и запуск первой программы на языке ассемблера RISC-V, реализующей вывод строки через системные вызовы Linux.

Приобретённые навыки работы с инструментами сборки GNU Make и CMake, а также с отладчиком GDB имеют ключевое значение для дальнейшего

программирования на ассемблере. Умение организовать процесс компиляции и компоновки с помощью makefile, а также пошагово отслеживать выполнение инструкций, анализировать содержимое регистров и памяти позволяет эффективно разрабатывать и отлаживать низкоуровневый код.

Таким образом, выполненная работа формирует базовые компетенции для дальнейшего изучения программирования на ассемблере RISC-V, взаимодействия с операционной системой Linux и понимания внутреннего устройства вычислительных систем на уровне машинных команд.

8.7. Варианты заданий

В рамках практической работы необходимо выполнить:

1. Настроить среду для дальнейшей разработки под RISC V.
2. Изменить текстовую строку в примере программы из пункта 1.4 в формате <ФИО_номер группы_номер в списке группы>.
3. Настроить и скомпоновать программу, написанную в предыдущем пункте.
4. Изменить исходный код программы так, чтобы в качестве точки и глобального определения использовать “main_app” вместо “_start” и заменить исходный код выхода инструкцией “j main_app”. Скомпоновать и выполнить в пошаговом режиме с демонстрацией содержимого регистров.
5. Создать makefile под каждую из двух версии программы с поддержкой релизной (финальной), отладочной версии программы.
6. Продемонстрировать каждый шаг преподавателю и отразить результат выполнения в итоговом отчете.

9. ВЗАИМОДЕЙСТВИЕ АССЕМБЛЕРА И ВЫСОКОУРОВНЕВЫХ ЯЗЫКОВ

9.1. Введение

В предыдущей практической работе была реализована простая программа вывода строки и завершение работы, что в Linux выполнялось через прямые системные вызовы. При работе на высокоуровневых языках программист обычно не сталкивается с ними напрямую: стандартные библиотеки среды выполнения предоставляют удобные обёртки и создают впечатление, что операции ввода-вывода являются частью языка.

В этой практической работе будет рассмотрено использование таких библиотек, правила вызова функций операционной системы, обработка кодов ошибок и применение компилятора GNU C для более удобного обращения к системным вызовам. Ранее многие приложения писались полностью на ассемблере ради максимальной производительности – например, VisiCalc или первые видеоигры. Сегодня основная часть программ реализуется на высокоуровневых языках, а ассемблер используется точечно – для оптимизации критически важных участков и взаимодействия с аппаратурой.

Таким образом, цель текущей работы – освоить механизм взаимодействия программ с ядром Linux через системные вызовы и изучить совместное использование ассемблера и высокоуровневого кода для повышения эффективности программ.

9.2. Системные службы Linux

Linux – мощное и гибкое ядро с более чем 25-летней историей, работающее от встроенных устройств до суперкомпьютеров. Одним из ключевых факторов его успеха является большой набор системных вызовов (более 400), обеспечивающих доступ к службам ядра. Полный их обзор выходит за рамки работы, поэтому здесь рассматриваются только основы: механизм вызова, соглашения и примеры использования.

В предыдущих заданиях использовались лишь два системных вызова: вывод текста и завершение программы. Системные вызовы Linux используют отдельное соглашение, отличное от вызова обычных функций. Обращение к ядру выполняется через инструкцию ecall, которая генерирует контролируемое исключение и передаёт управление ядру.

Параметры размещаются так:

- a0–a6 – до семи аргументов вызова;
- a7 – номер системного вызова;
- ecall – переход в ядро;
- a0 – возвращаемый код (результат или ошибка).

При ошибке возвращается отрицательное значение, соответствующее константам из errno.h (например, EACCES = 13). Если данных много, обычно передают указатель на структуру в памяти.

Каждый системный вызов Linux имеет свой уникальный номер. Например, номер вызова для выхода из программы – 93, а для записи в файл – 64 (иногда встречается 04 в старых документациях). Использовать такие «магические числа» напрямую неудобно, поэтому в программах принято применять символьные имена, определенные в заголовочных файлах языка С. Номера системных вызовов задаются в файле: "/usr/include/asm-generic/unistd.h". Например, "#define NR_write 64", где NR_write – символическое имя вызова операции записи в файл. Аналогично определяются номера для всех остальных системных вызовов.

Системные вызовы возвращают код результата:

- положительное число или ноль – успешное выполнение;
- отрицательное число – ошибка, соответствующая коду из заголовочного файла:
  - /usr/include/errno.h
  - /usr/include/asm-generic/errno.h
  - /usr/include/asm-generic/errno-base.h

Например, при вызове open для открытия файла:

- В случае успеха возвращается дескриптор файла (небольшое положительное число).
- В случае ошибки возвращается отрицательный код ошибки, соответствующий одной из констант в errno.h.

Для работы с Linux-системными вызовами на ассемблере используются те же константы и символические имена, что и в С. При необходимости параметры передаются через структуры, аналогично функциям среды выполнения С.

Многие службы Linux принимают в качестве параметров указатели на блоки памяти, содержимое которых описывается структурами языка С. Поэтому при работе на ассемблере нужно воспроизводить эти структуры в памяти.

Например, системный вызов nanosleep позволяет программе приостановить выполнение на заданное время – Листинг 9.1. Структура struct timespec показана в Листинге 9.2.

Листинг 9.1 – системный вызов nanosleep

```
int nanosleep(const struct timespec *req, struct timespec *rem);
```

Листинг 9.2 – Структура struct timespec

```
struct timespec {
     time_t tv_sec;   /* секунды */
     long tv_nsec;    /* наносекунды */
};
```

Зная, что это два 64-битных числа, ее можно представить на ассемблере следующим образом – Листинг 9.3. Адреса этих полей загружаются в регистры как первые два параметра системного вызова – Листинг 9.4.

Листинг 9.3 – два 64-битных числа

```
timespecsec:  .dword 0
timespecnano: .dword 100000000
```

Листинг 9.4 – Загрузка двух параметров системного вызова в регистры

```
la a0, timespecsec
la a1, timespecnano
```

Перейдем к оберткам вызовов функций. Чтобы не работать с регистрами и параметрами вручную каждый раз, можно создавать обертки для системных вызовов:

- в языке C уже есть готовые обертки (wrappers) для всех служб Linux, о которых подробно рассказывается в главе 9 «Взаимодействие с языками C и Python»;
- на ассемблере можно сделать собственную библиотеку вызовов, используя макрофункции GNU, но реализовать нужно только те вызовы, которые действительно необходимы.

Такой подход упрощает программирование и позволяет со временем создать удобную коллекцию функций для работы с Linux-службами, не дублируя всю библиотеку среды выполнения C. Рассмотрим программу,

использующую коллекцию служб. Программа для конвертации содержимого текстового файла в верхний регистр. Для этого используется функция toupper, а также применяются циклы и инструкции if для обработки текста. Для работы с файлами необходимо подключить библиотеку файлового ввода-вывода и создать файл fileio.S, содержащий макрофункции для открытия, чтения, записи и закрытия файлов (Листинг 9.5).

Листинг 9.5 – Макрофункции для работы с файлами

```
# Макрофункции для работы с файлами
# fd - регистр с дескриптором файла
# a0-a3, a7 используются для системных вызовов Linux
# Код возврата - в a0

#include <asm/unistd.h>

.equ O_RDONLY, 0
.equ O_WRONLY, 1
.equ O_CREAT,  0100
.equ O_EXCL,   0200
.equ S_RDWR,   0666
.equ AT_FDCWD, -100

.macro  openFile     fileName, flags
     li   a0, AT_FDCWD
     la   a1, \fileName
     li   a2, \flags
     li   a3, S_RDWR      # RW access rights
     li   a7, __NR_openat
     ecall
.endm
.macro  readFile     fd, buffer, length
     mv   a0, \fd# file descriptor
     la   a1, \buffer
     li   a2, \length
     li   a7, __NR_read
     ecall
.endm
.macro  writeFile    fd, buffer, length
     mv   a0, \fd         # file descriptor
     la   a1, \buffer
     li   a2, \length
     li   a7, __NR_write
     ecall
.endm
```

Продолжение листинга 9.5

```
.macro  flushClose    fd
#fsync syscall
     mv    a0, \fd
     li    a7, __NR_fsync
     ecall

#close syscall
     mv    a0, \fd
     li    a7, __NR_close
     ecall
.endm
```

Основная программа, main.S, управляет процессом конвертации. Она открывает входной и выходной файлы, считывает данные, вызывает функцию toupper и записывает результат (Листинг 9.6).

Листинг 9.6 – Главная функция программы

```
# Программа на ассемблере для конвертации строки
# в верхний регистр с помощью вызова функции.
#
# a0-a2, a7 - используются макрофункциями для вызовов Linux
# s1 - дескриптор входного файла
# s2 - дескриптор выходного файла
# t0 - количество прочитанных символов

#include <asm/unistd.h>
#include "fileio.S"

.equ  BUFFERLEN, 250

.global _start              # Provide program starting address to
linker

_start: openFileinFile, O_RDONLY
     mv         s1, a0          # save file descriptor
     bgez       a0, nxtfil      # pos number file opened ok
     li         a1, 1           # stdout
     la         a2, inpErrsz    # Error msg
     lw         a2, 0(a2)
     writeFile  a1, inpErr, a2  # print the error
     j          exit

nxtfil: openFileoutFile, O_CREAT+O_WRONLY
```

Продолжение листинга 9.6

```
     mv         s2, a0          # save file descriptor
     bgez       a0, loop        # pos number file opened ok
     li         a1, 1
     la         a2, outErrsz
     lw         a2, 0(a2)
     writeFile  a1, outErr, a2
     j          exit

# loop through file until done.
loop: readFile   s1, buffer, BUFFERLEN
     mv         t0, a0          # Keep the length read
     li         t1, 0           # Null terminator for string

     # setup call to toupper and call function
     la         a0, buffer      # first param for toupper
     add        a1, a0, t0      # addr to put null, buffer + len
     sb         t1, 0(a1)       # put null at end of string.
     la         a1, outBuf
     jal        toupper
     writeFile  s2, outBuf, t0

     li         t1, BUFFERLEN
     beq        t0, t1, loop

     flushClose s1
     flushClose s2

# Setup the parameters to exit the program
# and then call Linux to do it.
exit: li    a0, 0               # Use 0 return code
     li    a7, __NR_exit
     ecall                     # Call Linux to terminate

.data
inFile:  .asciz  "main.S"
outFile: .asciz  "upper.txt"
buffer:    .fill BUFFERLEN + 1, 1, 0
outBuf:    .fill BUFFERLEN + 1, 1, 0
inpErr: .asciz  "Failed to open input file.\n"
inpErrsz: .word  .-inpErr
outErr:    .asciz     "Failed to open output file.\n"
outErrsz: .word .-outErr
```

Файлы .S компилируются с помощью GCC, а не as, чтобы использовать возможности препроцессора C, включая директивы #include и #define. Это

позволяет подключать стандартные заголовочные файлы, такие как unistd.h – Листинг 9.7.

Листинг 9.7 – Makefile для сборки

```
UPPEROBJS = main.o upper.o

ifdef DEBUG
DEBUGFLGS = -g
else
DEBUGFLGS =
endif

all: upper

%.o : %.S
     gcc -mno-relax $(DEBUGFLGS) -c $< -o $@

%.o : %.s
     as -mno-relax $(DEBUGFLGS) $< -o $@

upper: $(UPPEROBJS)
     ld -o upper $(UPPEROBJS)
```

В этой программе используется файл upper.S, содержащий версию кода конвертации текста в верхний регистр с использованием функции – Листинг 9.8.

Листинг 9.8 – Перевод строки в верхний регистр

```
# Функция на языке ассемблера для конвертации строки
# в верхний регистр.
#
# a1 – адрес выходной строки
# a0 – адрес входной строки
# s0 – исходный адрес выходной строки (для вычисления длины)
# t2 – текущий обрабатываемый символ
# t3 – временный регистр для сравнений

.global mytoupper          # Allow other files to call this routine

toupper:
     addi  sp, sp, -16     # allocate 16 bytes on stack
     sd    ra, 8(sp)       # push return address
     sd    s0, 0(sp)       # push s0 register
     mv    s0, a1          # save original outstr for len calc
# The loop is until null (zero) character is encountered.
loop: lb    t2, 0(a0)       # load character
```

Продолжение листинга 9.8

```
     addi  a0, a0, 1       # increment buffer pointer
# If x7 > 'z' then goto cont
     li    t3, 'z'         # load 'z' for comparison
     bgt   t2, t3, cont    # branch if letter > 'z'?
# Else if x7 < 'a' then goto end if
     li    t3, 'a'         # load 'a' for comparison
     blt   t2, t3, cont    # goto to end if not lowercase
# If we got here then the letter is lower case, so convert it.
     addi  t2, t2, ('A'-'a')
cont:# end if
     sb    t2, 0(a1)       # store character to output str
     addi  a1, a1, 1       # increment buffer for next char
     li    t3, 0           # load 0 char for comparison
     bne   t2, t3, loop    # loop if character isn't null

# Setup the parameters to print our hex number
# and then call Linux to do it.
     sub   a0, a1, s0      # get the len by sub'ing the pointers

     ld    s0, 0(sp)       # pop s0
     ld    ra, 8(sp)       # pop ra
     addi  sp, sp, 16      # deallocate stack space
     ret                   # Return to caller
```

Для компиляции двух файлов .S применяется GCC, а не as. Многие считают GCC компилятором только для языка C, но на самом деле это GNU Compiler Collection – набор компиляторов, поддерживающий несколько языков, включая ассемблер.

Файлы с расширением .S (заглавная буква S) обрабатываются препроцессором C перед ассемблером, что позволяет использовать директивы #include и #define, а также стандартные заголовочные файлы, такие как unistd.h. Это делает номера системных вызовов и константы Linux читаемыми и удобными для работы.

Важно учитывать, что существует потенциальный конфликт между синтаксисом комментариев RISC-V (#) и препроцессором C. Если в файле .S комментарии начинаются с #, препроцессор может попытаться их обработать, что приведет к ошибкам. Поэтому такие комментарии нужно оформлять корректно.

Ядро Linux содержит как исходный код на C, так и на ассемблере. При определении констант, используемых обеими базами кода, они исключаются из дублирования, чтобы избежать ошибок.

Файлы .s (строчные) компилируются как чистый ассемблерный код, без обработки препроцессором C. Размер скомпонованной программы составляет всего около 3 Кб, что подчеркивает компактность и контроль над каждым байтом при программировании на чистом ассемблере.

Системный вызов Linux openat является типичным примером службы, работающей через системный интерфейс ядра. Он принимает четыре параметра:

1. Дескриптор каталога. Это дескриптор папки, внутри которой нужно открыть файл. Если используется значение AT_FDCWD, то файл открывается в текущем рабочем каталоге.
2. Имя файла. Указывается в виде строкового литерала, заканчивающегося символом NULL.
3. Флаги открытия. Определяют режим работы: чтение, запись, создание файла и т. д. Флаги задаются через директивы .equ, совпадающие по именам с константами из стандартной библиотеки C.
4. Режим доступа. Используется при создании файла. Значения соответствуют восьмеричным кодам, аналогичным параметрам команды chmod.

Результатом вызова является дескриптор файла (положительное число) или отрицательный код ошибки. Как и во многих других службах Linux, успешные и ошибочные результаты объединены в одно возвращаемое значение. В стандартной библиотеке C существуют обертки open и openat. Функция open фактически вызывает системный вызов openat, передавая в качестве первого параметра AT_FDCWD, что делает работу с файлами более удобной на уровне языка C.

Во многих учебных книгах вопросу обработки ошибок уделяют мало внимания, так как в примерах стараются показать только основную идею и не загромождать листинги лишними деталями. Однако в реальных программах проверка ошибок – обязательная часть работы. В этой программе проверяются коды возврата вызовов открытия файлов, поскольку именно эти операции чаще всего могут завершиться неудачно: файла может не существовать, путь может быть указан неверно или у программы нет прав на его открытие. Обычно проверяют код возврата *каждого* системного вызова, но здесь показаны только ключевые проверки – чтобы не перегружать пример. После вызова openFile дескриптор файла нужно сохранить в регистре, который не будет перезаписываться – Листинг 9.9.

Листинг 9.9 – Сохранения дескриптора файла

```
mv s1, a0             # сохранить дескриптор файла
```

Далее проверяется, является ли результат успешным (код возврата ≥ 0) – Листинг 9.10.

Листинг 9.10 – Открытие файла

```
bgez a0, nxtfil       # файл успешно открыт → переход
```

Если условие не выполняется, значит openFile вернул отрицательное значение – код ошибки. Чтобы вывести сообщение об ошибке, используется макрофункция writeFile, которая записывает строку в stdout (дескриптор 1), а затем выполняется выход из программы – Листинг 9.11.

Листинг 9.11 – Вывод сообщения об ошибке

```
li a1, 1              # stdout
la a2, inpErrsz       # длина сообщения
lw a2, 0(a2)          # загрузить длину
writeFile a1, inpErr, a2
exit
```

Сообщение об ошибке определяется в секции .data следующим образом – Листинг 9.12.

Листинг 9.12 – Пример кода сообщения об ошибке

```
inpErr:    .asciz "Failed to open input file.\n"
inpErrsz:  word . - inpErr
```

Такая конструкция использует особенность ассемблера GNU:

- символ “.” обозначает текущий адрес,
- поэтому “. – inpErr” вычисляет длину строкового литерала автоматически.

Это позволяет не подсчитывать количество символов вручную, что удобно при работе с длинными сообщениями об ошибках.

В реальных приложениях обычно существует отдельный модуль обработки ошибок, который занимается логированием или выводом сообщений. Такой подход позволяет сделать обработку ошибок более гибкой,

а также не загромождать основной исходный код. Сложность заключается в том, что код обработки ошибок часто не покрывается тестированием – поэтому в реальных системах иногда проявляются проблемы в тех ветках программы, которые серверами не проверялись.

Работа программы организована в виде простого цикла. Каждый проход выполняет следующие действия:

1. Чтение блока данных – из входного файла считывается до 250 символов.
2. Добавление завершающего NULL – в конец прочитанного фрагмента записывается 0, чтобы сформировать корректную строку.
3. Вызов функции toupper – строка преобразуется в верхний регистр.
4. Запись результата – полученные символы сохраняются в выходном файле.
5. Проверка конца файла – если прочитанный блок меньше запрошенного объема, цикл завершается.

Фрагмент проверки выхода из цикла показан в Листинге 9.13.

Листинг 9.13 – Фрагмент цикла обработки ошибки

```
li t1, BUFFERLEN
beq t0, t1, loop      # если t0 == BUFFERLEN → продолжаем цикл
```

Здесь регистр t0 содержит количество символов, фактически прочитанных вызовом readFile. Возможные ситуации:

- если t0 == BUFFERLEN, значит буфер полностью заполнен, возможно есть еще данные – выполняется переход к следующей итерации цикла;
- если t0 < BUFFERLEN, значит достигнут конец файла (количество символов меньше) или произошла ошибка чтения. Конец файла → t0 ≥ 0. Ошибка → t0 < 0

В обоих случаях дальнейшая обработка не требуется, и программа переходит к завершению. Такой подход делает цикл компактным, а проверка на окончание файла выполняется автоматически через returned value системного вызова read.

9.3. Взаимодействие ассемблера и C

В стандартной среде выполнения языка C точкой входа является метка _start, которая выполняет инициализацию окружения, а затем вызывает

функцию main. Если оставить собственную метку _start в файле .S, возникнет конфликт определений, поскольку стандартная библиотека также содержит _start. Кроме того, завершение программы будет выполняться не системным вызовом exit, а возвратом из main, чтобы среда выполнения C смогла выполнить завершающие действия (освобождение памяти, очистку состояний и т. д.).

Замечание: опция -mno-relax в данном случае не требуется, так как среда выполнения C сама инициализирует глобальный указатель, обеспечивая совместимость оптимизаций.

Чтобы подключить стандартную среду выполнения C и библиотеку, обычно модифицируют команду линковки (ld). Однако есть более простой путь: скомпилировать ассемблерный файл с помощью gcc, который автоматически вызывает ассемблер GNU и подключает runtime C.

Пример компиляции: “gcc -o myprogram myprogram.S”. В этом процессе gcc вызывает as для сборки myprogram.S, а затем ld для линковки, автоматически включая среду выполнения C.

Использование стандартного runtime языка C обеспечивает доступ к множеству инструментов: функциям работы со строками (NULL-terminated), средствам динамического выделения памяти, а также удобным оберткам над системными вызовами Linux. Это упрощает интеграцию кода на ассемблере с высокоуровневыми компонентами и делает возможной мультиязычную архитектуру программы.

Одним из важных преимуществ подключения стандартной среды выполнения C является возможность удобного вывода диагностических данных. В ранних примерах для печати содержимого регистра использовалась собственная ассемблерная подпрограмма, однако подключение C-библиотеки позволяет упростить этот процесс и снизить объем вспомогательного кода.

Чтобы не писать десятки одинаковых команд вручную, используются макрофункции GNU Assembler и режим altmacro, позволяющий программно создавать повторяющиеся инструкции. С помощью цикла макросов можно автоматически сгенерировать весь блок сохранения/восстановления регистров, превращая номер цикла в макропараметр.

Такое решение делает вызов отладочных макросов безопасным – они не нарушают состояние регистров в вызывающем коде и оставляют стек в корректном состоянии.

Вызов printf из ассемблера организуется по тем же правилам, что и при написании кода на C. Пример вызова: “printf("*%c = %32ld, 0x%016lx\n", reg, xreg, xreg);”. Аргументы функции размещаются в регистрах:

- a0 – адрес строки формата;
- a1 – первый параметр (reg);
- a2 – второй параметр (xreg);
- a3 – третий параметр (xreg).

Строка формата использует стандартные спецификаторы:

- %c – символ (номер регистра);
- %d – десятичное число;
- %x – шестнадцатеричное число;
- %016lx – 64-битное long-значение, выровненное нулями до 16 цифр;
- %32ld – значение в поле шириной 32 символа.

Важно сначала загрузить числовые значения регистров в a2 и a3, поскольку во время передачи других аргументов эти регистры могут быть перезаписаны. Если выводимый регистровый номер совпадает с a2 или a3, одна из инструкций mv просто ничего не изменит – это не ошибка.

Строка формата передается как строковый литерал в сегменте .data, а сама функция вызывается через команду call printf.

Макрофункция printStr предназначена для вывода строкового литерала. Поскольку язык ассемблера напрямую не работает со строками, литералы встраиваются в исходный код с помощью директивы .ascii или .asciz.

Сразу после строки рекомендуется использовать директиву .align, чтобы инструкции были выровнены по словам. Это улучшает производительность загрузки данных, так как выровненные данные читаются быстрее процессором.

Стоит учитывать, что вставка большого количества строковых литералов прямо в секцию кода не является лучшей практикой, однако для макрофункций это наиболее простой способ реализации вывода. В финальной версии программы отладочные строки обычно удаляются, чтобы не увеличивать размер кода и не создавать излишних смещений.

Если количество литералов становится слишком большим, необходимо либо сокращать их длину, либо удалять часть строк, чтобы сохранить компактность и корректное смещение данных в памяти.

Часто в разработке создается основная часть приложения на языке C, а отдельные функции реализуются на языке ассемблера для повышения производительности. Если следовать стандартному протоколу вызова

функций и организации стека, C не будет различать функции на ассемблере и функции на C. В качестве примера рассмотрим вызов функции toupper, реализованной на языке ассемблера. В Листинге 9.14 приведен код на C, который вызывает ассемблерную функцию mytoupper.

Листинг 9.14 – Программа на C для вызова функции toupper на ассемблере

```
#include <stdio.h>

#define MAX_BUFFSIZE 255

extern int mytoupper(const char *in, char *out);

int main()
{
      char *str = "This is a test.";
      char outBuf[MAX_BUFFSIZE];
      int len;

      len = mytoupper( str, outBuf );
      printf("Before str: %s\n", str);
      printf("After str: %s\n", outBuf);
      printf("Str len = %d\n", len);
      return(0);
}
```

Для сборки программы используется makefile – Листинг 9.15.

Листинг 9.15 – Файл makefile

```
uppertst: uppertst.c upper.S
      gcc -o uppertst uppertst.c upper.S
```

Для корректного использования функции на C необходимо определить ее прототип следующим образом: “extern int mytoupper(char *, char *);”.

Такое определение привычно программистам на C и обычно помещается в заголовочный .h файл. При компиляции и запуске программы вывод будет следующим – Листинг 9.16.

Строка была успешно преобразована в верхний регистр. Длина строки оказалась на единицу больше ожидаемой, поскольку учитывается завершающий символ NULL. Для совместимости с другими функциями на C рекомендуется вычитать единицу из длины строки.

Исходный код на ассемблере можно оставлять в отдельных объектных файлах (.o), однако удобнее объединять их в одну библиотеку. В этом случае пользователю достаточно подключить одну библиотеку, а не десятки отдельных файлов. В Linux есть два подхода:

1. Статическая библиотека – связывается с программой на этапе компоновки.
2. Динамическая (общая) библиотека – существует отдельно и может использоваться несколькими приложениями.

Листинг 9.16 – Результат вызова программы

```
ptsyngalev@ubuntu:~/lab7$ make
gcc -o uppertst uppertst.c upper.S
ptsyngalev@ubuntu:~/lab7$ ./uppertst
Before str: This is a test.
After str: THIS IS A TEST.
Str len = 16
```

Для создания статической библиотеки используется команда ar, которая объединяет несколько объектных файлов в один, обычно называемый lib<имя>.a. Этот файл затем включается в команду компоновки программы. Пример makefile для создания статической библиотеки показан в Листинге 9.17. Результат сборки показан в Листинге 9.18. Отличие от предыдущего примера в том, что сначала компилируется ассемблерный файл (upper.S) в объектный (upper.o), а затем создается библиотека libupper.a. Для распространения библиотеки следует включать также заголовочный файл с определениями функций на C и документацию.

Листинг 9.17 – Makefile для сборки upper в статическую библиотеку

```
LIBOBJS = upper.o

all: uppertst2

%.o : %.S
     as $(DEBUGFLGS) $< -o $@

libupper.a: $(LIBOBJS)
     ar -cvq libupper.a upper.o

uppertst2: uppertst.c libupper.a
     gcc -o uppertst2 uppertst.c libupper.a
```

Листинг 9.18 – Вызов makefile'a

```
ptsyngalev@ubuntu:~/lab7$ make
as upper.S -o upper.o
ar -cvq libupper.a upper.o
gcc -o uppertst2 uppertst.c libupper.a
ptsyngalev@ubuntu:~/lab7$ ./uppertst2
Before str: This is a test.
After str: THIS IS A TEST.
Str len = 16
```

Даже если библиотека предназначена только для внутреннего использования, такой подход облегчает совместное использование кода в команде и сокращает дублирование работы.

Динамические библиотеки более технологичны, чем статические: их код хранится отдельно от исполняемого файла и загружается динамически операционной системой Linux по мере необходимости. Это позволяет использовать одну библиотеку несколькими приложениями и упрощает управление версиями.

Для создания общей библиотеки используют опцию -shared и параметр -Wl,-soname, чтобы задать имя библиотеки. Библиотека должна находиться в одной из директорий файловой системы, где Linux ищет разделяемые библиотеки (например, /usr/local/lib). После компоновки библиотеку можно переместить в эту папку и создать символические ссылки на ее версии.

Пример makefile для компоновки и использования общей библиотеки приведен в Листинге 9.19. Результат запуска показан в Листинге 9.20.

Листинг 9.19 – Makefile для сборки и использования общей библиотеки

```
LIBOBJS = upper.o

all: uppertst3

%.o : %.S
	as $(DEBUGFLGS) $< -o $@

libup.so.1.0: $(LIBOBJS)
	gcc -shared -Wl,-soname,libup.so.1 -o libup.so.1.0 $(LIBOBJS)
	sudo mv libup.so.1.0 /usr/local/lib
	sudo ln -sf /usr/local/lib/libup.so.1.0 /usr/local/lib/libup.so.1
	sudo ln -sf /usr/local/lib/libup.so.1.0 /usr/local/lib/libup.so
	sudo ldconfig
```

Продолжение листинга 9.19

```
uppertst3: libup.so.1.0
     gcc -o uppertst3 uppertst.c –L. -lup
```

Листинг 9.20 – Результат сборки и запуска

```
....
ptsyngalev@ubuntu:~/lab7$ make -B
ptsyngalev@ubuntu:~/lab7$ ./uppertst3
Before str: This is a test.
After str: THIS IS A TEST.
Str len = 16
```

При использовании objdump исходный код функции mytoupper в исполняемом файле не отображается. Вместо этого видна таблица переходов (PLT), которая позволяет загрузчику динамически исправлять адрес функции при запуске – Листинг 9.21.

Листинг 9.21 – Результат сборки и запуска

```
600: 00002e17   auipc t3, 0x2
604: a283e03    ld t3, -1496(t3)
608: 000e0367   jalr t1, t3
60c: 00000013   nop
```

Таким образом, вызов ассемблерной функции из C работает аналогично статической версии, но код функции хранится отдельно, а компоновка и загрузка выполняются динамически. Это обеспечивает гибкость в управлении версиями библиотек и совместное использование кода разными приложениями.

Компилятор GNU C позволяет вставлять ассемблерные инструкции непосредственно в код на C с помощью конструкции asm. Это дает возможность напрямую использовать регистры и переменные C, интегрируя низкоуровневый код в высокоуровневую программу. Пример встраивания функции toupper на ассемблере в программу на C приведен в Листинге 9.22.

Листинг 9.22 – Пример встраивания ассемблерной подпрограммы в C

```
//
// C program to embed our Assembly
// toupper routine inline.
//

#include <stdio.h>

#define MAX_BUFFSIZE 255

int main() {
      char *str = "This is a test.";
      char outBuf[MAX_BUFFSIZE];
      int len;

      asm (
            "mv t4, %2\n"
            "loop:     lb    t2, 0(%1)\n"
            "addi %1, %1, 1\n"
            "li   t3, 'z'\n"
            "bgt  t2, t3, cont\n"
            "li   t3, 'a'\n"
            "blt  t2, t3, cont\n"
      "addi t2, t2, ('A'-'a')\n"
            "cont:     sb    t2, 0(%2)\n"
            "addi %2, %2, 1\n"
            "li   t3, 0 \n"
      "bne  t2, t3, loop\n"
            "sub  %0, %2, t4\n"
            : "=r" (len)
            : "r" (str), "r" (outBuf)
            : "t2", "t3", "t4", "a0", "a1"
      );

      printf("Before str: %s\n", str);
      printf("After str: %s\n", outBuf);
      printf("Str len = %d\n", len);
      return(0);
}
```

Пояснения к конструкции asm

- Ассемблерный шаблон – строковый литерал с кодом на ассемблере, в котором %0, %1 и т.д. заменяются на соответствующие переменные C.
- Выходные операнды (=r(len)) – регистры, результат работы которых сохраняется в переменные C.

- Входные операнды ("r"(str), "r"(outBuf)) – переменные C, которые будут загружены в регистры перед выполнением ассемблерного кода.
- Изменяемые регистры (t2, t3, t4, a0, a1) – регистры, используемые внутри asm, чтобы компилятор знал, какие из них нельзя использовать для других операций во время выполнения блока.
- Ключевое слово volatile (при необходимости) запрещает оптимизации компилятора, если ассемблерный код имеет побочные эффекты.

Программа компилируется как обычная C-программа: “gcc -o uppertst4 uppertst4.c”

При запуске результат будет аналогичен предыдущим примерам с внешней ассемблерной функцией: строка преобразуется в верхний регистр, а длина строки сохраняется в переменной len. Использование встроенного ассемблера позволяет создавать гибридный код, полностью интегрированный в C, при этом компилятор заботится о загрузке и сохранении регистров, что обеспечивает корректность работы программы.

9.4. Взаимодействие ассемблера и Python

Если функция на C или ассемблере следует стандартному протоколу вызова функций Linux, ее можно вызывать из других языков программирования. В Python .global toupper для этого используется модуль ctypes, который позволяет загружать общие библиотеки и вызывать функции из них. Статические библиотеки использовать нельзя, так как Python интерпретируемый язык, но динамические библиотеки (.so) работают корректно. Ранее была создана общая библиотека с функцией mytoupper (Листинги 9.19 – 9.20). Пример ее вызова из Python приведен в Листинге 9.23.

Листинг 9.23 – Вызов функции mytoupper из Python

```
from ctypes import *

# Загрузка общей библиотеки
libupper = CDLL("/usr/local/lib/libup.so")

# Определение типов аргументов и возвращаемого значения
libupper.mytoupper.argtypes = [c_char_p, c_char_p]
libupper.mytoupper.restype = c_int

# Подготовка входных и выходных буферов
inStr = create_string_buffer(b"This is a test!")
outStr = create_string_buffer(250)
```

Продолжение листинга 9.23

```
# Вызов функции
length = libupper.mytoupper(inStr, outStr)

# Вывод результата
print(inStr.value.decode())
print(outStr.value.decode())
print(length)
```

Пояснения:

1. Импортируется модуль ctypes для работы с общими библиотеками.
2. Общая библиотека загружается через CDLL. Хотя название CDLL происходит из Windows, оно используется и в Linux.
3. argtypes и restype задают типы аргументов и возвращаемого значения функции, что повышает безопасность вызова и позволяет Python корректно преобразовывать данные.
4. Строки в Python являются немутируемыми и кодируются в Unicode. Для совместимости с C и ассемблером создаются ASCII-буферы с помощью create_string_buffer, а строковый литерал указывается с префиксом b.
5. В буферы передаются входные и выходные данные функции. После выполнения вызова можно получить преобразованную строку и длину результата.

9.5. Заключение

Выполнение практической работы позволило закрепить знания о механизмах взаимодействия программ на языке ассемблера с ядром Linux через системные вызовы, а также освоить методы совместного использования ассемблерного и высокоуровневого кода. Студент познакомился с соглашениями о вызовах, работой с файловым вводом-выводом через макрофункции и обработкой кодов ошибок системных служб.

Приобретённые навыки создания статических и динамических библиотек на ассемблере, их вызова из программ на языках C и Python, а также встраивания ассемблерных инструкций непосредственно в код на C с помощью конструкции asm имеют ключевое значение для разработки высокопроизводительных приложений. Понимание принципов компоновки,

экспорта символов и протокола вызова функций позволяет создавать гибридные программы, в которых критически важные участки реализованы на ассемблере, а основная логика – на высокоуровневом языке.

Таким образом, выполненная работа формирует базовые компетенции для дальнейшего изучения мультиязычной разработки, оптимизации производительности программ и понимания внутренней организации стандартных библиотек и среды выполнения операционной системы Linux.

9.6. Варианты задания

Пояснения, изменить строковый литерал по следующему шаблону: “<ФИО>_<номер группы>_вариант_<номер варианта>”, строка должна быть в нижнем регистре. Например: «иванов_иван_иванович_икбо_01_25_вариант_5».

Задание №1. Используя Листинги 9.8, 9.14 – 9.15 собрать и выполнить программу преобразования строкового литерала.

Задание №2. Используя Листинги 9.8, 9.14, 9.17 собрать статическую библиотеку и выполнить программу преобразования строкового литерала.

Задание №3. Используя Листинги 9.19 собрать динамическую библиотеку и выполнить программу преобразования строкового литерала из Листинга 9.22 с использованием вызова функции ассемблера в языке C. Для вызова программы использовать “gcc -o uppertst4 uppertst4.c”

Задание №4. Используя собранную динамическую библиотеку из задания №3 и Листинг 9.23 выполнить программу преобразования строкового литерала с использованием вызова функции ассемблера в языке Python.

10. АССЕМБЛИРОВАНИЕ И ДИЗАССЕМБЛИРОВАНИЕ, ОПТИМИЗАЦИЯ ИСХОДНОГО КОДА УЯЗВИМОСТИ В НИЗКОУРОВНЕВЫХ ПРОГРАММАХ

10.1. Введение

Современные программные системы сталкиваются с широким спектром угроз, связанных как с техническими методами взлома, так и с ошибками разработчиков. В текущей практической работе под взломом понимается получение несанкционированного доступа к компьютерам и сетям через использование уязвимостей программного обеспечения. Цель изучения таких методов заключается не в их применении, а в понимании механизмов атак, чтобы уметь предотвращать их и защищать собственный программный код. Важно осознавать, что подложные данные, неверная обработка входных значений, некорректные проверки или ошибки в работе с памятью могут открыть злоумышленникам путь к эксплуатации программы. Поэтому каждый разработчик должен знать, как обнаруживаются слабые места в коде и каким образом хакеры используют их в своих целях. Социальная инженерия также является распространенным типом взлома, однако она выходит за рамки данной работы. Оптимизация в современных программных системах нередко требует нестандартного мышления и может предполагать не только удаление лишних инструкций, но и изменение структуры алгоритма. В ходе практической работы также будут изучены типичные приемы оптимизации, позволяющие сделать реализацию функции преобразования строки в верхний регистр более быстрой и компактной, а также проанализированы возникающие при этом компромиссы.

Таким образом, работа направлена на освоение основ анализа, чтения и понимания ассемблерного кода. Изучение чужого исходного кода является эффективным способом повышения квалификации, поскольку оно расширяет знания о приемах программирования, стилистике и архитектуре программ. В рамках работы рассматриваются источники и примеры реального ассемблерного кода архитектуры RISC-V, включая фрагменты из ядра Linux. На их основе демонстрируются методы оптимизации и структурные преобразования программ. Также, значимая часть работы посвящена оптимизации кода на уровне ассемблера. Рассматриваются методы повышения эффективности подпрограмм, включая создание более быстрых

условных инструкций и использование конструктивных шаблонов оптимизации.

10.2. Оптимизация кода

Рассмотрим процесс оптимизации кода, на примере подпрограммы, выполняющей конвертацию символов строкового литерала в верхний регистр (uppercase). Базовый вариант функции использует два ветвления для проверки того, находится ли символ в диапазоне 'a'–'z'. Хотя такой подход корректен и позволяет продемонстрировать работу условных переходов, он неэффективен с точки зрения производительности: каждое ветвление приводит к возможному нарушению конвейера выполнения инструкций и замедлению работы программы.

Исходная логика программы соответствует следующему псевдокоду из Листинга 10.1. Реализация на ассемблере – Листинг 10.2. При такой реализации используется «обратная логика»: программа уходит в ветвления, если символ не является строчной буквой. Такой подход полезен в учебных целях, однако с точки зрения оптимизации нежелателен из-за наличия двух условных переходов.

Листинг 10.1 – Псевдокод программы сравнения диапазонов

```
IF (t2 > 'a') AND (t2 < 'z') THEN
     t2 = t2 - ('a' - 'A')
END IF
```

Листинг 10.2 – Код программы сравнения диапазонов на ассемблере

```
# Если t2 > 'z', то перейти к cont
li t3, 'z'            # загрузить 'z' для сравнения
bgt t2, t3, cont# ветвление, если буква > 'z'?

# Иначе, если t2 < 'a', перейти в end if
li t3, 'a'            # загрузить 'a' для сравнения
blt t2, t3, cont      # переход в конец, если не в нижнем регистре

# если программа выполнилась до этого места, то буква в нижнем регистре
addi t2, t2, ('A' - 'a')
cont:
# end if
```

Попробуем упростить сравнения диапазонов. Стандартная техника устранения лишнего сравнения – сдвиг диапазона. Если из всех значений предварительно вычесть 'a', то нижняя граница автоматически становится равной нулю – Листинг 10.3.

Если рассматривать t4 как беззнаковое число, то проверка t4 >= 0 становится избыточной, так как все беззнаковые числа неотрицательны. Следовательно, остается только одна проверка – Листинг 10.4. Это означает, что сравнение можно сократить до одной инструкции bltu, что уменьшает количество ветвлений и ускоряет выполнение подпрограммы.

Листинг 10.3 – Псевдокод упрощенной программы сравнения диапазонов

```
t4 = t2 - 'a'
IF (t4 >= 0) AND (t4 < ('z' - 'a')) THEN
     t2 = t2 - ('a' - 'A')
END IF
```

10.4 – Код упрощенной программы сравнения диапазонов на ассемблере

```
T4 <= (z – 'a')
```

Оптимизированная версия помещается в файл функции toupper2.S и представлена в Листинге 10.5. Для сборки этого и последующего примеров используется Makefile из Листинга 10.6.

Листинг 10.5 – Функция toupper2.S

```
#
# Функция на языке ассемблера для конвертации символов
# строкового литерала в верхний регистр
#
# a1 - адрес выходного строкового литерала
# a0 - адрес входного строкового литерала
# t2 - текущий обрабатываемый символ
# t3 - временный регистр для сравнения
# t4 - минус 'a' для сравнения < 26
# t5 - исходная строка для расчета ее длины
#
.global toupper            # Разрешить другим файлам вызывать
подпрограмму
toupper:
    mv t5, a1              # сохранить исходную outstr для расчета
длины
```

Продолжение листинга 10.5

```
# Цикл выполняется до тех пор,
# пока не встретится нулевой (null) символ
Loop:
     lb t2, 0(a0)          # загрузить символ
     addi a0, a0, 1        # увеличить указатель буфера на 1

     # Нужно проверить, что 'a' <= t2 <= 'z'
     # Для этого сначала вычтем 'a'
     addi t4, t2, -'a'
     # Если t4 >= 26, перейти в end if
     li t3, 26             # загрузить 'z'-'a' для сравнения
     bltu t3, t4, cont          # перейти в конец, если не в нижнем
регистре

     # если программа выполнилась до этого места,
     # то буква в нижнем регистре, поэтому конвертировать ее
     addi t2, t2, ('A'-'a')
cont:                       # end if
     sb t2, 0(a1)          # сохранить символ в выходном строковом
литерале
     addi a1, a1, 1        # увеличить буфер на 1 для следующего
символа

     li t3, 0              # загрузить 0-символ для сравнения
     bne t2, t3, loop      # повторить итерацию, если символ не null

# Настроить параметры для вывода шестнадцатеричного числа,
# затем вызвать Linux
     sub a0, a1, t5        # получить длину путем вычитания указателей
     ret                   # вернуться к вызывающему элементу исходного
кода
```

Листинг 10.6 – Makefile сборки

```
UPPER2OBJS = main.o upper2.o
UPPER3OBJS = upper3.o

ifdef DEBUG
DEBUGFLGS = -g
else
DEBUGFLGS =
endif
```

Продолжение листинга 10.6

```
LSTFLGS =

all: upper2 upper3

%.o : %.S
     as $(DEBUGFLGS) -march=rv64i $(LSTFLGS) $< -o $@

upper2: $(UPPER2OBJS)
     ld -o toupper2 $(UPPER2OBJS)

upper3: $(UPPER3OBJS)
     ld -o toupper3 $(UPPER3OBJS)
```

Более эффективным способом оптимизации программного кода является сужение области решаемой задачи. Если подпрограмма должна работать только с алфавитными символами и не обязана корректно обрабатывать весь набор ASCII-символов, то часть проверок можно полностью исключить. Это позволяет значительно сократить количество инструкций и полностью удалить сравнительные операции и условные переходы.

Например, различие между строчными и заглавными латинскими буквами в ASCII заключается в наличии установленного бита 0x20 у строчных символов. Для заглавных букв этот бит всегда сброшен. Следовательно, перевод строчного символа в верхний регистр может быть выполнен простой логической операцией: “AND t2, t2, 0xDF “.

Маска 0xDF (1101 1111) позволит очистить этот бит, тем самым выполняя конвертацию без анализа диапазонов и без ветвлений. Однако такой метод подходит только для букв, для других ASCII-символов он может привести к искажению данных. В Листинге 10.7 показан пример подпрограммы toupper, реализованной как макрофункция. Макрос устраняет накладные расходы на вызов подпрограммы, а конвертация выполняется через маску 0xDF, без ветвлений.

В этом примере часть символов конвертируется корректно, однако некоторые специальные символы искажаются – это ожидаемый эффект при использовании побитовой очистки бита для символов, которые не являются буквами. Символ пробела имеет шестнадцатеричное значение 0x20. Если к

нему применить AND 0xDF, то результатом будет 0x00 – символ null. Поэтому пробел в этом примере завершил бы цикл обработки. Такая техника применима лишь к задачам, где не требуется точная обработка всего набора ASCII.

Отдельно следует отметить, что подпрограмма реализована в виде макрофункции, что позволяет избежать накладных расходов на вызовы процедур и еще сильнее сократить количество инструкций. Такой подход широко используется при низкоуровневой оптимизации, когда важны каждые несколько тактов выполнения.

Листинг 10.7 – Функция toupper3.S

```
#
# Функция на языке ассемблера для конвертации символов
# строкового литерала в верхний регистр.
#
# a0 - адрес входного строкового литерала
# a1 - адрес выходного строкового литерала
# t2 - текущий обрабатываемый символ
# t5 - исходный выходной строковый литерал для расчета длины строки
#

.global _start             # Точка входа для редактора связей
.macro toupper inputstr, outputstr
     la a0, \inputstr
     la a1, \outputstr
     mv t5, a1              # сохранить адрес исходной строки
                            # для расчета длины
# Цикл выполняется до тех пор, пока не будет встречен
# нулевой (null) символ
Loop:
     lb t2, 0(a0)           # загрузить символ
     addi a0, a0, 1         # увеличить указатель буфера на 1

     andi t2, t2, 0xdf      # очистить бит по адресу 0x20

     sb t2, 0(a1)           # сохранить символ в выходном строковом литерале
     addi a1, a1, 1         # увеличить буфер для следующего символа на 1

     li t3, 0               # загрузить 0-символ для сравнения
     bne t2, t3, Loop       # повторить итерацию, если символ не равен null

     # Настроить параметры для вывода шестнадцатеричного числа
     # и затем вызвать Linux
     sub a0, a1, t5         # получить длину путем вычитания указателей
```

Продолжение листинга 10.7

```
.endm

_start:
     toupper instr, outstr

     # Настроить параметры для вывода результирующего строкового литерала
     # и затем вызвать Linux
     mv a2, a0            # Кодом возврата является длина строки
     li a0, 1             # 1 = StdOut
     la a1, outstr        # выводимый строковый литерал
     li a7, 64            # системный вызов операции записи Linux
     ecall                # вывести строковый литерал


     # Задать параметры для завершения работы программы
     # и вызвать Linux для завершения
     li a0, 0             # Использовать код возврата
     li a7, 93           # Код 93 служебной команды завершает программу
     ecall                # завершить программу

.data
instr:
     .asciz "ThisIsRatherALargeVariableNameAaZz//['[\n"
outstr:
     .fill 255, 1, 0
```

10.3. Ассемблирование и дизассемблирование

Работа с Linux и GCC предоставляет разработчику важное преимущество – открытый исходный код. Это означает, что изучение внутреннего устройства ядра Linux, а также реализаций компилятора GCC, включая ассемблерные вставки и низкоуровневые функции, полностью доступно. Исходники можно получить напрямую из официальных репозиториев GitHub. Внутри ядра Linux имеется ряд каталогов, содержащих машинно-зависимые компоненты для архитектуры RISC-V: arch/riscv/include/asm, arch/riscv/lib, arch/riscv/kernel и arch/riscv/kvm. В GCC аналогичные элементы находятся в libgcc/config/riscv. Ассемблерный код размещен в файлах с расширением .S (с заглавной буквой), что позволяет использовать директивы препроцессора C и подключать заголовочные файлы. Разделения по разрядности (32/64 бита) нет – архитектурные различия сглаживаются условными конструкциями препроцессора, такими как #if riscv_xlen == 32.

Изучение этих исходников помогает понять, как ядро реализует критические операции – от инициализации процессора до обработки прерываний и функций среды выполнения. Поскольку стандартная библиотека C полностью доступна лишь после загрузки всей системы, ядро Linux включает собственные оптимизированные версии ключевых функций, особенно тех, что работают со строками. Многие структуры данных в ядре выравниваются по границе слова, что позволяет эффективно использовать пословные операции. В качестве примера рассмотрим функции strcmp из каталога arch/riscv/lib ядра Linux 6.7. Она реализована в двух вариантах: базовый побайтовый алгоритм (Листинг 10.8) и ускоренная версия (Листинг 10.9), использующая расширение Zbb для битовых манипуляций. Если аппаратная поддержка Zbb доступна, функция переходит на оптимизированный пословный алгоритм, который выполняется быстрее и активнее использует возможности процессора. Если же расширение отсутствует, используется простой универсальный цикл, сравнивающий байты один за другим. Такое разделение логики – пример внимательной архитектурной оптимизации: ядро выбирает максимально быстрый путь, не теряя совместимости. Разбор этой функции демонстрирует методологию написания высокоэффективного ассемблерного кода в Linux: использование макрофункций, проверка аппаратных возможностей, жесткая оптимизация циклов, минимизация ветвлений и учет выравнивания данных.

Листинг 10.8 – Код функции strcmp.S

```
	/*
	* Returns
	*	a0 - comparison result, value like strcmp
	*
	* Parameters
	*	a0 - string1
	*	a1 - string2
	*
	* Clobbers
	*	t0, t1
	*/

.global strcmp1		# Allow other files to call this routine

strcmp1:
1:
	lbu  t0, 0(a0)
```

Продолжение листинга 10.8

```
      lbu   t1, 0(a1)
      addi  a0, a0, 1
      addi  a1, a1, 1
      bne   t0, t1, 2f
      bnez  t0, 1b
      li    a0, 0
      ret
2:
      /*
      * strcmp only needs to return (< 0, 0, > 0) values
      * not necessarily -1, 0, +1
      */
      sub   a0, t0, t1
      ret
```

Листинг 10.9 – Код функции strcmp_zbb.S

```
/*
* Variant of strcmp using the ZBB extension if available.
* The code was published as part of the bitmanip manual in Appendix A.
*/

.global strcmp_zbb     # Allow other files to call this routine

.equ  SZREG, 8

strcmp_zbb:

.option push
.option arch,+zbb

      /*
      * Returns
      *     a0 - comparison result, value like strcmp
      *
      * Parameters
      *     a0 - string1
      *     a1 - string2
      *
      * Clobbers
      *     t0, t1, t2, t3, t4
      *
      * Parameters
      *     a0 - string1
      *     a1 - string2
      *
      * Clobbers
      *     t0, t1, t2, t3, t4
```

Продолжение листинга 10.9

```
     */

     or    t2, a0, a1
     li    t4, -1
     and   t2, t2, SZREG-1
     bnez  t2, 3f

     /* Main loop for aligned string.  */
     .p2align 3
1:
     ld    t0, 0(a0)
     ld    t1, 0(a1)
     orc.b t3, t0
     bne   t3, t4, 2f

     addi  a0, a0, SZREG
     addi  a1, a1, SZREG
     beq   t0, t1, 1b

     /* Synthesize (t0 >= t1) ? 1 : -1 in a branchless sequence. */
     sltu  a0, t0, t1
     neg   a0, a0
     ori   a0, a0, 1
     ret
2:
     /*
     * Found a null byte.
     * If words don't match, fall back to simple loop.
     */
     bne   t0, t1, 3f

     /* Otherwise, strings are equal. */
     li    a0, 0
     ret

     /* Simple loop for misaligned strings. */
     .p2align 3
3:
     lbu   t0, 0(a0)
     lbu   t1, 0(a1)
     addi  a0, a0, 1
     addi  a1, a1, 1
     bne   t0, t1, 4f
     bnez  t0, 3b
4:
     sub   a0, t0, t1
     ret

.option pop
```

Для сборки можно использовать Makefile из Листинга 10.10.

Листинг 10.10 – Makefile для сборки функции strcmp

```
STRCMPOBJS = main.o strcmp1.o strcmp2.o

ifdef DEBUG
DEBUGFLGS = -g
else
DEBUGFLGS =
endif
LSTFLGS =

all: strcmp

%.o : %.S
     as $(DEBUGFLGS) -mno-relax $(LSTFLGS) $< -o $@

strcmp: $(STRCMPOBJS)
     gcc -o strcmp $(STRCMPOBJS)
```

Продолжая разбор алгоритма сравнения строк в ядре Linux рассматривается реализация в двух вариантах, первый из них использует побайтовый подход. Он основан на инструкциях базового целочисленного набора, в частности lbu, что делает эту реализацию одинаково корректной и для 32-битных, и для 64-битных систем. Такой вариант легко понять и он работает всегда, включая случаи, когда данные не выровнены по границе слова. При необходимости ядро автоматически может переключиться на эту версию, если оптимизированный алгоритм не подходит. Вторая версия алгоритма рассчитана на значительно более высокую производительность. Она загружает данные целыми словами – например, по 8 байт за раз в 64-битных системах – что в идеальном случае ускоряет сравнение пропорционально размеру слова. Однако эта оптимизация применима только тогда, когда оба строковых литерала выровнены. Проверка выравнивания выполняется с помощью побитовых операций: адреса двух строк складываются, затем результат маскируется значением «размер слова минус один». Если получено ненулевое значение, значит хотя бы одна строка не выровнена, и алгоритм переходит к безопасному побайтовому методу.

Ключевой прием ускоренной версии связан с поиском нулевого байта, который завершает строку в стиле C. Проверка каждого байта по отдельности

привела бы к множественным сравнениям и ветвлениям, что снизило бы эффективность. Вместо этого используется малоизвестная, но крайне полезная инструкция расширения Zbb – orc.b. Она выполняет побитовое ИЛИ внутри каждого байта и превращает байт регистра в 0x00, если исходный байт был нулевым, или в 0xFF, если исходный байт содержал хотя бы один установленный бит. Благодаря этому можно одним сравнением определить наличие нулевого символа в любом из обработанных байтов.

Алгоритм загружает в регистр значение 1 (все биты равны 1), а затем сравнивает результат orc.b со значением -1. Если сравнение успешно, значит нулевых байтов нет и можно продолжать цикл пословного сравнения. Если же хотя бы один байт оказался нулевым, происходит переход на побайтовый алгоритм, который корректно завершает сравнение вплоть до символа \0. Такой подход исключает ошибочное сравнение байтов после нулевого символа и сохраняет семантику функции strcmp. Для использования инструкций расширения Zbb в ассемблерном коде включается директива option arch, +zbb. Без нее ассемблер не распознает операции расширения и выдаст ошибку. Этот механизм позволяет ядру Linux гибко использовать аппаратные возможности, обеспечивая максимальную производительность там, где это возможно, и корректную работу на всех остальных системах.

Исходный код ядра Linux существенно усложнен множеством вариантов компиляции, связанных с поддержкой разных процессоров и конфигураций. В примере со сравнением строк, ядро поддерживает обе архитектуры – 32-битную и 64-битную RISC-V – и предоставляет как базовую реализацию на инструкциях RV32I/RV64I, так и оптимизированный вариант, который использует инструкцию orc.b из расширения Zbb. Эти варианты переключаются автоматически с помощью специальных макрофункций.

Основным механизмом является макрофункция ALTERNATIVE, определенная в arch/riscv/include/asm/alternative-macros.h. Она позволяет компоновщику подставлять различные подпрограммы в зависимости от конфигурации ядра и наличия аппаратных расширений. Если расширение Zbb включено, в код будет вставлен ускоренный вариант функции; если нет – ядро оставит базовую версию, которая гарантированно работает везде. Для обработки различий между 32- и 64-битными архитектурами применяются макрофункции из arch/riscv/include/asm.h, которые выбирают корректные инструкции загрузки и сохранения регистров. Например, REG_L и REG_S автоматически подставляют ld/sd для 64 бит и lw/sw для 32 бит, обеспечивая единый код для обоих вариантов архитектуры. Кроме того, макрофункции

SYM_FUNC_START и SYM_FUNC_END, определенные в include/linux/linkage.h, гарантируют корректное выравнивание подпрограмм и экспорт символов в глобальное пространство имен. Эти конструкции скрывают платформенные детали и делают ассемблерные файлы более переносимыми и единообразными в структуре.

Рассмотрим пример генерации ассемблерного кода GCC на основе подпрограммы mytoupper на C (Листинг 10.11). Программа компилируется с полной оптимизацией: “gcc -O3 -o mytoupper upper.c”, после выполнения проанализируем вывод “objdump -d upper >od.txt” (листинг Листинг 10.12).

Листинг 10.11 – Программа mytoupper

```
#include <stdio.h>

int mytoupper(char *instr, char *outstr)
{
      char cur;
      char *orig_outstr = outstr;

      do
      {
            cur = *instr;
            if ((cur >= 'a') && (cur <='z'))
            {
                  cur = cur - ('a'-'A');
            }
            *outstr++ = cur;
            instr++;
      } while (cur != '\0');

      return( outstr - orig_outstr );
}

#define BUFFERSIZE 250

char *tstStr = "This is a test!";
char outStr[BUFFERSIZE];

int main()
{
      mytoupper(tstStr, outStr);
      printf("Input: %s\nOutput: %s\n", tstStr, outStr);

      return(0);
}
```

Листинг 10.12 – Псевдокод упрощенной программы сравнения диапазонов

```
0000000000010110 <main>:
      10110: 1101                   addi        sp,sp,-32
      10112: ec06                   sd          ra,24(sp)
      10114: 1000                   addi        s0,sp,32
      10116: fb042423               sw          zero,-88(s0)
      1011a: 05a00513               li          a0,90
      1011e: 05a00793               li          a5,90
      10122: 05a00613               li          a2,90

0000000000010126 <.L2>:
      10126: fbc42703               lw          a4,-68(s0)
      1012a: 0007071b               addiw       a4,a4,0
      1012e: 0007071b               addiw       a4,a4,0
      10132: 0007179b               slliw       a5,a4,0x0
      10136: 0007879b               addiw       a5,a5,0
      1013a: 0007c79b               zext.b      a5,a5
      1013e: 0007879b               addiw       a5,a5,0
      10142: 05f00713               li          a4,95
      10146: 00e7e63b               subw        a2,a5,a4
      1014a: 02064613               sltiu       a2,a2,32
      1014e: 02061613               slli        a2,a2,0x20
      10152: 02065613               srli        a2,a2,0x20
      10156: 00261613               slli        a2,a2,0x2
      1015a: 40c787bb               subw        a5,a5,a2
      1015e: 00078513               addi        a0,a5,0
      10162: 00000097               auipc       ra,0x0
      10166: 000000e7               jalr        ra,0(ra)

      1016a: fbc42703               lw          a4,-68(s0)
      1016e: 0017071b               addiw       a4,a4,1
      10172: fbd42423               sw          a4,-72(s0)
      10176: fbd42783               lw          a5,-72(s0)
      1017a: 07900713               li          a4,121
      1017e: fbe7dce3               bge         a5,a4,101b6 <.L4>
      10182: fbd42423               sw          a4,-72(s0)
      10186: fec42703               lw          a4,-20(s0)
      1018a: 0017071b               addiw       a4,a4,1
      1018e: fea42623               sw          a4,-20(s0)
      10192: fec42783               lw          a5,-20(s0)
      10196: 07a00713               li          a4,122
      1019a: fce7d0e3               blt         a5,a4,1015a <.L2>
      1019e: 00a00513               li          a0,10
      101a2: 00000097               auipc       ra,0x0
      101a6: 000000e7               jalr        ra,0(ra)
      101aa: 00000513               li          a0,0
      101ae: 00000097               auipc       ra,0x0
      101b2: 000000e7               jalr        ra,0(ra)

00000000000101b6 <.L4>:
      101b6: 00a00513               li          a0,10
      101ba: 00000097               auipc       ra,0x0
      101be: 000000e7               jalr        ra,0(ra)
      101c2: 00000513               li          a0,0
      101c6: 00000097               auipc       ra,0x0
      101ca: 000000e7               jalr        ra,0(ra)
```

Из результата видно, что компилятор встроил функцию mytoupper внутрь main, применил оптимизацию диапазона и заменил два сравнения одним – с использованием выражения addw a4, a5, -97. GCC также вставляет псевдоинструкцию zext.b, разворачивающуюся в andi, чтобы обеспечить корректную работу с типом char, подавляя нежелательное распространение знака. Хоть оптимизация компилятора заметна, но все равно присутствуют избыточные инструкции, которые можно удалить вручную.

Однако утилиты по типу objdump не позволяют целиком увидеть картину организации кода в уже скомпилированной программе. Для этого существуют специальные решения (коммерческие и свободно распространяемые) позволяющие декомпилировать исполняемый файл программы. Например, для этого можно использовать утилиту Ghidra, которая существенно упрощает изучение закрытых бинарных файлов, позволяя увидеть структуру программы на более высоком уровне, чем простой дизассемблер. В отличие от анализа "сырых" ассемблерных инструкций, Ghidra строит графы вызовов, определяет функции, статические данные, локальные переменные и пытается восстановить логику программы. Декомпилированный код редко совпадает с исходным – оптимизации компилятора могут изменить структуру циклов, уничтожить имена переменных и преобразовать условия в более низкоуровневые операции. Тем не менее, даже приблизительное восстановление исходного кода дает исследователю понимание того, как работает программа, есть ли в ней вредоносные фрагменты, какие данные она отправляет, какие системные вызовы использует и есть ли скрытая логика или "чит-коды".

Запуск Ghidra обычно сводится к импорту ELF-программы, настройке языка (например, RISCV:LE:64:RV64GC:gcc), запуску автоматического анализа и работе с декомпилятором. Один из типичных сценариев: сначала открыть Listing View для изучения машинного кода, затем – Decompile Window для получения C-подобного кода, и, при необходимости, – Symbol Tree для навигации по функциям. Аналитик может обойти точки входа, расставить метки, добавлять комментарии, изучить строки и глобальные объекты, а затем выполнить программу в Linux, чтобы проверить гипотезы о ее поведении. Такая техника особенно важна при проверке бинарников без исходников, аудите безопасности, анализе неизвестного сетевого трафика, изучении поведения проприетарных библиотек, а также исследовании старых игр или ПО для поиска скрытых возможностей.

10.4. Поиск уязвимостей в исходном коде

Одной из наиболее популярных и опасных уязвимостей на является переполнение буфера. Если функция записывает данные в массив, не контролируя длину входа, то при превышении размера буфера программа начинает перезаписывать соседние области памяти. Когда буфер расположен в стеке, под угрозой оказываются локальные переменные и, самое главное, адрес возврата функции. Подменив этот адрес заранее подготовленным значением, злоумышленник может перенаправить выполнение программы на произвольный участок памяти, включая специально вставленный машинный код (shellcode). Если же данные находятся в куче или сегменте данных, переполнение может испортить служебные структуры аллокатора, нарушить таблицы указателей или привести к аварийному завершению – все это тоже формирует основу для атак, включая DoS.

Для обнаружения таких уязвимостей важно анализировать участки кода, где выполняется запись в массивы без проверки длины: циклы копирования строк, функции обработки ввода, использование gets, небезопасных strcpy и конкатенаций. В контексте ранее рассматриваемого примера с преобразованием регистра строк функция корректно работает только при условии, что выходной буфер больше или равен входной строке. Если же вход длиннее, цикл записи поочередно перезаписывает память за пределами массива и потенциально затрагивает стековую рамку, что в реальных условиях позволяет изменить контрольный поток программы. Такие ошибки легко упустить в больших проектах, поэтому систематический анализ кода – статический, динамический, инструментальный, ревью.

Рассмотрим пример с кражей номеров кредитных карт (анализ переполнения буфера) с уязвимостью некоторого веб-приложения (Листинг 10.13), развернутого на сервере компании, выпускающей кредитные карты. Приложение использует вспомогательную программу для преобразования имен клиентов в верхний регистр. Такой подход выбран для ускорения обработки пользовательских запросов и формирования веб-страниц. На одной из страниц пользователю предлагается ввести свое имя, после чего приложение выполняет его конвертацию в верхний регистр. Важным является то, что веб-страница не проверяет длину введенных данных, передавая строку в функцию преобразования «как есть», что создает условия для возникновения ошибки переполнения буфера.

Дополнительно в приложении предусмотрена административная функциональность – например, утилита скачивания всех сохраненных

номеров кредитных карт для резервного копирования. Доступ к этой утилите разрешен только администраторам, а аутентификация выполняется по цифровому сертификату. Однако злоумышленник стремится получить административный доступ, обойдя процедуру проверки прав, подсовывая специально сформированные входные данные в поле ввода строки.

Согласно протоколам вызова подпрограмм, при переходе от одной функции к другой необходимо сохранять в стеке регистр возврата ra, чтобы избежать его перезаписи. В анализируемой программе присутствует промежуточная подпрограмма calltoupper, которая сохраняет ra в стеке и выделяет на стеке 16 байт под выходной буфер. При этом подпрограмма DownloadCreditCardNumbers, содержащая административный функционал, не должна быть доступна обычному пользователю. Однако, как будет показано далее, отсутствие проверки длины входных данных позволяет злоумышленнику сконструировать строку таким образом, чтобы перезаписать сохраненный в стеке ra и перенаправить выполнение в административную функцию.

В данном фрагменте используется первая версия функции toupper (Листинг 10.14), которая не выполняет проверку длины выходного буфера. Буфер размером 16 байт выделяется на стеке внутри подпрограммы calltoupper, однако входная строка может быть длиннее и, следовательно, функция toupper начинает записывать данные за пределы выделенной области. В результате переписываются элементы стека, включая сохраненный регистр ra.

Злоумышленник формирует строку в сегменте .data таким образом, чтобы после основной части входной строки следовало 8-байтовое значение – адрес административной функции DownloadCreditCardNumbers. Это значение записывается в ту область стека, где должен храниться сохраненный ra. Из-за этого после завершения работы toupper и выхода из calltoupper управление передается не в основную программу, а в административную функцию, хотя она явно не вызывалась.

Листинг 10.13 – Функция creditcard программы веб-приложения

```
#
# Assembler program to demonstrate a buffer
# overrun hacking attack.
#
# a0-a2 - parameters to Linux function services
# a1 - address of output string
```

Продолжение листинга 10.13

```
# a0 - address of input string
# a7 - Linux function number
#

.global _start             # Provide program starting address

DownloadCreditCardNumbers:
# Setup the parameters to print hello world
# and then call Linux to do it.
     li    a0, 1           # 1 = StdOut
     la    a1, getcreditcards # string to print
     li    a2, 30          # length of our string
     li    a7, 64          # linux write system call
     ecall                 # Call linux to output the string
     ret

calltoupper:
     addi  sp, sp, -16     # allocate 16 bytes on stack
     sd    ra, 0(sp)       # push return address
     addi  sp, sp, -16     # 16 bytes for outstr
     la    a0, instr       # start of input string
     mv    a1, sp          # address of output string

     call  toupper

aftertoupper:              # convenient label to use as a breakpoint
     addi  sp, sp, 16      # Free outstr
     ld    ra, 0(sp)       # pop ra
     addi  sp, sp, 16      # deallocate stack space
     ret

_start:

     call  calltoupper

# Setup the parameters to exit the program
# and then call Linux to do it.
     li    a0, 0           # Use 0 return code
     li    a7, 93          # Service command code 93 terminates this
program
     ecall                 # Call Linux to terminate the program

.data
instr:  .ascii  "This is our Test"    # Correct length string
     .quad 0x00000000000100e8   # overwrite for ra
getcreditcards: .asciz  "Downloading Credit Card Data!\n"
     .align 4
```

Листинг 10.14 – Функция toupper

```
#
# Assembly Language function to convert a string to
# all upper case.
#
# a1 - address of output string
# a0 - address of input string
# s0 - original output string for length calc.
# t2 - current character being processed
# t3 - temp register for comparisons
#

.global toupper            # Allow other files to call this routine

toupper:
     addi  sp, sp, -16     # allocate 16 bytes on stack
     sd    ra, 8(sp)       # push return address
     sd    s0, 0(sp)       # push s0 register
     mv    s0, a1          # save original outstr for len calc
# The loop is until null (zero) character is encountered.
loop: lb    t2, 0(a0)       # load character
     addi  a0, a0, 1       # increment buffer pointer
# If x7 > 'z' then goto cont
     li    t3, 'z'         # load 'z' for comparison
     bgt   t2, t3, cont    # branch if letter > 'z'?
# Else if x7 < 'a' then goto end if
     li    t3, 'a'         # load 'a' for comparison
     blt   t2, t3, cont    # goto to end if not lowercase
# If we got here then the letter is lower case, so convert it.
     addi  t2, t2, ('A'-'a')
cont: # end if
     sb    t2, 0(a1)       # store character to output str
     addi  a1, a1, 1       # increment buffer for next char
     li    t3, 0           # load 0 char for comparison
     bne   t2, t3, loop    # loop if character isn't null

# Setup the parameters to print our hex number
# and then call Linux to do it.
     sub   a0, a1, s0      # get the len by sub'ing the pointers

     ld    s0, 0(sp)       # pop s0
     ld    ra, 8(sp)            # pop ra
     addi  sp, sp, 16 # deallocate stack space
     ret                   # Return to caller
```

После компиляции и запуска программы наблюдается многократный вывод строки: “Downloading Credit Card Data!”. Однако этот вывод происходит несмотря на отсутствие прямого вызова DownloadCreditCardNumbers. Причина заключается в том, что подмененный адрес возврата снова приводит к выполнению административной функции, образуя бесконечный цикл до завершения программы аварийно. Исходный пользовательский интерфейс в листинг не включен – входные данные помещены непосредственно в секцию .data для упрощения демонстрации уязвимости.

Работа со стеком в демонстрируемом примере начинается в функции calltoupper, которая выделяет память под выходной буфер и сохраняет регистр возврата ra. Важно помнить, что указатель sp в архитектуре RISC-V всегда должен быть выровнен по границе 16 байт.

Первоначально sp содержит адрес 0x3ffffff030. После выполнения инструкции сохранения регистра ra стек изменяется следующим образом – Листинг 10.15.

Листинг 10.15 – Изменение стека

```
0x3ffffff030 – исходный указатель sp
0x3ffffff020 – 8 байт для ra
                8 байт нулей
(addi sp, sp, -16; sd ra, 0(sp))
0x3ffffff010 – 16-байтовый буфер для outstr
(addi sp, sp, -16)
```

Поскольку стек растет вниз, уменьшение значения sp означает выделение нового пространства. После размещения буфера адрес, переданный функции toupper как outstr, становится равным 0x3ffffff010. Подпрограмма toupper записывает символы в выходной буфер последовательно, увеличивая указатель. Из-за отсутствия проверки длины строки при переполнении функция начинает перезаписывать данные, находящиеся по адресу 0x3ffffff020, где расположен сохраненный регистр ra. Именно это позволяет злоумышленнику перезаписать адрес возврата значением, которое перенаправит выполнение программы на административную процедуру. В Листинге 10.16 показан фрагмент дампа функции программы веб-приложения.

Листинг 10.16 – Дамп функции программы веб-приложения

```
00000000000100e8 <DownloadCreditCardNumbers>:
100e8:     00100513   li a0, 1

0000000000010104 <calltoupper>:
10104:     ff010113   addi sp, sp, -16
10108:     00113023   sd ra, 0(sp)

0000000000010134 <_start>:
10134:     00000097   auipc ra, 0x0
10138:     fd0080e7   jalr -48(ra) # 10104 <calltoupper>
1013c:     00000513   li a0, 0
```

В начале выполнения основной функции «_start» используется инструкция «jalr», которая передает управление подпрограмме «calltoupper». В этот момент регистр «ra» получает значение адреса следующей инструкции – «0x1013c». Этот факт подтверждается выводом «objdump», где видно, что переход осуществляется на метку «calltoupper», а в регистре возврата содержится корректный адрес для последующего возврата.

При входе в «calltoupper» указатель стека «sp» содержит исходное значение «0x3ffffff030». Затем выполняются инструкции «addi sp, sp, -16» и «sd ra, 0(sp)». Эти операции уменьшают значение «sp» до «0x3ffffff020» и сохраняют текущее значение «ra» в стек. На этом этапе стек содержит следующую структуру: «0x3ffffff020: 0x0000000000010126», «0x3ffffff028: 0x0000000000000000»
что означает корректное помещение 8-байтового значения адреса возврата в стек.

Следующая операция – выделение выходного буфера. Для этого выполняется команда «addi sp, sp, -16», которая снова уменьшает «sp» – теперь до «0x3ffffff010», резервируя 16 байт под буфер для строки. После этого стек выглядит так: «0x3ffffff010: 0x0000000000000000 0x0000000000000000», «0x3ffffff020: 0x0000000000010126 0x0000000000000000». Буфер подготовлен, и подпрограмма «toupper» получает возможность записывать в него преобразованные символы.

Работа функции «toupper» приводит к критическому переполнению буфера. Подпрограмма корректно преобразует первые 16 байт строкового литерала «This is our Test», поскольку именно такой объем памяти был выделен под выходной буфер. Однако в исходной строке отсутствует завершающий нулевой байт, поэтому «toupper» продолжает обработку

последующего байта «0xe8». Поскольку этот байт не является символом нижнего регистра, он копируется в стек без изменений. Следующий байт равен нулю, что завершает работу цикла копирования. Таким образом, происходит запись данных за пределы буфера, и часть стека, содержащая сохраненный регистр «ra», оказывается перезаписанной. После выполнения «toupper» стек имеет вид: «0x3ffffff010: 0x2053492053494854 0x545345542052554f», «0x3ffffff020: 0x00000000000100e8 0x0000000000000000». Из этого видно, что значение регистра «ra» изменилось с корректного адреса «0x00010126» на «0x000100e8», который соответствует началу подпрограммы «DownloadCreditCardNumbers».

При завершении «calltoupper» выполняется последовательность команд «addi sp, sp, 16», «ld ra, 0(sp)», «addi sp, sp, 16» и «ret». На этапе загрузки «ra» в регистр попадает уже подмененное значение «0x100e8». Затем инструкция «ret» передает управление по этому адресу, что приводит к выполнению подпрограммы «DownloadCreditCardNumbers» вместо ожидаемого возврата в основную программу. В результате злоумышленник получает возможность вызвать защищенную административную функцию, которая должна быть доступна только авторизованным пользователям, – скачивание номеров кредитных карт.

Хакерская атака будет успешной благодаря ряду факторов:

- достаточно было изменить всего один байт адреса и завершающий NULL, чтобы получить корректный адрес административной функции;
- значение байта, который нужно было скопировать (0xe8), не являлось символом нижнего регистра, поэтому подпрограмма toupper не изменила его;
- переполнение произошло точно в области сохраненного регистра ra.

Если бы данные не находились в таком положении, злоумышленник все равно мог бы использовать разные варианты атаки, например, указывать адрес, не на начало, а на середину инструкции в целевой функции (обход пролога).

Поскольку выполнение попадает в DownloadCreditCardNumbers через инструкцию ret, регистр ra не обновляется. В конце подпрограммы снова выполняется ret, что приводит к переходу по тому же адресу – в начало DownloadCreditCardNumbers. Так формируется бесконечный цикл вывода строки: “Downloading Credit Card Data!”.

Смягчение уязвимостей, связанных с переполнением буфера, требует как корректных решений в исходном коде, так и использования механизмов операционных систем и компиляторов. Основная проблема возникает из-за функций наподобие «strcpy», которые копируют данные до первого нулевого байта и не контролируют длину источника. Попытки заменить «strcpy» более безопасными вариантами привели к появлению функций вроде «strncpy», где программист обязан передавать размер буфера назначения, однако такая замена порождает новую проблему: результирующая строка не завершается нулевым байтом, что создает риск дальнейших переполнений. Чтобы этого избежать, программист должен вручную дописывать «dest[num-1] = '\0'», но такая практика легко нарушается из-за спешки или человеческой ошибки. В ответ на это в BSD была предложена функция «strlcpy», которая гарантирует завершающий нуль, но она не является частью стандарта и отсутствует в GNU libc. Другие варианты, например, «strecpy», пытаются решить вопрос вложенных копирований, передавая указатель на конец буфера, однако такие функции также нестандартны и не входят в базовую библиотеку языка C.

Даже более безопасные варианты копирующих функций обладают побочным эффектом: если исходная строка не содержит нулевого байта и ее буфер меньше, чем буфер назначения, то копирование может захватить конфиденциальные данные, расположенные в памяти сразу после исходного буфера, и перенести их в область назначения, где они могут быть случайно выведены. Для устранения подобных рисков были разработаны функции семейства «strncpy_s», где указываются размеры обоих буферов, а результатом является код ошибки, позволяющий корректно обрабатывать аварийные ситуации. Обсуждение этих решений показывает, что улучшение безопасности стандартных библиотек связано с компромиссами: каждая новая функция решает одни проблемы, но создает ограничения в других областях, например, усложняет конкатенацию строк или нарушает переносимость. При проектировании безопасной подпрограммы преобразования регистра, такой как «uppercase», следует учитывать все эти технические нюансы, после чего уже рассматривать защитные возможности GNU-компилятора и операционной системы, которые дополняют меры, реализуемые на уровне исходного кода.

Использование позиционно-независимого исполняемого кода, или «PIE», стало одной из ключевых защит от эксплуатации переполнения буфера, поскольку при каждом запуске программы базовый адрес ее размещения меняется, а злоумышленник не может заранее знать, где находится целевая

функция. Однако в примере с уязвимостью, где выполнялась подмена адреса возврата на адрес «DownloadCreditCardNumbers», эксплойт все же сработал. Это произошло потому, что сама программа была собрана без поддержки PIE – редактор связей «ld» по умолчанию создает не-переразмещаемый бинарный файл, если явно не указана опция «-pie». Поэтому злоумышленник мог жестко прописать адрес подпрограммы в вводимых данных, и этот адрес совпадал с реальным размещением кода, позволяя переполнению стека изменить значение регистра «ra» и передать управление административной функции. Такой эксплойт был бы невозможен, если бы бинарный файл был переразмещаемым: при каждом запуске адрес «DownloadCreditCardNumbers» был бы случайным и неизвестным. Но в исходном примере использовалось прямое кодирование адреса в секции «.data», что гарантировало предсказуемость размещения программы и сделало атаку возможной.

Чтобы включить защиту PIE, необходимо собирать исполняемый файл с ключом «-pie», например: «ld -pie -o creditcardpie creditcard.o upper.o». После включения PIE загрузчик должен иметь возможность исправлять адреса, и для систем RISC-V на момент подготовки текста требовалось вручную копировать файл загрузчика: «sudo cp ld-linux-riscv64-lp64d.so.1 ld.so.1».

Если собрать программу с PIE, то запуск произведет иной результат: «./creditcardpie → Illegal instruction», или, в зависимости от содержимого памяти, «Segmentation fault». Это означает, что переполнение стека все еще происходит, но передача управления больше невозможна, так как подмененный адрес «ra» указывает на произвольную область памяти, находящуюся в другом месте после рандомизации. Атака перестает работать, но программа аварийно завершается, что создает риск отказа в обслуживании. Причиной невозможности корректного переразмещения программы часто становится жесткое кодирование адресов с помощью псевдоинструкции «li», которая разворачивается в цепочку «slli» и «addi» без записи переразмещения. В отличие от нее псевдоинструкция «la» генерирует корректную пару «auipc» и «add», создает записи reloc, и загрузчик может перемещать такие адреса. Поэтому для обеспечения безопасности хорошей практикой является не только включение «PIE», но и отказ от жестко закодированных адресов в ассемблере, так как современная защитная инфраструктура ASLR эффективна только при условии, что редактор связей и загрузчик имеют возможность корректно переразмещать программу. Хотя PIE не является абсолютной защитой, его использование значительно осложняет эксплуатацию

уязвимостей, вынуждая атакующего искать дополнительные слабости, а не опираться лишь на переполнение буфера.

Механизм стековых канареек (stack canaries) – одна из мер защиты, направленных на обнаружение переполнения буфера еще до того, как злоумышленник сможет выполнить свой эксплойт. Идея состоит в том, что компилятор вставляет в пролог функции специальный произвольный секретный маркер, а в эпилоге проверяет, сохранился ли он. Если маркер поврежден, программа немедленно аварийно завершается. Как и PIE, эта функциональность не активируется автоматически – ее необходимо включить при компиляции. Самый строгий вариант активируется флагом: “-fstack-protector-all”.

Пример программы и Makefile общей сборки показан в Листингах 10.17 – 10.18.

Листинг 10.17 – Программа на C перевода строки в верхний регистр

```
#include <stdio.h>

int dummy()
{
      return(2);
}

int mytoupper(char *instr, char *outstr)
{
      char cur;
      char *orig_outstr = outstr;

      dummy();
      do
      {
            cur = *instr;
            if ((cur >= 'a') && (cur <='z'))
            {
                  cur = cur - ('a'-'A');
            }
label:            *outstr++ = cur;
            instr++;
      } while (cur != '\0');
      return( outstr - orig_outstr );
}
```

Продолжение листинга 10.17

```
#define BUFFERSIZE 10

int main()
{
      char *tstStr = "This is a test!";
      char outStr[BUFFERSIZE];


      mytoupper(tstStr, outStr);
      printf("Input: %s\nOutput: %s\n", tstStr, outStr);

      return(0);
}
```

Листинг 10.18 – Makefile для сборки программ

```
CCOBJS = creditcard.o upper.o

ifdef DEBUG
DEBUGFLGS = -g
else
DEBUGFLGS =
endif

all: creditcard creditcardpie uppercanary

%.o : %.S
      gcc -mno-relax $(DEBUGFLGS) -c $< -o $@

%.o : %.s
      as -mno-relax $(DEBUGFLGS) $< -o $@

creditcard: $(CCOBJS)
      ld -o creditcard $(CCOBJS)

creditcardpie: $(CCOBJS)
      ld -pie -o creditcardpie $(CCOBJS)

uppercanary: upper.c
      gcc -o uppercanary -fstack-protector-all -O3 upper.c
```

Это приводит к вставке дополнительного кода в каждую функцию. После компиляции и запуска модифицированного примера программа действительно обнаруживает переполнение – Листинг 10.19.

Листинг 10.19 – Пример выполнения программы

```
ptsyngalev@ubuntu:~/lab8$ make
gcc -o uppercanary -fstack-protector-all -O3 upper.c
ptsyngalev@ubuntu:~/lab8$ ./uppercanary
Input: This is a test!xxxxxxxxxxxxxxxxxxxx
andeven longerandlongerandlonger
Output: THIS IS A TEST! XXXXXXXXXXXXXXXXXXXX
ANDEVENLONGERANDLONGERANDLONGER
* stack smashing detected *: terminated
Aborted
```

Компилятор GNU вставил в пролог и эпилог функции main дополнительные инструкции, обеспечивающие сохранение канарейки и проверку ее целостности. В Листинге 10.20 приведен фрагмент дизассемблированного кода функции main.

Листинг 10.20 – Дамп выполнения программы

```
0000000000000690 <main>:
690: 7179            add  sp, sp, -48
692: f022            sd   s0, 32(sp)
694: 0030            add  a2, sp, 8
696: 00002417        auipc s0, 0x2
69a: 9a43403         ld   s0, -1622(s0)   #
_stack_chk_guard@GLIBC_2.27
69e: f406            sd   ra, 40(sp)
6a0: 601c            ld   a5, 0(s0)
6a2: ec3e            sd   a5, 24(sp)
6a4: 4781            li   a5, 0

// тело функции …

6fc: 6762            ld   a4, 24(sp)
6fe: 601c            ld   a5, 0(s0)
700: 8fb9            xor  a5, a5, a4
702: 4701            li   a4, 0
704: e791            bnez a5, 710 <main+0x80>
706: 70a2            ld   ra, 40(sp)
708: 7402            ld   s0, 32(sp)
70a: 4501            li   a0, 0
70c: 6145            add  sp, sp, 48
70e: 8082            ret

710: f61ff0ef        jal  670 <_stack_chk_fail@plt>
```

В пролог было вставлено 8 инструкций, которых не было в оригинальной версии.

1) Выделение места в стеке: “add sp, sp, -48”.

Выделяются 48 байт под локальные данные функции *и* область для канарейки.

2) Сохранение регистра s0: “sd s0, 32(sp)”, s0 используется как указатель на глобальную переменную _stack_chk_guard.

3) Подготовка указателя на стековый кадр: “add a2, sp, 8”. Стандартная настройка кадра при ABI System V RISC-V.

4) Загрузка адреса глобальной канарейки: “auipc s0, 0x2”, “ld s0, -1622(s0)”. Двухинструкционный паттерн загрузки 64-битного адреса. В результате s0 указывает на _stack_chk_guard.

5) Сохранение адреса возврата: “sd ra, 40(sp)”. Защищаем ra – цель атак переполнения буфера.

6) Загрузка глобальной канарейки: “ld a5, 0(s0)”. Загружаем секретное случайное значение.

7) Запись канарейки в стек: “sd a5, 24(sp)”. Помещаем копию глобальной канарейки рядом с адресом возврата в стеке.

8) Очистка регистра, чтобы исключить утечку значения: “li a5, 0”. Чтобы не оставить канарейку в регистре.

В эпилоге компилятор вставил 11 инструкций.

1) Загрузка канарейки из стека: “ld a4, 24(sp)” – текущее значение, возможно поврежденное.

2) Загрузка эталонной канарейки из сегмента данных: “ld a5, 0(s0)”.

3) Сравнение через XOR: “xor a5, a5, a4”. Если они различаются → результат ≠ 0.

4) Очистка временного регистра: “li a4, 0”. Для защиты.

5) Условный переход: “bnez a5, 710”. Если XOR ≠ 0 → вызываем _stack_chk_fail.

6–10) Восстановление регистров и возврат: “ld ra, 40(sp)”, “ld s0, 32(sp)”, “li a0, 0”, “add sp, sp, 48”, “ret”.

11) Аварийное завершение: “jal _stack_chk_fail@plt”. Если канарейка повреждена → программа завершается.

Современные механизмы защиты стремятся предотвратить выполнение исходного кода в стеке, поскольку именно этот подход лежал в основе первых эксплойтов переполнения буфера. На полнофункциональных системах с архитектурой RISC-V, таких как VisionFive или QEMU, аппаратная защита

памяти позволяет помечать страницы как читаемые, записываемые и выполняемые, а Linux по умолчанию запрещает выполнение стека. Это делает классические эксплойты значительно сложнее и требует специальных условий или дополнительных уязвимостей, чтобы вновь включить исполняемость стека. Однако микроконтроллеры вроде ESP32-C3, имеющие минимальный набор инструкций и лишенные полноценной защиты памяти, остаются уязвимыми для выполнения произвольного кода из стека. Такие устройства широко используются в IoT, часто обрабатывают внешние данные и могут быть легко подменены злоумышленником, способным внедрить вредоносный трафик, вызывающий переполнение. Поэтому любые взаимодействия с внешними потоками данных требуют строгой проверки, особенно учитывая многочисленные реальные случаи успешных атак на IoT-устройства.

Несмотря на развитие методов предотвращения переполнения буфера, каждая из стратегий имеет свои компромиссы. Использование безопасных функций, проверок в системах контроля версий, PIE, NX-bit и других техник позволяет значительно снизить риск, но не устраняет его полностью и может снижать производительность. Дополнительную опасность представляют утечки адресов памяти в диагностических сообщениях, поскольку даже единичный вывод значения регистра помогает хакеру определить смещение PIE или другие детали размещения программы. Атаки типа Spectre и Meltdown также дают возможность косвенно считывать биты памяти через поведение кеша, увеличивая вероятность получения важных метаданных защиты – например, адресов памяти или стековых канареек. Если же какие-то общие библиотеки отключают PIE или были собраны без защиты, то вся программа теряет соответствующий уровень безопасности, что делает необходимым постоянную проверку итогового бинарного файла и используемых зависимостей. Даже при активировании всех доступных защит остается риск ошибок человеческого фактора: невнимательность, опечатки, неучтенные сценарии обработки данных.

В высокопроизводительных системах дополнительные проверки могут оказаться слишком дорогими, и тогда требуется архитектурное разделение: внешние слои должны тщательно валидировать данные, а оптимизированные подпрограммы – работать только с гарантированно корректными входами. Важную роль играют ревью исходного кода и анализ безопасности специалистами, знакомыми с техниками взлома. При этом недопустимо полагаться на проверку пользовательского интерфейса, поскольку браузерный JavaScript легко модифицируется злоумышленником, который может

обходить любые клиентские проверки, отправляя запросы напрямую на сервер. Сильные стороны защиты, такие как PIE, NX и стековые канарейки, дают значительную устойчивость, но остаются уязвимыми при неправильной конфигурации библиотек или частичном применении. Хакеры, объединяя отдельные недостатки, находят способы обойти даже комплексные меры защиты, поэтому безопасность всегда должна рассматриваться как процесс, требующий постоянного контроля и многослойного подхода.

10.5. Заключение

Выполнение практической работы позволило закрепить знания о методах оптимизации ассемблерного кода, приёмах ассемблирования и дизассемблирования программ, а также об основных уязвимостях низкоуровневого программного обеспечения. Студент познакомился с техниками сокращения числа ветвлений, использованием побитовых операций для ускорения обработки данных и анализом реального кода ядра Linux для архитектуры RISC-V.

Приобретённые навыки работы с утилитами objdump и Ghidra для анализа скомпилированных программ, а также понимание механизмов переполнения буфера и методов защиты от него — позиционно независимого кода (PIE), стековых канареек и запрета выполнения стека (NX bit) — имеют ключевое значение для разработки безопасного программного обеспечения. Изучение примера эксплуатации уязвимости с подменой адреса возврата наглядно демонстрирует, к каким последствиям приводит отсутствие проверки длины входных данных.

Таким образом, выполненная работа формирует базовые компетенции для дальнейшего изучения вопросов информационной безопасности, аудита исходного кода, а также понимания взаимосвязи между качеством низкоуровневого программирования и защищённостью современных вычислительных систем.

10.6. Варианты заданий

Задание №1. Собрать и запустить программы из Листингов 10.1 – 10.7 в одном общем цикле. Сравнить время запуска различных версий программ по переводу символов в верхний регистр. Для этого можно использовать службу “gettimeofday” или “time”.

Задание №2. Реализовать функцию преобразование строки из табл. 10.1 на C/C++. Скомпилировать программу и проанализировать ее ассемблерное

представление при помощи objdump. Сравнить результат выполнения со своей реализацией этой же функции на ассемблере. Сделать выводы.

Задание №3. Для скомпилированной программы из Задания №2 восстановить исходных код при помощи инструмента Ghidra. Проанализировать результат и сравнить с исходной функцией, исправить код “восстановленной” функции (при необходимости) и запустить.

Задание №4. Собрать программу из Листингов 10.13 – 10.14 включить режим PIE и проверить корректность выполнения, сделать выводы.

Варианты функций – табл. 4.7.

Таблица 10.1 – Варианты заданий

Вариант	**Пример входного строкового литерала**	**Задача обработки символов**
1	"Hello World!"	Конвертировать все буквы в верхний регистр
2	"Hello World!"	Конвертировать все буквы в нижний регистр
3	"R1SC-V 2025"	Все цифры заменить на символ #
4	"Lab Work 5"	Все пробелы заменить на _
5	"Test_String!"	Все символы, не являющиеся буквами, заменить на пробел
6	"Assembly123"	Все буквы конвертировать в верхний регистр, цифры оставить без изменений
7	"abcdefg"	Конвертировать в верхний регистр только первые 3 буквы
8	"RiscV_Lab"	Конвертировать в нижний регистр только последние 3 буквы
9	"123-456-789"	Заменить все - на /
10	"VarTest"	Конвертировать буквы в верхний регистр и добавить ! в конец строки
11	"Hello, RISC-V!"	Заменить все знаки препинания на .
12	"Lab5_Example"	Конвертировать все буквы в нижний регистр и все цифры на 0

11. ОРГАНИЗАЦИЯ ПАРАЛЛЕЛЬНЫЕ ВЫЧИСЛЕНИЯ

11.1. Введение

Современные требования к производительности вычислительных систем постоянно возрастают в связи с увеличением объемов обрабатываемых данных, сложности решаемых задач и необходимостью получения результатов в реальном времени. Традиционный подход к повышению производительности за счет увеличения тактовой частоты процессоров почти достиг физических ограничений, связанных с проблемами энергопотребления и отвода тепла. В связи с этим основным направлением развития вычислительных систем стало применение параллельных вычислений.

Параллельные вычисления представляют собой одновременное выполнение нескольких операций или процессов с целью ускорения решения общей задачи. Данный подход реализуется как в рамках одной вычислительной машины (многоядерные процессоры, графические ускорители), так и путем создания многопроцессорных систем и распределенных вычислительных кластеров.

Общей отличительной особенностью параллельных вычислительных систем является наличие средств, реализующих параллельную обработку за счет введения параллельных ветвей в вычислениях, что не предусматривалось классической архитектурой ЭВМ фон Неймана. Современные вычислительные системы используют различные уровни параллелизма – от микроуровня (конвейеризация) до уровня заданий (многопроцессорные системы).

11.2. История возникновения параллельных вычислений

Идея параллельной обработки информации возникла практически одновременно с появлением первых электронных вычислительных машин. Уже в 1950-х годах исследователи понимали, что последовательная архитектура фон Неймана имеет принципиальные ограничения производительности.

Первые шаги в области параллельных вычислений были сделаны в 1958 году, когда компания IBM представила компьютер IBM 704, в котором использовались независимые блоки для операций с плавающей точкой. В 1962 году появился суперкомпьютер Atlas, разработанный в Манчестерском университете, который использовал конвейерную обработку команд – один из первых примеров параллелизма на уровне инструкций.

Значительный прорыв произошел в 1960-х годах с появлением концепции многопроцессорных систем. В 1966 году Майкл Флинн предложил классификацию вычислительных систем, которая до сих пор является основной для описания параллельных архитектур. Классификация Флинна разделяет системы по способу организации потоков команд и данных.

В 1970-х годах компания Cray Research под руководством Сеймура Крея создала серию векторных суперкомпьютеров Cray-1, которые использовали SIMD-архитектуру для обработки больших массивов данных. Эти машины стали стандартом высокопроизводительных вычислений на несколько десятилетий.

1980-е годы ознаменовались развитием массивно-параллельных систем (MPP – Massively Parallel Processing). Компании Thinking Machines и nCUBE создали системы с тысячами параллельно работающих процессоров. Именно в этот период начались активные исследования в области программных моделей для параллельных вычислений.

В 1990-х годах произошла революция в доступности параллельных вычислений. Появление стандартов передачи сообщений MPI (Message Passing Interface) в 1994 году и OpenMP в 1997 году позволило программистам создавать переносимые параллельные приложения. В это же время начали активно развиваться вычислительные кластеры на базе обычных персональных компьютеров, соединенных высокоскоростной сетью.

2000-е годы стали эрой многоядерных процессоров. Компании Intel и AMD начали выпускать процессоры с несколькими вычислительными ядрами на одном кристалле. Это сделало параллельные вычисления доступными для массового пользователя. Параллельно развивались технологии GPGPU (General-Purpose computing on Graphics Processing Units), позволяющие использовать графические процессоры для общих вычислений.

Современный этап развития характеризуется гетерогенными вычислительными системами, объединяющими различные типы процессоров (CPU, GPU, FPGA), развитием облачных вычислений и технологий распределенной обработки больших данных (Big Data). Появление фреймворков таких как Apache Hadoop и Apache Spark позволило эффективно обрабатывать петабайты информации на кластерах из тысяч узлов.

11.3. Классификация параллельных вычислительных систем

11.3.1 Таксономия Флинна

Наиболее распространенной и фундаментальной является классификация, предложенная Майклом Флинном в 1966 году. Она основана на числе одновременных потоков команд и данных, обрабатываемых вычислительной системой. Классификация Флинна выделяет четыре основных класса архитектур.

SISD (Single Instruction Single Data) – одиночный поток команд и одиночный поток данных. Это классическая последовательная архитектура фон Неймана, в которой в каждый момент времени выполняется одна команда над одним элементом данных. К этому классу относятся традиционные однопроцессорные компьютеры без средств параллельной обработки. Схема организации SISD показана на рисунке 11.1.

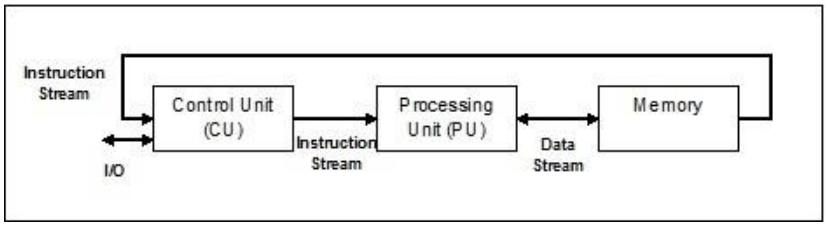


Рисунок 11.1 – Архитектура SISD

SIMD (Single Instruction Multiple Data) – одиночный поток команд и множественный поток данных. В системах этого класса одна команда одновременно выполняется над множеством элементов данных. Такая архитектура эффективна для задач, требующих одинаковой обработки больших массивов данных – обработка изображений, научные расчеты, графика. Примерами являются векторные процессоры, графические ускорители (GPU), а также векторные расширения современных процессоров (SSE, AVX для x86). Схема организации SIMD показана на рис. 11.2.

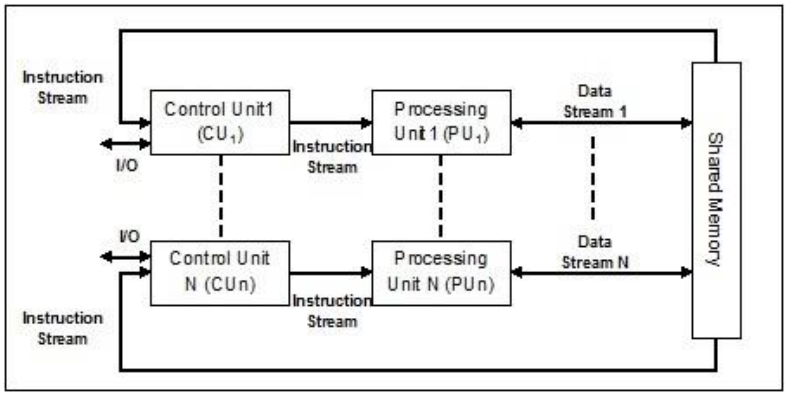


Рисунок 11.2 – Архитектура SIMD

MISD (Multiple Instruction Single Data) – множественный поток команд и одиночный поток данных. В этой архитектуре разные команды одновременно выполняются над одними и теми же данными. Данный класс имеет ограниченное практическое применение и используется в основном в отказоустойчивых системах, где несколько процессоров выполняют одни и те же вычисления для контроля корректности результатов. Схема организации MISD показана на рисунке 11.3.

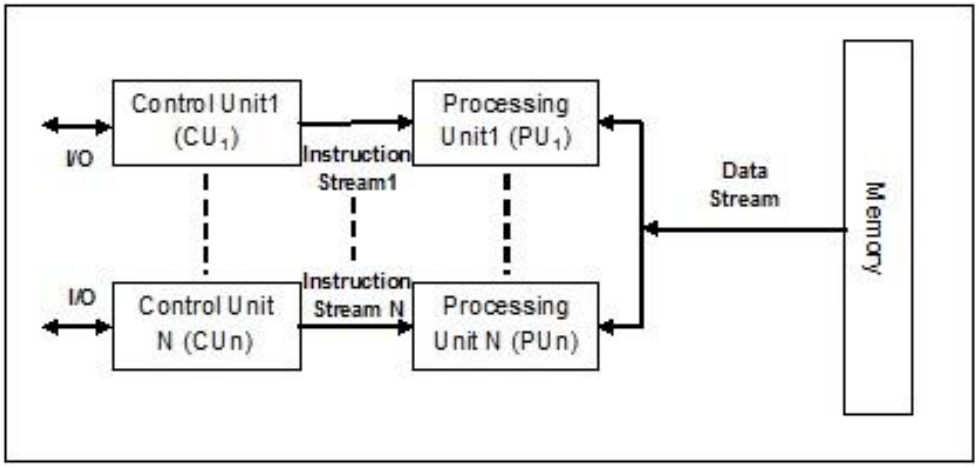


Рисунок 11.3 – Архитектура MISD

MIMD (Multiple Instruction Multiple Data) – множественный поток команд и множественный поток данных. Это наиболее общий и распространенный класс параллельных систем, в котором несколько процессоров независимо выполняют различные команды над различными данными. К этому классу относятся многопроцессорные системы, многоядерные процессоры и вычислительные кластеры. MIMD-системы обеспечивают наивысшую гибкость и могут эффективно решать широкий спектр задач. Схема организации MIMD показана на рисунке 11.4.

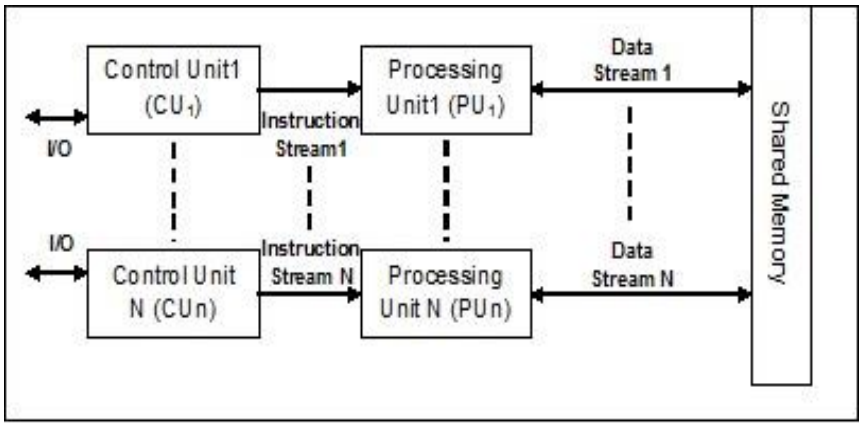


Рисунок 11.4 – Архитектура MIMD

11.3.2 Уровни параллелизма

Параллелизм в современных вычислительных системах может быть реализован на различных уровнях детализации. Выделяют следующие основные уровни параллелизма.

Микроуровень (уровень инструкций) – на этом уровне выполнение команды разделяется на фазы (выборка, декодирование, выполнение, запись результата), а фазы нескольких соседних команд могут выполняться одновременно за счет конвейеризации. Такой уровень параллелизма достижим даже на системах с одним процессором и является стандартной особенностью современных процессоров.

Уровень команд – выражается в параллельном выполнении нескольких независимых команд и достигается посредством размещения в процессоре сразу нескольких конвейеров. Данный уровень реализуется в суперскалярных процессорах, которые могут начинать выполнение нескольких команд за один такт.

Уровень потоков (нитей) – задача разбивается на части, которые могут выполняться параллельно (потоки или нити выполнения). Этот уровень реализуется на многоядерных процессорах и параллельных вычислительных системах. Потоки могут работать с общей памятью и требуют синхронизации доступа к разделяемым ресурсам.

Уровень процессов – несколько независимых процессов одновременно выполняется на нескольких процессорах, практически не взаимодействуя друг с другом или взаимодействуя через передачу сообщений. Такой уровень реализуется на многопроцессорных и многомашинных вычислительных системах.

Уровень заданий – несколько независимых заданий (программ) одновременно выполняются на разных вычислительных узлах. Это наиболее грубый уровень параллелизма, типичный для распределенных систем и вычислительных грид-систем.

11.3.3 Классификация по организации памяти

С точки зрения организации памяти параллельные системы MIMD-класса делятся на два основных типа: системы с общей памятью и системы с распределенной памятью. Такая классификация является критически важной для понимания моделей программирования параллельных приложений.

В табл. 11.1 представлено сравнение основных характеристик систем с общей и распределенной памятью.

Таблица 11.1 – Сравнение систем с общей и распределенной памятью

Характеристика	Общая память	Распределённая память
Адресное пространство	Единое	Раздельное для каждого узла
Обмен данными	Через разделяемые переменные	Через передачу сообщений
Синхронизация	Блокировки, семафоры	Синхронные и асинхронные сообщения
Масштабирование	Ограниченное, эффективность снижается при увеличении числа ядер	Хорошо масштабируется, эффективность сохраняется
Производительность сети и особенности	Низкие задержки доступа, но возможны конфликты доступа, кэш-промахи и влияние NUMA	Зависит от сети; выше задержки, требуется оптимизация обмена

11.4. Системы с общей памятью

Системы с общей памятью (Shared Memory Systems) – это многопроцессорные системы, в которых все процессоры имеют доступ к единому адресному пространству памяти. Любой процессор может читать и записывать данные в любую область памяти. Взаимодействие между процессорами осуществляется через разделяемые переменные, что делает программирование относительно простым и интуитивным.

11.4.1 Архитектуры систем с общей памятью

Системы с общей памятью делятся на несколько подтипов в зависимости от организации доступа к памяти.

UMA (Uniform Memory Access) – системы с равномерным доступом к памяти. В таких системах все процессоры имеют одинаковое время доступа к любой области памяти. Это достигается за счет использования общей шины или коммутатора, связывающего процессоры с блоками памяти. UMA-системы также называют SMP (Symmetric MultiProcessing) – симметричные многопроцессорные системы. Главным ограничением UMA является масштабируемость – при увеличении числа процессоров возникают конфликты за доступ к общей шине.

NUMA (Non-Uniform Memory Access) – системы с неравномерным доступом к памяти. В NUMA-архитектуре память физически распределена между процессорами, но логически представляет единое адресное пространство. Каждый процессор имеет быстрый доступ к локальной памяти и более медленный доступ к памяти других процессоров через межпроцессорные связи. NUMA-системы обеспечивают лучшую

масштабируемость по сравнению с UMA, так как снижают конкуренцию за доступ к памяти.

CC-NUMA (Cache-Coherent NUMA) – системы NUMA с аппаратной поддержкой когерентности кэш-памяти. Когерентность означает, что при изменении данных одним процессором все копии этих данных в кэшах других процессоров автоматически обновляются или аннулируются. Это гарантирует корректность параллельных вычислений, но требует дополнительных аппаратных средств и снижает производительность при интенсивном обмене данными.

Типичная схема организации системы с общей памятью представлена на рис. 11.5.

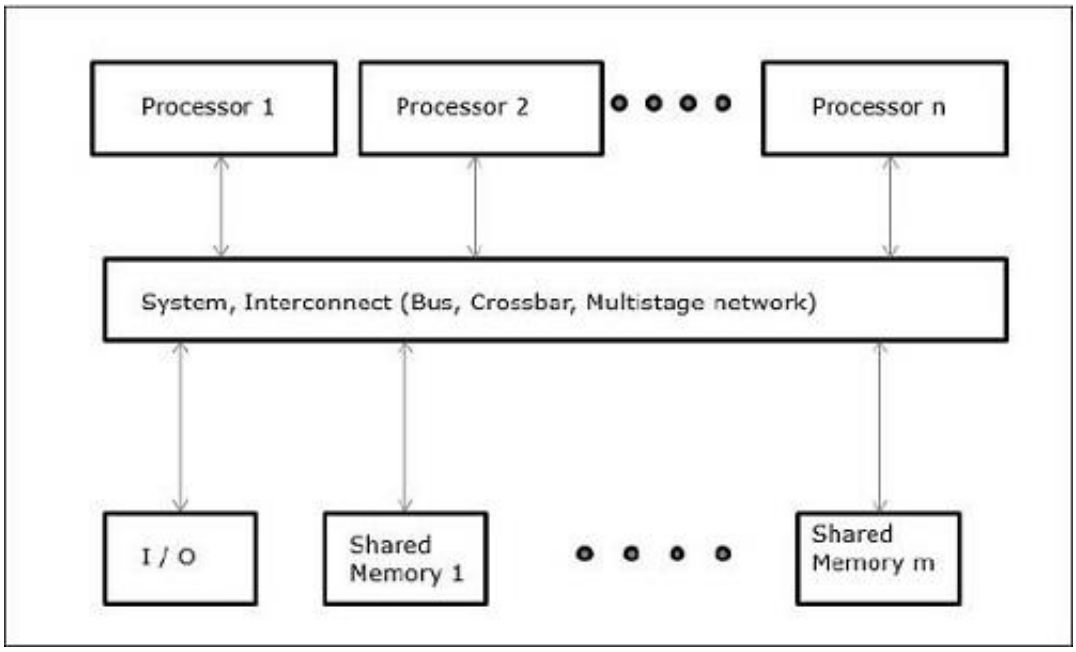


Рисунок 11.5 – Схема организации системы с общей памятью (UMA)

11.4.2 Технология OpenMP

OpenMP (Open Multi-Processing) – это стандарт программирования для систем с общей памятью, поддерживающий языки C, C++, Fortran и, через специальные библиотеки, Python. OpenMP представляет собой набор директив компилятора, библиотечных функций и переменных окружения, позволяющих создавать многопоточные приложения.

Основные преимущества OpenMP включают:

- простоту использования – добавление параллелизма в последовательную программу требует минимальных изменений кода;

- инкрементальный параллелизм – можно распараллеливать программу постепенно, тестируя каждый этап;
- переносимость – стандарт поддерживается большинством современных компиляторов;
- производительность – низкие накладные расходы на создание и синхронизацию потоков.

Модель программирования OpenMP основана на концепции fork-join параллелизма. Программа начинается как один главный поток (master thread), который при встрече директивы параллельной области создает (fork) команду дополнительных потоков (worker threads). После завершения параллельной области потоки синхронизируются и объединяются (join) обратно в главный поток, как показано на рисунке 11.6.

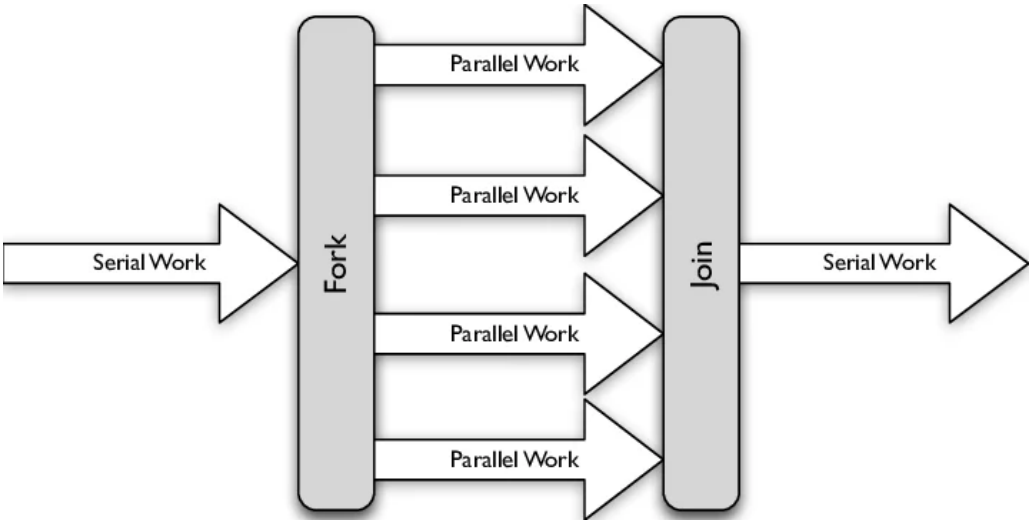


Рисунок 11.6 – Модель fork-join в OpenMP

В Python для работы с OpenMP используется библиотека multiprocessing или специализированные расширения, компилируемые с поддержкой OpenMP через Cython или аналогичные инструменты. Однако для учебных целей часто используется модуль multiprocessing.Pool, который реализует схожую модель параллелизма.

11.4.3 Синхронизация и проблемы параллельного программирования

При программировании систем с общей памятью возникают специфические проблемы, связанные с одновременным доступом нескольких потоков к разделяемым данным.

Гонка данных (Race Condition) – ситуация, когда результат вычислений зависит от относительной скорости выполнения потоков. Это

происходит, когда несколько потоков одновременно пытаются изменить общую переменную без должной синхронизации.

Взаимная блокировка (Deadlock) – ситуация, когда два или более потока бесконечно ждут освобождения ресурсов, захваченных друг другом. Для предотвращения взаимных блокировок необходимо соблюдать порядок захвата ресурсов.

Ложное разделение (False Sharing) – проблема производительности, возникающая, когда независимые данные, используемые разными потоками, располагаются в одной строке кэш-памяти. Изменение данных одним потоком приводит к аннулированию кэша у других потоков, даже если они работают с разными переменными.

Для решения этих проблем используются механизмы синхронизации:

- критические секции – участки кода, в которых одновременно может находиться только один поток;
- атомарные операции – операции, выполняющиеся как единое неделимое действие;
- барьеры – точки синхронизации, где все потоки должны дождаться друг друга перед продолжением;
- блокировки (locks) и семафоры – примитивы синхронизации для управления доступом к ресурсам.

11.5. Системы с рапределенной памятью

Системы с распределенной памятью (Distributed Memory Systems) состоят из множества вычислительных узлов, каждый из которых имеет собственную локальную память. Узлы соединены между собой сетью передачи данных. В отличие от систем с общей памятью, процессоры не имеют прямого доступа к памяти других узлов – обмен данными осуществляется исключительно через передачу сообщений по сети.

11.5.1 Архитектуры систем с распределенной памятью

MPP (Massively Parallel Processing)– массивно-параллельные системы, состоящие из большого числа (сотни и тысячи) относительно простых процессорных узлов, соединенных специализированной высокоскоростной сетью. MPP-системы проектируются как единое целое, с оптимизированной топологией связей (решетка, тор, гиперкуб, толстое дерево).

Кластерные системы – вычислительные системы, построенные из стандартных серверных узлов (часто на базе обычных серверов),

объединенных коммуникационной сетью (Ethernet, InfiniBand). Кластеры отличаются от MPP более низкой стоимостью, модульностью и возможностью постепенного наращивания мощности. Современные суперкомпьютеры в основном построены по кластерной архитектуре.

Грид-системы – географически распределенные гетерогенные вычислительные ресурсы, объединенные в единую систему через глобальные сети. Грид используется для решения задач, не требующих интенсивного обмена данными между узлами.

Схема организации системы с распределенной памятью показана на рисунке 11.7.

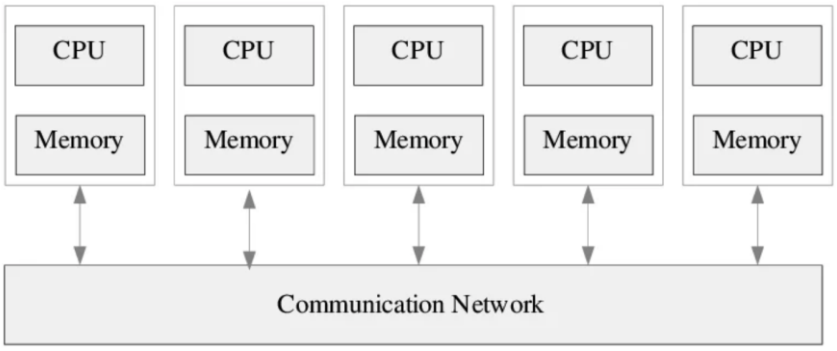


Рисунок 11.7 – Схема организации кластерной системы с распределенной памятью

11.5.2 Технология MPI

MPI (Message Passing Interface) – это стандарт программирования для систем с распределенной памятью, определяющий синтаксис и семантику библиотечных функций для передачи сообщений между процессами. MPI был разработан в 1994 году и стал де-факто стандартом для параллельного программирования на кластерах и суперкомпьютерах.

Основные характеристики MPI:

- переносимость – приложения работают на различных платформах без изменений;
- производительность – эффективная реализация коммуникаций для различных типов сетей;
- функциональность – богатый набор функций для различных схем обмена данными;
- масштабируемость – поддержка систем от нескольких до сотен тысяч процессоров.

Модель программирования MPI основана на следующих принципах:

- программа состоит из множества независимых процессов, каждый из которых имеет собственное адресное пространство;
- процессы взаимодействуют путем явной передачи сообщений;
- каждый процесс имеет уникальный идентификатор (ранг) в пределах коммуникатора;
- коммуникатор определяет группу процессов, которые могут обмениваться сообщениями.

В Python для работы с MPI используется библиотека mpi4py, которая предоставляет Python-интерфейс к стандартным реализациям MPI (Open MPI, MPICH).

11.5.3 Типы коммуникаций в MPI

MPI поддерживает два основных типа коммуникаций: точка-точка (point-to-point) и коллективные (collective).

Коммуникации точка-точка – обмен данными между двумя процессами. Основные функции:

- функция Send/Recv – синхронная отправка и прием сообщений;
- функция Isend/Irecv – асинхронная (неблокирующая) отправка и прием;
- функция Sendrecv – одновременная отправка и прием для избежания взаимных блокировок.

Коллективные коммуникации – операции, в которых участвует группа процессов. Основные типы:

- процесс Broadcast (широковещательная рассылка) – один процесс отправляет данные всем остальным;
- процесс Scatter (распределение) – один процесс распределяет части массива между всеми процессами;
- процесс Gather (сбор) – один процесс собирает данные от всех процессов;
- процесс Reduce (редукция) – выполнение операции (сумма, максимум и т. д.) над данными от всех процессов;
- процесс Allreduce – редукция с отправкой результата всем процессам;
- процесс Barrier (барьер) – синхронизация всех процессов.

Схема работы некоторых коллективных операций представлена на рис. 11.8.

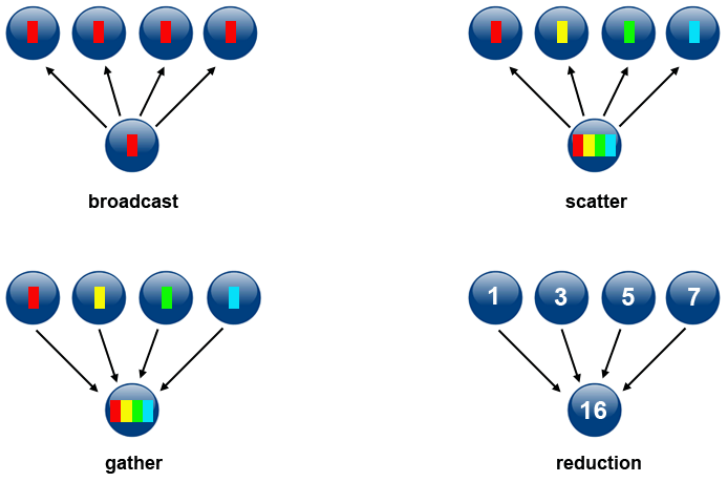


Рисунок 11.8 – Коллективные операции MPI: Broadcast, Scatter, Gather

11.5.4 Топологии и производительность

Производительность систем с распределенной памятью существенно зависит от характеристик сети передачи данных. Основные параметры сети:

- задержка (latency) – время инициализации передачи сообщения;
- пропускная способность (bandwidth) – скорость передачи данных;
- топология – схема соединения узлов.

Современные кластеры используют высокоскоростные сети InfiniBand, обеспечивающие задержку порядка 1-5 микросекунд и пропускную способность до 200 Гбит/с. Для эффективного использования систем с распределенной памятью необходимо минимизировать объем и частоту коммуникаций, что достигается за счет:

- использования локальных вычислений вместо передачи данных если возможно;
- применения асинхронных коммуникаций для перекрытия обмена и вычислений;
- использования коллективных операций вместо множества операций точка-точка;
- оптимизации размера сообщений для конкретной сети.

11.6. Метрики и оценка эффективности параллельных систем

11.6.1 Ускорение, эффективность, избыточность, стоимость, масштабируемость

Для оценки эффективности параллельных вычислений используется набор количественных характеристик, позволяющих сравнивать различные

реализации алгоритмов и аппаратные платформы. Основные метрики включают ускорение, эффективность, избыточность вычислений и масштабируемость.

Ускорение S(p) показывает, во сколько раз параллельная программа на p процессорах выполняется быстрее последовательной версии: $S(p) = T_1 / T_p$, где T_1 – время выполнения последовательной программы (или параллельной на одном процессоре), T_p – время выполнения параллельной программы на p процессорах. Идеальное (линейное) ускорение достигается при $S(p) = p$, что означает, что программа на p процессорах выполняется ровно в p раз быстрее. На практике линейное ускорение недостижимо из-за накладных расходов на взаимодействие процессов, синхронизацию и наличия последовательных участков кода. Иногда наблюдается линейное ускорение $S(p) > p$, которое может быть вызвано эффектами кэширования (данные лучше умещаются в кэш при распределении по процессорам) или особенностями алгоритма.

Эффективность E(p) характеризует степень использования вычислительных ресурсов и определяется как отношение ускорения к числу процессоров: $E(p) = S(p) / p = T_1 / (p * T_p)$. Эффективность изменяется в диапазоне от 0 до 1 (или от 0% до 100%). Значение $E(p) = 1$ означает идеальное использование ресурсов, когда все процессоры постоянно заняты полезной работой. На практике эффективность обычно снижается с ростом числа процессоров из-за увеличения доли времени, затрачиваемого на коммуникации и синхронизацию.

Избыточность вычислений O(p) – это дополнительное время, которое тратится в параллельной программе по сравнению с идеальным случаем: $O(p) = p * T_p - T_1$. Эта метрика показывает суммарные потери производительности на всех процессорах. Избыточность может быть вызвана:

- коммуникациями между процессорами;
- синхронизацией и ожиданием;
- дублированием вычислений;
- неравномерным распределением нагрузки (дисбаланс нагрузки).

Относительная избыточность вычисляется как: $O_rel(p) = O(p) / T_1 = (p * T_p - T_1) / T_1 = p / S(p) – 1$.

Стоимость параллельного алгоритма C(p) определяется как произведение времени выполнения на число процессоров: $C(p) = p * T_p$. Оптимальный параллельный алгоритм имеет стоимость, равную времени выполнения лучшего последовательного алгоритма: $C(p) = T_1$, что эквивалентно $E(p) = 1$.

Масштабируемость характеризует способность системы сохранять эффективность при увеличении числа процессоров. Можно выделить два типа масштабируемости. Сильная масштабируемость (strong scaling) – способность ускорять решение задачи фиксированного размера при увеличении числа процессоров. Идеально масштабируемая система удваивает производительность при удвоении числа процессоров. Слабая масштабируемость (weak scaling) – способность решать пропорционально более крупные задачи при увеличении числа процессоров с сохранением времени выполнения. При идеальной слабой масштабируемости удвоение числа процессоров позволяет решить вдвое большую задачу за то же время.

11.6.2 Закон Амдала

Закон Амдала, сформулированный Джином Амдалом в 1967 году, устанавливает теоретический предел ускорения параллельной программы в зависимости от доли последовательного кода. Если f – доля последовательного кода ($0 \leq f \leq 1$), то максимальное ускорение на p процессорах ограничено: $S_Amdahl(p) = 1 / (f + (1-f)/p)$. При $p \to \infty$ предельное ускорение составляет $S_max = 1/f$. Например, если 10% кода последовательны ($f = 0.1$), максимальное ускорение не превысит 10 раз независимо от числа процессоров. Закон Амдала подчеркивает важность минимизации последовательных участков для достижения высокой производительности на большом числе процессоров.

Закон Амдала исходит из предположения о фиксированном размере задачи (сильная масштабируемость) и часто дает пессимистичную оценку возможного ускорения.

11.6.3 Закон Густафсона

Закон Густафсона, предложенный Джоном Густафсоном в 1988 году, предлагает более оптимистичный взгляд на масштабируемость параллельных программ. В отличие от закона Амдала, закон Густафсона рассматривает модель слабой масштабируемости, где размер задачи увеличивается пропорционально числу процессоров.

Если α – доля последовательного времени выполнения на параллельной системе, то ускорение определяется как: $S_Gustafson(p) = \alpha + p(1 - \alpha) = p - \alpha(p - 1)$.

В отличие от закона Амдала, ускорение по Густафсону растет почти линейно с увеличением числа процессоров. Это объясняется тем, что при

увеличении вычислительных ресурсов пользователи склонны решать более крупные задачи, а не просто быстрее решать задачи фиксированного размера.

Закон Густафсона утверждает, что масштабированное ускорение (scaled speedup) может быть близко к линейному, если параллельная часть задачи масштабируется соответственно числу процессоров. Это более реалистичная модель для многих практических приложений, таких как научные симуляции, обработка больших данных и машинное обучение.

11.6.4 Закон Сана-Ная

Закон Сан-Ная, разработанный Сюэ-Хуэй Сун и Лайонелом Ни в 1990 году, представляет собой обобщенную модель, которая учитывает как фиксированные, так и масштабируемые части задачи. Этот закон объединяет подходы Амдала и Густафсона, добавляя учет накладных расходов на коммуникации.

Модель Сан-Ная учитывает три компоненты:

- последовательная часть (не масштабируется);
- параллельная часть (масштабируется с числом процессоров);
- коммуникационные издержки (растут с числом процессоров).

Ускорение по закону Сан-Ная определяется как: $S_{Sun-Ni}(p) = p / (1 + \alpha(p-1) + \beta\sqrt{p})$, где α – доля последовательного кода, β – коэффициент коммуникационных издержек.

Закон Сан-Ная показывает, что реальная производительность параллельных систем ограничена не только последовательными участками кода, но и накладными расходами на межпроцессорные коммуникации. При малых β ускорение близко к модели Густафсона, а при больших β – к модели Амдала или даже хуже.

Эта модель особенно актуальна для систем с распределенной памятью, где коммуникационные издержки играют значительную роль. Параметр β зависит от характеристик сети (задержка и пропускная способность) и паттернов обмена данными в алгоритме.

11.7. Заключение

В данной работе были рассмотрены основные подходы к организации параллельных вычислений и архитектуры параллельных систем. Параллельные вычисления являются ключевым направлением развития современной вычислительной техники из-за роста числа ядер процессоров. Были рассмотрены основные метрики производительности: ускорение,

эффективность, избыточность и масштабируемость. Также были рассмотрены законы Амдала, Густафсона и Сан-Ная, описывающие ограничения и возможности параллельных систем. На примере реализации метода трапеций была показана эффективность распараллеливания вычислений. Показано, что эффективность параллелизации зависит от соотношения вычислений и обмена данными между процессами.

11.8. Варианты заданий

В качестве примера использования параллельных вычислений возьмем задачу численного интегрирования функции методом трапеций и напишем программу в трех вариантах: последовательном, с использованием OpenMP (многопоточность) и с использованием MPI (распределенная память). Программа должна также вычислять метрики производительности и визуализировать результаты для сравнения времени выполнения.

Метод трапеций является одним из простейших численных методов для вычисления определенного интеграла. Суть метода заключается в аппроксимации площади под кривой функции суммой площадей трапеций. Для функции f(x) на интервале [a, b] интеграл приближенно вычисляется по формуле:

$$I_{\text{трапеция } n} = \frac{h}{2} * \sum_{i=0}^{n} k_i f(x_i), h = \frac{b-a}{n}, x_i = a + i * h, i = \overline{0,n},$$

$$k_i = \begin{cases} 1, i = 0 \\ 2, i = \overline{1, n-1} \\ 1, i = n \end{cases}$$

Чем больше число разбиений n, тем точнее результат, но и тем больше требуется вычислений. Это делает задачу подходящей для параллелизации, так как вычисление значений функции в разных точках независимо.

Демонстрационная прогрмма состоит из следующих основных функций – Листинги 11.1-11.9.

Листинг 11.1 – calculate_metrics.py

```
def calculate_metrics(time_seq, times_parallel, num_processors_list):
      """
      Вычисляет метрики производительности параллельных программ.

      Параметры:
      ----------
      time_seq : float
```

Продолжение листинга 11.1

```
            Время выполнения последовательной программы
      times_parallel : list of float
            Список времен выполнения параллельной программы для разного
числа процессоров
      num_processors_list : list of int
            Список количества процессоров/потоков

      Возвращает:
      -----------
      list of dict
            Список словарей с метриками для каждого числа процессоров.
            Каждый словарь содержит:
            - processors: int - число процессоров
            - time: float - время выполнения
            - speedup: float - ускорение S(p) = T₁ / Tₚ
            - efficiency: float - эффективность E(p) = S(p) / p × 100%
            - overhead: float - избыточность O(p) = p × Tₚ - T₁
            - cost: float - стоимость C(p) = p × Tₚ

      Формулы:
      --------
      - Ускорение: S(p) = T₁ / Tₚ
      - Эффективность: E(p) = S(p) / p
      - Избыточность: O(p) = p × Tₚ - T₁
      - Стоимость: C(p) = p × Tₚ

      Пример:
      -------
      >>> metrics = calculate_metrics(10.0, [5.2, 2.7], [2, 4])
      >>> print(metrics[0]['speedup'])
      1.92
      >>> print(metrics[0]['efficiency'])
      96.0
      """
      metrics = []

      for p, time_p in zip(num_processors_list, times_parallel):
            # Ускорение
            speedup = time_seq / time_p if time_p > 0 else 0

            # Эффективность (в процентах)
            efficiency = (speedup / p * 100) if p > 0 else 0

            # Избыточность
            overhead = p * time_p - time_seq

            # Стоимость
            cost = p * time_p

            metrics.append({
```

Продолжение листинга 11.1

```
                'processors': p,
                'time': time_p,
                'speedup': speedup,
                'efficiency': efficiency,
                'overhead': overhead,
                'cost': cost
            })

    return metrics
```

Листинг 11.2 – main.py

```
import time
import math
import numpy as np
from pathlib import Path

from .sequential_integration import sequential_integration
from .parallel_integration_openmp import parallel_integration_openmp
from .calculate_metrics import calculate_metrics
from .print_metrics_table import print_metrics_table
from .plot_scaling_laws import plot_scaling_laws
from .visualize_results import visualize_results

try:
    from .parallel_integration_mpi import parallel_integration_mpi,
MPI_AVAILABLE
except ImportError:
    MPI_AVAILABLE = False

# ========== ПАРАМЕТРЫ ВАРИАНТА (ИЗМЕНЯЕМАЯ ЧАСТЬ) ==========
VARIANT_NUM = 1                     # Номер варианта (1-30)
FUNCTION_TYPE = 'sin'               # Тип функции: 'sin', 'x2', 'exp'
FUNCTION_NAME = 'k × sin(x)'        # Название функции для вывода
K = 10.0                            # Параметр функции
A = 4.0                             # Нижняя граница интегрирования
B = math.pi                         # Верхняя граница интегрирования
N = 10_000_000                      # Число разбиений
THREADS_LIST = [2, 4, 8]            # Потоки/процессы для теста (значения
используются и для MPI)
OUTPUT_DIR = 'results'              # Директория для результатов
# ============================================================

def function_to_integrate(x, k, function_type='sin'):
    """
    Вычисляет значение функции в точке x.

    Параметры:
    ----------
    x : float
        Точка, в которой вычисляется значение
    k : float
```

Продолжение листинга 11.2

```
            Коэффициент функции
      function_type : str
            Тип функции: 'sin', 'x2', 'exp'
      """
      if function_type == 'sin':
            return k * np.sin(x)
      elif function_type == 'x2':
            return k * x * x
      elif function_type == 'exp':
            return k * np.exp(-x)
      else:
            raise ValueError(f"Неизвестный тип функции: {function_type}")

def estimate_sequential_fraction(metrics):
      """
      Оценивает долю последовательного кода из экспериментальных данных.
      Использует закон Амдала: α = (1/S - 1/p) / (1 - 1/p)
      """
      if not metrics:
            return 0.1

      last_metric = metrics[-1]
      p = last_metric['processors']
      S = last_metric['speedup']

      if p > 1 and S > 0:
            alpha = (1/S - 1/p) / (1 - 1/p)
            return max(0.01, min(0.3, alpha))
      else:
            return 0.1

def main():
      """Главная функция программы."""
      globals()
      print("\n" + "="*70)
      print("ПРАКТИЧЕСКАЯ РАБОТА №11")
      print("Организация параллельных вычислений")
      print("Численное интегрирование методом трапеций")
      print("="*70)

      print(f"\nВариант: {VARIANT_NUM}")
      print(f"Функция: {FUNCTION_NAME}")
      print(f"Параметр k: {K}")
      print(f"Границы интегрирования: [{A:.4f}, {B:.4f}]")
      print(f"Число разбиений: {N:,}")
      print(f"Потоки для тестирования: {THREADS_LIST}")

      # Создание директории для результатов
      output_path = Path(OUTPUT_DIR)
      output_path.mkdir(parents=True, exist_ok=True)
```

Продолжение листинга 11.2

```
      # ========== 1. ПОСЛЕДОВАТЕЛЬНАЯ ВЕРСИЯ ==========
      print("\n" + "="*70)
      print("1. ПОСЛЕДОВАТЕЛЬНАЯ ВЕРСИЯ")
      print("="*70)

      start = time.time()
      result_seq = sequential_integration(A, B, N, K, FUNCTION_TYPE)
      time_seq = time.time() - start

      print(f"Результат интегрирования: {result_seq:.10f}")
      print(f"Время выполнения: {time_seq:.4f} секунд")

      # ========== 2. ПАРАЛЛЕЛЬНАЯ ВЕРСИЯ (OpenMP) ==========
      print("\n" + "="*70)
      print("2. ПАРАЛЛЕЛЬНАЯ ВЕРСИЯ (OpenMP)")
      print("="*70)

      times_openmp = []
      results_openmp = []

      for num_threads in THREADS_LIST:
            print(f"\nТестирование с {num_threads} потоками...")
            start = time.time()
            result = parallel_integration_openmp(A, B, N, K, FUNCTION_TYPE,
num_threads)
            time_parallel = time.time() - start

            times_openmp.append(time_parallel)
            results_openmp.append(result)

            speedup = time_seq / time_parallel if time_parallel > 0 else 0
            efficiency = (speedup / num_threads) * 100

            print(f"  Результат: {result:.10f}")
            print(f"  Время: {time_parallel:.4f}s | Ускорение:
{speedup:.2f}x | Эффективность: {efficiency:.1f}%")

      # ========== 3. ПАРАЛЛЕЛЬНАЯ ВЕРСИЯ (MPI) ==========
      print("\n" + "="*70)
      print("3. ПАРАЛЛЕЛЬНАЯ ВЕРСИЯ (MPI)")
      print("="*70)

      times_mpi = []
      results_mpi = []

      if MPI_AVAILABLE:
            print("MPI доступен! Запуск тестов...")
            for np in THREADS_LIST:
                  print(f"  mpiexec -n {np} python -m
src.parallel_integration_mpi {VARIANT_NUM}")
            print()
```

Продолжение листинга 11.2

```
            # Локальное тестирование MPI (однопроцессорный режим)
            for num_proc in THREADS_LIST:
                  print(f"\nЛокальный тест с имитацией {num_proc}
процессов...")
                  start = time.time()
                  try:
                        result_mpi, _, _ = parallel_integration_mpi(A, B, N,
K, FUNCTION_TYPE)
                        time_mpi = time.time() - start

                        # Для локального теста используем время OpenMP как
приближение
                        time_mpi_approx =
times_openmp[THREADS_LIST.index(num_proc)]
                        times_mpi.append(time_mpi_approx)
                        results_mpi.append(result_mpi)

                  print(f"  Результат: {result_mpi:.10f}")
                        print(f"  Время (локальный тест): {time_mpi:.4f}s")
                  except Exception as e:
                        print(f"  Ошибка: {e}")

      times_mpi.append(times_openmp[THREADS_LIST.index(num_proc)])
                        results_mpi.append(result_seq)
      else:
            print("MPI не доступен. Пропуск тестирования.")
            print("\nДля установки MPI:")
            print("  Linux: sudo apt install mpich && pip install mpi4py")
            print("  macOS: brew install open-mpi && pip install mpi4py")
            print("  Windows: установите Microsoft MPI + pip install
mpi4py")

            # Используем данные OpenMP как заглушку
            times_mpi = times_openmp.copy()
            results_mpi = results_openmp.copy()

      # ========== 4. ВЫЧИСЛЕНИЕ МЕТРИК ==========
      print("\n" + "="*70)
      print("4. ВЫЧИСЛЕНИЕ МЕТРИК ПРОИЗВОДИТЕЛЬНОСТИ")
      print("="*70)

      # Метрики для OpenMP
      print("\n--- OpenMP (multiprocessing) ---")
      metrics_openmp = calculate_metrics(time_seq, times_openmp,
THREADS_LIST)
      print_metrics_table(metrics_openmp, VARIANT_NUM)

      # Метрики для MPI
      print("\n--- MPI (распределенная память) ---")
      metrics_mpi = calculate_metrics(time_seq, times_mpi, THREADS_LIST)
```

Продолжение листинга 11.2

```
      print_metrics_table(metrics_mpi, VARIANT_NUM)

      # ========== 5. ВИЗУАЛИЗАЦИЯ РЕЗУЛЬТАТОВ ==========
      print("\n" + "="*70)
      print("5. ВИЗУАЛИЗАЦИЯ РЕЗУЛЬТАТОВ")
      print("="*70)

      # График сравнения для OpenMP
      times_for_viz_openmp = [time_seq, times_openmp[0], times_openmp[-1]]
      labels_for_viz_openmp = [
            'Последовательная',
            f'OpenMP ({THREADS_LIST[0]} потока)',
            f'OpenMP ({THREADS_LIST[-1]} потоков)'
      ]

      save_prefix_openmp = str(output_path / 'openmp_variant')
      visualize_results(times_for_viz_openmp, labels_for_viz_openmp,
VARIANT_NUM,
                              metrics_openmp, save_prefix_openmp,
technology_type='OpenMP')

      # График сравнения для MPI
      times_for_viz_mpi = [time_seq, times_mpi[0], times_mpi[-1]]
      labels_for_viz_mpi = [
            'Последовательная',
            f'MPI ({THREADS_LIST[0]} процессов)',
            f'MPI ({THREADS_LIST[-1]} процессов)'
      ]

      save_prefix_mpi = str(output_path / 'mpi_variant')
      visualize_results(times_for_viz_mpi, labels_for_viz_mpi, VARIANT_NUM,
                              metrics_mpi, save_prefix_mpi,
technology_type='MPI')

      print(f"\nСоздано графиков: 4")
      print(f"  {save_prefix_openmp}_{VARIANT_NUM}_comparison.png")
      print(f"  {save_prefix_openmp}_{VARIANT_NUM}_metrics.png")
      print(f"  {save_prefix_mpi}_{VARIANT_NUM}_comparison.png")
      print(f"  {save_prefix_mpi}_{VARIANT_NUM}_metrics.png")

       # ========== 6. ПОСТРОЕНИЕ ГРАФИКОВ ЗАКОНОВ МАСШТАБИРУЕМОСТИ
==========
      print("\n" + "="*70)
      print("6. ПОСТРОЕНИЕ ГРАФИКОВ ЗАКОНОВ МАСШТАБИРУЕМОСТИ")
      print("="*70)

      # Оценка параметров для OpenMP
      estimated_alpha_openmp = estimate_sequential_fraction(metrics_openmp)
      estimated_beta = 0.01

      print(f"\n--- Параметры для OpenMP ---")
```

Продолжение листинга 11.2

```
      print(f"Оценочная доля последовательного кода:
{estimated_alpha_openmp*100:.1f}%")
      print(f"Оценочный коэффициент коммуникаций: {estimated_beta}")

      scaling_path_openmp = str(output_path /
f'scaling_laws_openmp_variant_{VARIANT_NUM}.png')
      plot_scaling_laws(estimated_alpha_openmp, estimated_beta,
                              max_processors=32,
save_path=scaling_path_openmp, technology_type='OpenMP')

      # Оценка параметров для MPI
      estimated_alpha_mpi = estimate_sequential_fraction(metrics_mpi)
      estimated_beta_mpi = 0.02  # Для MPI коммуникационные издержки обычно
выше

      print(f"\n--- Параметры для MPI ---")
      print(f"Оценочная доля последовательного кода:
{estimated_alpha_mpi*100:.1f}%")
      print(f"Оценочный коэффициент коммуникаций: {estimated_beta_mpi}")

      scaling_path_mpi = str(output_path /
f'scaling_laws_mpi_variant_{VARIANT_NUM}.png')
      plot_scaling_laws(estimated_alpha_mpi, estimated_beta_mpi,
                              max_processors=32, save_path=scaling_path_mpi,
technology_type='MPI')

      print(f"\nСоздано графиков законов: 2")
      print(f"  ✓ {scaling_path_openmp}")
      print(f"  ✓ {scaling_path_mpi}")

      # ========== 7. ИТОГОВЫЙ ОТЧЕТ ==========
      print("\n" + "="*70)
      print("РАБОТА ЗАВЕРШЕНА УСПЕШНО!")
      print("="*70)

      print(f"\nВсего сохранено файлов: 6")
      print(f"\nOpenMP (общая память):")
      print(f"  • {save_prefix_openmp}_{VARIANT_NUM}_comparison.png")
      print(f"  • {save_prefix_openmp}_{VARIANT_NUM}_metrics.png")
      print(f"  • {scaling_path_openmp}")

      print(f"\nMPI (распределенная память):")
      print(f"  • {save_prefix_mpi}_{VARIANT_NUM}_comparison.png")
      print(f"  • {save_prefix_mpi}_{VARIANT_NUM}_metrics.png")
      print(f"  • {scaling_path_mpi}")

      print(f"\n{'='*70}")
      print("СРАВНЕНИЕ МЕТОДОВ ДЛЯ ПАРАЛЛЕЛИЗАЦИИ ВЫЧИСЛЕНИЙ")
      print(f"{'='*70}")

      print(f"\nOpenMP (общая память):")
```

Продолжение листинга 11.2

```
      print(f"  • Время последовательное: {time_seq:.4f}s")
      print(f"  • Максимальное ускорение: {max(m['speedup'] for m in
metrics_openmp):.2f}x")
      print(f"  • Максимальная эффективность: {max(m['efficiency'] for m in
metrics_openmp):.1f}%")
      print(f"\nMPI (распределенная память):")

      print(f"  • Время последовательное: {time_seq:.4f}s")
      print(f"  • Максимальное ускорение: {max(m['speedup'] for m in
metrics_mpi):.2f}x")
      print(f"  • Максимальная эффективность: {max(m['efficiency'] for m in
metrics_mpi):.1f}%")

      print(f"\n{'='*70}\n")

if __name__ == "__main__":
      main()
```

Листинг 11.3 – parallel_integration_mpi.py

```
import sys
import time
import math
import numpy as np

try:
      from mpi4py import MPI
      MPI_AVAILABLE = True
except ImportError:
      MPI_AVAILABLE = False
      print("Warning: mpi4py не установлен. MPI-версия недоступна.")

def function_to_integrate(x, k, function_type='sin'):
      """
      Вычисляет значение функции в точке x.

      Параметры:
      ----------
      x : float
            Точка, в которой вычисляется значение
      k : float
            Коэффициент функции
      function_type : str
            Тип функции: 'sin', 'x2', 'exp'
      """
      if function_type == 'sin':
            return k * np.sin(x)
      elif function_type == 'x2':
            return k * x * x
      elif function_type == 'exp':
            return k * np.exp(-x)
      else:
```

Продолжение листинга 11.3

```
            raise ValueError(f"Неизвестный тип функции: {function_type}")

def parallel_integration_mpi(a, b, n, k, function_type='sin'):
      """
      MPI-реализация метода трапеций для численного интегрирования.

      Параметры:
      ----------
      a : float
            Нижняя граница интегрирования
      b : float
            Верхняя граница интегрирования
      n : int
            Число разбиений интервала
      k : float
            Параметр функции (коэффициент k)
      function_type : str
            Тип функции: 'sin', 'x2', 'exp'

      Возвращает:
      -----------
      tuple (float, int, int)
             (результат интегрирования, ранг процесса, общее число
процессов)

      Описание:
      ---------
      Каждый MPI-процесс вычисляет интеграл на своей части интервала [a, b].
      Результаты собираются на процессе с рангом 0 с помощью операции
MPI.reduce.

      Алгоритм:
      1. Разбить интервал [a, b] на части по числу процессов
      2. Каждый процесс вычисляет интеграл на своей части
      3. Процесс 0 собирает результаты и суммирует их (операция reduce)
      """
      if not MPI_AVAILABLE:
            raise RuntimeError("mpi4py не установлен. Установите: pip
install mpi4py")

      comm = MPI.COMM_WORLD
      rank = comm.Get_rank()
      size = comm.Get_size()

      h = (b - a) / n

      # Определяем локальные границы для каждого процесса
      local_n = n // size
      local_a = a + rank * local_n * h
      local_b = local_a + local_n * h
```

Продолжение листинга 11.3

```
      # Локальное интегрирование на каждом процессе
      local_integral = 0.0

      for i in range(local_n):
            x = local_a + i * h
            local_integral += function_to_integrate(x, k, function_type)

      # Обработка граничных значений
      # Только первый процесс добавляет левую границу
      if rank == 0:
            local_integral += 0.5 * function_to_integrate(a, k,
function_type)

      # Только последний процесс добавляет правую границу
      if rank == size - 1:
            local_integral += 0.5 * function_to_integrate(b, k,
function_type)

      # Умножаем локальный результат на шаг
      local_integral *= h

      # Сбор результатов на процессе 0 (операция редукции - суммирование)
      total_integral = comm.reduce(local_integral, op=MPI.SUM, root=0)

      return total_integral, rank, size
```

Листинг 11.4 – parallel_integration_openmp.py

```
import math
import time
import numpy as np
from multiprocessing import Pool

def function_to_integrate(x, k, function_type='sin'):
      """
      Вычисляет значение функции в точке x.

      Параметры:
      ----------
      x : float
            Точка, в которой вычисляется значение
      k : float
            Коэффициент функции
      function_type : str
            Тип функции: 'sin', 'x2', 'exp'
      """
      if function_type == 'sin':
            return k * np.sin(x)
      elif function_type == 'x2':
            return k * x * x
      elif function_type == 'exp':
            return k * np.exp(-x)
```

Продолжение листинга 11.4

```
      else:
            raise ValueError(f"Неизвестный тип функции: {function_type}")

def _compute_partial_sum(args):
      """
      Вспомогательная функция для вычисления частичной суммы.
      Используется для параллельного выполнения через multiprocessing.

      Параметры:
      ----------
      args : tuple
             (start_idx, end_idx, a, h, variant_param, function_type)

      Возвращает:
      -----------
      float
            Частичная сумма значений функции
      """
      start_idx, end_idx, a, h, variant_param, function_type = args
      partial_sum = 0.0

      for i in range(start_idx, end_idx):
            x = a + i * h
            partial_sum += function_to_integrate(x, variant_param,
function_type)

      return partial_sum

def parallel_integration_openmp(a, b, n, variant_param, function_type='sin',
num_threads=4):
      """
      Параллельная реализация метода трапеций с использованием
многопоточности.
      Имитирует OpenMP через multiprocessing.

      Параметры:
      ----------
      a : float
            Нижняя граница интегрирования
      b : float
            Верхняя граница интегрирования
      n : int
            Число разбиений интервала
      variant_param : float
            Параметр функции (коэффициент k)
      function_type : str
            Тип функции: 'sin', 'x2', 'exp'
      num_threads : int
            Число параллельных потоков/процессов

      Возвращает:
```

Продолжение листинга 11.4

```
      -----------
      float
            Приближенное значение интеграла

      Описание:
      ---------
      Задача разбивается на num_threads частей, каждая из которых
      обрабатывается отдельным процессом. Результаты объединяются.
      Используется модель fork-join параллелизма.
      """
      h = (b - a) / n

      # Вычисляем граничные значения (выполняется в главном потоке)
      integral = 0.5 * (function_to_integrate(a, variant_param,
function_type) +
                              function_to_integrate(b, variant_param,
function_type))

      # Разбиваем работу между потоками
      chunk_size = (n - 1) // num_threads
      tasks = []

      for i in range(num_threads):
            start_idx = 1 + i * chunk_size
            end_idx = 1 + (i + 1) * chunk_size if i < num_threads - 1 else n
            tasks.append((start_idx, end_idx, a, h, variant_param,
function_type))

      # Параллельное вычисление частичных сумм
      with Pool(processes=num_threads) as pool:
            partial_sums = pool.map(_compute_partial_sum, tasks)

      # Объединение результатов
      integral += sum(partial_sums)
      integral *= h

      return integral
```

Листинг 11.5 – plot_scaling_laws.py

```
import numpy as np
import matplotlib.pyplot as plt

def plot_scaling_laws(sequential_fraction, comm_overhead=0.01,
max_processors=32, save_path='scaling_laws_comparison.png',
technology_type=''):
      """
      Строит графики законов Амдала, Густафсона и Сан-Ная.
      Параметры:
      ----------
      sequential_fraction : float
            Доля последовательного кода (α), 0 ≤ α ≤ 1
```

Продолжение листинга 11.5

```
      comm_overhead : float
            Коэффициент коммуникационных издержек (β)
      max_processors : int
            Максимальное число процессоров для графика
      save_path : str
            Путь для сохранения графика
      technology_type : str
            Тип технологии ('OpenMP' или 'MPI') для отображения в заголовке

      Законы:
      -------
      1. Закон Амдала: S(p) = 1 / (f + (1-f)/p)
            - Пессимистичная модель для фиксированного размера задачи
            - Максимальное ускорение ограничено 1/f

      2. Закон Густафсона: S(p) = p - α(p - 1)
            - Оптимистичная модель для масштабируемых задач
            - Почти линейное ускорение

      3. Закон Сан-Ная: S(p) = p / (1 + α(p-1) + β√p)
            - Реалистичная модель с учетом коммуникаций
            - Учитывает накладные расходы на обмен данными
      """
      processors = np.arange(1, max_processors + 1)

      # Закон Амдала
      speedup_amdahl = 1 / (sequential_fraction + (1 - sequential_fraction)
/ processors)

      # Закон Густафсона
      speedup_gustafson = processors - sequential_fraction * (processors - 1)

      # Закон Сан-Ная
      speedup_sunni = processors / (1 + sequential_fraction * (processors -
1) + comm_overhead * np.sqrt(processors))

      # Идеальное линейное ускорение
      speedup_ideal = processors

      # Построение графика
      plt.figure(figsize=(12, 7))
      plt.plot(processors, speedup_ideal, 'k--', linewidth=2.5,
                  label='Идеальное ускорение (линейное)', alpha=0.7)
      plt.plot(processors, speedup_amdahl, 'r-', linewidth=2.5,
                  label=f'Закон Амдала (α={sequential_fraction})',
marker='o', markersize=4)
      plt.plot(processors, speedup_gustafson, 'g-', linewidth=2.5,
                  label=f'Закон Густафсона (α={sequential_fraction})',
marker='s', markersize=4)
      plt.plot(processors, speedup_sunni, 'b-', linewidth=2.5,
      plt.plot(processors, speedup_sunni, 'b-', linewidth=2.5,
```

Продолжение листинга 11.5

```
                  label=f'Закон Сан-Ная (α={sequential_fraction},
β={comm_overhead})',
                  marker='^', markersize=4)

      # Формирование заголовка с типом технологии
      tech_suffix = f' ({technology_type})' if technology_type else ''

      plt.xlabel('Число процессоров (p)', fontsize=13, fontweight='bold')
      plt.ylabel('Ускорение S(p)', fontsize=13, fontweight='bold')
      plt.title(f'Сравнение законов масштабируемости параллельных
вычислений{tech_suffix}', fontsize=14, fontweight='bold')
      plt.grid(True, alpha=0.3, linestyle='--')
      plt.legend(fontsize=11, loc='upper left')

      # Добавляем аннотации для ключевых точек
      p_16 = 15  # индекс для 16 процессоров
      if p_16 < len(processors):
            plt.annotate(f'Амдал: {speedup_amdahl[p_16]:.1f}x',
                        xy=(processors[p_16], speedup_amdahl[p_16]),
                        xytext=(processors[p_16] + 3, speedup_amdahl[p_16] - 2),
                        fontsize=9, color='red',
                        arrowprops=dict(arrowstyle='->', color='red', lw=1))
            plt.annotate(f'Густафсон: {speedup_gustafson[p_16]:.1f}x',
                        xy=(processors[p_16], speedup_gustafson[p_16]),
                        xytext=(processors[p_16] - 8,
speedup_gustafson[p_16] + 2),
                        fontsize=9, color='green',
                        arrowprops=dict(arrowstyle='->', color='green',
lw=1))

      # Добавляем информацию о пределах
      max_amdahl = 1 / sequential_fraction
      plt.axhline(y=max_amdahl, color='red', linestyle=':', linewidth=1.5,
alpha=0.5)
      plt.text(max_processors * 0.7, max_amdahl + 1,
                  f'Предел Амдала: {max_amdahl:.1f}x',
                  fontsize=10, color='red', fontweight='bold')
      plt.xlim(1, max_processors)
      plt.ylim(0, min(max_processors + 5, max(speedup_gustafson) + 5))
      plt.tight_layout()
      plt.savefig(save_path, dpi=300, bbox_inches='tight')
      plt.show()
      print(f"График законов масштабируемости сохранен: {save_path}")

      return {
            'amdahl': speedup_amdahl[-1],
            'gustafson': speedup_gustafson[-1],
            'sunni': speedup_sunni[-1],
            'ideal': speedup_ideal[-1],
            'max_amdahl': max_amdahl
      }
```

Листинг 11.6 – print_metrics_table.py

```
def print_metrics_table(metrics, variant_num):
      """
      Выводит красиво отформатированную таблицу с метриками
производительности.

      Параметры:
      ----------
      metrics : list of dict
            Список метрик, полученный из calculate_metrics()
      variant_num : int
            Номер варианта задания

      Вывод:
      ------
      Таблица с колонками:
      - Процессоры (p)
      - Время (Tp)
      - Ускорение (S(p))
      - Эффективность (E(p), %)
      - Избыточность (O(p), c)
      - Стоимость (C(p))

      Дополнительный анализ:
      - Максимальное ускорение
      - Максимальная эффективность
      - Оценка масштабируемости
      """
      print("\n" + "="*95)
      print(f"ТАБЛИЦА МЕТРИК ПРОИЗВОДИТЕЛЬНОСТИ (Вариант {variant_num})")
      print("="*95)

      # Заголовок таблицы
      print(f"{'Процессоры':^12} | {'Время (c)':^12} | {'Ускорение':^12} | "
                  f"{'Эффективность':^15} | {'Избыточность':^15} |
{'Стоимость':^12}")
      print(f"{'p':^12} | {'Tp':^12} | {'S(p)':^12} | {'E(p), %':^15} | "
                  f"{'O(p), c':^15} | {'C(p)':^12}")
      print("-"*95)

      # Строки с данными
      for m in metrics:
            print(f"{m['processors']:^12} | {m['time']:^12.4f} |
{m['speedup']:^12.2f} | "

                  f"{m['efficiency']:^15.1f} | {m['overhead']:^15.4f} |
{m['cost']:^12.4f}")

      print("="*95)

      # Дополнительный анализ
```

Продолжение листинга 11.6

```
                  "{m['efficiency']:^15.1f} | {m['overhead']:^15.4f} |
{m['cost']:^12.4f}")

      print("="*95)

      # Дополнительный анализ
      print("\nАНАЛИЗ РЕЗУЛЬТАТОВ:")

      # Находим максимумы
      best_speedup = max(metrics, key=lambda x: x['speedup'])
      best_efficiency = max(metrics, key=lambda x: x['efficiency'])

      print(f"• Максимальное ускорение: {best_speedup['speedup']:.2f}x "
                  f"на {best_speedup['processors']} процессорах")
      print(f"• Максимальная эффективность:
{best_efficiency['efficiency']:.1f}% "
                  f"на {best_efficiency['processors']} процессорах")

      # Оценка масштабируемости
      if len(metrics) >= 2:
            scaling_factor = metrics[-1]['speedup'] / metrics[0]['speedup']
            proc_increase = metrics[-1]['processors'] /
metrics[0]['processors']

            print(f"• Масштабируемость: при увеличении процессоров в
{proc_increase:.0f} раз, "
                  f"ускорение выросло в {scaling_factor:.2f} раз")

            # Классификация масштабируемости
            scaling_quality = scaling_factor / proc_increase
            if scaling_quality > 0.7:
                  status = "Хорошая масштабируемость"
            elif scaling_quality > 0.4:
                  status = "Умеренная масштабируемость"
            else:
                  status = "Слабая масштабируемость"
            print(f"  status} (коэффициент: {scaling_quality:.2f})")

      # Оценка эффективности использования ресурсов
      avg_efficiency = sum(m['efficiency'] for m in metrics) / len(metrics)
      print(f"• Средняя эффективность использования процессоров:
{avg_efficiency:.1f}%")

      if avg_efficiency > 80:
            print("  Отличное использование ресурсов")
      elif avg_efficiency > 60:
            print("  Хорошее использование ресурсов")
      else:
            print("  Требуется оптимизация")

      print("="*95)
```

Листинг 11.7 – sequential_integration.py

```
import math
import time
import numpy as np

def function_to_integrate(x, k, function_type='sin'):
      """
      Вычисляет значение функции в точке x.

      Параметры:
      ----------
      x : float
            Точка, в которой вычисляется значение
      k : float
            Коэффициент функции
      function_type : str
            Тип функции: 'sin', 'x2', 'exp'
      """
      if function_type == 'sin':
            return k * np.sin(x)
      elif function_type == 'x2':
            return k * x * x
      elif function_type == 'exp':
            return k * np.exp(-x)
      else:
            raise ValueError(f"Неизвестный тип функции: {function_type}")

def sequential_integration(a, b, n, variant_param, function_type='sin'):
      """
      Последовательная реализация метода трапеций для численного
интегрирования.

      Параметры:
      ----------
      a : float
            Нижняя граница интегрирования
      b : float
            Верхняя граница интегрирования
      n : int
            Число разбиений интервала
      variant_param : float
            Параметр функции (коэффициент k)
      function_type : str
            Тип функции: 'sin', 'x2', 'exp'

      Возвращает:
      -----------
      float
            Приближенное значение интеграла

      Описание метода:
```

Продолжение листинга 11.7

```
      # Умножаем на ширину трапеции
      integral *= h

      return integral
```

Листинг 11.8 – visualize_results.py

```
import matplotlib.pyplot as plt

def visualize_results(times, labels, variant_num, metrics=None,
save_prefix='variant', technology_type=''):
      """
      Создает графики сравнения времени выполнения и метрик
производительности.

      Параметры:
      ----------
      times : list of float
            Список времен выполнения
      labels : list of str
            Список меток для каждого метода
      variant_num : int
            Номер варианта для заголовка
      metrics : list of dict, optional
            Метрики производительности из calculate_metrics()
      save_prefix : str
            Префикс для имен сохраняемых файлов
      technology_type : str
            Тип технологии ('OpenMP' или 'MPI') для отображения в заголовках
      """
      # График 1 и 2: Основное сравнение
      fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

      colors = ['#FF6B6B', '#4ECDC4', '#45B7D1']

      # Формирование заголовка с типом технологии
      tech_suffix = f' ({technology_type})' if technology_type else ''

      # График 1: Столбчатая диаграмма времен
      bars = ax1.bar(labels, times, color=colors, edgecolor='black',
linewidth=1.5)
      ax1.set_ylabel('Время выполнения (секунды)', fontsize=12)
      ax1.set_title(f'Сравнение времени выполнения (Вариант
{variant_num}){tech_suffix}',
                        fontsize=14, fontweight='bold')
      ax1.grid(axis='y', alpha=0.3)

      for bar, time_val in zip(bars, times):
            height = bar.get_height()
            ax1.text(bar.get_x() + bar.get_width()/2., height,
                  f'{time_val:.4f}s',
                  ha='center', va='bottom', fontsize=10)
```

Продолжение листинга 11.8

```
      # График 2: Ускорение
      speedup = [times[0] / t for t in times]
      bars2 = ax2.bar(labels, speedup, color=colors, edgecolor='black',
linewidth=1.5)
      ax2.axhline(y=1, color='red', linestyle='--', label='Базовая линия')
      ax2.set_ylabel('Ускорение (раз)', fontsize=12)
      ax2.set_title('Ускорение относительно последовательной версии',
                  fontsize=14, fontweight='bold')
      ax2.grid(axis='y', alpha=0.3)
      ax2.legend()

      for bar, speed_val in zip(bars2, speedup):
            height = bar.get_height()
            ax2.text(bar.get_x() + bar.get_width()/2., height,
                  f'{speed_val:.2f}x',
                  ha='center', va='bottom', fontsize=10)

      plt.tight_layout()
      plt.savefig(f'{save_prefix}_{variant_num}_comparison.png', dpi=300)
      plt.show()

      # Дополнительная визуализация метрик
      if metrics and len(metrics) > 1:
            fig2, ((ax3, ax4), (ax5, ax6)) = plt.subplots(2, 2, figsize=(14,
10))

            processors = [m['processors'] for m in metrics]
            speedups = [m['speedup'] for m in metrics]
            efficiencies = [m['efficiency'] for m in metrics]
            overheads = [m['overhead'] for m in metrics]

            # График 3: Ускорение vs процессоры
            ax3.plot(processors, speedups, 'o-', color='#4ECDC4',
linewidth=2,
                  markersize=8, label='Реальное ускорение')
            ax3.plot(processors, processors, '--', color='red', linewidth=2,
                  label='Идеальное (линейное)')
            ax3.set_xlabel('Число процессоров', fontsize=11)
            ax3.set_ylabel('Ускорение S(p)', fontsize=11)
            ax3.set_title(f'Зависимость ускорения от числа
процессоров{tech_suffix}', fontsize=12, fontweight='bold')
            ax3.grid(alpha=0.3)
            ax3.legend()

            # График 4: Эффективность
            ax4.plot(processors, efficiencies, 's-', color='#FF6B6B',
                  linewidth=2, markersize=8)
            ax4.axhline(y=100, color='red', linestyle='--', linewidth=2,
                        label='Идеальная (100%)')
            ax4.set_xlabel('Число процессоров', fontsize=11)
```

Продолжение листинга 11.8

```
            ax4.set_title(f'Зависимость эффективности от числа
процессоров{tech_suffix}',
                        fontsize=12, fontweight='bold')
            ax4.grid(alpha=0.3)
            ax4.legend()

            # График 5: Избыточность
            ax5.plot(processors, overheads, '^-', color='#45B7D1',
                  linewidth=2, markersize=8)
            ax5.set_xlabel('Число процессоров', fontsize=11)
            ax5.set_ylabel('Избыточность O(p), секунды', fontsize=11)
            ax5.set_title(f'Зависимость избыточности от числа
процессоров{tech_suffix}',
                  fontsize=12, fontweight='bold')
            ax5.grid(alpha=0.3)

            # График 6: Таблица метрик
            ax6.axis('tight')
            ax6.axis('off')
            table_data = [['p', 'T(p), c', 'S(p)', 'E(p), %']]
            for m in metrics:
                  table_data.append([
                  f"{m['processors']}",
                  f"{m['time']:.3f}",
                  f"{m['speedup']:.2f}",
                  f"{m['efficiency']:.1f}"
            ])
      table = ax6.table(cellText=table_data, cellLoc='center', loc='center',
                              colWidths=[0.15, 0.25, 0.25, 0.25])
            table.auto_set_font_size(False)
            table.set_fontsize(10)
            table.scale(1, 2)

            for i in range(4):
                  table[(0, i)].set_facecolor('#4ECDC4')
            table[(0, i)].set_text_props(weight='bold', color='white')

            ax6.set_title('Сводная таблица метрик', fontsize=12,
                              fontweight='bold', pad=20)

            plt.tight_layout()
            plt.savefig(f'{save_prefix}_{variant_num}_metrics.png', dpi=300)
            plt.show()

      print(f"\n{'='*60}")
      print(f"Результаты для варианта {variant_num}")
      print(f"{'='*60}")
      for label, t, s in zip(labels, times, speedup):
            print(f"{label:30s}: {t:.4f}s (ускорение: {s:.2f}x)")
      print(f"{'='*60}")
```

Листинг 11.9 – pyproject.toml

```
[project]
name = "parallel-integration"
version = "1.0.0"
description = "Организация параллельных вычислений"
authors = [
    {name = "student first name last name", email = "student@edu.mirea.ru"}
]
readme = "README.md"
requires-python = ">=3.8"
license = {text = "MIT"}
keywords = ["parallel computing", "numerical integration", "OpenMP", "MPI",
"HPC"]
classifiers = [
      "Development Status :: 4 - Beta",
      "Intended Audience :: Education",
      "Programming Language :: Python :: 3",
      "Programming Language :: Python :: 3.8",
      "Programming Language :: Python :: 3.9",
      "Programming Language :: Python :: 3.10",
      "Programming Language :: Python :: 3.11",
      "Programming Language :: Python :: 3.12",
      "Topic :: Scientific/Engineering",
      "Topic :: Education",
]

dependencies = [
      "numpy>=1.21.0",
      "matplotlib>=3.5.0",
      "mpi4py>=3.1.0; platform_system != 'Windows'",
]

[project.optional-dependencies]
dev = [
      "pytest>=7.0.0",
      "pytest-cov>=4.0.0",
      "black>=23.0.0",
      "flake8>=6.0.0",
      "mypy>=1.0.0",
]
windows = [
      # MPI для Windows нужно установить отдельно
]

[project.scripts]
parallel-integration = "src.main:main"

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.hatch.build.targets.wheel]
```

Продолжение листинга 11.9

```
packages = ["src"]

[tool.black]
line-length = 100
target-version = ['py38', 'py39', 'py310', 'py311']
```

Общая структура проекта показана на рис. 11.10.

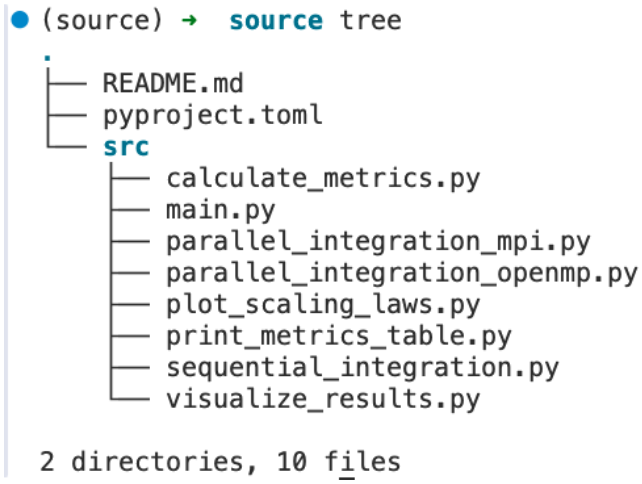


Рисунок 11.10 – Структура проекта

Таблица с вариантами – табл. 11.2.

Таблица 11.2 – Таблица вариантов

Вариант	Функция f(x)	k	a	b	n	Потоки/процессы
1	k*sin(x)	2.0	0	π	10 000 000	[2, 4, 8]
2	k*sin(x)	3.5	0	2π	15 000 000	[2, 4, 6]
3	k*sin(x)	1.5	0	π/2	8 000 000	[2, 3, 6]
4	k*sin(x)	4.0	0	3π	20 000 000	[2, 4, 8]
5	k*sin(x)	2.5	π/4	π	12 000 000	[2, 4, 6]
6	k*sin(x)	1.0	0	4π	25 000 000	[4, 8, 12]
7	k*sin(x)	3.0	0	π/2	10 000 000	[2, 4, 8]
8	k*sin(x)	2.2	π/6	π	14 000 000	[2, 4, 6]
9	k*sin(x)	1.8	0	2π	18 000 000	[3, 6, 9]
10	k*sin(x)	5.0	0	π	20 000 000	[4, 8, 16]
11	k*x²	1.0	0	10	10 000 000	[2, 4, 8]
12	k*x²	0.5	0	20	15 000 000	[2, 4, 6]

Продолжение табл. 11.2

13	k*x²	2.0	0	5	8 000 000	[2, 3, 6]
14	k*x²	1.5	1	10	12 000 000	[2, 4, 6]
15	k*x²	0.8	0	15	20 000 000	[4, 8, 12]
16	k*x²	1.2	2	12	18 000 000	[3, 6, 9]
17	k*x²	0.6	0	8	10 000 000	[2, 4, 8]
18	k*x²	1.8	0	6	14 000 000	[2, 4, 6]
19	k*x²	0.9	1	15	16 000 000	[4, 8, 16]
20	k*x²	2.5	0	4	12 000 000	[2, 4, 6]
21	k*e^{-x}	1.0	0	5	10 000 000	[2, 4, 8]
22	k*e^{-x}	2.0	0	3	8 000 000	[2, 3, 6]
23	k*e^{-x}	1.5	0	4	12 000 000	[2, 4, 6]
24	k*e^{-x}	3.0	0	2	15 000 000	[2, 4, 8]
25	k*e^{-x}	0.8	0	6	20 000 000	[4, 8, 12]
26	k*e^{-x}	2.5	0.5	5	18 000 000	[3, 6, 9]
27	k*e^{-x}	1.2	0	4	10 000 000	[2, 4, 8]
28	k*e^{-x}	1.8	0	3	14 000 000	[2, 4, 6]
29	k*e^{-x}	2.2	1	6	16 000 000	[4, 8, 16]
30	k*e^{-x}	3.5	0	2	12 000 000	[2, 4, 6]

Для запуска выполнить следующие действия:

1. Установить менеджер. Если ОС дистрибутив Linux или macOS, то команды: «curl -LsSf https://astral.sh | sh» и «source $HOME/.local/bin/env». Если ОС Windows – «powershell -c "irm https://astral.sh | iex"».
2. Перейти в рабочую папку. Если ОС дистрибутив Linux или macOS: « cd /path/to/dir», если ОС Windows: «cd C:\path\to\dir», где «path\to\dir» это путь до вашей рабочей директории.
3. Создать окружение «uv env». После нужно активировать окружение. Если ОС дистрибутив Linux или macOS: «source .venv/bin/activate», если ОС Windows: «.venv\Scripts\activate».
4. Установить зависимости из pyproject.toyml: «uv pip install -e .».
5. Запустить программу «python -m src.main».

Для запуска также нужно изменить параметры в функции main взяв их для выбранного варианта из табл. 11.2. Пример показан в Листинге 11.10.

Листинг 11.10 – пример конфигурации варианта в main.py

```
# ========== ПАРАМЕТРЫ ВАРИАНТА (ИЗМЕНЯЕМАЯ ЧАСТЬ) ==========
VARIANT_NUM = 1                        # Номер варианта (1-30)
FUNCTION_TYPE = 'sin'                  # Тип функции: 'sin', 'x2', 'exp'
FUNCTION_NAME = 'k × sin(x)'           # Название функции для вывода
K = 10.0                                # Параметр функции
A = 4.0                                # Нижняя граница интегрирования
B = math.pi                            # Верхняя граница интегрирования
N = 10_000_000                         # Число разбиений
THREADS_LIST = [2, 4, 8]              # Потоки/процессы для теста
(значения используются и для MPI)
OUTPUT_DIR = 'results'                 # Директория для результатов
# ============================================================
```

Результаты запуска показаны в Листинге 11.11 и на рис. 11.8-11.13.

Листинг 11.11 – Результат запуска демонстрационного варианта

```
(source) ➜  source python -m src.main

======================================================================
ПРАКТИЧЕСКАЯ РАБОТА №11
Организация параллельных вычислений
Численное интегрирование методом трапеций
======================================================================

Вариант: 1
Функция: k × sin(x)
Параметр k: 10.0
Границы интегрирования: [4.0000, 3.1416]
Число разбиений: 10,000,000
Потоки для тестирования: [2, 4, 8]

======================================================================
1. ПОСЛЕДОВАТЕЛЬНАЯ ВЕРСИЯ
======================================================================
Результат интегрирования: 3.4635637914
Время выполнения: 3.4344 секунд

======================================================================
2. ПАРАЛЛЕЛЬНАЯ ВЕРСИЯ (OpenMP)
======================================================================

Тестирование с 2 потоками...
  Результат: 3.4635637914
  Время: 2.2987s | Ускорение: 1.49x | Эффективность: 74.7%

Тестирование с 4 потоками...
  Результат: 3.4635637914
  Время: 1.6907s | Ускорение: 2.03x | Эффективность: 50.8%

Тестирование с 8 потоками...
  Результат: 3.4635637914
```

Продолжение листинга 11.11

```
  Время: 2.9532s | Ускорение: 1.16x | Эффективность: 14.5%

======================================================================
3. ПАРАЛЛЕЛЬНАЯ ВЕРСИЯ (MPI)
======================================================================
MPI доступен! Запуск тестов...
  mpiexec -n 2 python -m src.parallel_integration_mpi 1
  mpiexec -n 4 python -m src.parallel_integration_mpi 1
  mpiexec -n 8 python -m src.parallel_integration_mpi 1

Локальный тест с имитацией 2 процессов...
  Результат: 3.4635644410
  Время (локальный тест): 3.2038s

Локальный тест с имитацией 4 процессов...
  Результат: 3.4635644410
  Время (локальный тест): 3.1732s

Локальный тест с имитацией 8 процессов...
  Результат: 3.4635644410
  Время (локальный тест): 3.1676s

======================================================================
4. ВЫЧИСЛЕНИЕ МЕТРИК ПРОИЗВОДИТЕЛЬНОСТИ
======================================================================

--- OpenMP (multiprocessing) ---

================================================================================
==================
ТАБЛИЦА МЕТРИК ПРОИЗВОДИТЕЛЬНОСТИ (Вариант 1)
================================================================================
==================
 Процессоры  |  Время (с)   |  Ускорение   |  Эффективность  |  Избыточность
|  Стоимость
     p       |      Tp      |     S(p)     |     E(p), %     |     O(p), c
|     C(p)
--------------------------------------------------------------------------------
------------------
     2       |    2.2987    |     1.49     |      74.7       |     1.1629
|    4.5973
     4       |    1.6907    |     2.03     |      50.8       |     3.3283
|    6.7627
     8       |    2.9532    |     1.16     |      14.5       |     20.1911
|   23.6255
================================================================================
==================

АНАЛИЗ РЕЗУЛЬТАТОВ:
• Максимальное ускорение: 2.03x на 4 процессорах
• Максимальная эффективность: 74.7% на 2 процессорах
```

Продолжение листинга 11.11

```
• Масштабируемость: при увеличении процессоров в 4 раз, ускорение выросло в
0.78 раз
  Слабая масштабируемость (коэффициент: 0.19)
  Средняя эффективность использования процессоров: 46.7%
  требуется оптимизация
================================================================================
==================

--- MPI (распределенная память) ---

================================================================================
==================
ТАБЛИЦА МЕТРИК ПРОИЗВОДИТЕЛЬНОСТИ (Вариант 1)
================================================================================
==================
Процессоры  |  Время (с)   |  Ускорение   |  Эффективность  |  Избыточность
|  Стоимость
     p      |      Tp      |     S(p)     |     E(p), %     |     O(p), c
|     C(p)
--------------------------------------------------------------------------------
------------------
     2      |    2.2987    |     1.49     |      74.7       |     1.1629
|    4.5973
     4      |    1.6907    |     2.03     |      50.8       |     3.3283
|    6.7627
     8      |    2.9532    |     1.16     |      14.5       |     20.1911
|   23.6255
================================================================================
==================
АНАЛИЗ РЕЗУЛЬТАТОВ:
• Максимальное ускорение: 2.03x на 4 процессорах
• Максимальная эффективность: 74.7% на 2 процессорах
• Масштабируемость: при увеличении процессоров в 4 раз, ускорение выросло в
0.78 раз
  Слабая масштабируемость (коэффициент: 0.19)
  Средняя эффективность использования процессоров: 46.7%
  Требуется оптимизация
================================================================================
==================

============================================================
5. ВИЗУАЛИЗАЦИЯ РЕЗУЛЬТАТОВ
============================================================

============================================================
Результаты для варианта 1
============================================================
Последовательная              : 3.4344s (ускорение: 1.00x)
OpenMP (2 потока)             : 2.2987s (ускорение: 1.49x)
OpenMP (8 потоков)            : 2.9532s (ускорение: 1.16x)
============================================================
```

Продолжение листинга 11.11

```
============================================================
Результаты для варианта 1
============================================================
Последовательная              : 3.4344s (ускорение: 1.00x)
MPI (2 процессов)             : 2.2987s (ускорение: 1.49x)
MPI (8 процессов)             : 2.9532s (ускорение: 1.16x)
============================================================

Создано графиков: 4
  results/openmp_variant_1_comparison.png
  results/openmp_variant_1_metrics.png
  results/mpi_variant_1_comparison.png
  results/mpi_variant_1_metrics.png

  results/mpi_variant_1_metrics.png

======================================================================
6. ПОСТРОЕНИЕ ГРАФИКОВ ЗАКОНОВ МАСШТАБИРУЕМОСТИ
======================================================================

--- Параметры для OpenMP ---
Оценочная доля последовательного кода: 30.0%
Оценочный коэффициент коммуникаций: 0.01
График законов масштабируемости сохранен:
results/scaling_laws_openmp_variant_1.png

--- Параметры для MPI ---
Оценочная доля последовательного кода: 30.0%
Оценочный коэффициент коммуникаций: 0.02
График законов масштабируемости сохранен:
results/scaling_laws_mpi_variant_1.png

Создано графиков законов: 2
  results/scaling_laws_openmp_variant_1.png
  results/scaling_laws_mpi_variant_1.png

======================================================================
РАБОТА ЗАВЕРШЕНА УСПЕШНО!
======================================================================

Всего сохранено файлов: 6

OpenMP (общая память):
  • results/openmp_variant_1_comparison.png
  • results/openmp_variant_1_metrics.png
  • results/scaling_laws_openmp_variant_1.png

MPI (распределенная память):
  • results/mpi_variant_1_comparison.png
  • results/mpi_variant_1_metrics.png
```

Продолжение листинга 11.11

```
   • results/scaling_laws_mpi_variant_1.png

================================================================================
СРАВНЕНИЕ МЕТОДОВ ДЛЯ ПАРАЛЛЕЛИЗАЦИИ ВЫЧИСЛЕНИЙ
================================================================================

OpenMP (общая память):
   • Время последовательное: 3.4344s
   • Максимальное ускорение: 2.03x
   • Максимальная эффективность: 74.7%

MPI (распределенная память):
   • Время последовательное: 3.4344s
   • Максимальное ускорение: 2.03x
   • Максимальная эффективность: 74.7%

================================================================================
```

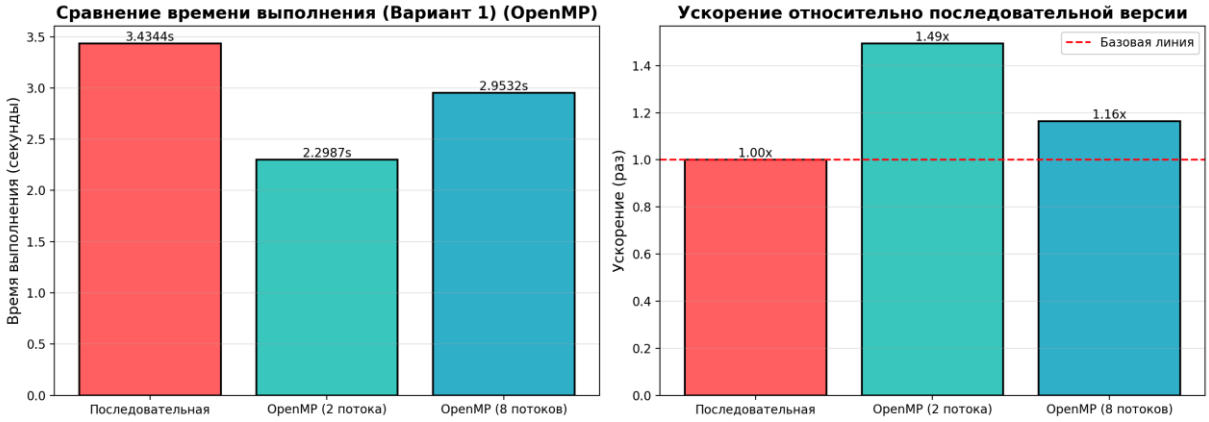
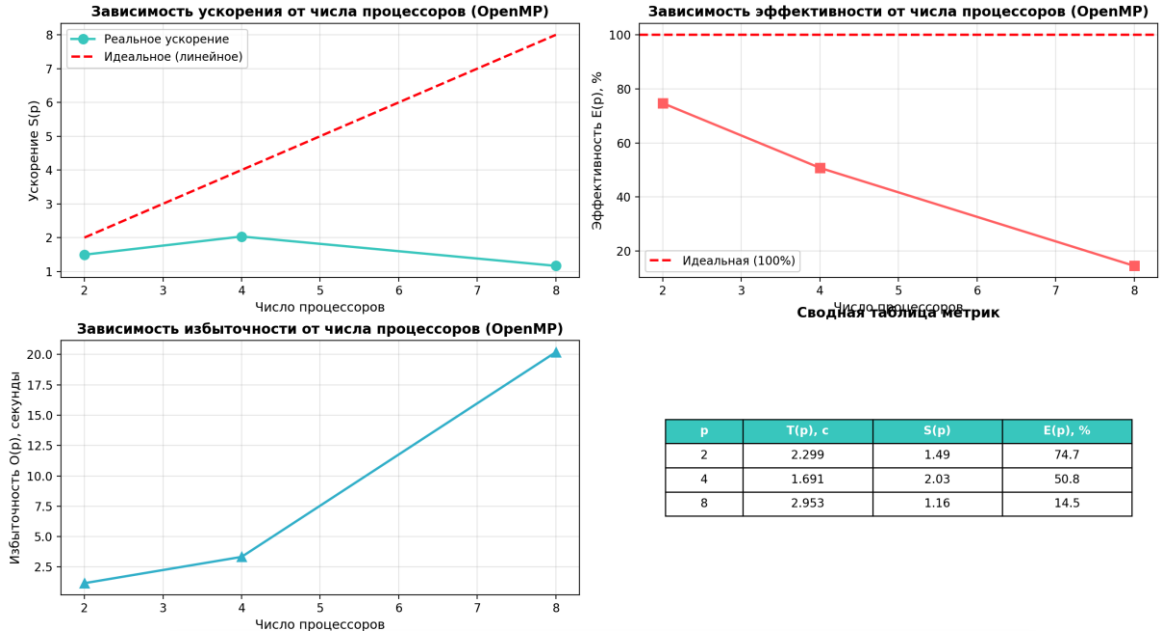


Рисунок 11.8 – Оценка времени до и после распараллеливания на OpenMP



p	T(p), с	S(p)	E(p), %
2	2.299	1.49	74.7
4	1.691	2.03	50.8
8	2.953	1.16	14.5

Рисунок 11.9 – Метрики эффективности после распараллеливания на OpenMP

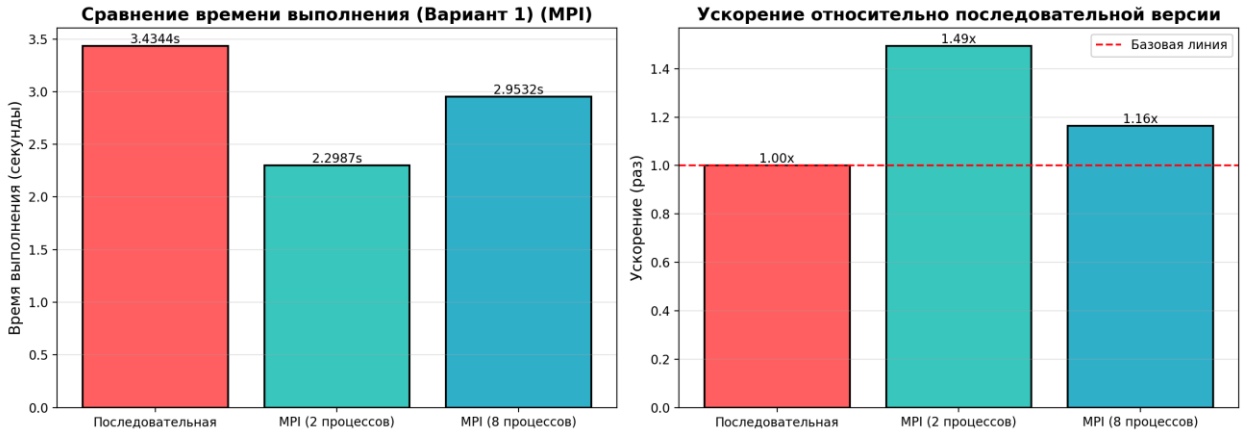
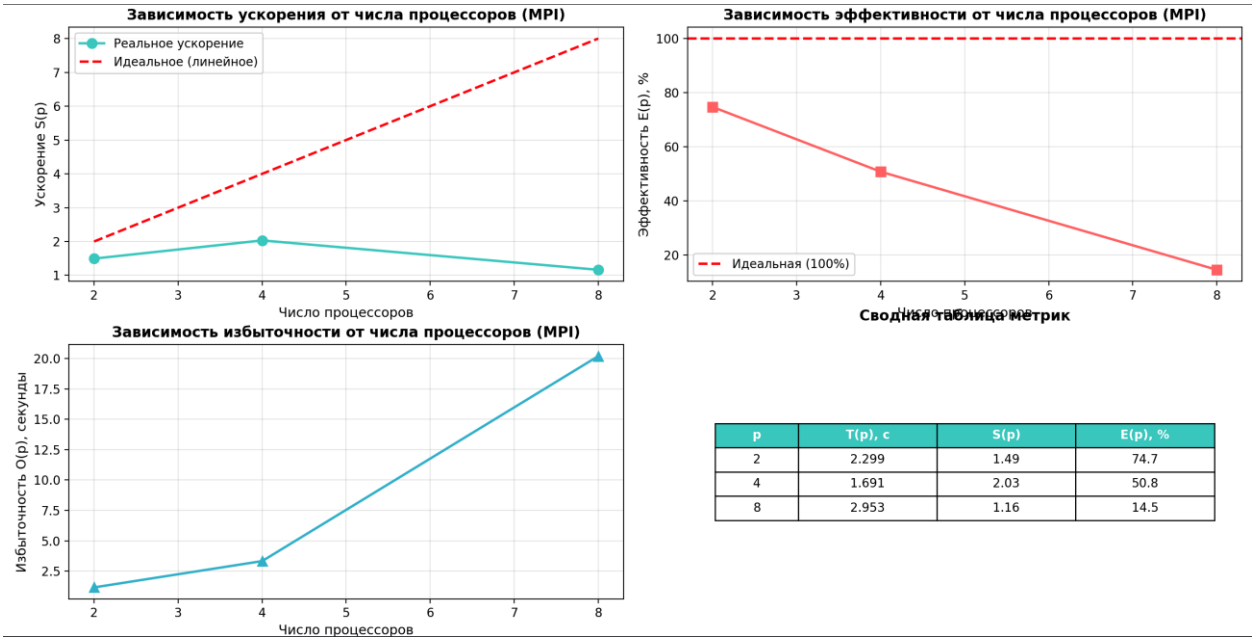


Рисунок 11.10 – Оценка времени до и после распараллеливания на MPI



Сводная таблица метрик

p	T(p), с	S(p)	E(p), %
2	2.299	1.49	74.7
4	1.691	2.03	50.8
8	2.953	1.16	14.5

Рисунок 11.11 – Метрики эффективности после распараллеливания на MPI

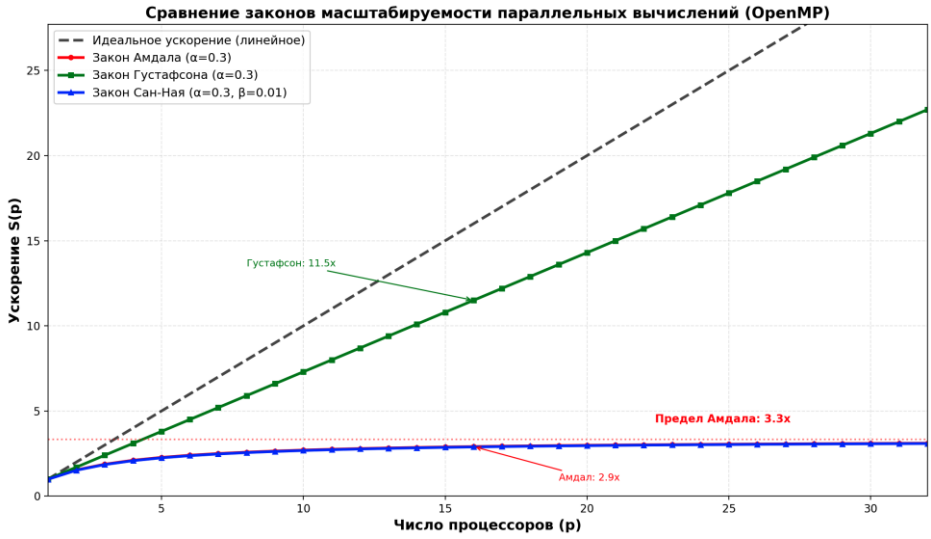


Рисунок 11.12 – Сравнение масштабируемости по законам Амдала, Густафсона, Сана-Ная для реализации на OpenMP

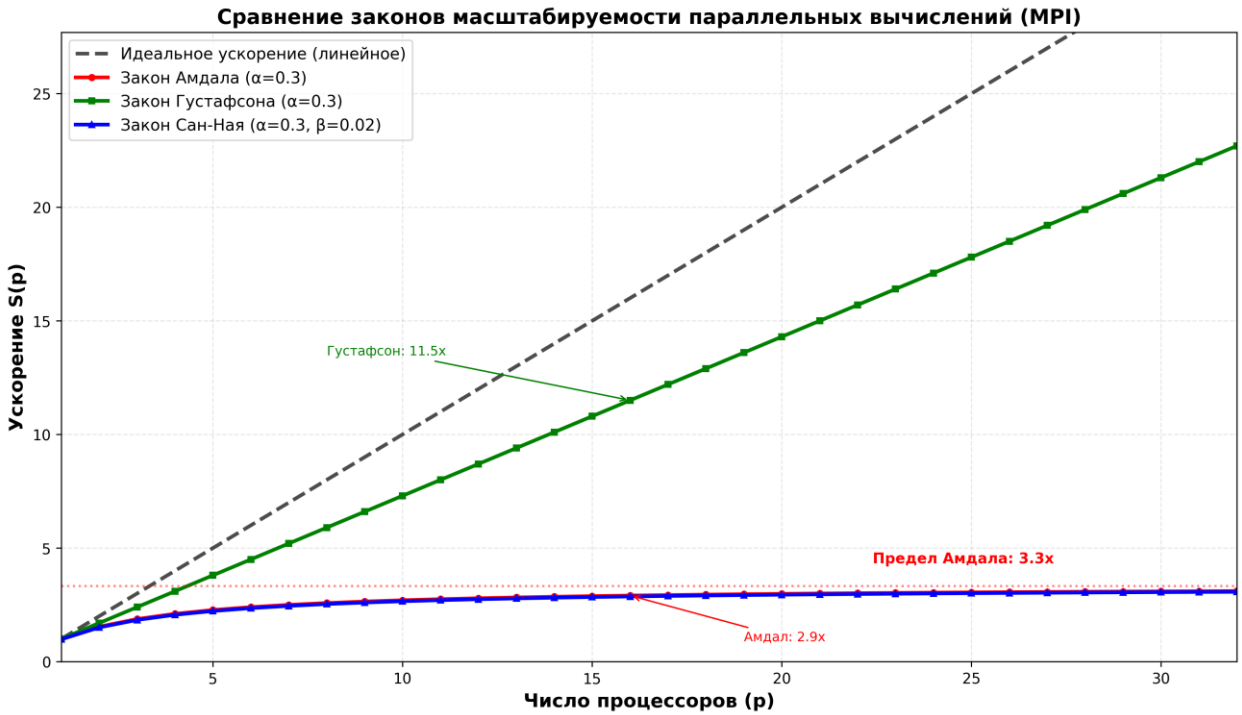


Рисунок 11.13 – Сравнение масштабируемости по законам Амдала, Густафсона, Сана-Ная для реализации на MPI

Задание для самостоятельной работы:

Задание №1 Собрать демонстрационный проект и установить все зависимости.

Задание №2 Запустить демонстрационный вариант, ознакомится с результатами запуска.

Задание №3 Изменить функцию в файле «function_to_integrate.py» функцией из персонального варианта в табл. 11.2. Провести оценку эффективности и объяснить результат запуска.

12. ГРАФИЧЕСКИЙ ПРОЦЕССОР

12.1. Введение

Одной из самых популярных карт расширений является графическая карта, или просто – видеокарта. Видеокарты делятся на два основных вида: встроенные и дискретные. Встроенные графические адаптеры – это как правило совокупность из некоторого числа специальных графических ядер, установленных в корпус центрального процессора. Благодаря такому решению, в центральном процессоре выполняются не только его основные задачи, но и ведется обработка графики. Стоит учитывать, что такое решение привносит ряд осложнений и неудобств. Например, вместо того, чтобы использовать пространство кристалла процессора под его нужды (больше физических ядер, увеличенный кэш, буферы и пр.), инженеры жертвуют им в угоду графических ядер, которые из-за своего строения не могут выполнять рядовые задачи, присущие ЦП. Вторым осложнением является то, что работа с современной графикой, процесс сложный и, порой, требующий колоссальных вычислительных мощностей, что невозможно организовать при встроенной графике, ввиду ограниченности физического пространства. Из этого вытекает тот факт, что производительность будет сравнительно невысокой, и для работы со сложной графикой такие решения не подойдут. Еще одним осложнением является тот факт, что имеющаяся ОЗУ в компьютере будет использоваться сразу и CPU, и GPU, что увеличит на неё нагрузку и уменьшит доступный рабочий объем и для одного, и для второго. В итоге получается устройство, которое может и выполнять различные процессы, и при этом генерировать изображение, но ни первое, ни второе оно не делает хорошо (если сравнивать с раздельным вариантом). С другой стороны, такой подход позволяет сэкономить на физическом размере и на системе охлаждения. Соответственно, беря во внимание вышеперечисленные особенности, такие решения чаще всего применяются в моноблоках, ноутбуках и ряде других мобильных устройствах, где мобильность и компактность гораздо важнее мощности и производительности. В то же время стоит путать более сложные решения, где на одной материнской плате (например, в ноутбуках топового сегмента) распаяны сразу и ЦП, и ГП. Такое решение уже будет содержать дискретную видеокарту (так как это отдельный чип), со своей отдельной памятью.

Дискретная видеокарта – напротив, является специализированной платой расширения, задача которой, сугубо работа с графическими

вычислениями, в последнее время к этому добавилось еще и машинное обучение, и ряд других неграфических задач. Видеокарта является полноценным компьютером в компьютере, где есть свой процессор, своя оперативная память, свое ПЗУ (свой BIOS), свое охлаждение, своя подсистема питания и прочее. В данном разделе мы будет рассматривать сугубо дискретные видеокарты, так как они представляют наибольший интерес. А встроенная графика, как правило строится на тех же принципах, что и дискретные, только в меньшем объеме.

Немного истории: Потребность в отдельном устройстве, который напрямую работал бы с изображениями, не возникала довольно долго. Со всеми задачами справлялся ЦП. В самые ранние времена становления компьютерной техники интерфейсами вывода было многочисленные лампы и светодиоды. Позже, когда появились мониторы, примерно 1960-х годах, когда для вывода информации стали использовать электронно-лучевые трубки. Там не было сложных изображений, а вывод информации осуществлялся в основном с помощью символов. Позже стали задумываться об удобном интерфейсе для взаимодействия пользователя с устройством. В 1970-х года появились первые монохромные интерфейсы (GUI в современном понимании). Позже в 1980-х появились, по сути, первые предки видеокарт (рисунок 12.1). Это были платы расширения, которые в основном позволяли выводить информацию в монохромном текстовом виде. Основным производителем подобных решений была компания IBM.

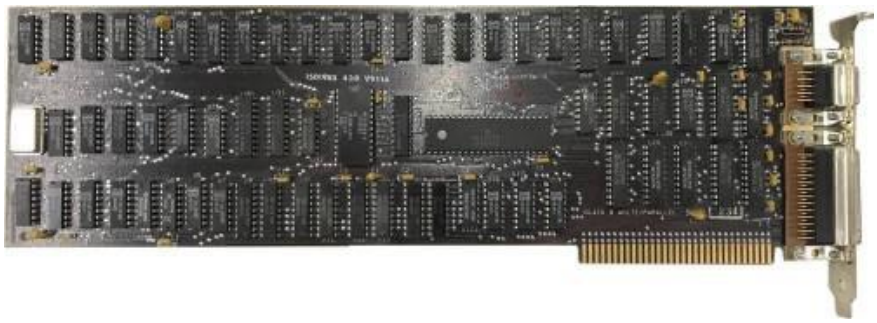


Рисунок 12.1 – Monochrome Display Adapter

В середине 1980-х произошел скачок развития в сторону цветных пользовательских интерфейсов. Появилась первая Windows и ряд других. В 1981 году, все так же компания IBM, стала производить уже продвинутые версии своих «Видеокарт» – CGA – Color Graphics Adapter (1981 год), поддерживавшую разрешение 320x200 пикселей и до 16 цветов. Затем последовала Enhanced Graphics Adapter (EGA) в 1984 году, которая предлагала

разрешение 640x350 пикселей и 16 цветов из палитры в 64 цвета. Все эти карты расширения поддерживали низкое разрешение, по современным меркам, и ограниченное количество цветов, но именно они положили начало индустрии видеокарт. Пиком развития стала VGA – Video Graphics Adapter – который, в последствии стал стандартом вывода изображений на несколько лет. Видеоадаптеры этого поколения обладали разрешением 640×480 пикселей и поддерживали палитру из 256 цветов (8bit). Например, модель 8514/A обеспечивала разрешение в 1024×768 с той же 8-битной цветовой глубиной. Следующим шагом в развитии стал стандарт SVGA (Super VGA), который позволил выводить изображение в разрешении 800×600 точек с поддержкой уже 65 536 цветов. 1990 год ознаменовал следующий этап развития графических карт, и резкий скачек 3D технологий. Развитие получили и популярные программные библиотеки, которые используются и по сей день, например такие вещи как DirectX и OpenGL. Позже стали активно появляться коммерческие компании, специализирующихся на разработке видеокарт, произошло зарождение будущих гигантов индустрии: NVIDIA и ATI Radeon. Уже в период с 1996 по 2001 год на рынке видеокарт развернулась первая конкуренция между тремя ведущими производителями (ATI, NVIDIA и 3Dfx), каждый из которых боролся за лидерство. В 1999 году компания NVIDIA представляет первую видеокарту линейки GeForce – NVIDIA GeForce 256 (рисунок 12.2). Данная модель обладала 220 нанометровым техпроцессом, 32 МБ оперативной памяти, 128 битной шиной и всего 4 блоками текстурной обработки и 4 блоками растеризации.



Рисунок 12.2 – Видеокарта NVIDIA Geforce 256

Для компьютеров, ставших уже персональными стали появляться специализированные приложения для работы с графикой, приложения для редактирования видео и фото, а видеоигры стали переходить из состояния 2D и псевдо 3D в полноценное 3D, с выходом первой Quake, так как возможности видеокарт позволили генерировать и выводить изображение в реальном времени. С момента 2000-х годов сильных изменений в индустрии было много, производительность видеокарт постоянно растет, появляется множество новых технологий, совершенствуются старые. Архитектурно, самым важным скачком в развитии видеокарт, можно отметить переход к универсальным потоковым процессорам (вместо их разделения на пиксельные и вершинные), естественно происходит существенное уменьшение типоразмеров кристаллов почти с каждым новым поколением. Компания 3Dfx обанкротилась к 2002 году, а большая часть ее сотрудников и интеллектуальной собственности перешла к NVIDIA, так на рынке осталось только два игрока. Сейчас на рынок видеокарт пытается выйти и Intel, имевшая до этого опыт как в создании дискретных видеокарт, так и интегрированных решений. На более поздних этапах своего развития эти компании каждый пришли к некоторому своему стандарту, по которым они делают свои процессоры.

12.2. Составляющие части современных видеокарт

Как было сказано выше, современная видеокарта, это компьютер в компьютере. Естественно, в видеокартах от тех или иных производителей применяются, некоторые, собственные решения, отличающие конкретную видеокарту от остальных. Стоит, однако, понимать, разницу между производителями графических процессоров и видеокарт. Первые – это группа компаний (знакомые всем NVIDIA, AMD и, с недавних пор, Intel), которые производят непосредственно сам процессор для графической карты, который непосредственно отвечает за процесс обработки графики. Вторые, в свою очередь, устанавливают полученное решение на печатную плату, подводят к нему питание, устанавливают оперативную память, присоединяют систему охлаждения, упаковывают все это в красивый корпус и выпускают на потребительский рынок. Зачастую в этот момент, перепрошивается ПЗУ (конкретно BIOS), например в рамках одной и той же модели может быть обычная и OC (OverClocked – разогнанная) версии.

Разберем компоненты, входящие в состав современной видеокарты. Составляющие части практически любой видеокарты показан на рисунке 12.3,

ниже. Это процессор, память, питание, порты вывода, интерфейс PCI-E x16, система охлаждения и корпус.

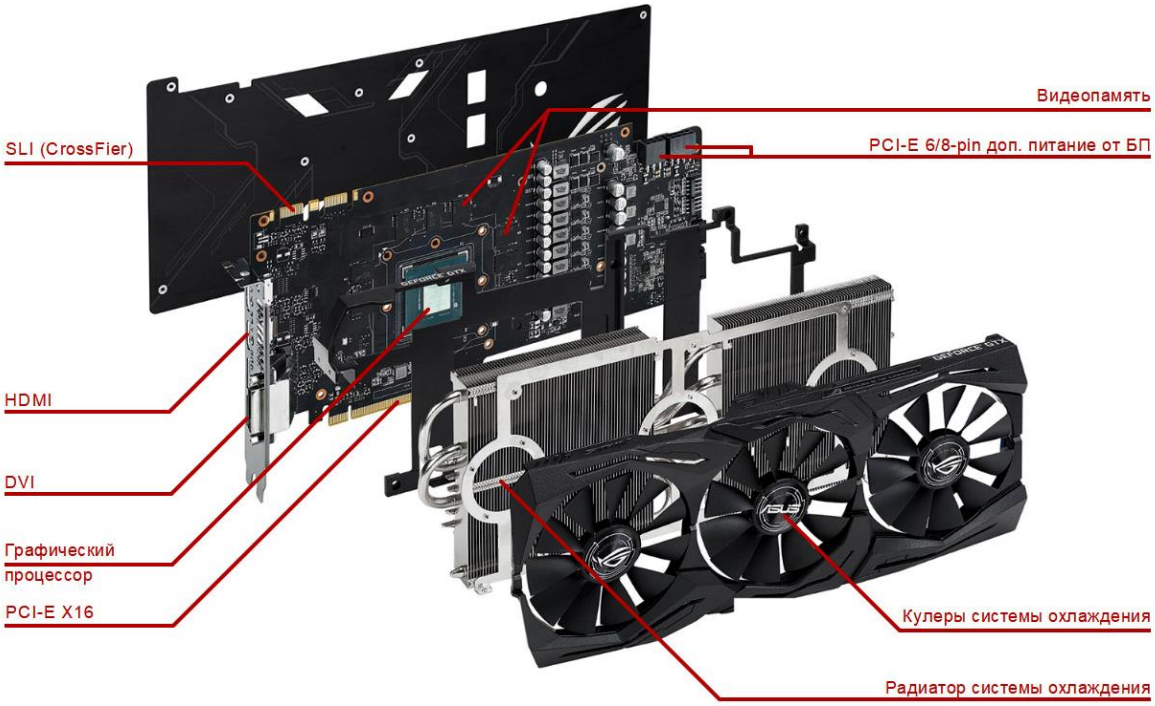


Рисунок 12.3 – Основные составляющие части видеокарты

Графический процессор видеокарты. Графический процессор или GPU – Graphical Processing Unit (рисунок 12.4) – сердце любой видеокарты, которое позволяет, выполняя миллионы операций в секунду, выводить на экран монитора фотореалистичное изображение, в высоком разрешении и большой частотой обновления.



Рисунок 12.4 – Графический процессор

GPU – комплексное устройство, косвенно по архитектуре, напоминающее CPU, но с многими существенными отличиями. И первый, и второй, имеют в своем составе вычислительные блоки (АЛУ), задача которых выполнять множество логических и математических операций. Однако у графического процессора они узко специализированные и отличаются по набору команд, из-за чего GPU не может выполнять задачи CPU.

Основная задача CPU — выполнение потоков команд, причем делать это необходимо максимально быстро, но без нарушения последовательности инструкций, входящих в эти потоки. Процессор спроектирован так, чтобы обрабатывать несколько таких потоков одновременно либо разделять один поток на независимые части, выполнять их параллельно, а затем объединять результаты обратно в исходном порядке. Важно понимать, что команды в потоке не всегда жестко зависят друг от друга; процессор ищет независимые инструкции для параллельного выполнения. Это одна из причин, по которой современные CPU содержат множество исполнительных блоков, а основной упор в архитектуре делается на предотвращение простоев конвейера при сохранении корректности вычислений. Для достижения высокой скорости обработки потоков необходимы глубокая конвейеризация и большие объемы кэш-памяти, о чем уже упоминалось при разборе центрального процессора.

GPU – напротив, работает не с цепочками взаимосвязанных команд, а с огромными массивами независимых и повторяющихся математических данных. Задачи сводятся к обработке этих массивов с целью получения координат пикселей для последующего их отображения на экране. На заре развития видеокарт, когда машинное обучение, криптовалюты и криптография еще не были развиты и доступны, задачей видеокарты было сугубо обработка трехмерной графики. Преобразование 3D сцен в плоское изображение, будь то архитектурные презентации или видеоигры. 3D-графика, если очень поверхностно и грубо описывать ее принцип работы и не вдаваться в детали, тонкости, сложности и современные технологии и алгоритмы, сводится к обработке множества различных поверхностей (полигонов) некоторого объекта или их набора, на которые направлены лучи света. На полигоны, как правило, наложена текстура – плоское изображение, как правило многослойное, содержащее информацию о цвете и типе имитируемого материала. Например, если имитируется металлическая поверхность, то лучи отражаются целиком, если же поверхность имитирует, например, дерево, то лучи поглощаются, в результате чего отражения не возникает. Обработка теней, Ray-Tracing, анизотропная фильтрация,

хроматические аберрации и различные алгоритмы сглаживания также относятся к сложным математическим алгоритмам, задача которых сделать итоговое изображение более качественным и реалистичным. Все вышеописанное – задачи линейной алгебры, просчетом которых и занята видеокарта в момент работы. Благодаря тому, что исходные данные как правило одинаковые и простые (координаты вершин объекта и пикселя на экране монитора), а задачи между собой не связанные (один пиксель никак не связан с другим) видеокарта может выполнять огромное количество подобных операций в секунду.

Тут следует отметить, что раньше в графических процессорах шейдерные ядра (вычислительные блоки) делились на два подтипа: пиксельные и вершинные. Первые, вычисляли пространственные координаты вершин полигонов, необходимые для переноса 3D-картинки в двухмерное изображение на мониторе (процесс растеризации). Вторые занимались вычислением цвета каждого пикселя этого изображения с учетом данных, полученных при обработке объекта. В 2006 году были разработаны универсальные шейдерные ядра (CUDA в случае с NVIDIA и CU в случае с AMD), которые могут выполнять обе операции, еще и генерировать новую геометрию на основе имеющихся вершин. Из этого вытекает основное архитектурное различие от центрального процессора, в GPU вычислительных блоков на несколько порядков больше, чем в классическом CPU. Современные топовые CPU имеют до 24 физических ядер, в то время как, в графических картах это количество может достигать 20000. При этом большинство современных CPU, могут обрабатывать не более двух потоков на ядро, а у GPU, её специальные потоковые процессоры, способны обрабатывать тысячи потоков одновременно, а переход с одного потока на другой не вызывает проблем, ввиду отсутствия взаимосвязи между расчётами. Различия между архитектурами GPU и CPU можно изобразить следующим образом (рисунок 12.5).

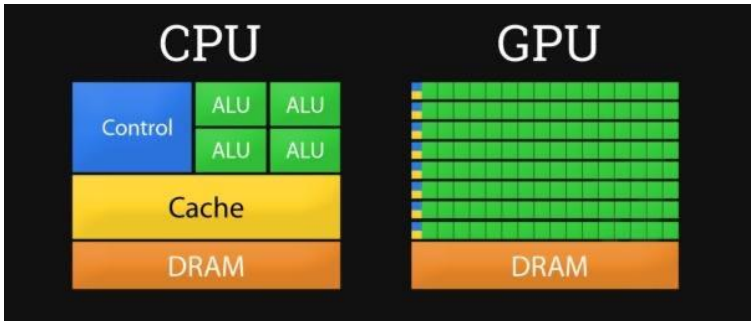


Рисунок 12.5 – Логическое устройство процессоров

Следующим важным компонентом графического процессора являются текстурные блоки (TMU – Texture Mapping Unit). Их задача – корректное наложение имеющейся текстуры на полигональную модель объекта, с последующей фильтрацией (билинейной, трилинейной или анизотропной), для корректного отображения на экране, объектов, находящихся под углом к камере наблюдателя.

ROP – Render Output Unit или блок растеризации– завершающий элемент в генерации изображения для вывода на экран. Его задача полученную 3D геометрию, с учетом фильтров, цвета и позиции отобразить на 2D плоскость и применить метод сглаживания, чтобы итоговое изображение выглядело как реальный объект, а не набор цветных пикселей. Чем сложнее и точнее метод сглаживания, тем качественней будет итоговый результат.

Для некоторых современных видеокарт характерно наличие блоков матричных вычислений, и ядер трассировки лучей. Задача первых – работа с матрицами, в частности это их перемножение, что является достаточно сложной и немаловажной математической операцией. Немного алгебры: математический объект, который описывает соотношения между другими математическими объектами, связанными друг с другом называется тензором, обычно он изображается в виде массива чисел. Тензор может иметь разную размерность, например, тензор «0x0» – это скаляр, то есть число, тензор «1x0» – это вектор, а тензор «1x1» — это матрица (рисунок 12.6).

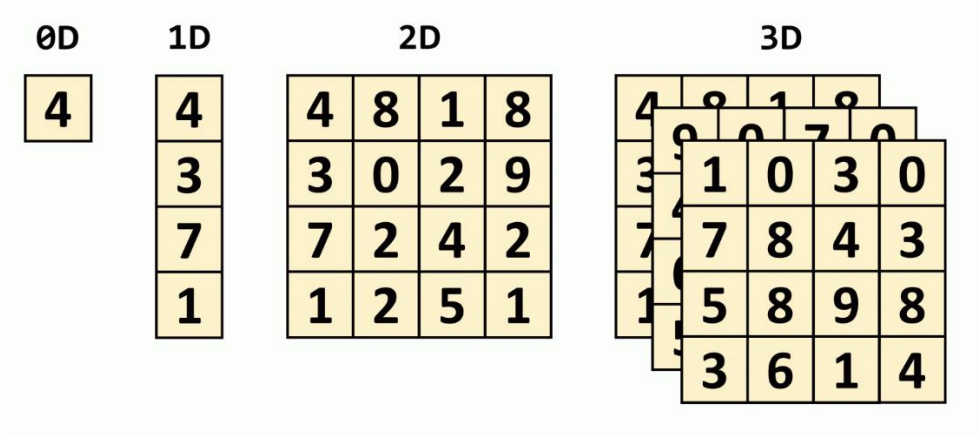


Рисунок 12.6 – Основные виды тензоров

Работа с тензорами и есть основная задача тензорных ядер, откуда и вытекает их название. Например, вычисление произведения матриц вида 4 на 4, вырастает в огромное количество сложений и умножений (рисунок 12.7), которое, конечно, можно выполнить и на обычных шейдерных ядрах, или даже

на CPU, однако скорость выполнения такого вычисления будет существенно ниже, чем на специально отведенных для этого ядрах.

$$\mathbf{C} = \mathbf{A} \times \mathbf{B}$$

$$\mathbf{C} = \begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{bmatrix} \times \begin{bmatrix} \alpha & \beta & \gamma & \delta \\ \varepsilon & \zeta & \eta & \theta \\ \kappa & \lambda & \mu & \nu \\ \pi & \tau & \psi & \omega \end{bmatrix}$$

$$\mathbf{C} = \begin{bmatrix} (a \times \alpha) + (b \times \varepsilon) + (c \times \kappa) + (d \times \pi) & (a\beta) + (b\zeta) + (c\lambda) + (d\tau) & (a\gamma) + (b\eta) + (c\mu) + (d\psi) & (a\delta) + (b\theta) + (c\nu) + (d\omega) \\ (e\alpha) + (f\varepsilon) + (g\kappa) + (h\pi) & (e\beta) + (f\zeta) + (g\lambda) + (h\tau) & (e\gamma) + (f\eta) + (g\mu) + (h\psi) & (e\delta) + (f\theta) + (g\nu) + (h\omega) \\ (i\alpha) + (j\varepsilon) + (k\kappa) + (l\pi) & (i\beta) + (j\zeta) + (k\lambda) + (l\tau) & (i\gamma) + (j\eta) + (k\mu) + (l\psi) & (i\delta) + (j\theta) + (k\nu) + (l\omega) \\ (m\alpha) + (n\varepsilon) + (o\kappa) + (p\pi) & (m\beta) + (n\zeta) + (o\lambda) + (p\tau) & (m\gamma) + (n\eta) + (o\mu) + (p\psi) & (m\delta) + (n\theta) + (o\nu) + (p\omega) \end{bmatrix}$$

Рисунок 12.7 – Схема умножений матриц 4 на 4

Продемонстрированный выше пример содержит 64 умножения и 48 сложений; при этом, каждое небольшое произведение даёт значение, которое нужно где-то хранить, прежде чем его можно будет сложить с другими тремя небольшими произведениями, чтобы позже можно было сохранить окончательное значение тензора. Теперь добавим тот факт, что значения могут быть с плавающей точкой (и разной точностью FP32, FP16, FP8, FP4) и для корректных вычислений таких тензоров будет важна точность – что еще сильней усложнит вычисления. В мире графики одновременно необходимо передавать и обрабатывать огромные объёмы информации в виде векторов. Благодаря своей способности параллельной обработки GPU идеально подходят для обработки тензоров. В результате чего все современные графические процессоры поддерживают функциональность под названием GEMM (General Matrix Multiplication). Выглядит это как последовательное перемножение с последующим сложением трех различных матриц (рисунок 12.8), с определенными ограничениями.

$$\mathbf{D} = (\mathbf{A} \times \mathbf{B}) + \mathbf{C}$$

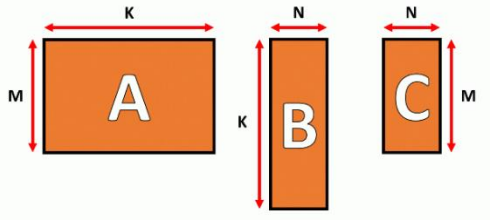


Рисунок 12.8 – Схема GEMM

Изначально такие ядра использовались сугубо для профессиональных вычислений в физике, математике, моделировании жидкостей и, конечно, для машинного обучения. Однако в последнее время эту технологию адаптировали для реализации «умного» масштабирования в видеоиграх (DLSS, FSR, XeSS). Эффективность этих технологий обуславливается пониженной точностью при обработке изображения. Сразу отметим, что тензорные ядра, как отдельные структуры присутствуют только в процессорах от компаний NVIDIA и Intel, у первых они называются тензорные ядра, у вторых – матричные ядра (XMX). Это не значит, что у моделей линейки AMD графический процессор не может выполнять такие вычисления, реализация в их процессорах немного иная, и не имеет отдельно выделенных ядер для таких вычислений, поэтому есть механизм, который позволяет переключить режим работы основных ядер в режим матричных вычислений. Такая реализация, к сожалению, снижает общую производительность.

Последние, это RT-ядра, ставшие популярными с приходом 20-й линейки видеокарт от NVIDIA. Трассировка лучей – процесс, при котором имитируется процесс распространения лучей из реального мира. Создается исходный луч света, которой отражается от поверхности в определенном порядке, в зависимости от ее типа. На основании отражения или преломления генерируется новый луч (в случайном направлении), а может и не один, со своими собственными направлениями, который взаимодействует с другими лучами и объектами, генерируя новые лучи. Стоит понимать, что в реальном жизни каждый луч света и его поведение «обрабатывается законами физики», а в виртуальном мире алгоритмами и просчетами. Данные о всех обрабатываемых лучах, преломлениях и цветах нужно где-то хранить и одновременно обрабатывать их поведение, желательно в реальном времени для поддержания реалистичности происходящего (например, когда объекты находятся в движении). Все это еще должно приемлемую частоту кадров, что является крайне затруднительным и ресурсоемким процессом. Упрощенное представление процесса трассировки лучей изображен на рисунке ниже (рисунок 12.9).

Также важно отметить, что в реальном мире таких лучей много, и каждый из них взаимодействует друг с другом и с окружающей средой, при трассировке лучей для оптимизации процесса, моделируется только часть лучей.

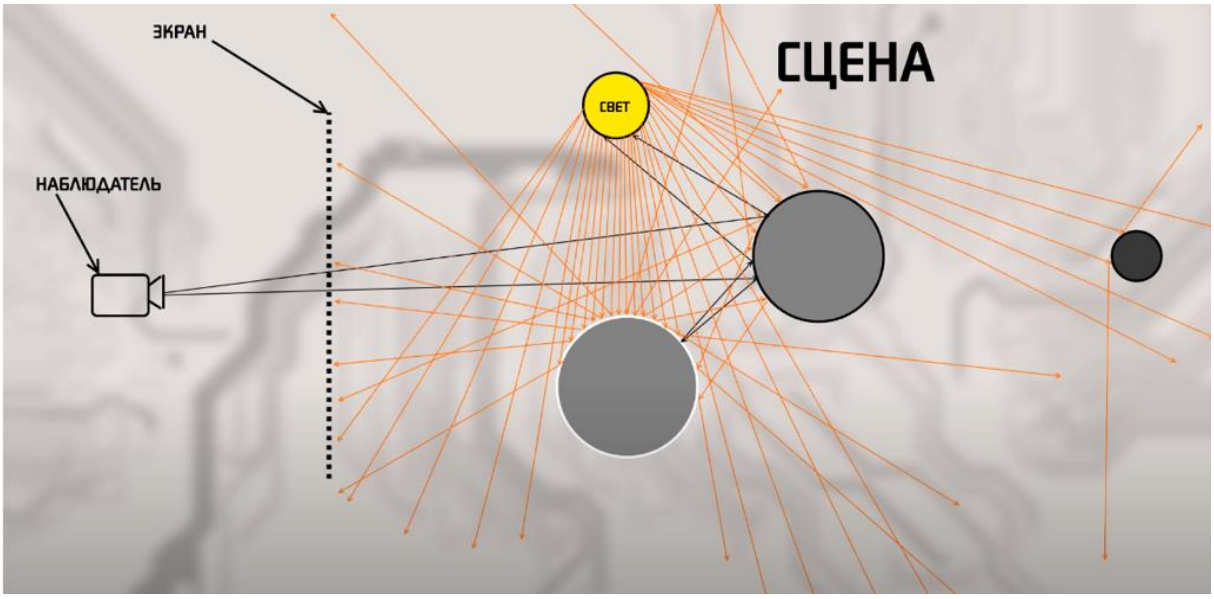


Рисунок 12.9 – Визуализация распространения лечей в процессе трассировки

Процесс трассировки лучей, не вдаваясь в математический хаос, который происходит в каждый момент времени их расчета, можно описать следующим простым алгоритмом:

- Генерация первичных лучей:

Из позиции виртуальной камеры испускаются лучи, соответствующие каждому пикселю итогового изображения. Их направление зависит от угла обзора камеры и итогового разрешения. Опционально могут создаваться дополнительные лучи создаваться для устранения эффекта "лесенки" (как при сглаживании) или моделирования размытия в глубине резкости.

- Определение пересечений с объектами:

Для каждого луча проверяется, пересекает ли он какие-либо объекты в виртуальном пространстве. При наличии пересечений фиксируются набор данных: координаты, нормаль к поверхности, учитываются свойства материала и др.

- Расчёт освещения:

Освещённость в этой точке вычисляется на основе характеристик материала, наличия, температуры и расположения источников света. После чего применяются локальные модели освещения (например, модель Фонга) или более комплексные и сложные методы (например, фотонные карты).

- Трассировка вторичных лучей:

Для имитации отражений и преломлений из точки пересечения выпускаются дополнительные лучи, в случайном направлении. Процесс

может повторяться рекурсивно, чтобы учесть многократные переотражения света.

- Моделирование теней:

Для проверки затенения испускаются тестовые лучи от точки пересечения к источникам света, если он перекрывается другим объектом, точка считается находящейся в тени. Мягкие тени создаются путем анализа множества лучей, направленных к площадным источникам света.

- Финальное формирование цвета пикселя:

Итоговый цвет определяется через усредненное значение вкладов всех лучей, попавших в пиксель. Дополнительно может применяться постобработка: тональная коррекция, размытие, имитация глубины резкости и др.

Еще одним ключевым архитектурным различием является кэш память, у современных видеокарт она сильно зависит от конкретной архитектуры и производителя. Например, у RTX 5090 – только два уровня кэш памяти (небольшой L1, которые находится в каждом SM и огромный общий L2), в то время как у RX 7900 XTX – их 3 (L1 для каждого CU, L2 поделен между массивами CU и L3 – своя технология, называемая Infinity Cache, которая является промежуточной между CPU и ОЗУ). Подробно рассмотрим отличия ниже, при рассмотрении архитектурных подходов.

Память видеокарты. Подсистема оперативной памяти основана на DDR технологии, однако, слегка измененной. GDDRx – Graphics Double Data Rate – это ускоренная и расширенная версия уже знакомой нам оперативной памяти. Здесь стоит дополнить имеющиеся знания об оперативной памяти новым понятием – предвыборка (Xn-prefetch, где X-объем). Данная технология, во многом, и позволила достичь тех частот, на которых сейчас и работает память, она же легла в основу понятий «реальной» и «эффективной» частот работы ОЗУ. Суть в том, что для отправки данных два раза за такт их нужно заранее к этому подготовить, для этого информация, которая должна быть передана, заносится в специальный буфер ввода/вывода, который работает на реальной частоте памяти. Далее, в зависимости от частоты шины данные передаются в процессор, соответственно, если частота шины равна частоте ядра, то данные передаются с удвоенной частотой, что и изображено на рисунке ниже (рисунок 12.10).

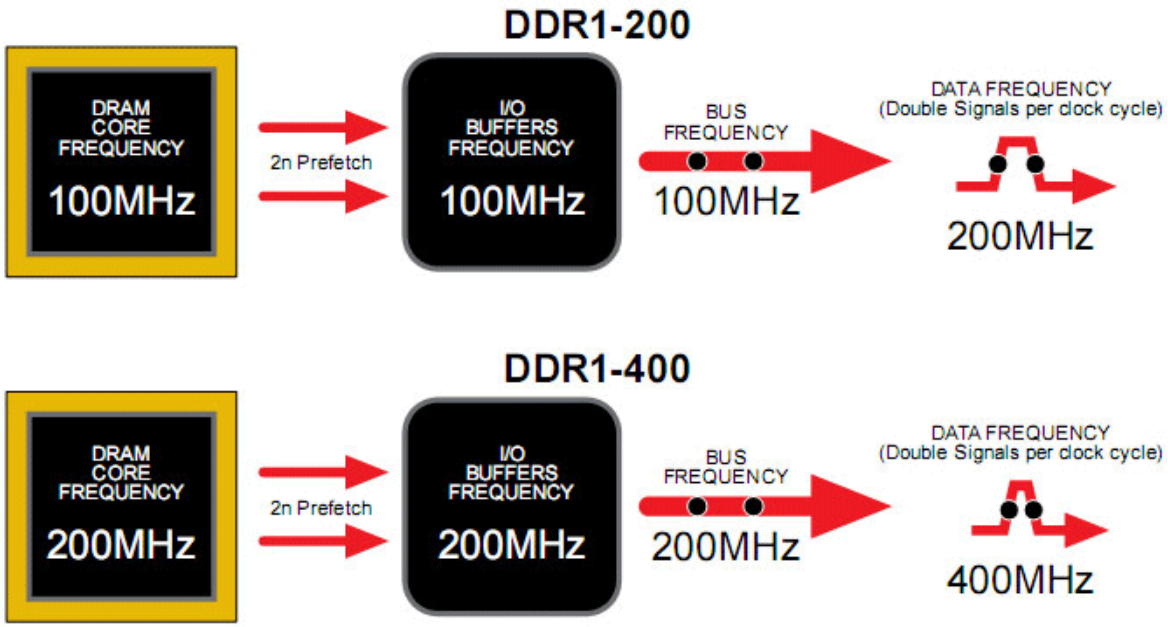


Рисунок 12.10 – Технология prefetch в DDR

В последующих итерациях памяти размер выборки увеличили, и для DDR2, уже использовалась 4n-prefetch. Например, для DDR2-800, шина и буфер работали на частоте 400 МГц, а сама память работала на частоте 200 МГц. В оперативной памяти DDR3 и DDR4 уже используется выборка 8n-prefetch. Соответственно, шина и буфер работают на частоте, которая в четыре раза превышает частоту самой памяти. Реальная скорость передачи данных выше частоты шины все так же в два раза. На этом же принципе разработана GDDR4. Из более-менее современных – GDDR5 – которая, основана на том же принципе, однако способна передавать данные четыре раза за такт против двух у предшественников. Возможность появилась благодаря отделению частоты передачи адресов и команд от частоты передачи данных, которая вдвое выше. Это позволило передавать одновременно два импульса, в каждом из которых, как и раньше, два бита данных – на фронте и на спаде. Таким образом, эффективная частота памяти у GDDR5 в четыре раза превышает частоту шины. В памяти GDDR5X реализовали предвыборку формата 16n-prefetch, пропускная способность увеличилась, а частота немного снизилась. Самые массовые виды на данный момент – это GDDR6 и GDDR6X. Основным отличием GDDR6 стало то, что каждый 32-битный канал данных разделен на два 16-битных (рис. 12.11).

Figure 2: GDDR5 8n Prefetch Memory Architecture

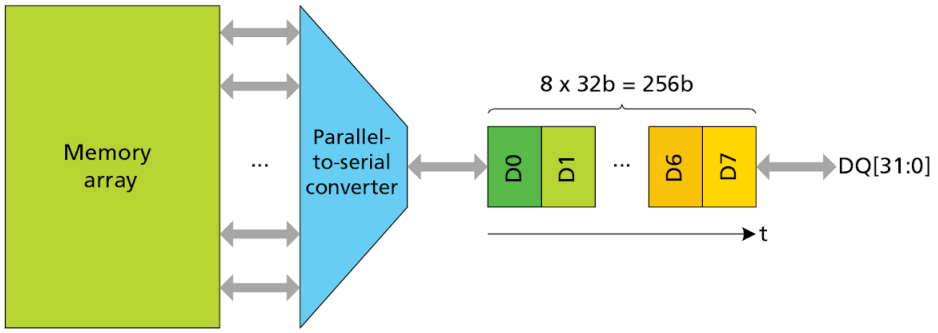


Figure 3: GDDR5X 16n Prefetch Memory Architecture

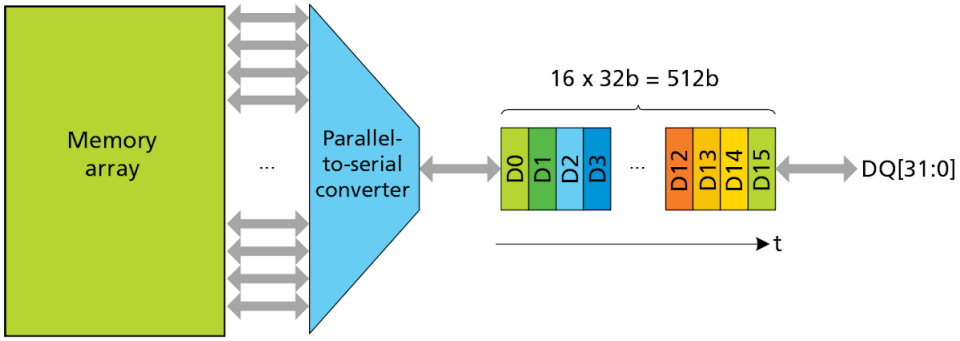


Figure 4: GDDR6 2-Channel 16n Prefetch Memory Architecture

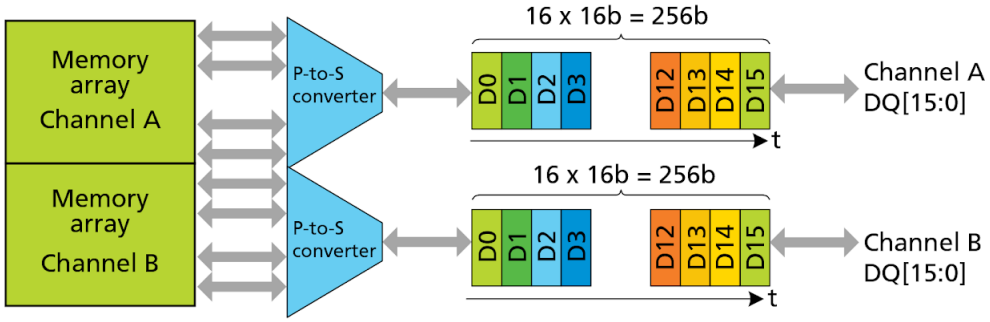


Рисунок 12.11 – Технология prefetch в GDDR

Благодаря этому количество одновременных запросов к памяти увеличилось, но непосредственно на пропускную способность это не влияет. Основной прирост дали повышенные частоты, по сравнению с предшественником. В GDDR6X, прирост достигается благодаря новой технологии кодирования PAM4 – 4 Pulse Amplitude Modulation – четырехуровневая амплитудно-импульсная модуляция, суть которой в использовании двух битов за такт вместо одного, как у всех остальных. Все ключевые характеристики можно отобразить в табл. 12.1, и увидеть разницу в реальных скоростях работы памяти, а также соотношения реальных и эффективных частот.

Таблица 12.1 – Таблица характеристик памяти типа GDDR

Тип памяти	**Бит за такт по горизонтали импульса**	**Бит за такт по вертикали импульса**	**Выборок n-prefetch**	**Количество передаваемых бит / байт из памяти за один такт**	**Соотношение эффективной частоты к частоте шины памяти**	**Соотношение эффективной частоты к частоте ядра памяти**
GDDR6X	4 (WCK)	2 (PAM4)	16 (8x2)	64 / 8	16 : 1	64 : 1
GDDR6	4 (WCK)	1	16 (8x2)	32 / 4	8 : 1	32 : 1
GDDR5X	4 (WCK)	1	16	32 / 4	8 : 1	32 : 1
GDDR5	4 (WCK)	1	8	16 / 2	4 : 1	16 : 1
GDDR4	2	1	8	8 / 1	2 : 1	8 : 1
GDDR3	2	1	4	4 / 0.5	2 : 1	4 : 1

На сами чипы памяти бывают на 1 Гб или на 2 Гб, располагаются они вокруг графического процессора (рисунок 12.12).



Рисунок 12.12 – Расположение модулей памтяи на плате

Релиз 50-й линейки RTX видеокарт от компании NVIDIA, принес новый перспективный стандарт памяти GDDR7. В нем применена технология кодирования PAM-3, которая позволяет передавать 3 бита информации за такт. Частоты передачи также возросли, что позволило увеличить пропускную способность до 128 ГБ/с. При всем при этом, память стала проще в изготовлении и еще более энергоэффективной.

Для управления и отслеживания характеристик на материнской плате видеокарты располагается модуль ПЗУ, в котором хранятся собственный видео-BIOS, шрифты, служебные таблицы и ряд другой информации, из-за

сложности восстановления видео-BIOS’а, их на плате может располагаться до 2 шт. В некоторых игровых моделях вместо резервного BIOS’а устанавливается «разогнанный» (OC BIOS), чтобы увеличить быстродействие карты.

Питание видеокарты. Питание материнской платы видеокарты осуществляется через шину PCIe, по крайней мере в ранних моделях видеокарт, и пока его хватало. PCIe может обеспечивать до 75 Вт питания. С течением времени мощность графических процессоров росла, в результате чего, стало понятно, что для новых видеокарт потребуются разъемы дополнительного питания. Так появился разъем стандарта 8pin, расположили его в верхнем правом углу платы. Стандартно такой разъем дает 150 Вт электропитания, но из-за того, что некоторые модели видеокарт потребляют меньше энергии, такое питание является для них избыточным. Для решения этой проблемы 2pin сделали опциональными, так появился разъем 6pin. Потребление 6 pin разъема уменьшилось до 75 Вт (рисунок 12.13). Забегая вперед, для блоков питания, чтобы не делать множество отдельных кабелей для каждого конкретного интерфейса, создали специальный кабель 6+2pin.



Рисунок 12.13 – Разъемы питания видеокарты

С выходом на рынок линейки видеокарт 40-й серии от компании NVIDIA, появился новый стандарт питания 12VHPWR. Сам порт имеет 12pin, позже их число увеличилось до 16, но разветвлялся на 3–4 стандартных 8pin разъема. Такая конфигурация позволила увеличить мощность до 600 Вт (рисунок 12.14), но также сэкономить на физическом пространстве.



Рисунок 12.14 – Кабель питания 12VHPWR

Интерфейсы видеокарты. Разделим все интерфейсы на две логические группы: интерфейсы вывода (VGA, HDMI, DP– из самых популярных) и интерфейс ввода, в большинстве случаев он один и позволяет видеокарте общаться с CPU – это PCIe (Peripheral Component Interconnect-Express) на 16 линий. С момента своего появления он постоянно наращивает пропускную способность и прошел путь от 2.5 Гт/с, до внушительных 32 Гт/с (рисунок 12.15).

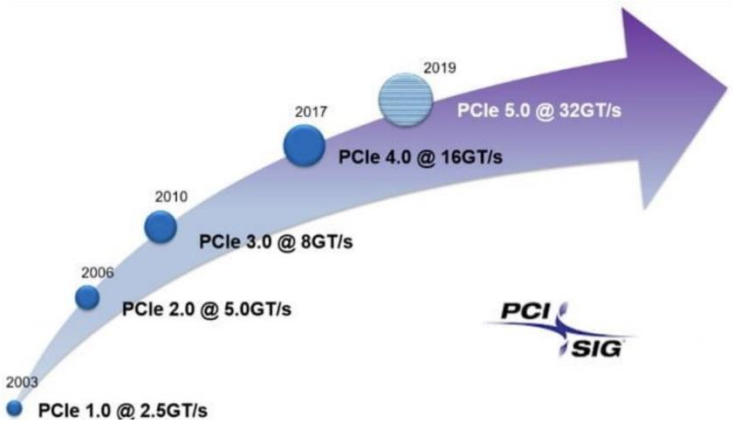


Рисунок 12.15 – История развития технологии PCIe

PCIe – компьютерная последовательная шина, имеющая принцип соединения «точка-точка», в которой применяются высокопроизводительный протокол последовательной передачи данных. PCIe основывается на программной модели обычной шины PCI, но функционирует в полнодуплексном режиме (одновременная передача данных в обе стороны), по независимым каналам. Такие каналы называются линиями PCIe, причем от числа линий (x1, x2, x4, x8, x16, в редких x24 или x32) напрямую зависит пропускная способность шины, измеряемая, как правило, в гигатранзакциях в секунду (ГТ/с). Пропускная способность шины может быть также указана в гигагерцах или в гигабитах в секунду. Все эти понятия взаимосвязаны, отражают одну и ту же характеристику и могут быть выражены друг через друга. Например, NVIDIA RTX 3090Ti имеет скорость работы памяти 21000 МТ/с, что соответствует 21 ГГц, 21 Гбиту/с и 21 ГТ/с. Касаемо применения данный шины по числу линий, то PCIe x32 используется редко и только в ряде особых и специальных случаев, а соответствующие продукты массового производства почти равны нулю; PCIe x12 в основном применяются в серверных решениях; PCIe x2 используется в основном для внутренних

интерфейсов. Наиболее популярными до сих пор остаются варианты x1/x4/x8/x16. За время своего существования шина PCIe прошла уже 5 этапов эволюции, каждый раз увеличивая пропускную способность, увеличивая тем самым эффективность.

- PCIe 1.0 появился в 2003 году, со скоростью передачи данных в каждой полосе 2.5 ГТ/с. В максимальном варианте на 16 полос пропускная способность составляла 4 ГБ/с Использовался он для сетевых карт и некоторых накопителей. Для видеокарт тогда еще стандартным интерфейсом оставался AGP (Advanced Graphics Port).
- PCIe 2.0 на момент 2007 года имел уже 5 ГТ/с по одной полосе. В реализации на 16 полос пропускная способность составила 8 ГБ/с.
- В PCIe 3.0 важным изменением стал новый принцип кодировки, если в первой и второй ревизиях использовалась кодировка 8/10 (информация разбивается на 8 битные сегменты, и дополняются до 10 битных кодировок), что обеспечивало надежную передачу информации, за счет применения управляющих битов, то в этой ревизии, и в почти всех последующих, используется 128/130 (работает по тому же принципу, только с сильно большим количеством бит). Скорость возросла до 8 ГТ/с. Расширилась сфера применения, появилась возможность подключения твердотельных накопителей через данную шину. Однако, стоит отметить, что шина в материнской плате одна, и чем больше устройств к ней подключается, тем меньше, может быть, производительность отдельных компонентов.
- PCIe 4.0 привнес еще большую масштабируемость, уменьшение задержек и поддержку виртуализации. Пропускная способность выросла до 16 ГТ/с.
- PCIe 5.0 самый новый и эффективный стандарт, который увеличил максимальную ширину шины до 24, а пропускную способность до 32 ГТ/с.

Сейчас, активно в разработке находится PCIe 6-го поколения, принцип работы которого должен измениться. Пропускная способность вновь должна удвоиться до 64 ГТ/с, а принцип кодирования информации должен перейти на более современный и инновационный PAM-4. Важно также и то, что данный интерфейс, по своей сути взаимозаменяем, и, в отличии от системной памяти, где разные поколения физически разделены ключами, не имеет таких ограничений. Исходя из этого, видеокарту, которая поддерживает PCIe 4.0 может быть установлена в материнскую плату, имеющую шину PCIe 3.0, и наоборот, и она даже будет корректно работать, однако не будет выдавать заявленных производителем характеристик, так как будет ограничена шиной.

Интерфейсов вывода графической информации – много, но для видеокарт стандартными остаются VGA, DVI, HDMI и Displayport (рисунок 12.16).



Рисунок 12.16 – Стантартные разъемы для видеокарт

VGA – разработанный для одноименного стандарта Video Graphics Array, в далеком 1987 году и иногда используется по сей день несмотря на то, что активно вытесняется более новыми и производительными стандартами. Это разъем, содержащий 15 pin, расположенных в три строки, он служит для передачи аналогового сигнала, в отличии от всех остальных. Глубина цвета – 18 бит, который использует 6 бит на канал (R, G, B) предоставляя возможность отображать 262144 комбинации цветов. Максимальное теоретическое разрешение –2048x1536 при частоте в 30–60 Гц. При этом наиболее распространенным разрешением все-таки является 1280x1024 или 1024x768, где поддерживается стабильные 60 Гц и минимальные помехи и искажения.

DVI – Digital Video Interface – разработан в 1999 году. Как ясно из названия, интерфейс стал цифровым, увеличил свое теоретическое разрешение до 2560×1600, возросли рабочие частоты до стабильных 60 Гц в режиме FullHD, появилась поддержка 144 Гц. Сам стандарт подразделяется на несколько видов, каждый из которых используется в разных задачах: DVI-I (Integrated) поддерживает как цифровую передачу, так и аналоговую, DVI-D (Digital) – поддерживает только цифровую передачу, и DVI-A (Analog) – только аналоговую. В нем также реализованы два варианта передачи данных Single link и Dual link, в первом случае возможна передача изображения с разрешением в 2К (2560x1440) при 60 Гц, во втором случае изображение с тем же разрешением можно выводить при 75–120 Гц, а изображение в FullHD в 144 Гц. Глубина цвета – 24 бит по 8 бит на каждый канал, что дает 16777216 комбинаций цветов.

HDMI – High-Definition Multimedia Interface – самый популярный интерфейс на сегодняшний день. Позволяет передавать как цифровые видеоданные, так и аудиоданные. Сам стандарт постоянно развивается и

сейчас уже насчитывает более 10 версий. Самыми популярными являются версии 1.4 и 2.0. Максимально возможное разрешение для стандарта HDMI 2.1 – 10К (10240x4320). Возможная глубина цвета увеличилась до 30/36/48 и более бит, в частности, благодаря добавлению поддержки HDR (High Dynamic Video Range) использующей 16 бит на канал, которая позволяет выдавать более детализированное и насыщенное изображение, с натуральными цветами и глубокими тенями. Среди прочих достоинств этого стандарта – поддержка 3D.

Displayport – в некотором роде основной конкурент HDMI. Это также цифровой интерфейс, но имеющий более низкое напряжение работы (3.3 В против 5В у HDMI и DVI), и слегка большую полосу передачи данных чем у HDMI. Теоретическое разрешение у него также выше, чем у конкурента 16К (15360x8640). Также имеется поддержка HDR. За счет большей пропускной способности (77.37 Гбит/с против 48 Гбит/с) имеет лучшую работу с высокой частотой кадров при высоких разрешениях. Интересной особенностью является поддержка Multi-Stream – подключения нескольких мониторов через каскад. В разъеме предусмотрена адаптивная синхронизация, что позволяет технологиям типа FreeSync (от AMD) и G-Sync (от NVIDIA), подстраивающие частоту обновления монитора под фактическую частоту кадров работать стабильнее, чем через HDMI.

Для более удобного отображения соберем все данные в табл. 12.2.

Таблица 12.2 – Сравнительная таблица характеристик видеоразъемов

Характеристика	**VGA (D-Sub)**	**DVI**	**HDMI**	**DisplayPort (DP)**
Тип сигнала	Аналоговый	Цифровой/Аналоговый	Цифровой	Цифровой
Макс. разрешение	2048×1536 (QXGA)	2560×1600 (Dual Link)	10K@120 Гц (HDMI 2.2)	16K@60 Гц (DP 1.4)
Макс. частота (1080p)	60–85 Гц	144 Гц (Dual Link)	240 Гц (HDMI 2.1)	360 Гц (DP 1.4)
Макс. пропускная способность	~400 МГц (пиксельная)	7.92 Гбит/с (Dual Link)	48 Гбит/с (HDMI 2.1)	77.37 Гбит/с (DP 2.1)
Поддержка HDR	Нет	Нет	Да (HDMI 2.0a+)	Да (DP 1.4+)
Аудиопередача	Нет	Теоретически возможна	Да (до 32 каналов)	Да (до 8 каналов)

Продолжение табл. 12.2

Адаптивный Sync	Нет	Нет	FreeSync (HDMI 2.1+)	FreeSync, G-Sync
Multi-Stream (MST)	Нет	Нет	Нет	Да (поддержка нескольких мониторов)
Разъемы	15-pin D-Sub	DVI-I, DVI-D, DVI-A	HDMI Type A/C	DP, Mini DP, USB-C (Alt Mode)
Год появления	1987	1999	2002	2006
Актуальность	Устарел	Устарел (редко используется)	Основной для ТВ/консолей	Основной для ПК/мониторов

12.3. Основные различия в архитектурных подходах у компаний AMD, NVIDIA и Intel

В последние несколько лет, основными и единственными лидерами на рынке видеокарт являются AMD и NVIDIA. С 2022 года на рынок выходит еще одна компания – Intel, представив свою собственную линейку видеокарт ARC. Рассмотрим основные тенденции в разработке видеокарт за последние несколько лет, выделим основные отличия в архитектурных взглядах между лидирующими компаниями, сравним их между собой и новым игроком на рынке, выделим преимущества и недостатки описанных подходов. Важным отличием, которое сразу можно выделить, является то, что ориентированность компаний уже изначально отличается. NVIDIA, делает основную ставку на искусственный интеллект и неграфические вычисления, которые дают видимое преимущество в профессиональных задачах, связанных с суперкомпьютерами, большими вычислениями, нейро-медициной и с обучением моделей ИИ. При всем при этом с графическими вычислениями (дизайн, рендеры, архитектура и видеоигры), данные графические процессоры тоже справляются на отлично. Графические процессоры от AMD, хоть и позволяют работать со всем вышеперечисленными задачами, все-таки находится на догоняющей позиции, сильно уступая в этом плане, и больше ориентирована на рынок графических вычислений и современных видеоигр. Рассмотрим основные архитектурные подходы к созданию своих процессоров у этих компаний, и разберемся почему каждые из них хороши в своих, конкретных задачах.

NVIDIA. Основным преимуществом графических процессоров компании «зеленых» являются потоковые мультипроцессоры (SM – Streaming Multiprocessor), разработанные еще в далеком 2006 году, ознаменовавшие переход от разделенных шейдерных ядер к универсальным, а позже получившие высокопроизводительные тензорные ядра и RT-ядра, о которых говорилось ранее.

SM – это основной вычислительный модуль, на котором строится остальной процессор, в его состав входит определённое число CUDA ядер, несколько тензорных ядер и RT-ядро (начиная с семейства Turing), L1 кэш, несколько блоков текстурной обработки, регистры и различные блоки управления.

Например, для RTX 2080, построенная на архитектуре Turing, для чипа TU104 в состав одного SM модуля входит 64 ядра CUDA, 8 тензорных ядер и 1 RT-ядро и 4 TMU. Общее число SM – 46 шт., следовательно общее число ядер – 2944 шт. Для сравнения, в относительно новой RTX 4090, построенной уже на Ada Lovelace 128 CUDA-ядра, 4 тензорных ядра (нового, уже 4-го поколения), 1 RT-ядро (третьего поколения) и 4 TMU на один модуль, при этом количество самих SM – увеличилось до 128 шт, а общее число ядер стало – 16384 шт. Схематически структура одного потокового мультипроцессора для RTX 4090 выглядит следующим образом (рисунок 12.17). Для сравнения приведем структуру SM для RTX 2080, на рис. 12.18, соответственно.

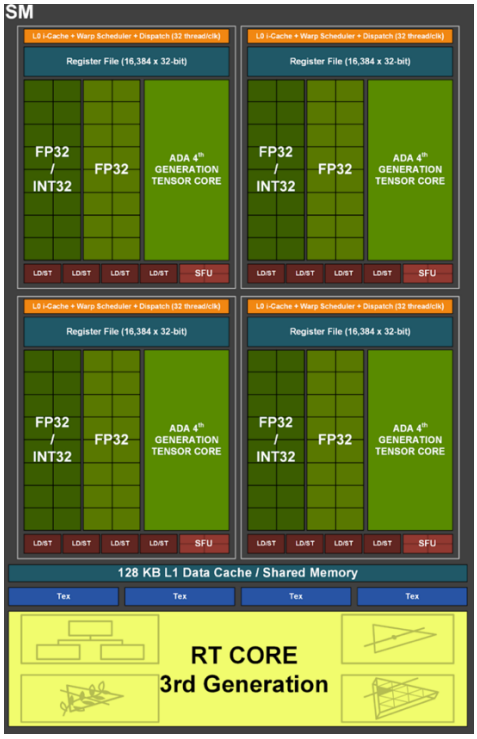


Рисунок 12.17 – Устройсто мультипроцессора на примере 4090

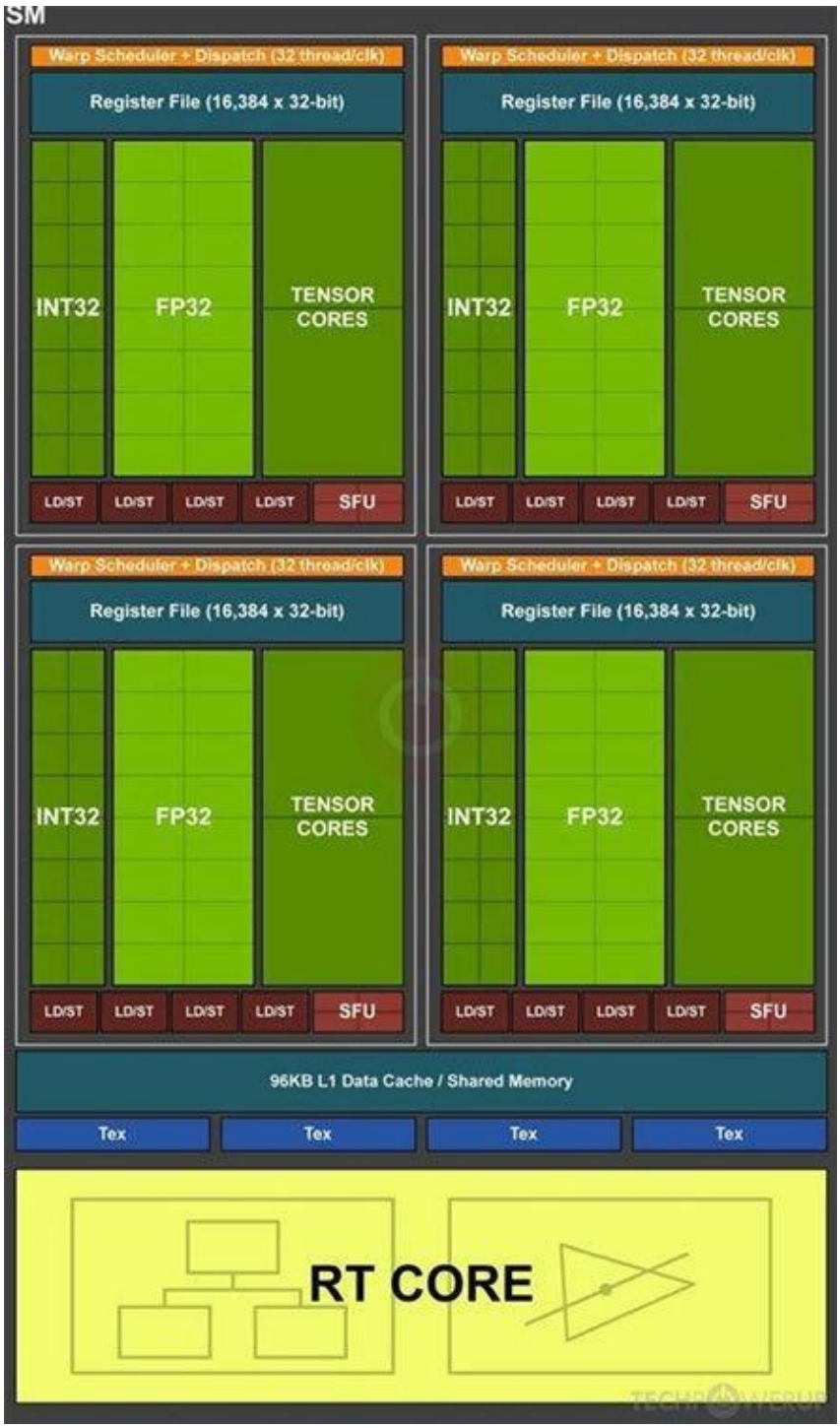


Рисунок 12.18 – Устройсто мультипроцессора на примере RTX 2080 (TU102)

Два SM-модуля, объединяются в кластер обработки текстур (TPC – Texture Processing Cluster), а 6 кластеров в совокупности с 14 блоками растровой обработки объединяются в графический вычислительный кластер (GPC – Graphical Processing Cluster) – мощная вычислительная единица, содержащая в себе все необходимое для работы с графикой (рисунок 12.19).

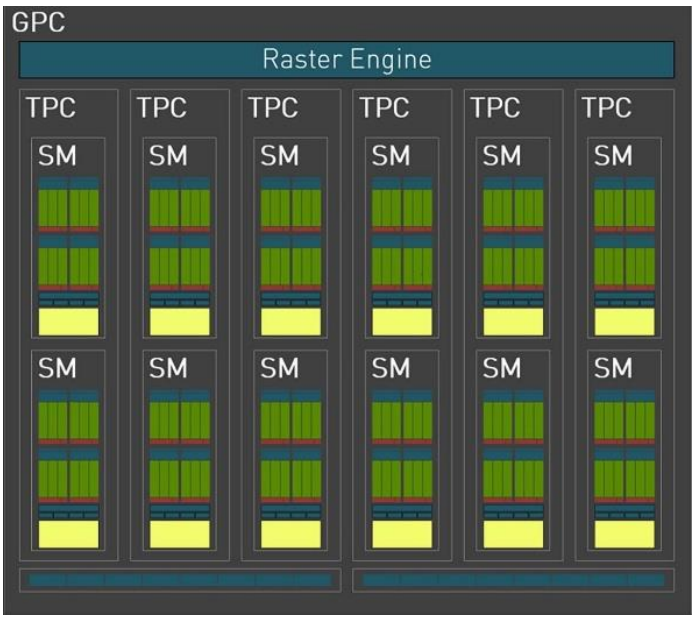


Рисунок 12.19 – Устройсто кластера на примере 4090

Совокупность GPC (12 шт в случае RTX 4090), огромного кэша второго уровня (L2), контроллеров памяти, контроллера шины PCI, GigaThread Engine (мощного и сложного планировщика задач) и ряда вспомогательных управляющих элементов дает нам итоговый графический процессор AD102 (рисунок 12.20). Для сравнения приведем архитектуру процессора TU102 установленного в рассмотренной ранее RTX 2080 Ti и даже архитектуры процессора GP-102, который ставился в GTX 1080 Ti (рисунки 12.21 -12.22 соответственно).



Рисунок 12.20 – Устройсто процессора на примере RTX4090 (AD102)

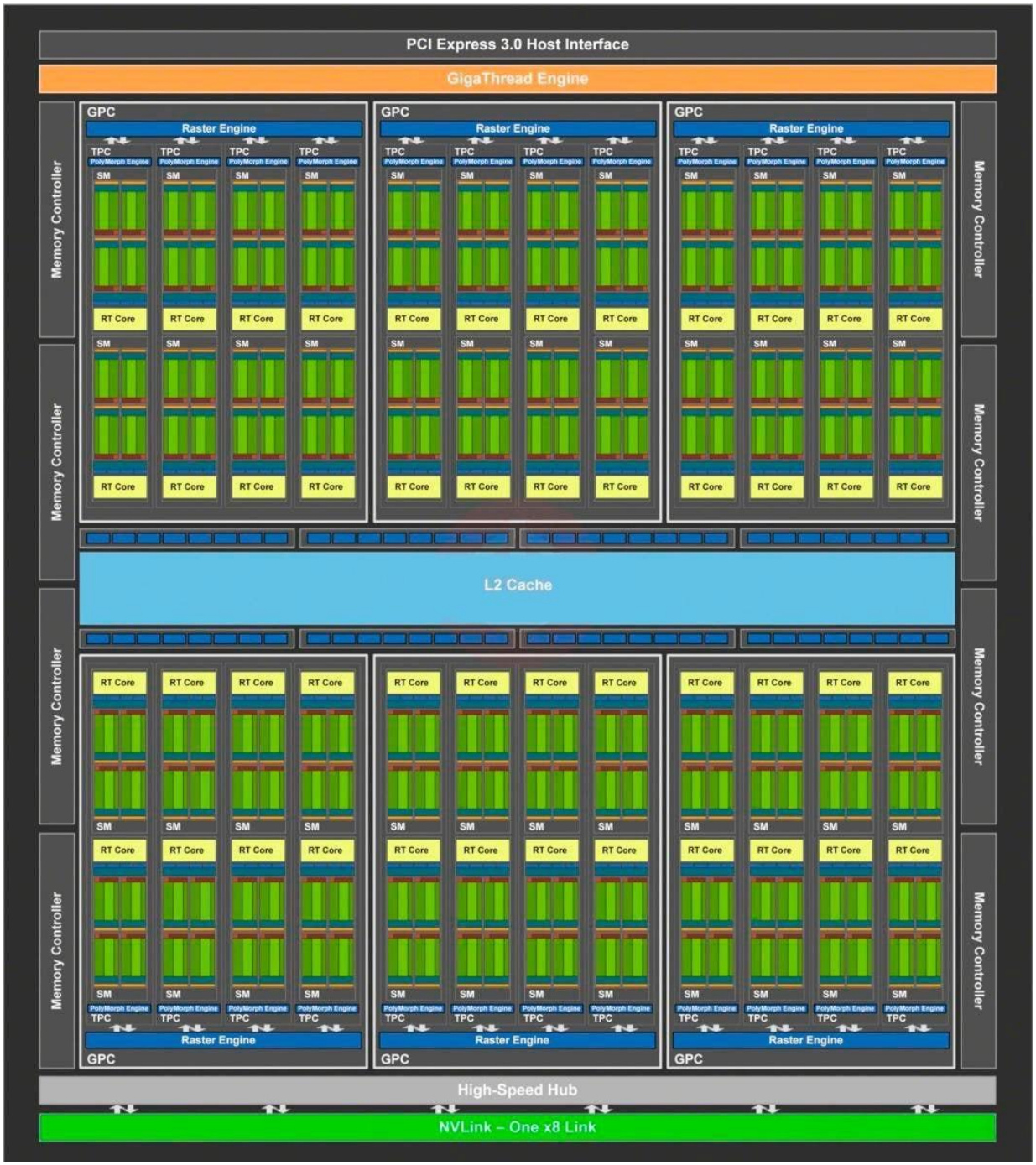


Рисунок 12.21 – Устройство процессора на примере RTX2080 (TU102)



Рисунок 12.22 – Устройсто процессора на примере GTX1080 (GP102)

Важное замечание по поводу указанных характеристик процессоров. В различных источниках указываются различное количество SM и CUDA-ядер, причиной тому отделение эталонного чипа, определяющего общую архитектуру и устанавливающегося в профессиональные решения, от коммерческих вариантов. Действительно – для чипа AD102 – число CUDA-ядер должно быть 18432, однако, если посмотреть на все коммерческие модели, то в них установлен не чисто AD102, а AD-120–300-A1, в котором только 16432 ядра. Сделано так для того, чтобы отделить профессиональный сегмент видеокарт от коммерческого, а также для того, чтобы удешевить производство. Такая же ситуация и с частотами, на которых работают GPU, в коммерческих вариантах могут встречаться разные частоты для одного и того же GPU для разделения бюджетных версий и более дорогих OC (OverClocked) – с повышенными частотами.

Если говорить о младших моделях, то благодаря модульной конструкции GPU, NVIDIA уменьшает количество GPC, контроллеров памяти и управляющих элементов. Например, процессор AD106 который устанавливается в RTX 4060 Ti, имеет 3 GPC, что суммарно дает 36 SM, 48 ROP, 44 TMU, 4 контроллера памяти (рисунок 12.23).



Рисунок 12.23 – Устройсто процессора на примере RTX4060Ti

Как видно из рисунков выше архитектура графических процессоров у NVIDIA за почти 10 лет не претерпела глобальных изменений, усложнились сами потоковые процессоры, изменилось программное обеспечение, повысились частоты работы памяти и самого процессора. Даже с релизом 50-й линейки RTX архитектура осталась верна всем предыдущим (рисунок 12.24), вновь получив улучшенные и ускоренные версии всех своих компонентов (5-е поколение тензорных ядер и 4-е поколение RT-ядер), а в части программных улучшений – более серьезное внедрение ИИ.



Рисунок 12.24 – Устройсто процессора на примере RTX5090

В частности, NVIDIA перешла на чиплетную компоновку (о которой будет сказано позже), как у ее основных конкурентов. Среди прочих изменений – поддержка GDDR7, с большими частотами, добавление AI-Management Processor – планировщика на базе ИИ, для лучшей оптимизации нагрузки на GPU, оптимизация общего энергопотребления, поддержка новых форматов данных FP4 и FP6 и интеграция технологии Neural Shaders для поддержки работы DLSS 4.0, но все это за счет энергопотребления, которое может достигать 500–600 Вт.

Отдельного упоминания стоит технология SLI (Scalable Link Interface), на базе NVlink, которая позволяла объединять ресурсы нескольких видеокарт в одну более производительную систему. Применялась она для сугубо профессиональных задач, но с приходом на коммерческий рынок, была переориентирована на графические вычисления. Ее новая задача состояла в том, чтобы разделить генерацию кадров на две (или более) карты по отдельности, либо пополам по горизонтали, когда в режиме SFR (одна карточка генерирует верхнюю половину, вторая – нижнюю), либо на четные и нечетные кадры, но целиком, в режиме AFR. При этом характеристики карточек не удваивались, то есть все ресурсы оставались доступными в рамках одной карты. Также применение такого подхода накладывало существенные ограничения, например материнская плата должна была иметь 2 полноценных слота PCIe x16, а целевое ПО, например видеоигры, должны были внедрять профили SLI для того, чтобы уметь использовать такой режим. Особую популярность эта технология набрала в период GTX 10-й серии и RTX 20-й серии, хотя первые наработки были еще в 1998 году. За программную связку отвечал отдельный контроллер, он указан на рисунке 12.21 ярко зеленым цветом, физически на плате был добавлен специальный интерфейс, а также коннектор для связи видеокарт, называемый мостом. Прирост

производительности в графических вычислениях не был существенным, поэтому данная технология в коммерческих решениях продержалась не долго. Последняя ее итерация была представлена в RTX 3090, но производительность одной такой карты уже позволяла выдавать качественную картинку при уже высоких разрешениях, и с высокой частотой кадров.

AMD. Главный оппонент для NVIDIA, была и остается компания AMD. Подход к созданию GPU у них достаточно сильно отличается от «зеленых». Основная архитектура, которая используется в последнее время – RDNA, эксплуатирующаяся с 2019 года и прошедшая уже 4 поколения. В отличии от «зеленых», компания AMD в своем подходе к разработке GPU, зачастую принимают неоднозначные решения, даже в рамках одного поколения.

В основе вычислительных мощностей GPU от «красных», лежат CU – Computing Units, которые были разработаны еще в 2012 году для архитектуры GCN (Graphics Core Next). Некоторый аналог CUDA. В одном таком блоке содержались 4 TMU и 64 SP (Streaming Processor – потоковый процессор), поделенных на четыре группы, L1-кэш на 16 КБ памяти, и ряд необходимых блоков управления. Каждая из них работает со своим потоком команд, которые задает планировщик исполнения, имеющийся внутри CU (12.25).

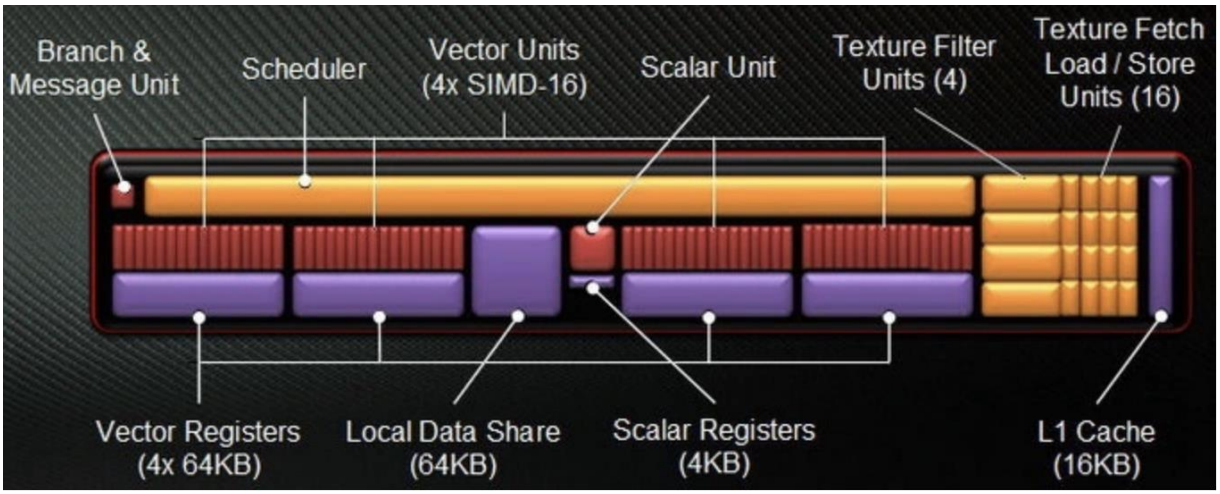


Рисунок 12.25 – Устройсто мультипроцессора у AMD

Общая архитектура чипа GCN1 содержала 32 CU, что в сумме давало 2048 SP и 128 TMU блоков, общая архитектура такого процессора, представлена на рисунке 12.26, ниже. Среди остального оборудования были 32 блока растеризации (ROP), и 6 контроллеров памяти по 64 бита каждый, что давало возможность работы с 384 битной шиной GDDR5 памяти, кэш L2 составлял 768 КБ. Управляют ими процессор графических команд и два

движка асинхронных вычислений (ACE), у оппонентов за это отвечает GMU. Он отвечает за регулировку очередей команд, не привязываясь к конкретным вычислительным блокам, позволяя эффективнее нагружать ресурсы процессора.

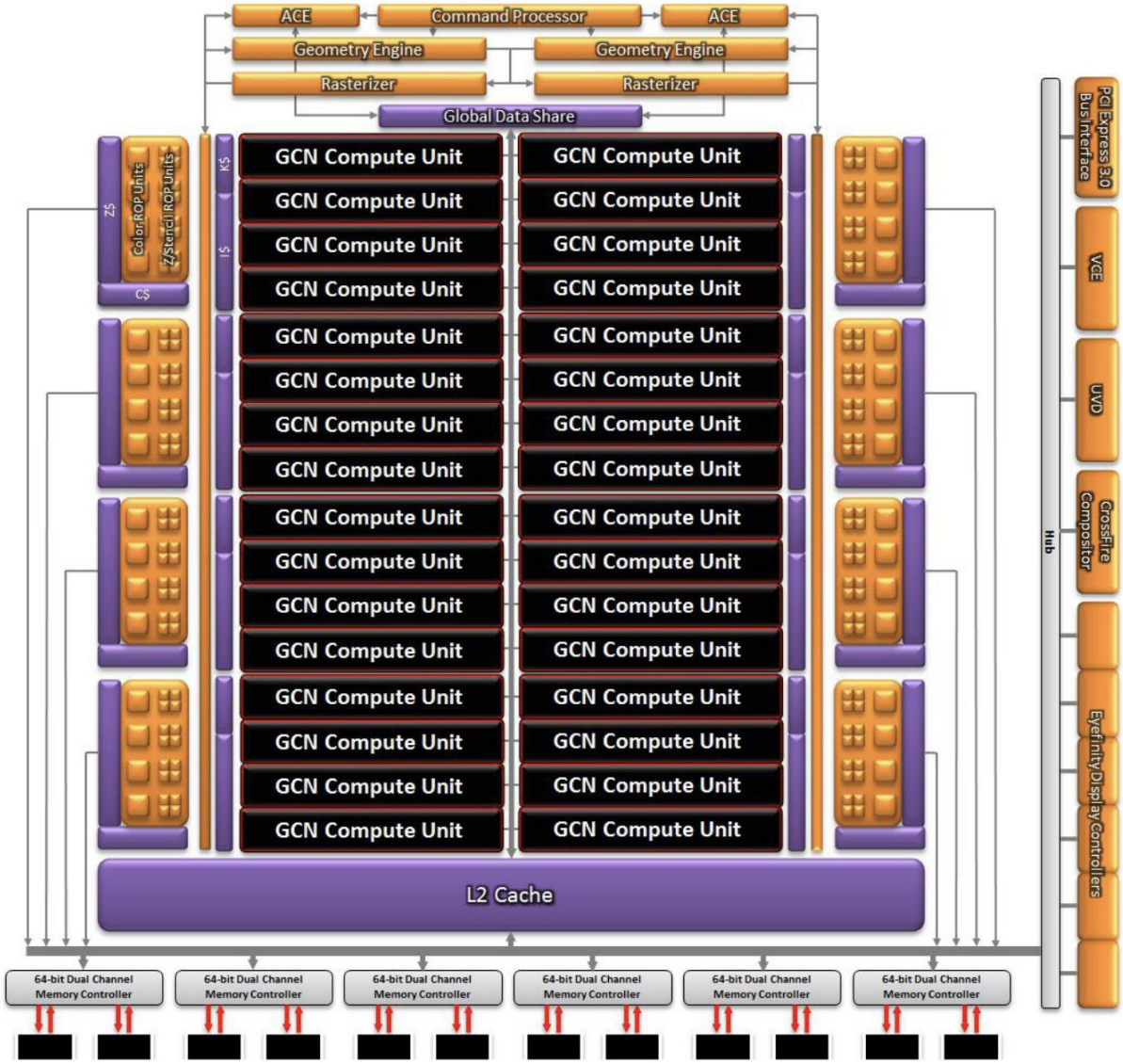


Рисунок 12.26 – Устройсто процессора на архитектуре GCN1

Позже, в 2013 году, AMD представила архитектуру GCN2, где главным изменением стал переход от одного общего «вычислительного модуля», на несколько поменьше. CU были разделены на 4 группы, по 11 штук, каждая из которых содержала 16 блоков растеризации. Такое решение, получило название шейдерный процессор, или Shader Engine (SE). Общее число всех главных компонентов равнялось 44 CU, в которых 2816 SP и 176 TMU, число ROP равнялось 64, кэш L2 увеличился до 1024 КБ. На рисунке 12.27,

изображен процессор Hawaii, для видеокарты RX 290X. Среди прочих нововведений добавилась еще пара контроллеров памяти, что расширило общую шину до 512 бит, количество блоков асинхронных вычислений (ACE) тоже прибавилось, составив 8 штук.

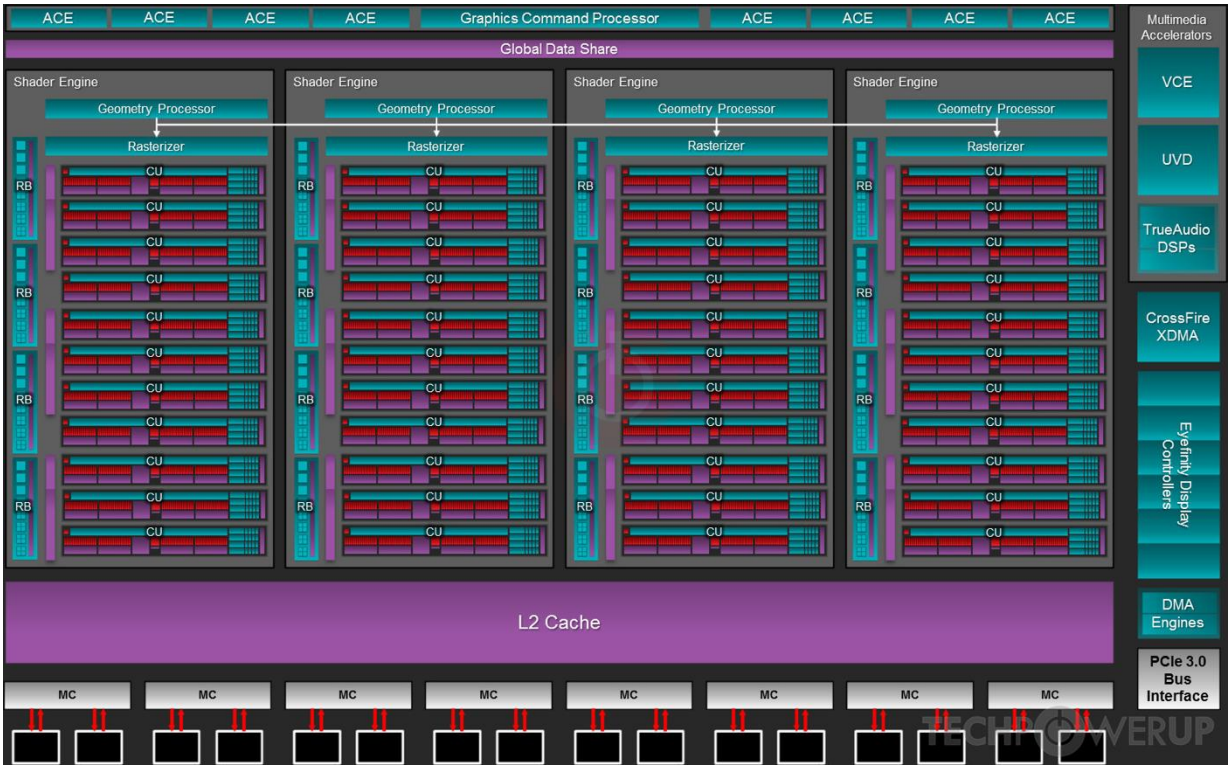


Рисунок 12.27 – Устройсто процессора на архитеутуре GCN2

Примерно в это же время у «красных», как и у «зеленых», был пик моды на объединение нескольких видеокарт в одну вычислительную систему с целью увеличения производительности. У AMD данная технология называлась CrossfireX, но работала по схожему принципу, через отдельный контроллер и мост. Как и в случае с NVIDIA данная технология долго не прожила и в коммерческих решениях популярности не сыскала.

В последующие несколько лет, изменений в архитектуре особо не происходило, в, конкретно, следующем поколении GCN3 (рисунок 12.28), например для архитектуры Fiji, возросло количество CU (до 64 шт.), вместе с ними увеличилось и количество TMU (до 256 шт.), также в два раза увеличился и L2 кэш. Была впервые реализована поддержка перспективной и инновационной памяти типа HBM (High Bandwith Memory), благодаря чему GPU смог работать большими объемами памяти и с шиной шириной в 4096 бит.



Рисунок 12.28 – Устройсто процессора на архитеутуре GCN3

Главным изменением четвертого поколения (GCN4), помимо программных и алгоритмических, стало применение меньшего техпроцесса (14 нм против 28 нм у предыдущей версии). Это позволило уменьшить физические размеры кристалла, и улучшить энергопотребление.

Самая противоречивая и, возможно, недооцененная итерация графических процессоров от AMD – это поколение GCN5 (рисунок 12.29) и серия видеокарт Vega. Необычный новый ГП (по сути, являющийся развитием идей GCN3) производился по техпроцессу 14 нм и использовал следующее поколение быстрой памяти – HBM2. Это сделало возможным использование больших объемов памяти, так, например, Vega 64 и Vega 56 имели по 8 ГБ, что было достаточно большим объемом. У конкурента, такими объемами обладали только старшие решения (GTX 1070 Ti и 1080). В иерархии памяти объем этой памяти теперь мог использоваться как высокопроизводительный кэш. Общий объем памяти мог расширяться за счет системной благодаря новому контроллеру High Bandwidth Cache Controller (HBCC)**,** встроенному непосредственно в чип. Он использует страницы небольшого размера, помещая неактивные в системную память (ОЗУ). При необходимости запрашивая их постранично. Поэтому такой способ куда эффективнее обычного вытеснения ресурсов в память, применяемого у других ГП. Частоты работы процессора увеличились, по сравнению с Fiji, а вот ширина шины памяти – уменьшилась в два раза. Увеличилось количество блоков геометрии и тесселяции, их стало 4 против 2 у предыдущих моделей. Вычислительные блоки тоже подверглись доработке: теперь ГП полную аппаратную поддержку

FP32, FP16, INT16, INT8. Эта способность получила название Rapid Packed Math, а сами вычислительные блоки получили новое название NCU (Next Generation CU). Общее количество блоков при этом осталось неизменным со времен Fiji – 64 ROP и 64 NCU, в которых 4096 SP и 256 TMU. Немаловажным было внедрение Infinity Fabric – системной шины, которая соединяла компоненты GPU.

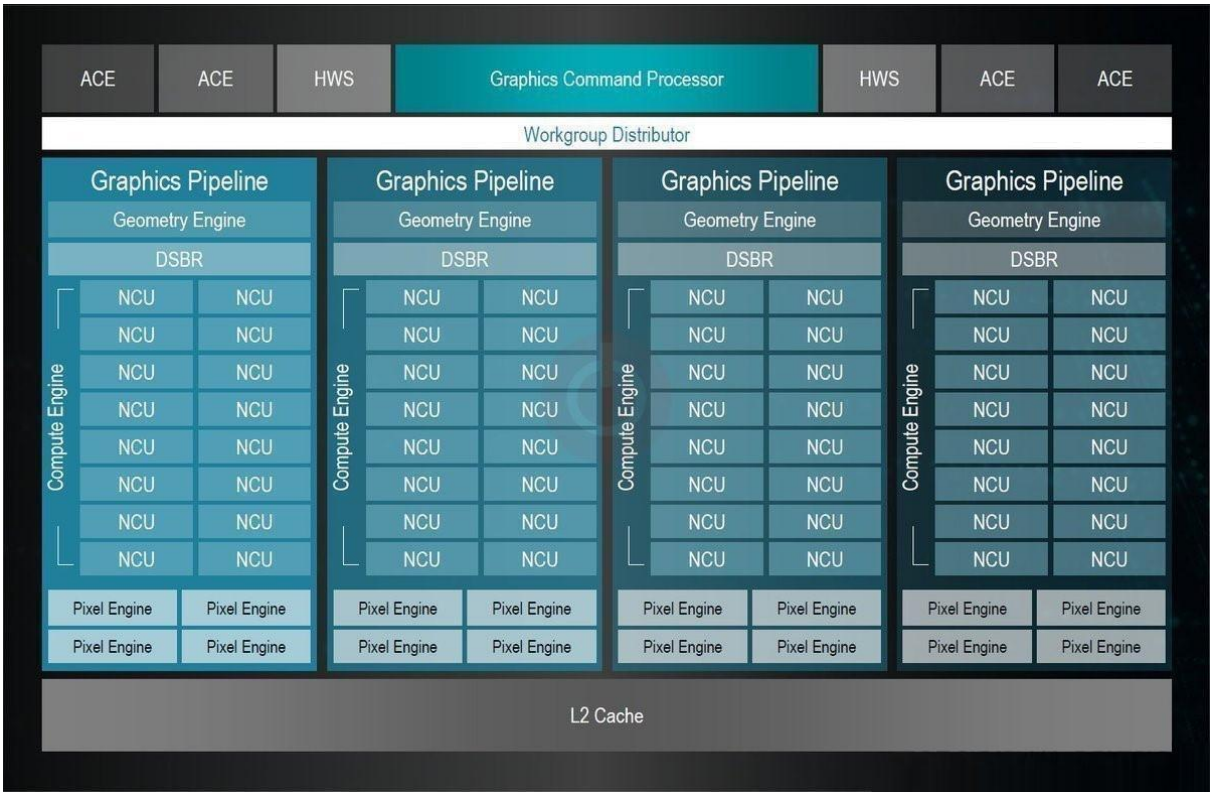


Рисунок 12.29 – Устройсто процессора на архитеутуре GCN5

Если подводить некоторый итог, то серия Vega, как и архитектура GCN5 стала гибче, быстрее и стала больше ориентирована на неграфические вычисления**,** при этом в видеоиграх Vega не часто отставала от NVIDIA, несмотря на высокую задержку и повышенное энергопотребление. Именно поэтому AMD вскоре перешла к Radeon DNA.

RDNA – стала новой вехой в эпохе развития графических процессоров AMD и стала основной на многие годы вперед. Тут стоит отметить важную особенность, что в отличии от оппонентов, «красные» переориентировались больше в сторону графических вычислений и видеоигр, в частности, отойдя от неграфических вычислений. Переход случился в 2019 году (у «зеленых» в это время выпускалась серия видеокарт на архитектуре Turing, и уже дебютировала 20-я серия видеокарт, ознаменовавшая переход к RTX), было

переработано почти что все, начиная с CU, заканчивая остальной компоновкой. Новые CU по-прежнему содержали 64 SP, однако их поделили на 2 группы, каждая из которых получила собственный планировщик (рисунок 12.30).

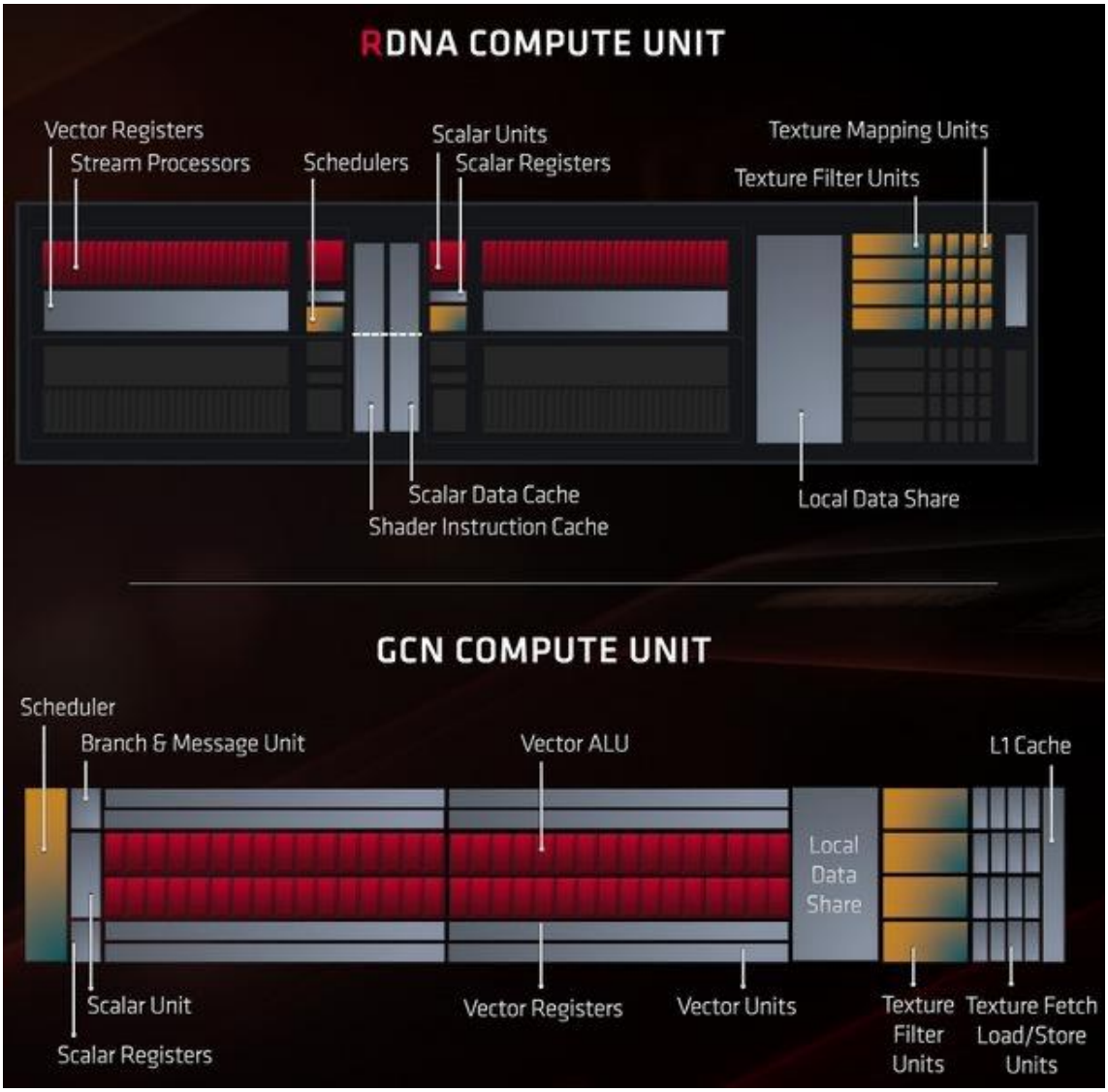


Рисунок 12.30 – Различия между CU архитектуры GCN и RDNA

Такой подход позволил еще эффективнее использовать ресурсы GPU. Если в GCN часть SP могла простаивать при определенной нагрузке, то в RDNA количество таких ситуаций свели к минимуму. Для этой же цели были увеличены и ускорены внутренние кэши. В итоге, на одной частоте блок RDNA около полутора раз быстрее аналогичного блока GCN. При этом все чипы получили поддержку PCIe 4.0, а частотные характеристики приблизились к 2 ГГц, появилась поддержка памяти GDDR6, что позволило приблизить пропускную способность к значениям HBM памяти у Vega. Ну и

последним важным изменением стал переход на 7 нм техпроцесс (у GPU «зеленых» в это время было 12 нм).

Первым вариантом GPU нового поколения стал чип Navi 10, применявшийся в RX5700. Его архитектура стала более централизованной и выглядела следующим образом – рисунок 12.31.

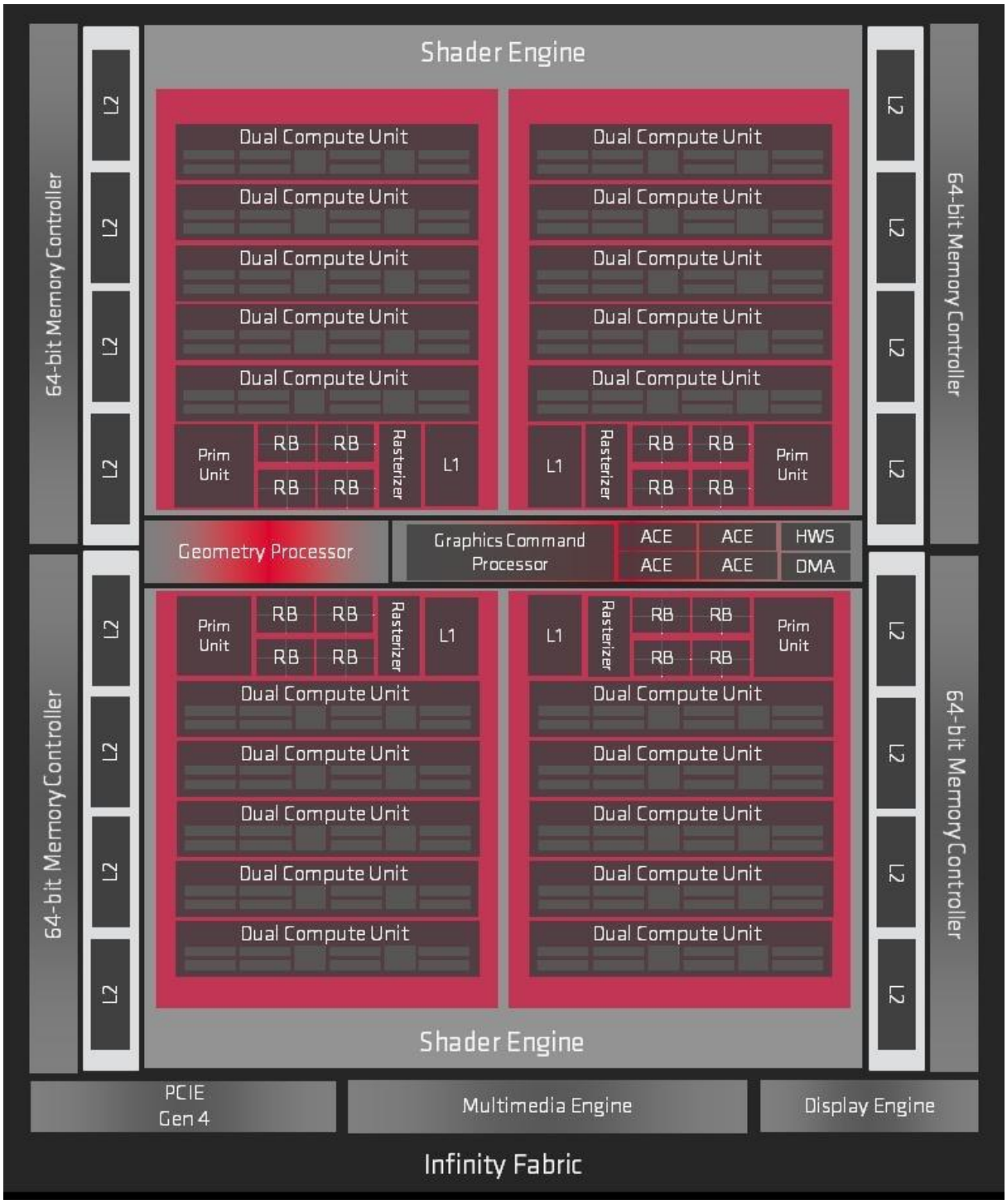


Рисунок 12.31 – Устройсто процессора на примере RX5700

Он имеет 64 ROP и 40 CU, разделенных на 2 шейдерных блока, состоящих из 2560 SP и 160 TMU. Управляют ими процессор команд, планировщик графических задач и четыре блока ACE. Четыре геометрических движка и блока тесселяции теперь находятся в одном процессоре обработки геометрии. Шина памяти уменьшилась до 256-бит. Появился новый уровень кэш-памяти L0 (32 КБ), который располагался в CU, L1 (128 КБ) – стал общим для массива CU, а L2 (8192 КБ) – разделился на 4 части, для каждого контроллера памяти, соответственно. По своим характеристикам, если сравнивать с аналогами у «зеленых», новая видеокарта AMD – RX5700 XT заняла позицию между RTX 2070 и RTX 2070 SUPER. Говоря об остальных вариантах 5000-й серии, то компания AMD принимала странные решения относительно младших моделей этой линейки. Так, например, RX5500 XT, основанная на GPU Navi 14, который обладает 1536 SP, 128-битной шиной памяти GDDR6, имела урезанным интерфейс PCI-E 4.0 x8, вместо x16. Активными в новинке осталось 1408 SP, что не помешало ей достигнуть уровня популярной RX580, на смену которой она и была разработана. Позже, была выпущена версия RX5600, которая имела схожую конфигурацию с RX5700, но с урезанной шиной памяти (192 бита), вследствие чего и с меньшим ее объемом. Таким образом линейка RDNA-1 составила неплохую конкуренцию «бюджетным» вариантам RTX 20-й, видеокарт от NVIDIA, однако ответа старшим моделям не было.

Ситуацию решено было изменить с приходом 6000-й линейки и переходом на улучшенную архитектуру RDNA-2. Главной идеей которой, было внедрение технологий трассировки лучей в свои видеокарты, для этого в новых чипах Navi 21 (RX 6800 и RX 6800 XT) в каждый CU было добавлен блок трассировки лучей, в задачи которого входило найти пересечения лучей и отправить их на обработку в потоковые процессоры. Такой процесс оказался хоть и проще, но значительно медленнее, чем у «зеленых», так как там для этого были предусмотрены отдельные ядра. Основная архитектура чипа также претерпела ряд существенных изменений, например появился еще один уровень кэш памяти – Infinity Cache (можно назвать его L3, имел объем 128 Кб), а L2, наоборот, уменьшился до 4096 КБ и стал использоваться как общий для 2-х блоков. L1 – остался общим для массива CU и имел 128 КБ, а L0 – также остался в CU и имел 32 КБ, описанные характеристики актуальны для старших моделей 6800 и 6800 XT и чипа Navi 21, в частности (рисунок 12.32). Однако тенденция осталась общей для всей линейки. Шейдерных блоков стало 4, против 2 у предыдущего поколения. В каждом блоке находилось по 20 CU.

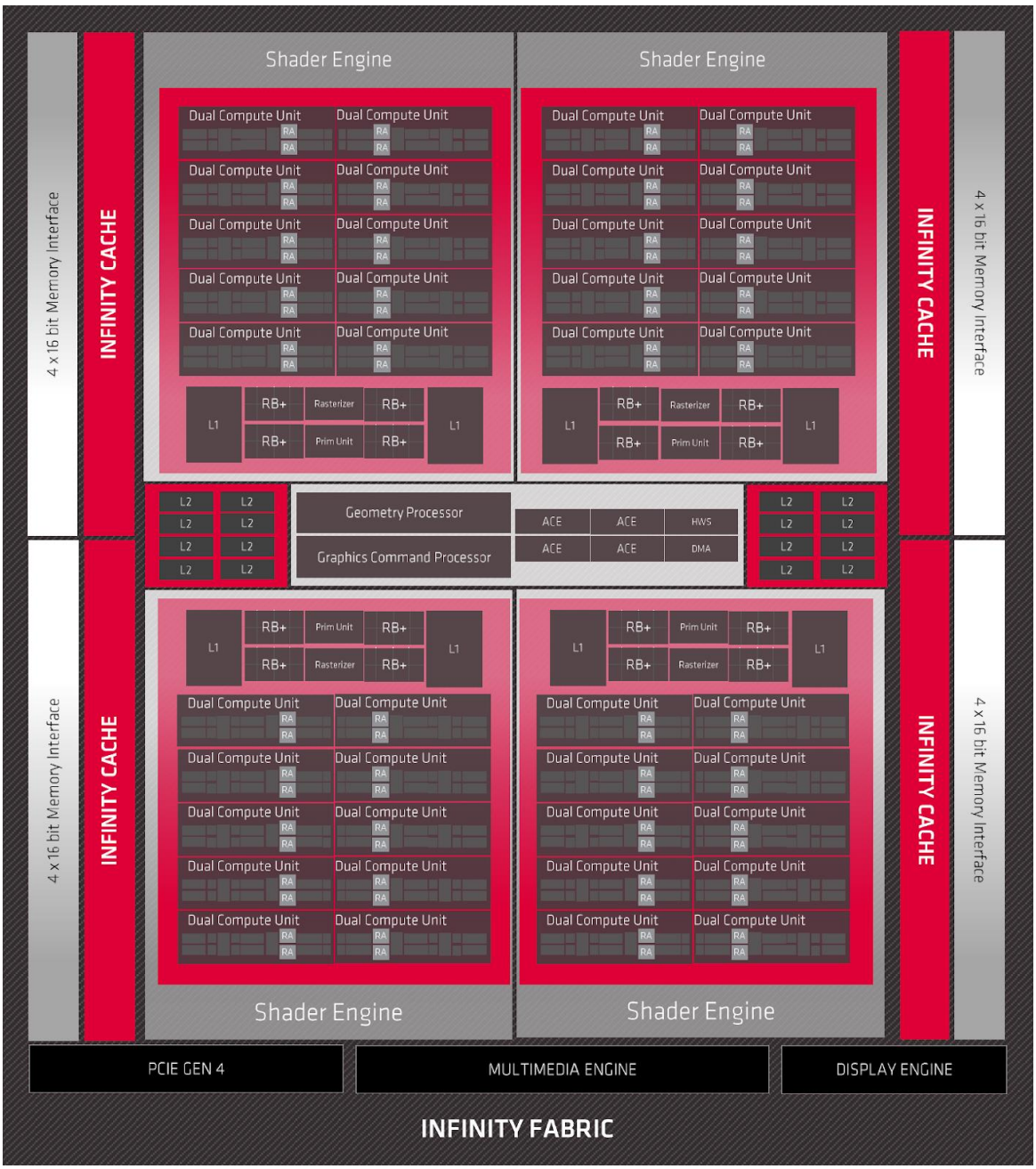


Рисунок 12.32 – Устройсто процессора на примере RX6800XT

Упомянутые GPU оснащены 128 ROP и 80 CU, формирующих 5120 SP и 320 TMU. Шина памяти 256-битная, к которой подключено 16 ГБ GDDR6 с полосой пропускания в 512 ГБ/с. Стоит отметить, что в этих решениях применено странное решение, в RX6800XT была просто отключена десятая часть блоков, а в RX6800 – целая четверть, однако у старшей модели линейки RX6900 XT, чип был полноценный. Старшие карты успешно конкурировали с RTX3080 и даже с RTX3090, а младшая ощутимо опережала народную RTX 3070. При высокой производительности и большом объеме видеопамяти недостаток у 6000-го семейства был лишь один – они сильно проигрывали

«зеленым» при включении трассировки лучей, так как реализация у них была медленнее, из-за того процесс обработки отличался, задействуя основные ядра.

Для «средних» моделей RX6700 и RX6700XT, чип Navi 21 – «разрезали» пополам. Navi 22 – имел 40 CU, 64 ROP и 160 TMU, урезанную шину памяти до 192 бит, и меньшими объемами кэш памяти. Для младших моделей (RX6600 и RX6600 XT) и самых младших моделей (RX6500) тенденция урезания исходного чипа сохранилась, так, например, у младшей модели линейки было всего 16 CU.

RDNA 3.0 представила чиплетную компоновку, которая до этого активно применялась в CPU, от все тех же AMD, где и зарекомендовала себя. Чиплет – это отдельный функциональный блок процессора. Несколько чиплетов соединяются в единый вычислительный блок, заменяя традиционный монолитный дизайн. Вместо того создавать все компоненты процессора, сразу на едином чипе (так называемый монолитный подход), разрабатываются и производятся отдельные секции, которые затем собираются в единый корпус с помощью комплексной системы соединений, образуя готовый процессор. Такой подход впервые применили именно инженеры из AMD при создании серверных процессоров серии Threadripper и Ryzen 1000-й серии. Архитектура приобрела следующий вид (рисунок 12.33).

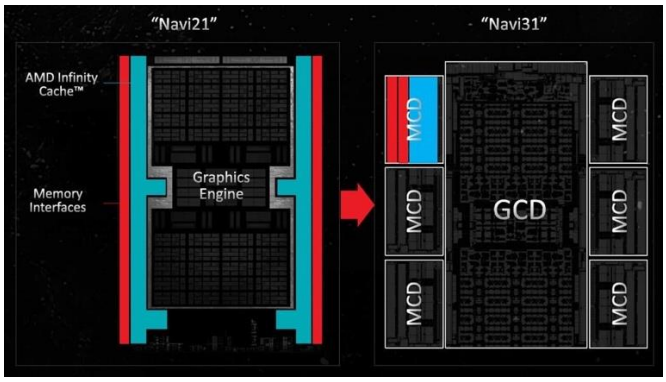


Рисунок 12.33 – Чипленая компановка архитектуры Navi31

Переход к чиплетной компоновке в первую очередь позволил упростить производство GPU, разделив основные компоненты на логические составляющие. Как показано выше, из основного чипа, отделили все, что связанно с внешней памятью – Infinity Cache и контроллеры памяти, назвали такой компонент MCD (Memory Control Die). Остальные компоненты, присущие предыдущим поколениям RDNA, остались почти что неизменными и преобразовались в GCD (Graphics Control Die). Такое решение позволило

комбинировать готовые элементы, собирая их в необходимом количестве и под нужные задачи.

Первым на рынке появились старшие модели 7000-й линейки – RX 7900XT и RX 7900XTX, имевшие в своей основе чип. Navi 31 (рисунок 12.34). Данный чип получил один производительный GCD и 6 MCD. При этом GCD перешел на еще меньший техпроцесс в 5 нм, а блоки MCD остались на 6 нм техпроцессе.

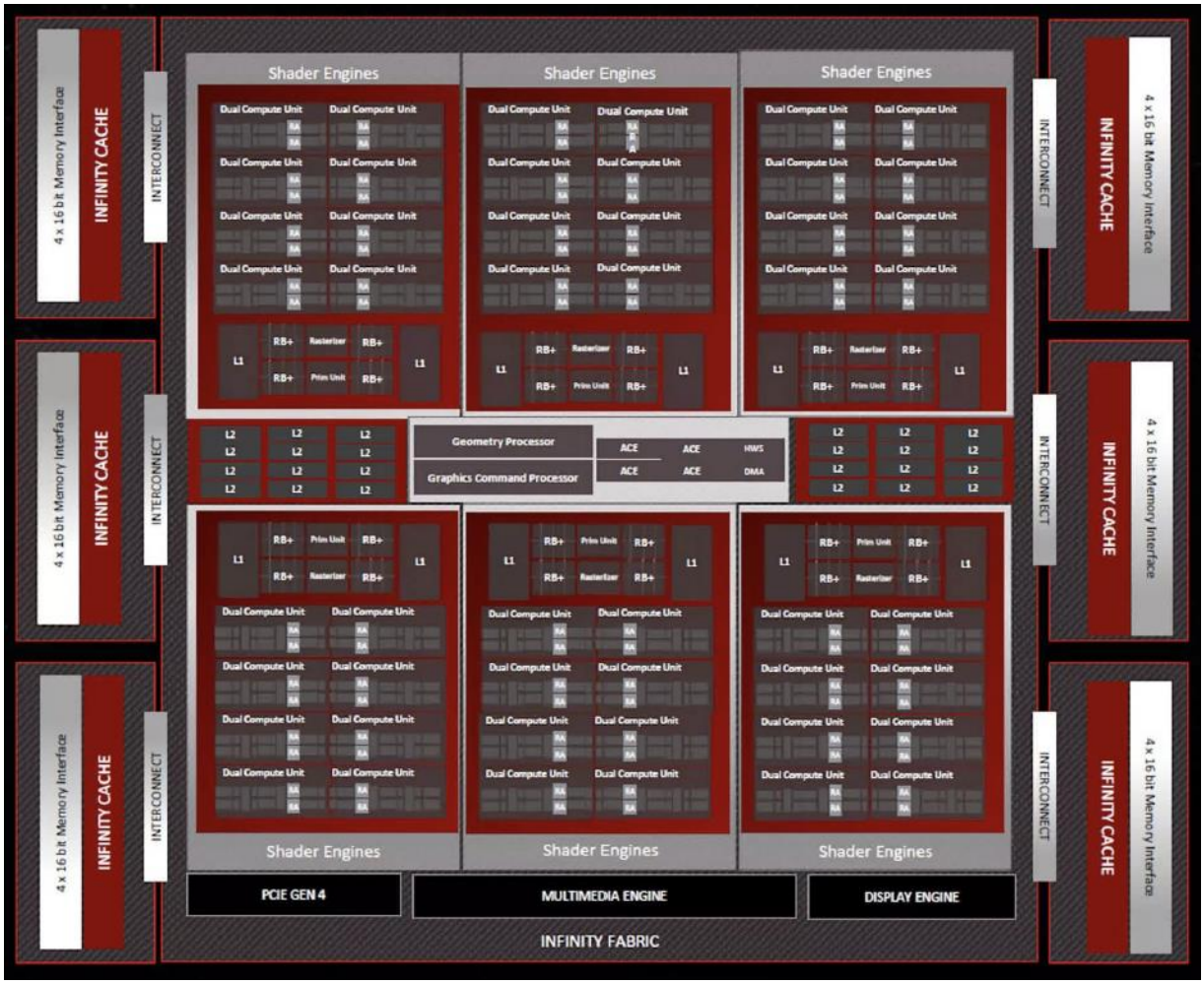


Рисунок 12.34 – Устройсто процессора на примере RX7900XT

Из важных изменений в GCD добавилось еще 2 шейдерных блока. Количество CU в одном блоке уменьшилось и стало равно 16. Каждый CU получил свой собственный блок асинхронных вычислений, улучшенную версию блоков трассировки лучей. Хоть и принцип их обработки остался прежним, сами CU теперь включают в себя 128 блоков вычислений с плавающей запятой и 64 блока целочисленных вычислений, тогда как во всех прошлых поколениях их было поровну – 64. Полная конфигурация процессора составила 96 CU, 192 ROP и 384 TMU, вся кэш память стала больше, L0 – 64

КБ, L1 – 256 КБ, L2 – 6144 КБ, а L3 (он же Infinity Cache) стал 96 КБ на один MCD, но взамен этого удвоил частоту. Значительно ускорилась и подсистема памяти: 384-битная шина вкупе с GDDR6 позволила достигнуть пропускной способности в 960 ГБ/с. По чистой производительности (с отключенной трассировкой лучей) новые флагманы расположись на одном уровне с RTX 4080. Традиционно, средние модели 7000-й линейки (RX7700 и RX7700 XT) получили упрощенные GCD, с меньшим числом CU и меньшим числом MCD, вследствие чего и с меньшими объемами памяти. В младших моделях (RX7500, RX7600 и RX7600 XT), вовсе отказались от чиплетной компоновки, выпустив «улучшенную» версию процессора Navi 23.

Пиком развития, на данный момент, является RDNA 4.0, представленная совсем недавно в 2025 году. Пока что в линейке только одна видеокарта RX9070 и её XT версия, и, скорее всего, это промежуточная модель линейки. GPU в этих моделях носит название Navi 48, и имеет ряд существенных изменений (рисунок 12.35).

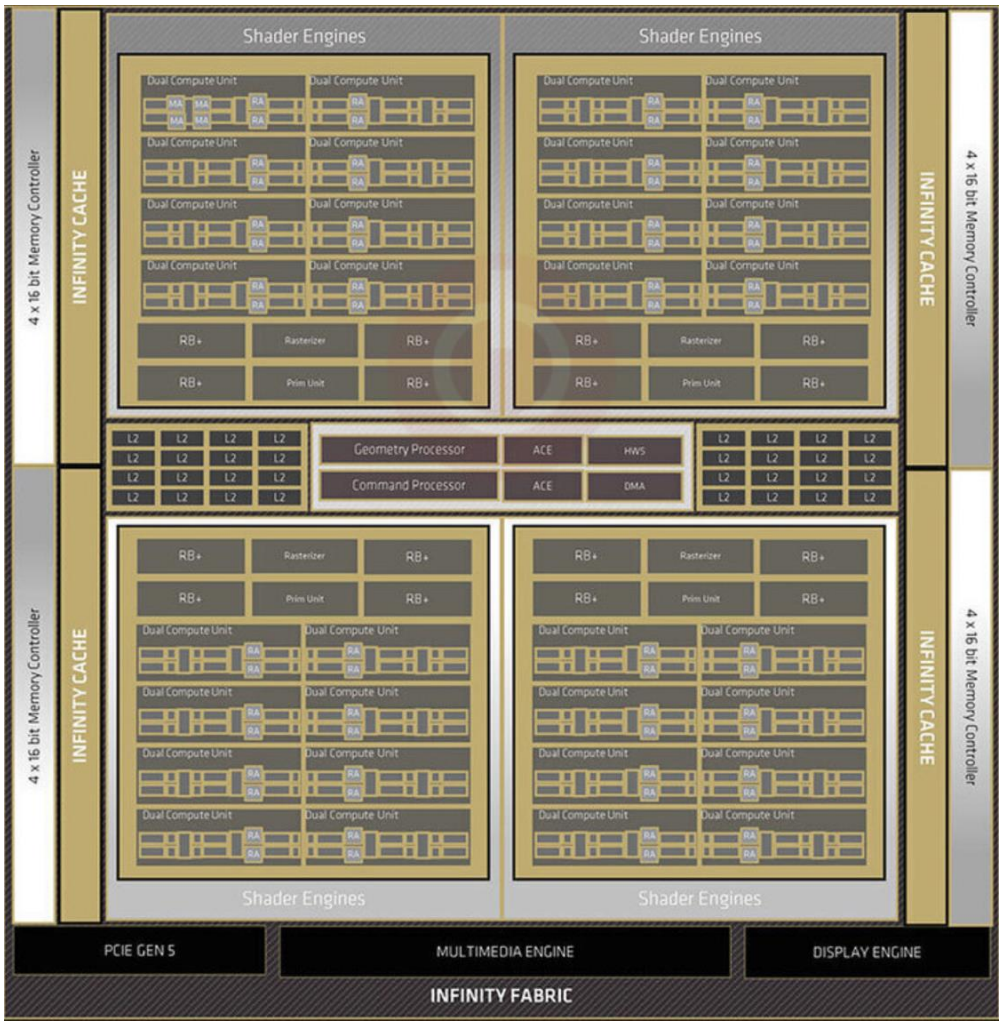


Рисунок 12.35 – Устройсто процессора на примере RX9070

Общая архитектура, как всегда, осталась прежней, но вернулась к монолитному исполнению. CU вновь претерпели изменения и переименовались в CE (Compute Engine), произошел отказ от одного уровня кэш памяти, а конкретно от L1 (по сути, он остался в составе CE, раньше это был L0), добавилась возможность компрессии данных в памяти, поддержка машинного обучения для обработки изображения, поддержка PCIe 5.0, но самым главным изменением стало добавление отдельных тензорных ядер. Конкретно Navi 48 имеет 4 шейдерных блока, 64 CU, 128 ROP, 256 TMU и 128 тензорных ядер. В чистой производительности новинка от AMD расположилась возле RTX 5070 Ti, и лишь чуть-чуть превосходит прошлого лидера линейки 7900 XT, что и является доказательством того, что ответа на старшие модели линейки пока что нет.

Intel. Последним игроком на рынке графических процессоров и видеокарт является Intel, который представил свою новую линейку Arc на архитектуре Xe-HPG в 2022 году. Несмотря на то, что основное направление деятельности данной компании – центральные процессоры, опыт в создании видеокарт у компании из лагеря «синих» был, еще в далеком 1998 году, однако тогда популярности и признания они не получили. Дальнейшая работа с графическими вычислениями тоже велась, правда, на полях интегрированной графики. Компания начала движение к возвращению на рынок дискретных графических решений ещё в 2019 году, когда анонсировала создание принципиально новой архитектуры Xe.

Первое воплощение этой технологии, в упрощённом варианте Xe-LP, появилось в начале 2021 года – оно стало частью графического ядра процессоров Core 11 поколения. Что же касается полноценной версии Xe-HPG, ориентированной на более производительные системы, то её релиз состоялся годом позже, с выходом первой линейки дискретных видеокарт Arc A, известных под кодовым именем Alchemist. В основу легли Xe-core – некоторый аналог CU от «красных» и SM от «зеленых». В их состав входит 16 элементов, соединенных общей шиной и планировщиком, состоящих из XVE (Xe Vector Engine) и XMX (Xe Matrix eXtentions), которые вкупе с регистровым файлом и кэшем L1 образуют вычислительную единицу графического процессора (рисунок 12.36).



Рисунок 12.36 – Устройсто мультипроцессора фирмы Intel

Внутри XMX (блока матричных вычислений) находятся по 8 блоков FP и INT. Все виды расчетов – плавающие, целочисленные и матричные, могут производиться параллельно, не мешая друг другу.

Более крупным объединением является модуль Render Slice (аналог уже Shader Engine или GPC), содержащий 4 Xe-ядра, четыре блока управления потоками, 4 блока трассировки лучей (RTU), блок работы с геометрией, 16 ROP и 32 TMU, образуя схему на рисунке 12.37.



Рисунок 12.37 – Устройсто кластера фирмы Intel

В итоге, в этой архитектуре, было разработано два чипа: ACM-G10 и ACM-G11, оба производились по 7нм техпроцессу. Первый (старший) получил 8 блоков Render Slice и 256 битную шину памяти, второй получил только 2 блока и 96 битную шину. Итоговый вид чипа G10 изображен на рис. 12.38.



Рисунок 12.38 – Устройсто процессора на примере A770

Применение свое они нашли в A-серии первого поколения видеокарт (A300, A500 и A700). По производительности, флагман серии A770 не сильно ушел от GTX 1080 Ti, RX6600, уступая по производительности на 5–10% RX7600 и RTX4060, в графических вычислениях.

Спустя два года, в 2024, на рынок выходит вторая серия видеокарт «синих», на улучшенной архитектуре Xe-HPG2. Общий принцип остался прежним, однако XVE и XMX были переработаны. Вместо попарного объединения XVE движков, Intel разработала один, общий, но с удвоенным количеством функциональных блоков (рисунок 12.39). XMX также получил удвоение своих функциональных блоков: по 16 блоков FP/INT.

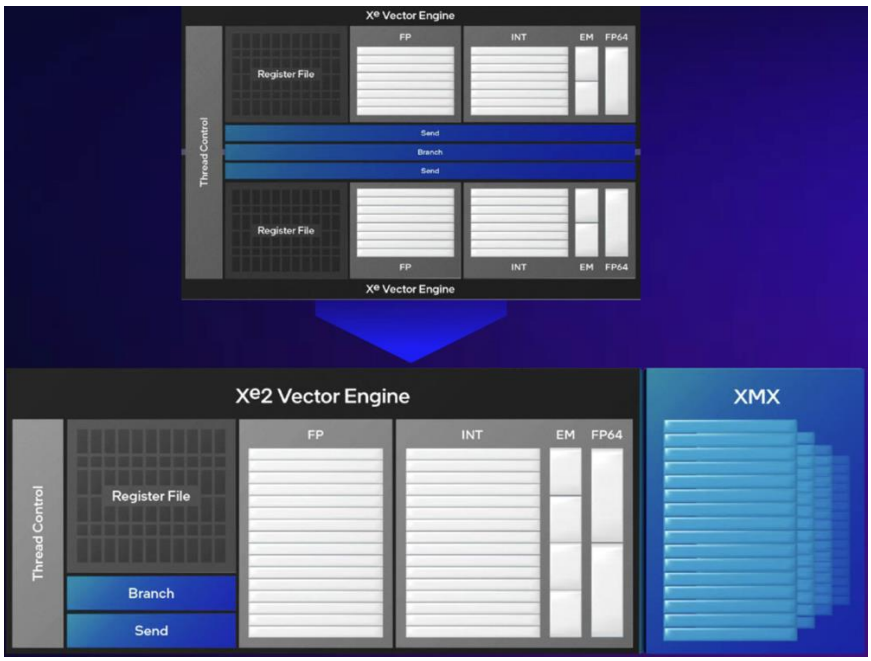


Рисунок 12.39 – Изменения в болка XMX и XVE

Итого, новое Xe2-core ядро, получило уменьшение общего XVE и XMX (с 16 до 8) и увеличение объема L1 кэш (до 256 КБ), строение такого ядра изображено на рисунке 12.40. Внутри блоков Render Slice, особых изменений не произошло, он содержал в себе все тот же набор компонентов.

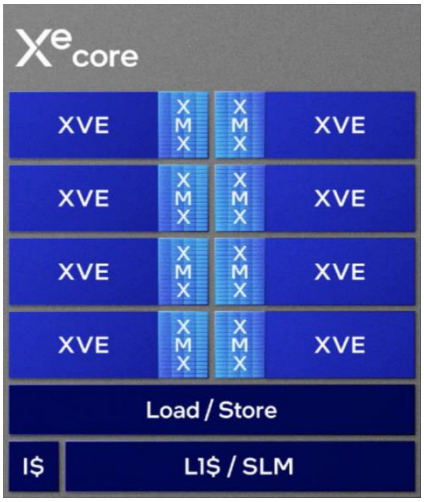


Рисунок 12.40 – Устройсто мультипроцессора 2-ого поколения фирмы Intel

Итоговый чип BGM-G21 (рисунок 12.41), лег в основу новой линейки видеокарт Intel, и имел 5 блоков Render Slice (вместо 8 у предыдущей модели), 20 новых Xe-core, 160 тензорных ядер, 20 RT-ядер, 160 TMU и 80 ROP. L2 кэш вырос до 18 МБ.



Рисунок 12.41 – Устройсто процессора на примере B580

Видеокарт на его основе только 2 B570 и B580. Вторая, являясь флагманом линейки, не сильно выигрывает по графической производительности у RX7600 и, местами, проигрывает RTX 3060 Ti, в профессиональных задачах она также не выделяется.

Пока что, компания только вышла на рынок и не пытается откусить кусок пирога у закоренелых гигантов. Вышедшие на рынок решения – нельзя назвать плохими, однако они не лишены недостатков, среди которых процессорозависимость и отсутствие подходящей оптимизации у большинства приложений. Все ПО, оптимизируется под новые стандарты и гигантов индустрии, а у Intel, как у нового игрока на сцене, эти механизмы еще не до конца отлажены, и, в ряде приложений, видеокарты могут не раскрывать своего полного потенциала. Говоря о неграфических вычислениях, то с уверенность можно сказать, что для конкуренции на этом направлении, в процессорах этой компании пока что маловато ядер и, как следствие, производительности.

Разобравшись в архитектурных подходах трех основных компаний и их отличиях, можно ли однозначно сказать какой из них лучше? С одной стороны, ответ очевиден, NVIDIA является безусловным лидером в мире как графических вычислений и трассировки лучей, так и машинного обучения и неграфических вычислений, что активно используется в различных профессиональных задачах. Среди недостатков, графических процессоров

этой компании, можно выделить разве что высокую конечную стоимость и колоссальное энергопотребление.

AMD в 2019 году отказалась от конкуренции в сфере неграфических вычислений и сориентировала свое внимание на производительности в графических вычислениях и видеоиграх, где им удалось достичь неплохих результатов. Например, текущий флагман «красных», 7900 XTX составляет неплохую конкуренцию флагманам «зеленых», не сильно уступая по чистой производительности в видеоиграх, уступая RTX 4090 на 15–20% в зависимости от конкретного проекта. Архитектурный подход AMD, позволяет делать видеокарты со значительно меньшим энергопотреблением, а следовательно, меньшим нагревом, в том числе и из-за того, что не применяется более «горячая» память типа GDDR6X или GDDR7, главным же преимуществом компании AMD можно назвать низкую рыночную стоимость конечного продукта, что позволяет получать схожую производительность, но с меньшими затратами.

Intel – пока что не раскрыла своего потенциала, но определенно зарекомендовала себя. Для окончательных выводов стоит подождать пока компания представит более производительные чипы, для конкуренции с текущими топами или предтопами других компаний. Сейчас же, это неплохие и достаточно бюджетные аналоги младших моделей линеек AMD и NVIDIA, которые особо не выделяются как в графических, так и в неграфических вычислениях.

Общий же посыл современных архитектур процессоров схож и, как видно из блок-схем процессоров, не сильно отличается как в рамках одной компании, так и между ними. Intel, если посмотреть, и вовсе взяла за основу устоявшуюся архитектуру NVIDIA, немного изменив ее под свои нужды. Обусловлено это все тем фактом, что алгоритмические основы обработки изображений, заложенные еще очень давно, особо не меняются, однако растет количество обрабатываемых операций и сложность итогового изображения, что естественным образом повышает нагрузку. Аппаратная производительность современных GPU, равно как и CPU упирается в их типоразмер, где дальнейшее его уменьшение дается все труднее, и вызывает множество проблем, с которыми нужно как-то бороться. Среди этих проблем наиболее остро стоит нагрев. Постоянный рост производительности требует увеличения площади кристалла. Например, площадь поверхности современного GPU (Die size) достигает 700 мм^2, в то время как у CPU только 250 мм^2. При таких размерах крайне сильно вырастает TDP, при размере в 700

мм², он составляет около 575 Вт, в то время как у CPU при его размерах максимальный TDP около 250 – 230 Вт. Технологии искусственного интеллекта, активно внедряемые в программной стороне вопроса, добавляющие генерацию кадров или умный апскейлинг для графических вычислений, позволяют немного сгладить и улучшить производительность в больших разрешениях (2К или 4К), однако до совершенства им еще далеко.

Исходя из всего вышеописанного этого нельзя однозначно сказать какой из подходов лучше или хуже, они просто разные, и преследуют разные цели, от которых и зависит выбор той или иной архитектуры. Где-то более предпочтительными будут AMD ввиду экономичности, как в плане ресурсов, так и в плане затрат; где-то NVIDIA – дорогие, но флагманские решения в неграфических вычислениях и трассировке лучей; Intel же, простые и эффективные решения для офисных и не затратных графических задач.

12.4. Несколько слов о блокчейне и майнинге

Концепция криптовалюты впервые была описана в статье некого Сатоши Накамото, вышедшей еще 2008 г. под названием "Bitcoin: пиринговая электронная денежная система" ("Bitcoin: A Peer-to-Peer Electronic Cash System"). В статье излагались математическая и криптографическая основы системы для выполнения децентрализованных финансовых транзакций. Централизованная финансовая система, как известно, зависит от уполномоченных организаций, таких как правительства и банки, которые осуществляют мониторинг и контроль деятельности системы.

Основной концепцией криптовалюты является полное отсутствие централизованного регулятора и опора на равноправные сетевые узлы, конкурентное взаимодействие которых организовано специальным образом, чтобы поддерживать стабильную работу всей системы. Система открыта и любой желающий может присоединиться к сети в качестве равноправного участника. Информация системы хранится в блокчейне – распределенном реестре, содержащем криптографически защищенные записи обо всех транзакциях (операциях в сети от имени ее участников), которые проводились с криптовалютой с момента появления системы. Как следует из названия, блокчейн (blockchain) – это цепочка блоков, каждый блок которой, содержит криптографически защищенные описания множества транзакций между пользователями сети. Каждый равноправный узел сети может в любое время запросить и получить полную копию блокчейна. В ходе подключения к сети нового участника система должна загрузить и проверить все транзакции

блокчейна, начиная с самой первой, и до конца блокчейна, это обязательный шаг. Начиная с первого блока, размещенного в начале цепочки при ее создании, каждый последующий блок содержит ссылку на блок, непосредственно предшествующий ему в цепочке. Криптографические методы используются для того, чтобы сохранить неизменными все блоки и транзакции внутри них, а также исключить возможность подделки информации и ссылок между блоками. На рисунке 12.42, ниже, показано упрощенное представление конца цепочки блокчейна.



Рисунок 12.42 – Цепочка блокчейна

Каждая транзакция влечет за собой небольшую (но необязательную) комиссию, выплачиваемую первому майнеру, который успешно выполнит работу, необходимую для добавления набора транзакций. Например, в наиболее популярной криптовалюте Bitcoin применяется алгоритм хеширования SHA-256, который работает с блоками данных, длина которых кратна 512 битам. Результатом вычисления SHA-256 является 256-битное хеш-значение, которое чаще всего представляется в виде 64 шестнадцатеричных символов. Это крайне надежный алгоритм шифрования данных, который применяется не только для получения криптовалюты, но и хранения паролей, цифровых подписей и пр. Основные этапы работы этого алгоритма можно описать так. Выбирается исходный блок данных, который может иметь любую длину от 1 до $(2^{64}–1)$ бит, также называемый сообщением. Дополняется до длины, кратной 512 битам по следующему алгоритму:

1. Добавить 1 бит к сообщению.

2. Добавить к сообщению минимальное количество нулевых битов таким образом, чтобы длина сообщения в битах была на 64 меньше, чем значение, кратное 512.

3. Добавить к сообщению 64-битное целое число без знака, содержащее длину исходного сообщения в битах.

SHA-256 применяет простые логические и математические операции: И, ИЛИ, исключающее ИЛИ, НЕ, сложение целых чисел, сдвиг вправо, вращение вправо и объединение битов, как правило такие операции выполняются над 32 битными числами. Смешивание операций в более сложные, с добавлением предопределенных констант для кодирования сообщения, позволяет получить случайный вид итогового хэш-значения. Алгоритм SHA-256 обрабатывает каждую 512-битную секцию входных данных последовательно. После дополнения сообщения 512 бит каждой секции разделяются на 64 слова по 32 бита в каждом. Обработка каждой секции, по сути, представляет собой процедуру кодирования, которая многократно смешивает биты в словах посредством серии логических операций. Алгоритм выполняет цикл из 64 проходов, содержащий серию интенсивных логических и математических операций над словами данных в каждой секции.

Сетевые узлы, которые выполняют эти вычисления, называют системами майнинга, а людей, которые ими владеют – майнерами. Добровольное включение комиссии в каждую транзакцию повышает вероятность того, что майнеры включат транзакцию в следующий блок, тем самым ускорив ее прохождение. Добавление блока в блокчейн – это задача, намеренно требующая больших вычислительных затрат. Операция хеширования, выполняемая при майнинге, заключается в подборе хэш-значения путем перебора большого количества различных значений.

Для хранения своих средств пользователи используют специальные приложения, именуемые цифровыми кошельками. Цифровой кошелек отслеживает баланс криптовалюты своего владельца и помогает организовывать переводы другим пользователям в сети. В кошельке хранится секретный ключ, используемый владельцем для доступа к принадлежащим ему средствам. Когда пользователь инициирует транзакцию для перевода некоторого количества валюты другому пользователю или получения перевода от него, в блокчейн добавляется соответствующая запись об этом, которая помимо одобрения отправителя транзакции должна быть проверена остальными участниками сети.

Сам блокчейн является общедоступным, но содержащаяся в нем информация не идентифицирует ни отправителей, ни получателей, каким-либо иным способом, кроме как номером цифрового ключа, связанного с цифровым кошельком. Этот номер никак не связан с личностью пользователя, и при желании может быть изменен, для сохранения анонимности.

Для того чтобы конвертировать некоторое количество криптовалюты в более традиционную валюту, например доллары США, или обратно, существуют финансовые учреждения, уполномоченные на работу в той или иной валюте, они выполняют принимают на свой счет транзакцию за определенную стоимость в реальной валюте. В качестве альтернативы пользователь может перевести некоторое количество валюты другому пользователю в обмен на взаимно согласованное количество наличных денег.

Так как в основе майнинга, например биткоина, лежит выполнение огромного множества вычислений по алгоритму SHA-256, то для увеличения эффективности и скорости обработки всего процесса подбора значения, возникает необходимость в больших вычислительных мощностях. С одной стороны, для этого подходит любой CPU или микропроцессор, способные обрабатывать большое количество операций в секунду. Но даже самые топовые процессоры на сегодняшний день не могут выдавать хорошие результаты, находясь на уровне 1.5 MH/s. Данная характеристика MegaHash/second – мегахеш в секунду, отражает количество рассчитываемых хешей в секунду, и является показателем эффективности устройств для майнинга. Показатель в полтора мегахеша в секунду, по современным меркам является крайне скудным, по сравнению с GPU или специализированных плат. GPU – другое дело, как известно, они имеют в своем распоряжении огромное количество вычислительных ядер и колоссальную многопоточность, например RTX 4090 выдает около 120 MH/s. Именно по этой причине во время «бума» майнинга, биткоина в 2021 году, активно использовались именно видеокарты, которые на то время имели около 40 – 50 MH/s. Однако лидером в майнинге данной криптовалюты, в настоящее время, является майнинг на ASIC (Application-Specific Integrated Circuit) – Специализированных интегральных схемах, например, Antminer S19 Pro вышедший в релиз в 2020 году (или его более новая версия), которая способна выполнять 110 TH/s, что в 1000 раз больше, чем у RTX 4090. Конечно, одними биткоинами криптовалюты не ограничиваются и, после пика биткоинов, появилось множество других криптовалют, которые работают по своим алгоритмам и которые могут требовать отличные вычислительные ресурсы для своего

выполнения. Технологии же блокчейна, давно уже вышли за пределы криптовалют и сейчас используются в различных сферах, в основном связанных с безопасностью или финансами.

12.5. Заключение

В результате изучения данной темы было изучено из чего состоит современная видеокарта, как развивались графические процессоры и какими характеристиками они обладают. Были отдельно рассмотрены основные различия в архитектурных подходах разных передовых компаний, рассмотрены их сильные и слабые стороны. Отдельно была упомянута развивающаяся технология RTX, и ее аппаратная реализация. В качестве дополнения были рассмотрены базовые принципы блокчейна, а также причины использования графических накопителей для его организации.

12.6. Варианты заданий

Работа с современным графическим процессором представляет собой комплексную инженерную задачу. В реальных прикладных системах разработчику необходимо учитывать выбор графического API, выделение и копирование памяти, подготовку буферов и текстур, синхронизацию между CPU и GPU, компиляцию шейдеров, размеры рабочих групп, ограничения драйверов и различия аппаратных платформ. Такой набор факторов быстро превращает знакомство с графическими вычислениями в борьбу с инфраструктурой. По этой причине в учебных целях целесообразно использовать контролируемую модель, в которой основные идеи GPU-вычислений сохраняются, а второстепенные детали сведены к минимуму.

Именно такую роль выполняет разработанный тестовый стенд на основе WebGPU. Он не отменяет реальную архитектуру работы видеокарты, а показывает её в упрощённом и наглядном виде: студенты по-прежнему формируют алгоритм как независимую обработку множества пикселей. Далее предлагается сравнить последовательное исполнение на CPU и массово-параллельное исполнение на GPU, чтобы оценить влияние разрешения изображения на время расчёта и увидеть расхождения результатов, возникающие из-за вычислений с плавающей точкой.

Для работы с тестовым стендом потребуется язык WGSL (WebGPU Shading Language) – это язык шейдеров, используемый API WebGPU для описания программ, выполняемых на графическом процессоре. Исторически WGSL появился в рамках стандартизации WebGPU как специализированный

язык, предназначенный именно для безопасной и предсказуемой работы в веб-среде. В отличие от более низкоуровневых нативных решений, он изначально ориентирован на проверяемость кода, строгую типизацию и единообразное поведение в браузере. Благодаря этому WGSL хорошо подходит для учебной практики: студент видит структуру вычислительного ядра, но не тратит основное время на платформенные различия между Vulkan, Metal или Direct3D.

С практической точки зрения WGSL применяется в двух основных классах задач. Во-первых, это графический вывод, когда шейдеры участвуют в построении изображения на экране. Во-вторых, это вычислительные задачи, где GPU используется как массово-параллельный ускоритель для обработки данных, изображений, шума, фракталов, фильтров и элементов научной визуализации. Для данной практической работы наиболее важен именно второй режим, поскольку он позволяет изучать работу видеокарты

Разработанный стенд представляет собой web-приложение, в котором реализованы два независимых канала вычислений (рис. 12.42).

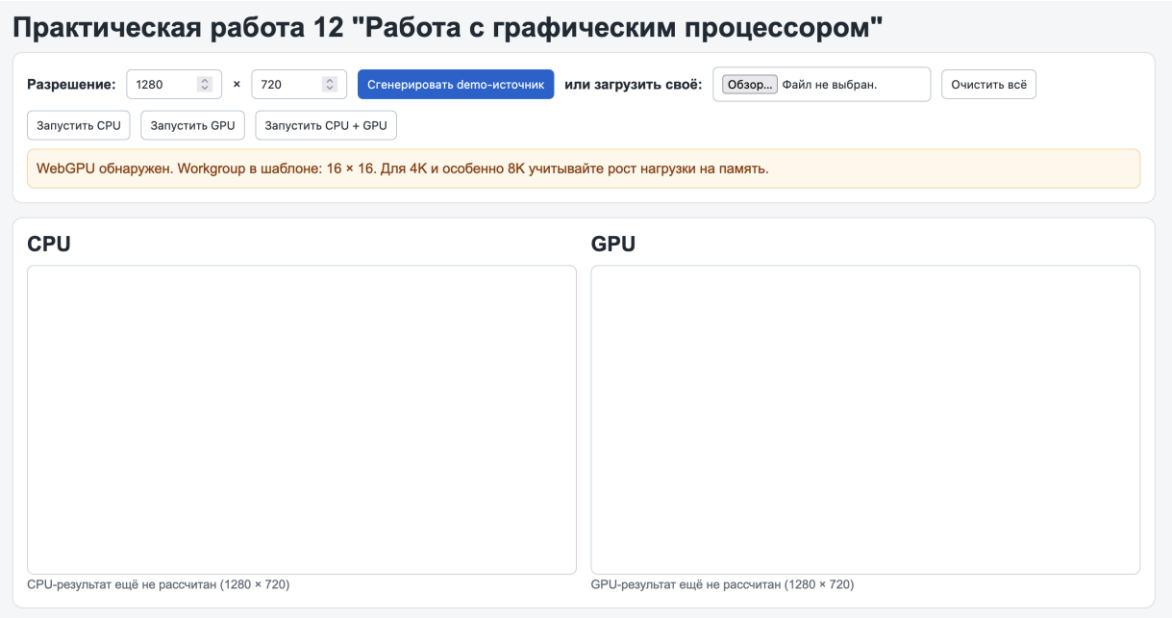


Рисунок 12.43 – Интерфейс web-приложения

Первый канал (слева) выполняет обработку или генерацию изображения на CPU с использованием языка JavaScript. Второй канал выполняет ту же задачу, но уже на GPU с использованием WebGPU и языка WGSL. На странице предусмотрены поля для задания разрешения, кнопка генерации демонстрационного источника, возможность загрузки собственного изображения, отдельные редакторы кода CPU и GPU, области

предварительного просмотра результатов и блок измерений. В интерфейсе отображаются время выполнения на CPU и GPU, коэффициент ускорения, а также метрики Mean diff и Max diff, показывающие различие между двумя результатами.

В ходе работы предлагается, используя указанные языки, сгенерировать изображение по заданным праметрам. Для каждого варианта имеется своя математическая модель, с описанием, представленным ниже. Для всех процедурных режимов удобно использовать одинаковую систему координат. Пусть x и y – координаты текущего пикселя, а W и H – ширина и высота изображения. Тогда нормированные координаты и координаты в диапазоне от −1 до 1 вычисляются по формуле (12.1).

$$u = x / (W - 1), \quad v = y / (H - 1), \quad c_x = 2u - 1, \quad c_y = 2v - 1. \qquad (12.1)$$

Первым вариантом является множество Мандельброта. Для него берётся комплексное число $c = c_x + i\, c_y$, начальное значение $z_0 = 0$, после чего выполняется итерационный процесс (12.2).

$$z_{n+1} = z_n^2 + c. \qquad (12.2)$$

Если в ходе итераций выполняется условие $|z_n|^2 > 4$, точка считается ушедшей из области устойчивости, а число итераций используется для выбора цвета.

Для множества Жюлиа схема почти та же, однако начальное значение берётся по координатам пикселя, а комплексная константа k задаётся заранее, как в формуле (12.3).

$$z_0 = c_x + i\, c_y, \quad z_{n+1} = z_n^2 + k. \qquad (12.3)$$

Фрактал Burning Ship строится модификацией предыдущей формулы. Перед возведением в квадрат используются модули действительной и мнимой частей: $z_{n+1} = (|Re\, z_n| + i|Im\, z_n|)^2 + c$.

Плазма и волновая интерференция удобны для изучения периодических функций. Интенсивность пикселя можно задать суммой синусоид по двум осям и радиальной составляющей (12.4).

$$I(u,v) = 0.5 + 0.25 \sin(2\pi f_x u + \varphi) + 0.25 \cos(2\pi f_y v - \varphi) + 0.25 \sin(2\pi f_r r),$$
$$\text{где } r = \sqrt{(c_x^2 + c_y^2)}. \qquad (12.4)$$

Шум value noise строится на решётке псевдослучайных значений с последующей интерполяцией между узлами. На практике студенту достаточно реализовать случайное значение в целочисленных узлах решётки и линейно либо сглаженно интерполировать его внутри клетки. Более сложный режим fBM clouds формируется как сумма нескольких октав шума $F(u,v) = \Sigma\, a^o\, n(2^o u,\, 2^o v), \quad o = 0..m-1,$ где n(u,v) – базовая шумовая функция, a – коэффициент уменьшения амплитуды. Радиальные кольца с шумовой деформацией определяются расстоянием до центра и добавочным шумовым смещением (12.5)

$$I(u,v) = 0.5 + 0.5\, \sin(2\pi f_r r + \alpha \cdot n(u,v) + \varphi). \qquad (12.5)$$

Шахматная сетка является самым простым дискретным генератором. Цвет клетки задаётся по чётности суммы индексов (12.6).

$$tile = (\lfloor u \cdot N_x \rfloor + \lfloor v \cdot N_y \rfloor) \bmod 2, \qquad (12.6)$$

а линии сетки получаются сравнением локальных координат внутри клетки с заданной толщиной. Для обработки загруженного изображения студент может использовать формулы яркости и усреднения соседних пикселей. В режиме grayscale яркость берётся как взвешенная сумма каналов $L = 0.299R + 0.587G + 0.114B,$ а для blur можно применять среднее по квадратному окну радиуса r (12.7):

$$C(x,y) = 1/(2r+1)^2 \cdot \Sigma\,\Sigma\, C(x+i, y+j), \quad i,j \in [-r, r]. \qquad (12.7)$$

После написания код под каждую конкретную функцию и запуска программы, можно увидеть следующий результат (рис. 12.44).

WGSL в сочетании с WebGPU даёт достаточно строгую и в то же время доступную модель вычислительного ядра, а сопоставление CPU- и GPU-версий одной и той же формулы наглядно показывает, почему массово-параллельные устройства эффективны в задачах пиксельного уровня. Набор вариантов на основе фракталов, шума, периодических функций и простых фильтров обеспечивает как базовый, так и расширенный уровень исследования.

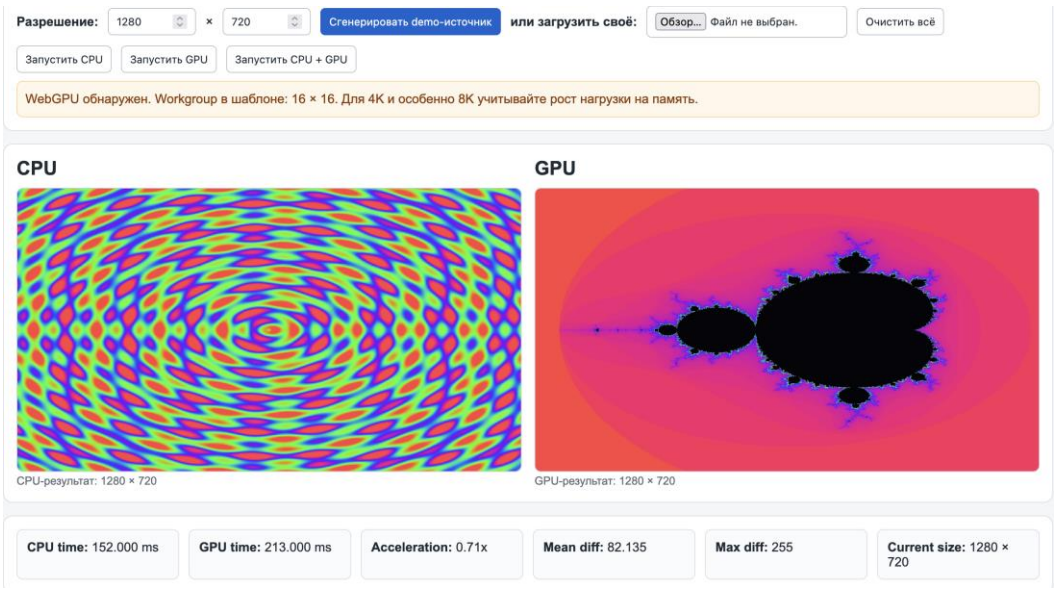


Рисунок 12.43 – Петля Мандельброта и плазма

Таблица 12.3 – Варианты заданий

			zoom=1.60; offsetX=-0.55; offsetY=0.00; maxIter=180; colorPower=0.65
			zoom=1.60; offsetX=-0.55; offsetY=0.00; maxIter=200; trapScale=35
			zoom=0.90; offsetX=-0.75; offsetY=0.10; maxIter=260; colorPower=0.55
12	2560×1440	fBM clouds	scale=7.0; density=0.48; gain=1.45; offset=0.8
13	1280×720	Plasma / wave interference	

Продолжение табл. 12.3

			zoom=1.80; offsetX=-0.62; offsetY=0.02; maxIter=240; trapScale=50

			zoom=1.20; offsetX=-0.50; offsetY=0.00; maxIter=220; colorPower=0.70
			zoom=1.35; offsetX=-0.74; offsetY=0.08; maxIter=180; trapScale=28
			zoom=2.40; offsetX=-0.10; offsetY=0.65; maxIter=320; colorPower=0.50

Для организации собственного стенда потребуется браузер, и язык python. В учебной работе используется максимально простой проект, состоящий из одного HTML-файла с интерфейсом и логикой (Листинг 12.1), а также вспомогательного Python-скрипта для локального запуска (Листинг 12.2). В комплекте проекта находятся index.html, в котором размещена вся лабораторная работа, start_server.py для старта локального HTTP-сервера,

Листинг 12.1 – код файла index.html

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>WebGPU Lab: Procedural GPU/CPU Sandbox</title>
  <style>
    :root {
      --bg: #f4f6f8;
```

```
    --panel: #ffffff;
    --border: #d9dee5;
    --text: #1f2933;
    --subtle: #5c6773;
    --accent: #0b63ce;
    --warn-bg: #fff7eb;
    --warn-border: #ffd29b;
    --log-bg: #11161d;
    --log-text: #d9f2ff;
  }
  * { box-sizing: border-box; }
  body {
    margin: 20px;
    font-family: Arial, sans-serif;
    background: var(--bg);
    color: var(--text);
  }
  h1, h2, h3 { margin: 0 0 10px 0; }
  .panel {
    background: var(--panel);
    border: 1px solid var(--border);
    border-radius: 12px;
    padding: 16px;
    margin-bottom: 16px;
  }
  .row {
    display: flex;
    flex-wrap: wrap;
    gap: 12px;
    align-items: center;
```

Продолжение листинга 12.1

```
    margin-bottom: 10px;
  }
  .row:last-child { margin-bottom: 0; }
  label { font-weight: 600; }
  select, button, input[type="file"], input[type="number"] {
    padding: 8px 10px;
    border: 1px solid #b8c2cc;
    border-radius: 6px;
    background: white;
    color: var(--text);
  }
  .size-input {
    width: 110px;
  }
```

```
button { cursor: pointer; }
button:hover { background: #eef3f7; }
.btn-primary {
  background: var(--accent);
  color: white;
  border-color: var(--accent);
}
.btn-primary:hover { background: #0954ab; }
.small {
  color: var(--subtle);
  font-size: 14px;
  line-height: 1.55;
}
.warning {
  color: #8a3b00;
  background: var(--warn-bg);
  border: 1px solid var(--warn-border);
  padding: 10px;
  border-radius: 8px;
  line-height: 1.45;
}
.grid-2 {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 16px;
}
.preview-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(320px, 1fr));
  gap: 16px;
}
.metrics {
```

Продолжение листинга 12.1

```
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(170px, 1fr));
  gap: 12px;
}
.metric {
  background: #fafbfd;
  border: 1px solid var(--border);
  border-radius: 8px;
  padding: 12px;
}
.editor {
  width: 100%;
  min-height: 340px;
  resize: vertical;
```

```
    border: 1px solid #b8c2cc;
    border-radius: 8px;
    padding: 12px;
    font-family: Consolas, Menlo, monospace;
    font-size: 13px;
    line-height: 1.5;
    background: #f9fbfd;
    color: #1b2733;
    tab-size: 2;
  }
  .preview-canvas {
    width: 100%;
    height: auto;
    aspect-ratio: 16 / 9;
    border: 1px solid #c9d2dc;
    background: white;
    border-radius: 8px;
    display: block;
  }
  .log {
    white-space: pre-wrap;
    background: var(--log-bg);
    color: var(--log-text);
    padding: 12px;
    border-radius: 8px;
    min-height: 150px;
    font-family: Consolas, monospace;
    font-size: 13px;
    line-height: 1.5;
  }
  .hint-box {
    background: #f8fbff;
```

Продолжение листинга 12.1

```
    border: 1px solid #dbe8f6;
    border-radius: 8px;
    padding: 12px;
    line-height: 1.5;
    margin-bottom: 10px;
  }
  details.hint-box summary {
    cursor: pointer;
    font-weight: 600;
  }
  details.hint-box[open] summary {
    margin-bottom: 8px;
  }
  code.inline {
```

```
    background: #eef3f7;
    padding: 2px 6px;
    border-radius: 5px;
    font-family: Consolas, monospace;
    font-size: 12px;
   }
   @media (max-width: 980px) {
    .grid-2 { grid-template-columns: 1fr; }
   }
  </style>
</head>
<body>
  <h1>Практическая работа 12 "Работа с графическим процессором"</h1>

  <div class="panel">
   <div class="row">
    <label for="resolutionWidth">Разрешение:</label>
    <input id="resolutionWidth" class="size-input" type="number" min="16" max="8192" step="1"
value="1280" />
    <span>×</span>
    <input id="resolutionHeight" class="size-input" type="number" min="16" max="8192" step="1"
value="720" />
    <button id="generateBtn" class="btn-primary">Сгенерировать demo-источник</button>
    <label for="imageInput">или загрузить своё:</label>
    <input id="imageInput" type="file" accept="image/*" />
    <button id="clearBtn">Очистить всё</button>
   </div>

   <div class="row">
    <button id="cpuBtn">Запустить CPU</button>
    <button id="gpuBtn">Запустить GPU</button>
    <button id="bothBtn">Запустить CPU + GPU</button>
```

Продолжение листинга 12.1

```
   </div>
   <div id="supportInfo" class="warning" style="margin-top:10px;"></div>
  </div>

  <div class="panel preview-grid">
   <div>
    <h2>CPU</h2>
    <canvas id="cpuCanvas" class="preview-canvas" width="960" height="540"></canvas>
    <div class="small" id="cpuInfo">CPU-результат ещё не рассчитан</div>
   </div>
   <div>
    <h2>GPU</h2>
    <canvas id="gpuCanvas" class="preview-canvas" width="960" height="540"></canvas>
    <div class="small" id="gpuInfo">GPU-результат ещё не рассчитан</div>
```

```
  </div>
 </div>

 <div class="panel metrics">
  <div class="metric"><strong>CPU time:</strong> <span id="cpuTime">—</span></div>
  <div class="metric"><strong>GPU time:</strong> <span id="gpuTime">—</span></div>
  <div class="metric"><strong>Acceleration:</strong> <span id="speedup">—</span></div>
  <div class="metric"><strong>Mean diff:</strong> <span id="meanDiff">—</span></div>
  <div class="metric"><strong>Max diff:</strong> <span id="maxDiff">—</span></div>
  <div class="metric"><strong>Current size:</strong> <span id="currentSize">—</span></div>
 </div>

 <div class="panel grid-2">
  <div>
   <h2>Код CPU (JavaScript)</h2>
   <details class="hint-box small">
    <summary>Подсказка по API CPU</summary>
    Тело функции должно вернуть <code class="inline">[r, g, b, a]</code> с диапазоном <code class="inline">0..1</code>.<br />
    Доступно: <code class="inline">sample(dx, dy)</code>, <code class="inline">sampleNorm(u, v)</code>, <code class="inline">rgba(...)</code>, <code class="inline">luminance(rgb)</code>, <code class="inline">fract(x)</code>, <code class="inline">mix(a,b,t)</code>, <code class="inline">smoothstep(a,b,x)</code>, <code class="inline">hash2(x,y)</code>, <code class="inline">noise2(x,y)</code>, <code class="inline">fbm(x,y)</code>, <code class="inline">palette(t)</code>, а также <code class="inline">x, y, width, height, uvx, uvy, cx, cy</code>.
   </details>
   <textarea id="cpuCode" class="editor"></textarea>
  </div>
  <div>
   <h2>Код GPU (WGSL)</h2>
   <details class="hint-box small">
```

Продолжение листинга 12.1

```
    <summary>Подсказка по API GPU</summary>
    Тело функции должно вернуть <code class="inline">vec4&amp;lt;f32&amp;gt;</code>.<br />
    Доступно: <code class="inline">sampleAt(ix, iy)</code>, <code class="inline">sampleNorm(uvx, uvy)</code>, <code class="inline">rgba(...)</code>, <code class="inline">luminance(rgb)</code>, <code class="inline">fractf(x)</code>, <code class="inline">mix1(a,b,t)</code>, <code class="inline">smoothstep1(a,b,x)</code>, <code class="inline">hash21(vec2)</code>, <code class="inline">noise2(vec2)</code>, <code class="inline">fbm(vec2)</code>, <code class="inline">palette(t)</code>, а также <code class="inline">x, y, uvx, uvy, cx, cy</code>.
   </details>
   <textarea id="gpuCode" class="editor"></textarea>
  </div>
 </div>
```

```
  <div class="panel">
    <h2>Лог</h2>
    <div id="log" class="log"></div>
  </div>

  <script>
    const cpuCanvas = document.getElementById('cpuCanvas');
    const gpuCanvas = document.getElementById('gpuCanvas');
    const cpuCtx = cpuCanvas.getContext('2d');
    const gpuCtx = gpuCanvas.getContext('2d');

    const imageInput = document.getElementById('imageInput');
    const resolutionWidthEl = document.getElementById('resolutionWidth');
    const resolutionHeightEl = document.getElementById('resolutionHeight');
    const clearBtn = document.getElementById('clearBtn');
    const cpuCodeEl = document.getElementById('cpuCode');
    const gpuCodeEl = document.getElementById('gpuCode');
    const cpuTimeEl = document.getElementById('cpuTime');
    const gpuTimeEl = document.getElementById('gpuTime');
    const speedupEl = document.getElementById('speedup');
    const meanDiffEl = document.getElementById('meanDiff');
    const maxDiffEl = document.getElementById('maxDiff');
    const supportInfo = document.getElementById('supportInfo');
    const logEl = document.getElementById('log');
    const currentSizeEl = document.getElementById('currentSize');
    const cpuInfoEl = document.getElementById('cpuInfo');
    const gpuInfoEl = document.getElementById('gpuInfo');

    let gpuDevice = null;
    let gpuAdapter = null;
    let sourcePixels = null;
    let sourceWidth = 0;
```

Продолжение листинга 12.1

```
    let sourceHeight = 0;
    let lastCpuPixels = null;
    let lastGpuPixels = null;

    const workgroupSizeX = 16;
    const workgroupSizeY = 16;
    const demoCpuCode = `const c = sample(0, 0);\nconst gray = luminance(c);\nreturn rgba(gray, gray, gray, c[3]);`;
    const demoGpuCode = `let c = sampleAt(i32(x), i32(y));\nlet gray = luminance(c.xyz);\nreturn rgba(gray, gray, gray, c.w);`;

    function log(msg) {
      const time = new Date().toLocaleTimeString();
      logEl.textContent += `[${time}] ${msg}\n`;
```

```
  logEl.scrollTop = logEl.scrollHeight;
}

function clamp01(v) {
  if (!Number.isFinite(v)) return 0;
  return Math.max(0, Math.min(1, v));
}

function rgba(r, g, b, a = 1.0) {
  return [clamp01(r), clamp01(g), clamp01(b), clamp01(a)];
}

function luminance(rgb) {
  return 0.299 * rgb[0] + 0.587 * rgb[1] + 0.114 * rgb[2];
}

function fract(x) {
  return x - Math.floor(x);
}

function mix(a, b, t) {
  return a * (1.0 - t) + b * t;
}

function smoothstep(edge0, edge1, x) {
  const denom = Math.max(1e-6, edge1 - edge0);
  const t = clamp01((x - edge0) / denom);
  return t * t * (3.0 - 2.0 * t);
}

function hash2(x, y) {
  return fract(Math.sin(x * 127.1 + y * 311.7) * 43758.5453123);
```

Продолжение листинга 12.1

```
}

function noise2(x, y) {
  const ix = Math.floor(x);
  const iy = Math.floor(y);
  const fx = fract(x);
  const fy = fract(y);
  const ux = fx * fx * (3.0 - 2.0 * fx);
  const uy = fy * fy * (3.0 - 2.0 * fy);
  const a = hash2(ix, iy);
  const b = hash2(ix + 1, iy);
  const c = hash2(ix, iy + 1);
  const d = hash2(ix + 1, iy + 1);
  return mix(mix(a, b, ux), mix(c, d, ux), uy);
```

```
}

function fbm(x, y) {
  let v = 0.0;
  let a = 0.5;
  let px = x;
  let py = y;
  for (let i = 0; i < 5; i++) {
    v += a * noise2(px, py);
    px = px * 2.03 + 17.13;
    py = py * 2.01 + 8.73;
    a *= 0.5;
  }
  return v;
}

function palette(t) {
  return [
    0.5 + 0.5 * Math.cos(6.2831853 * (t + 0.00)),
    0.5 + 0.5 * Math.cos(6.2831853 * (t + 0.33)),
    0.5 + 0.5 * Math.cos(6.2831853 * (t + 0.67)),
  ];
}

function setCurrentSize(width, height) {
  if (!Number.isFinite(width) || !Number.isFinite(height) || width <= 0 || height <= 0) {
    currentSizeEl.textContent = '—';
    return;
  }
  currentSizeEl.textContent = `${width} × ${height}`;
}
```

Продолжение листинга 12.1

```
function getRequestedResolution() {
  const width = Math.floor(Number(resolutionWidthEl.value));
  const height = Math.floor(Number(resolutionHeightEl.value));
  if (!Number.isFinite(width) || !Number.isFinite(height) || width < 16 || height < 16) {
    throw new Error('Разрешение должно быть целыми числами не меньше 16.');
  }
  if (width > 8192 || height > 8192) {
    throw new Error('Разрешение слишком большое. Максимум 8192 × 8192.');
  }
  resolutionWidthEl.value = String(width);
  resolutionHeightEl.value = String(height);
  return { width, height };
}
```

```
function clearMetrics() {
  cpuTimeEl.textContent = '—';
  gpuTimeEl.textContent = '—';
  speedupEl.textContent = '—';
  meanDiffEl.textContent = '—';
  maxDiffEl.textContent = '—';
  delete cpuTimeEl.dataset.value;
  delete gpuTimeEl.dataset.value;
  lastCpuPixels = null;
  lastGpuPixels = null;
}

function updateSpeedup() {
  const cpu = parseFloat(cpuTimeEl.dataset.value || 'NaN');
  const gpu = parseFloat(gpuTimeEl.dataset.value || 'NaN');
  if (!Number.isNaN(cpu) && !Number.isNaN(gpu) && gpu > 0) {
    speedupEl.textContent = (cpu / gpu).toFixed(2) + 'x';
  }
}

function compareOutputs(a, b) {
  if (!a || !b || a.length !== b.length) {
    meanDiffEl.textContent = '—';
    maxDiffEl.textContent = '—';
    return;
  }
  let sum = 0;
  let max = 0;
  const count = a.length;
  for (let i = 0; i < count; i++) {
    const d = Math.abs(a[i] - b[i]);
    sum += d;
```

Продолжение листинга 12.1

```
    if (d > max) max = d;
  }
  meanDiffEl.textContent = (sum / count).toFixed(3);
  maxDiffEl.textContent = max.toFixed(0);
}

function createDemoImage(width, height) {
  const pixels = new Uint8ClampedArray(width * height * 4);
  const cell = Math.max(8, Math.floor(Math.min(width, height) / 24));
  for (let y = 0; y < height; y++) {
    for (let x = 0; x < width; x++) {
      const i = (y * width + x) * 4;
      const nx = x / Math.max(1, width - 1);
```

```
      const ny = y / Math.max(1, height - 1);
      const checker = (((Math.floor(x / cell) + Math.floor(y / cell)) & 1) === 0) ? 1.0 : 0.35;
      const dx = x - width * 0.5;
      const dy = y - height * 0.5;
      const dist = Math.sqrt(dx * dx + dy * dy) / Math.max(1, Math.min(width, height) * 0.55);
      const radial = Math.max(0, 1.0 - dist);
      const waves = 0.5 + 0.5 * Math.sin(nx * 18.0) * Math.cos(ny * 12.0);
      const r = (0.15 + 0.85 * nx) * checker;
      const g = (0.15 + 0.85 * ny) * (0.65 + 0.35 * waves);
      const b = 0.15 + 0.7 * radial + 0.15 * waves;
      pixels[i + 0] = Math.round(clamp01(r) * 255);
      pixels[i + 1] = Math.round(clamp01(g) * 255);
      pixels[i + 2] = Math.round(clamp01(b) * 255);
      pixels[i + 3] = 255;
    }
   }
   return pixels;
  }

  function renderPixelsPreview(ctx, pixels, width, height) {
   const outW = ctx.canvas.width;
   const outH = ctx.canvas.height;
   const preview = new Uint8ClampedArray(outW * outH * 4);
   for (let y = 0; y < outH; y++) {
    const sy = Math.min(height - 1, Math.floor((y / outH) * height));
    for (let x = 0; x < outW; x++) {
     const sx = Math.min(width - 1, Math.floor((x / outW) * width));
     const si = (sy * width + sx) * 4;
     const di = (y * outW + x) * 4;
     preview[di + 0] = pixels[si + 0];
     preview[di + 1] = pixels[si + 1];
     preview[di + 2] = pixels[si + 2];
     preview[di + 3] = pixels[si + 3];
```

Продолжение листинга 12.1

```
    }
   }
   ctx.putImageData(new ImageData(preview, outW, outH), 0, 0);
  }

  function setSourceImage(pixels, width, height) {
   sourcePixels = pixels;
   sourceWidth = width;
   sourceHeight = height;
   setCurrentSize(width, height);
   clearMetrics();
   cpuCtx.clearRect(0, 0, cpuCanvas.width, cpuCanvas.height);
   gpuCtx.clearRect(0, 0, gpuCanvas.width, gpuCanvas.height);
```

```
    cpuInfoEl.textContent = `CPU-результат ещё не рассчитан (${width} × ${height})`;
    gpuInfoEl.textContent = `GPU-результат ещё не рассчитан (${width} × ${height})`;
  }

  function resizePixels(pixels, srcWidth, srcHeight, dstWidth, dstHeight) {
    const srcCanvas = document.createElement('canvas');
    srcCanvas.width = srcWidth;
    srcCanvas.height = srcHeight;
    const srcCtx = srcCanvas.getContext('2d');
    srcCtx.putImageData(new ImageData(new Uint8ClampedArray(pixels), srcWidth, srcHeight), 0, 0);

    const dstCanvas = document.createElement('canvas');
    dstCanvas.width = dstWidth;
    dstCanvas.height = dstHeight;
    const dstCtx = dstCanvas.getContext('2d');
    dstCtx.drawImage(srcCanvas, 0, 0, dstWidth, dstHeight);

    return new Uint8ClampedArray(dstCtx.getImageData(0, 0, dstWidth, dstHeight).data);
  }

  function applyRequestedResolutionToCurrentSource() {
    if (!sourcePixels) return;
    const { width, height } = getRequestedResolution();
    if (width === sourceWidth && height === sourceHeight) return;
    const resized = resizePixels(sourcePixels, sourceWidth, sourceHeight, width, height);
    setSourceImage(resized, width, height);
    log(`Источник приведён к ${width} × ${height}.`);
  }

  function clearWorkspace() {
    const hasSource = !!sourcePixels;
    cpuCodeEl.value = '';
    gpuCodeEl.value = '';
```

Продолжение листинга 12.1

```
    imageInput.value = '';
    clearMetrics();
    cpuCtx.clearRect(0, 0, cpuCanvas.width, cpuCanvas.height);
    gpuCtx.clearRect(0, 0, gpuCanvas.width, gpuCanvas.height);
    if (hasSource) {
      setCurrentSize(sourceWidth, sourceHeight);
      cpuInfoEl.textContent = `CPU-результат ещё не рассчитан (${sourceWidth} ×
${sourceHeight})`;
      gpuInfoEl.textContent = `GPU-результат ещё не рассчитан (${sourceWidth} ×
${sourceHeight})`;
      log('Код и результаты очищены. Текущий источник сохранён.');
      return;
    }
```

```
    setCurrentSize(NaN, NaN);
    cpuInfoEl.textContent = 'CPU-результат ещё не рассчитан';
    gpuInfoEl.textContent = 'GPU-результат ещё не рассчитан';
    log('Рабочая область очищена.');
  }

  function packPixelsToU32(bytes) {
    const count = bytes.length >>> 2;
    const packed = new Uint32Array(count);
    for (let i = 0, p = 0; i < count; i++, p += 4) {
      packed[i] = bytes[p] | (bytes[p + 1] << 8) | (bytes[p + 2] << 16) | (bytes[p + 3] << 24);
    }
    return packed;
  }

  function unpackU32ToPixels(packed) {
    const bytes = new Uint8ClampedArray(packed.length * 4);
    for (let i = 0, p = 0; i < packed.length; i++, p += 4) {
      const v = packed[i];
      bytes[p] = v & 0xff;
      bytes[p + 1] = (v >>> 8) & 0xff;
      bytes[p + 2] = (v >>> 16) & 0xff;
      bytes[p + 3] = (v >>> 24) & 0xff;
    }
    return bytes;
  }

  function compileCpuKernel(studentBody) {
    return new Function(
      'x', 'y', 'width', 'height', 'uvx', 'uvy', 'cx', 'cy',
      'sample', 'sampleNorm', 'rgba', 'luminance', 'clamp01', 'fract', 'mix', 'smoothstep', 'hash2', 'noise2',
'fbm', 'palette',
      studentBody
```

Продолжение листинга 12.1

```
    );
  }

  function processCpu(inputPixelsBytes, width, height) {
    const output = new Uint8ClampedArray(inputPixelsBytes.length);
    const kernel = compileCpuKernel(cpuCodeEl.value);

    function sampleAbs(ix, iy) {
      const sx = Math.max(0, Math.min(width - 1, ix));
      const sy = Math.max(0, Math.min(height - 1, iy));
      const idx = (sy * width + sx) * 4;
      return [
        inputPixelsBytes[idx] / 255,
```

```
        inputPixelsBytes[idx + 1] / 255,
        inputPixelsBytes[idx + 2] / 255,
        inputPixelsBytes[idx + 3] / 255,
       ];
     }

     for (let y = 0; y < height; y++) {
      const uvy = y / Math.max(1, height - 1);
      const cy = uvy * 2.0 - 1.0;
      for (let x = 0; x < width; x++) {
       const uvx = x / Math.max(1, width - 1);
       const cx = uvx * 2.0 - 1.0;
       const sample = (dx, dy) => sampleAbs(x + dx, y + dy);
       const sampleNorm = (u, v) => sampleAbs(Math.round(clamp01(u) * (width - 1)),
Math.round(clamp01(v) * (height - 1)));
       let out;
       try {
        out = kernel(x, y, width, height, uvx, uvy, cx, cy,
          sample, sampleNorm, rgba, luminance, clamp01, fract, mix, smoothstep, hash2, noise2, fbm,
palette);
       } catch (err) {
        throw new Error(`CPU kernel error at pixel (${x}, ${y}): ${err.message}`);
       }
       if (!Array.isArray(out) || out.length < 4) {
        throw new Error('CPU kernel must return [r, g, b, a].');
       }
       const idx = (y * width + x) * 4;
       output[idx] = Math.round(clamp01(out[0]) * 255);
       output[idx + 1] = Math.round(clamp01(out[1]) * 255);
       output[idx + 2] = Math.round(clamp01(out[2]) * 255);
       output[idx + 3] = Math.round(clamp01(out[3]) * 255);
      }
     }
```

Продолжение листинга 12.1

```
    return output;
   }

   async function initWebGPU() {
    if (gpuDevice) return gpuDevice;
    if (!navigator.gpu) {
     throw new Error('WebGPU не поддерживается в этом браузере или недоступен в текущем
режиме запуска.');
    }
    gpuAdapter = await navigator.gpu.requestAdapter();
    if (!gpuAdapter) {
     throw new Error('Не удалось получить GPUAdapter.');
    }
```

```
    gpuDevice = await gpuAdapter.requestDevice();
    gpuDevice.lost.then((info) => {
      log(`GPU device lost: ${info.message || 'no message'}`);
      gpuDevice = null;
    });
    return gpuDevice;
  }

  function buildShader(studentBody) {
    return `
struct Params {
  width: u32,
  height: u32,
  _pad0: u32,
  _pad1: u32,
};

@group(0) @binding(0) var<storage, read> inputPixels: array<u32>;
@group(0) @binding(1) var<storage, read_write> outputPixels: array<u32>;
@group(0) @binding(2) var<uniform> params: Params;

fn clamp01(v: f32) -> f32 {
  return clamp(v, 0.0, 1.0);
}

fn rgba(r: f32, g: f32, b: f32, a: f32) -> vec4<f32> {
  return vec4<f32>(clamp01(r), clamp01(g), clamp01(b), clamp01(a));
}

fn luminance(rgb: vec3<f32>) -> f32 {
  return dot(rgb, vec3<f32>(0.299, 0.587, 0.114));
}
```

Продолжение листинга 12.1

```
fn fractf(x: f32) -> f32 {
  return x - floor(x);
}

fn mix1(a: f32, b: f32, t: f32) -> f32 {
  return a * (1.0 - t) + b * t;
}

fn mix3(a: vec3<f32>, b: vec3<f32>, t: f32) -> vec3<f32> {
  return a * (1.0 - t) + b * t;
}
```

```
fn smoothstep1(edge0: f32, edge1: f32, x: f32) -> f32 {
  let denom = max(1e-6, edge1 - edge0);
  let t = clamp((x - edge0) / denom, 0.0, 1.0);
  return t * t * (3.0 - 2.0 * t);
}

fn hash21(p: vec2<f32>) -> f32 {
  return fractf(sin(dot(p, vec2<f32>(127.1, 311.7))) * 43758.5453123);
}

fn noise2(p: vec2<f32>) -> f32 {
  let i = floor(p);
  let f = vec2<f32>(fractf(p.x), fractf(p.y));
  let u = f * f * (vec2<f32>(3.0, 3.0) - 2.0 * f);
  let a = hash21(i + vec2<f32>(0.0, 0.0));
  let b = hash21(i + vec2<f32>(1.0, 0.0));
  let c = hash21(i + vec2<f32>(0.0, 1.0));
  let d = hash21(i + vec2<f32>(1.0, 1.0));
  return mix1(mix1(a, b, u.x), mix1(c, d, u.x), u.y);
}

fn fbm(seed: vec2<f32>) -> f32 {
  var p = seed;
  var a = 0.5;
  var v = 0.0;
  for (var i: i32 = 0; i < 5; i = i + 1) {
    v = v + a * noise2(p);
    p = p * vec2<f32>(2.03, 2.01) + vec2<f32>(17.13, 8.73);
    a = a * 0.5;
  }
  return v;
}

fn palette(t: f32) -> vec3<f32> {
```

Продолжение листинга 12.1

```
  return vec3<f32>(
    0.5 + 0.5 * cos(6.2831853 * (t + 0.00)),
    0.5 + 0.5 * cos(6.2831853 * (t + 0.33)),
    0.5 + 0.5 * cos(6.2831853 * (t + 0.67))
  );
}

fn unpackPixel(v: u32) -> vec4<f32> {
  let r = f32(v & 0xffu) / 255.0;
  let g = f32((v >> 8u) & 0xffu) / 255.0;
  let b = f32((v >> 16u) & 0xffu) / 255.0;
  let a = f32((v >> 24u) & 0xffu) / 255.0;
```

```
  return vec4<f32>(r, g, b, a);
}

fn packPixel(v: vec4<f32>) -> u32 {
  let c = vec4<u32>(round(clamp(v, vec4<f32>(0.0), vec4<f32>(1.0)) * 255.0));
  return c.x | (c.y << 8u) | (c.z << 16u) | (c.w << 24u);
}

fn sampleAt(ix: i32, iy: i32) -> vec4<f32> {
  let maxX = i32(params.width) - 1;
  let maxY = i32(params.height) - 1;
  let sx = u32(clamp(ix, 0, maxX));
  let sy = u32(clamp(iy, 0, maxY));
  let idx = sy * params.width + sx;
  return unpackPixel(inputPixels[idx]);
}

fn sampleNorm(u: f32, v: f32) -> vec4<f32> {
  let sx = i32(clamp(u, 0.0, 1.0) * max(1.0, f32(params.width - 1u)));
  let sy = i32(clamp(v, 0.0, 1.0) * max(1.0, f32(params.height - 1u)));
  return sampleAt(sx, sy);
}

fn studentKernel(x: u32, y: u32) -> vec4<f32> {
  let uvx = f32(x) / max(1.0, f32(params.width - 1u));
  let uvy = f32(y) / max(1.0, f32(params.height - 1u));
  let cx = uvx * 2.0 - 1.0;
  let cy = uvy * 2.0 - 1.0;
${studentBody}
}

@compute @workgroup_size(${workgroupSizeX}, ${workgroupSizeY}, 1)
fn main(@builtin(global_invocation_id) gid: vec3<u32>) {
  let x = gid.x;
```

Продолжение листинга 12.1

```
  let y = gid.y;
  if (x >= params.width || y >= params.height) {
    return;
  }
  let idx = y * params.width + x;
  outputPixels[idx] = packPixel(studentKernel(x, y));
}
`;
    }

    async function processGpu(inputPixelsBytes, width, height) {
      const device = await initWebGPU();
```

```
  const packedInput = packPixelsToU32(inputPixelsBytes);
  const bufferSize = packedInput.byteLength;
  const uniformSize = 16;

  const inputBuffer = device.createBuffer({
   size: bufferSize,
   usage: GPUBufferUsage.STORAGE | GPUBufferUsage.COPY_DST,
  });
  const outputBuffer = device.createBuffer({
   size: bufferSize,
   usage: GPUBufferUsage.STORAGE | GPUBufferUsage.COPY_SRC,
  });
  const readbackBuffer = device.createBuffer({
   size: bufferSize,
   usage: GPUBufferUsage.COPY_DST | GPUBufferUsage.MAP_READ,
  });
  const uniformBuffer = device.createBuffer({
   size: uniformSize,
   usage: GPUBufferUsage.UNIFORM | GPUBufferUsage.COPY_DST,
  });

  const shaderCode = buildShader(gpuCodeEl.value);
  const shaderModule = device.createShaderModule({ code: shaderCode });
  if (typeof shaderModule.getCompilationInfo === 'function') {
   const info = await shaderModule.getCompilationInfo();
   const errors = info.messages.filter((m) => m.type === 'error');
   if (errors.length > 0) {
    const msg = errors.map((m) => `${m.lineNum}:${m.linePos} ${m.message}`).join('\n');
    throw new Error('WGSL compilation error:\n' + msg);
   }
  }

  const bindGroupLayout = device.createBindGroupLayout({
   entries: [
```

Продолжение листинга 12.1

```
    { binding: 0, visibility: GPUShaderStage.COMPUTE, buffer: { type: 'read-only-storage' } },
    { binding: 1, visibility: GPUShaderStage.COMPUTE, buffer: { type: 'storage' } },
    { binding: 2, visibility: GPUShaderStage.COMPUTE, buffer: { type: 'uniform' } },
   ],
  });
  const pipelineLayout = device.createPipelineLayout({
   bindGroupLayouts: [bindGroupLayout],
  });
  const pipeline = device.createComputePipeline({
   layout: pipelineLayout,
   compute: { module: shaderModule, entryPoint: 'main' },
  });
```

```
    const uniformData = new ArrayBuffer(uniformSize);
    const u32 = new Uint32Array(uniformData);
    u32[0] = width;
    u32[1] = height;

    device.queue.writeBuffer(inputBuffer, 0, packedInput.buffer, packedInput.byteOffset,
packedInput.byteLength);
    device.queue.writeBuffer(uniformBuffer, 0, uniformData);

    const bindGroup = device.createBindGroup({
      layout: bindGroupLayout,
      entries: [
        { binding: 0, resource: { buffer: inputBuffer } },
        { binding: 1, resource: { buffer: outputBuffer } },
        { binding: 2, resource: { buffer: uniformBuffer } },
      ],
    });

    const encoder = device.createCommandEncoder();
    const pass = encoder.beginComputePass();
    pass.setPipeline(pipeline);
    pass.setBindGroup(0, bindGroup);
    pass.dispatchWorkgroups(Math.ceil(width / workgroupSizeX), Math.ceil(height /
workgroupSizeY));
    pass.end();
    encoder.copyBufferToBuffer(outputBuffer, 0, readbackBuffer, 0, bufferSize);

    const t0 = performance.now();
    device.queue.submit([encoder.finish()]);
    await device.queue.onSubmittedWorkDone();
    await readbackBuffer.mapAsync(GPUMapMode.READ);
    const rawCopy = readbackBuffer.getMappedRange().slice(0);
    const t1 = performance.now();
```

Продолжение листинга 12.1

```
    const packedOutput = new Uint32Array(rawCopy);
    const pixels = unpackU32ToPixels(packedOutput);
    readbackBuffer.unmap();

    inputBuffer.destroy();
    outputBuffer.destroy();
    readbackBuffer.destroy();
    uniformBuffer.destroy();

    return { output: pixels, timeMs: t1 - t0 };
  }
```

```
  async function renderResult(ctx, infoEl, pixels, width, height, label) {
    renderPixelsPreview(ctx, pixels, width, height);
    infoEl.textContent = `${label}: ${width} × ${height}`;
  }

  async function runCpu() {
    if (!sourcePixels) {
      alert('Сначала сгенерируйте demo-источник или загрузите изображение.');
      return;
    }
    if (!cpuCodeEl.value.trim()) {
      alert('Введите код CPU в редакторе.');
      return;
    }
    clearMetrics();
    const t0 = performance.now();
    const output = processCpu(sourcePixels, sourceWidth, sourceHeight);
    const t1 = performance.now();
    lastCpuPixels = output;
    await renderResult(cpuCtx, cpuInfoEl, output, sourceWidth, sourceHeight, 'CPU-результат');
    const time = t1 - t0;
    cpuTimeEl.textContent = `${time.toFixed(3)} ms`;
    cpuTimeEl.dataset.value = String(time);
    updateSpeedup();
    log(`CPU завершён: ${time.toFixed(3)} ms`);
  }

  async function runGpu() {
    if (!sourcePixels) {
      alert('Сначала сгенерируйте demo-источник или загрузите изображение.');
      return;
    }
    if (!gpuCodeEl.value.trim()) {
```

Продолжение листинга 12.1

```
      alert('Введите код GPU в редакторе.');
      return;
    }
    clearMetrics();
    try {
      const result = await processGpu(sourcePixels, sourceWidth, sourceHeight);
      lastGpuPixels = result.output;
      await renderResult(gpuCtx, gpuInfoEl, result.output, sourceWidth, sourceHeight, 'GPU-
результат');
      gpuTimeEl.textContent = `${result.timeMs.toFixed(3)} ms`;
      gpuTimeEl.dataset.value = String(result.timeMs);
      updateSpeedup();
```

```
      log(`GPU завершён: ${result.timeMs.toFixed(3)} ms`);
    } catch (err) {
      log('Ошибка GPU: ' + err.message);
      alert('Ошибка GPU: ' + err.message);
    }
  }

  async function runBoth() {
    if (!sourcePixels) {
      alert('Сначала сгенерируйте demo-источник или загрузите изображение.');
      return;
    }
    if (!cpuCodeEl.value.trim() || !gpuCodeEl.value.trim()) {
      alert('Введите код CPU и GPU в редакторах.');
      return;
    }
    clearMetrics();

    try {
      const t0 = performance.now();
      lastCpuPixels = processCpu(sourcePixels, sourceWidth, sourceHeight);
      const t1 = performance.now();
      await renderResult(cpuCtx, cpuInfoEl, lastCpuPixels, sourceWidth, sourceHeight, 'CPU-
результат');
      const cpuMs = t1 - t0;
      cpuTimeEl.textContent = `${cpuMs.toFixed(3)} ms`;
      cpuTimeEl.dataset.value = String(cpuMs);
      log(`CPU завершён: ${cpuMs.toFixed(3)} ms`);
    } catch (err) {
      log('Ошибка CPU: ' + err.message);
      alert('Ошибка CPU: ' + err.message);
      return;
    }
```

Продолжение листинга 12.1

```
    try {
      const gpuResult = await processGpu(sourcePixels, sourceWidth, sourceHeight);
      lastGpuPixels = gpuResult.output;
      await renderResult(gpuCtx, gpuInfoEl, lastGpuPixels, sourceWidth, sourceHeight, 'GPU-
результат');
      gpuTimeEl.textContent = `${gpuResult.timeMs.toFixed(3)} ms`;
      gpuTimeEl.dataset.value = String(gpuResult.timeMs);
      log(`GPU завершён: ${gpuResult.timeMs.toFixed(3)} ms`);
    } catch (err) {
      log('Ошибка GPU: ' + err.message);
      alert('Ошибка GPU: ' + err.message);
```

```
      return;
    }

    updateSpeedup();
    compareOutputs(lastCpuPixels, lastGpuPixels);
  }

  async function generateDemoImage() {
    const { width, height } = getRequestedResolution();
    const pixels = createDemoImage(width, height);
    setSourceImage(pixels, width, height);
    cpuCodeEl.value = demoCpuCode;
    gpuCodeEl.value = demoGpuCode;
    log(`Сгенерирован demo-источник ${width} × ${height} и загружен demo-код (CPU/GPU: grayscale).`);
  }

  function prepareInitialSource() {
    const { width, height } = getRequestedResolution();
    const pixels = createDemoImage(width, height);
    setSourceImage(pixels, width, height);
  }

  async function showSupportInfo() {
    if (!navigator.gpu) {
      supportInfo.textContent = 'WebGPU не обнаружен. GPU-часть не будет доступна. Используйте актуальный браузер с поддержкой WebGPU и запускайте страницу через localhost.';
      return;
    }
    try {
      const adapter = await navigator.gpu.requestAdapter();
      if (!adapter) {
        supportInfo.textContent = 'navigator.gpu есть, но получить GPUAdapter не удалось.';
        return;
      }
```

Продолжение листинга 12.1

```
      supportInfo.textContent = `WebGPU обнаружен. Workgroup в шаблоне: ${workgroupSizeX} × ${workgroupSizeY}. Для 4K и особенно 8K учитывайте рост нагрузки на память.`;
    } catch (err) {
      supportInfo.textContent = 'Ошибка при проверке WebGPU: ' + err.message;
    }
  }

  imageInput.addEventListener('change', async (event) => {
    const file = event.target.files?.[0];
    if (!file) return;
    try {
```

```
      const { width: targetWidth, height: targetHeight } = getRequestedResolution();
      const bitmap = await createImageBitmap(file);
      const canvas = document.createElement('canvas');
      canvas.width = targetWidth;
      canvas.height = targetHeight;
      const ctx = canvas.getContext('2d');
      ctx.drawImage(bitmap, 0, 0, targetWidth, targetHeight);
      const data = ctx.getImageData(0, 0, targetWidth, targetHeight).data;
      setSourceImage(new Uint8ClampedArray(data), targetWidth, targetHeight);
      log(`Загружено изображение ${bitmap.width} × ${bitmap.height}, приведено к ${targetWidth}
× ${targetHeight}`);
      bitmap.close();
     } catch (err) {
      log('Ошибка загрузки изображения: ' + err.message);
      alert('Ошибка загрузки изображения: ' + err.message);
     }
    });

    document.getElementById('generateBtn').addEventListener('click', generateDemoImage);
    clearBtn.addEventListener('click', clearWorkspace);
    document.getElementById('cpuBtn').addEventListener('click', runCpu);
    document.getElementById('gpuBtn').addEventListener('click', runGpu);
    document.getElementById('bothBtn').addEventListener('click', runBoth);
    resolutionWidthEl.addEventListener('change', () => {
     try {
      applyRequestedResolutionToCurrentSource();
     } catch (err) {
      alert(err.message);
     }
    });
    resolutionHeightEl.addEventListener('change', () => {
     try {
      applyRequestedResolutionToCurrentSource();
     } catch (err) {
      alert(err.message);
```

Продолжение листинга 12.1

```
     }
    });

    showSupportInfo();
    prepareInitialSource();
    log('Страница готова. Пишите код сразу или нажмите "Сгенерировать demo-источник".');
  </script>
</body>
</html>
```

Листинг 12.2 – Код python-скрипта

```
from http.server import ThreadingHTTPServer, SimpleHTTPRequestHandler
import os
import webbrowser

HOST = 'localhost'
PORT = 8000

if __name__ == '__main__':
    os.chdir(os.path.dirname(os.path.abspath(__file__)))
    url = f'http://{HOST}:{PORT}'
    print(f'Serving at {url}')
    try:
        webbrowser.open(url)
    except Exception:
        pass
    server = ThreadingHTTPServer((HOST, PORT), SimpleHTTPRequestHandler)
    server.serve_forever()
```

ЗАКЛЮЧЕНИЕ

В настоящем учебно-методическом пособии приведены материалы двенадцати практических занятий по дисциплине «Архитектура вычислительных машин и систем». Для каждого практического занятия имеются краткие теоретические сведения, дополняющие материал лекционных занятий, разбирается пример выполнения практических заданий, приводятся варианты заданий для самостоятельного решения студентами.

СПИСОК ИСТОЧНИКОВ И ЛИТЕРАТУРЫ

1. Программирование на языке ассемблера RISC-V [Электронный ресурс]: С. Смит; пер. А. Логунов; науч. ред. А. Ю. Романов. — М.: ДМК Пресс, 2024. — Электрон. опт. диск (ISO)

2. Цифровой синтез: RISC-V [Электронный ресурс]: А. Ю. Романов. — М.: ДМК Пресс, 2024. — Электрон. опт. диск (ISO)

3. Архитектура встраиваемых систем [Электронный ресурс]: Д. Лакамера; пер. В. Яценков; науч. ред. А. Ю. Романов. — М.: ДМК Пресс, 2023. — Электрон. опт. диск (ISO)

4. Проектирование реконфигурируемых систем в LabVIEW FPGA [Электронный ресурс]: Е. Д. Баран, А. Ю. Романов. — М.: ДМК Пресс, 2023. — Электрон. опт. диск (ISO)

5. Архитектура вычислительных машин и систем [Электронный ресурс]: учеб.-метод. пособие / Мусихин А.Г., Морозов В.А., Гололобов А.А. [и др.]. — М.: РТУ МИРЭА, 2024. — Электрон. опт. диск (ISO)

6. Схемотехника устройств компьютерных систем. Ч. 1 [Электронный ресурс]: лабораторный практикум / Н. А. Дуксин, Д. В. Люлява, И. Е. Тарасов. — М.: РТУ МИРЭА, 2023. — Электрон. опт. диск (ISO)

7. Инструментальные средства разработки программно-аппаратных комплексов [Электронный ресурс]: учебное пособие / И. Е. Тарасов. — М.: РТУ МИРЭА, 2021. — Электрон. опт. диск (ISO)

8. Разработка программно-аппаратного обеспечения информационных и автоматизированных систем [Электронный ресурс]: учебное пособие / Д. С. Потехин, И. Е. Тарасов. — М.: РТУ МИРЭА, 2022. — Электрон. опт. диск (ISO)

9. Архитектура вычислительных машин и систем [Электронный ресурс]: учебное пособие / А. Г. Мусихин, Н. А. Смирнов. — М.: РТУ МИРЭА, 2021. — Электрон. опт. диск (ISO).

10. Методы проектирования цифровых устройств в составе инфокоммуникационных систем [Электронный ресурс]: учебное пособие / Д. С. Потехин. — М.: РТУ МИРЭА, 2022. — Электрон. опт. диск (ISO)

11. Схемотехника устройств компьютерных систем. Ч. 1 [Электронный ресурс]: лабораторный практикум / Н. А. Дуксин, Д. В. Люлява, И. Е. Тарасов. — М.: РТУ МИРЭА, 2023. — Электрон. опт. диск (ISO)

12. Коников, А.И. Электронные вычислительные машины: учеб.-метод. пособие / О.М. Баранова; Нац. исследоват. Моск. гос. строит. ун-т; А.И.

Коников. — Электрон. изд. — Москва: Изд-во МИСИ-МГСУ, 2021. — 39 с.: ил. — Электрон. дан. и прогр. (5,2 Мб). — ISBN 978-5-7264-2859-8. — ISBN 978-5-7264-2860-4. — URL: https://lib.rucont.ru/efd/786123

13. Овчеренко, В.А. Периферийные устройства информационных систем. Физические принципы организации и интерфейсы ввода-вывода: учеб. пособие / В.Г. Токараев; В.А. Овчеренко. — Новосибирск: Изд-во НГТУ, 2018 — 75 с.: ил. — ISBN 978-5-7782-3625-7. — URL: https://lib.rucont.ru/efd/774730

14. Фомин, Д.В. Основы компьютерной электроники: учебное пособие / Д.В. Фомин. — 2-е изд. — Москва: Директ-Медиа, 2019. — 109 с. — ISBN 978-5-4499-0152-1. — URL: https://lib.rucont.ru/efd/798852

СВЕДЕНИЯ ОБ АВТОРАХ

Гололобов Александр Алексеевич, старший преподаватель кафедры вычислительной техники института информационных технологий Российского технологического университета — МИРЭА.

Гуличева Анастасия Алексеевна, старший преподаватель кафедры вычислительной техники института информационных технологий Российского технологического университета — МИРЭА.

Цынгалёв Павел Сергеевич, ассистент кафедры вычислительной техники института информационных технологий Российского технологического университета — МИРЭА.